When Averages Fail

Mondrian Conformal Prediction in ER Discrete-Event Simulation

G Venugopalan · Vipin Sudhakar · Rithvik Arulprakash · Harshith Kv

Faculty Guide: Akhil VM

Preface

Conformal prediction has, over the past two decades, become one of the principal distribution-free tools for attaching a reliable uncertainty guarantee to an otherwise unconstrained point predictor - a property that matters increasingly as machine-learned surrogate models are deployed in operational decision-making settings where a wrong-but-confident interval can be costly. This report examines that guarantee's behavior in one such setting - hospital emergency department operations - chosen because its underlying dynamics (discrete, stochastic, queueing-driven) differ structurally from the physics-simulation domains in which conformal prediction's surrogate-modeling use was first validated.

Standard conformal prediction (CP) guarantees marginal coverage - correctness averaged across an entire calibration distribution - but provides no guarantee within any specific subgroup or operating condition. Gopakumar et al. [1], whose validation of conformal prediction for surrogate-model uncertainty quantification spans several physics-simulation domains, name this marginal-versus-conditional gap as an explicit, untested limitation of their own results. This report tests whether that limitation holds - and whether Mondrian conformal prediction, which calibrates separately per category rather than pooling, closes it - in a domain not previously tested for this purpose: a discrete-event simulation of a hospital emergency department, calibrated on real arrival and triage-acuity data.

A gradient-boosting surrogate is trained to approximate the simulation's scenario-level outputs from two real, available covariates - staffing capacity and arrival-rate multiplier - and three core uncertainty quantification methods are compared on identical calibration and test data: a Gaussian process baseline, standard conformal prediction, and Mondrian conformal prediction. The central finding (Chapter 7) is that a single pooled conformal quantile silently fails the operationally critical scenario - an understaffed emergency department during a demand surge - with coverage as low as 68.2 percent against a 90 percent target, while Mondrian conformal prediction corrects this to 90.9-92.0 percent. The effect is confirmed statistically significant ($p < 0.001$ on every affected target) across 30 independent calibration/test draws and replicates at a second, independent hospital department with materially different volume and acuity characteristics (Chapter 9). Chapter 6 extends the conformal toolkit itself with four further methods evaluated on real data: conformalized

quantile regression and Mondrian-CQR (a stronger, width-adaptive baseline), conformal risk control (bounding operational overflow severity, not just its probability), adaptive conformal inference (recovering most of a static method's lost coverage under a live demand-surge stream without retraining), and likelihood-ratio weighted conformal prediction (a real, bounded partial remedy under known covariate shift). Chapter 8 tests the base paper's second stated limitation directly: conformal prediction's exchangeability assumption is stress-tested to destruction across two surrogate architectures and five surrogate architectures are benchmarked in Chapter 7, confirming the coverage collapse is not an artifact of one specific model. Chapter 10 translates these results into operational terms with a working prediction- interval dashboard and a capacity-planning optimization built directly on this project's own conformal intervals. This is, to the best of this project's literature review, the first empirical test of conformal prediction's marginal-coverage limitation outside the physics-simulation domains in which it was originally validated.

This report is organized as follows. Chapter 1 introduces the problem and motivates the choice of domain; Chapter 2 reviews the relevant literature; Chapter 3 states the research gap and problem addressed; Chapters 4 and 5 describe the methodology and implementation in full, including derivations of every theoretical result later chapters rely on; Chapter 6 extends the conformal toolkit with four further methods; Chapter 7 presents this report's central empirical results; Chapter 8 stress-tests every method against exchangeability violation; Chapter 9 tests cross-site generalization; Chapter 10 addresses operational deployment; and Chapter 11 synthesizes all of it with limitations and future scope. Readers primarily interested in the empirical findings rather than the underlying theory may proceed directly to Chapter 7 after Chapter 3.

TABLE OF CONTENTS

LIST OF ABBREVIATIONS

Abbreviation	Full Form
CP	Conformal Prediction
CQR	Conformalized Quantile Regression
DES	Discrete-Event Simulation
ED	Emergency Department
ER	Emergency Room
ESI	Emergency Severity Index (1 = most acute, 5 = least acute)
GBR	Gradient Boosting Regressor (HistGradientBoostingRegressor)
GP	Gaussian Process
LOS	Length of Stay
MAE	Mean Absolute Error
MLP	Multi-Layer Perceptron
OOD	Out-of-Distribution
R^2	Coefficient of Determination
RMSE	Root Mean Squared Error
UQ	Uncertainty Quantification

Chapter 1: The ER Uncertainty Dilemma

Surrogate Metamodeling and Operational Volatility in Emergency Healthcare

1.1 Background and Motivation

Emergency departments (EDs) are among the most heavily studied stochastic service systems in operations research, precisely because they combine three properties that make good decision-making difficult: highly variable arrival patterns, a service process whose duration depends on patient acuity in ways that are only partially observable at intake, and severe, immediate consequences for getting staffing or capacity decisions wrong. Overcrowding in an ED is not merely an inconvenience; a substantial body of clinical and health-services literature associates ED crowding with delayed treatment, increased rates of patients leaving without being seen, and worse downstream outcomes. At the same time, over-staffing relative to demand is costly and, given that clinical staff are a scarce resource in most health systems, frequently not even feasible to correct for by simply adding more capacity.

Because real EDs cannot be experimented on directly - a hospital cannot try five different staffing policies on the same day to see which produces the shortest waits - operations researchers and hospital administrators have long relied on models: analytic queueing theory on one end of the spectrum, and discrete-event simulation (DES) on the other. Queueing theory offers closed-form or semi-closed-form expressions for quantities such as expected wait time under simplifying assumptions (Poisson arrivals, exponential or otherwise well-behaved service times, a fixed number of homogeneous servers). These assumptions are mathematically convenient but often violated in a real ED, where service times are heavy-tailed and acuity-dependent, and where the practical quantity of interest - a full distribution of possible wait times under a given staffing policy, not just its mean - is exactly what a closed-form queueing result is least well suited to provide. DES relaxes these assumptions at the cost of losing closed-form tractability: a DES model can simulate patient-

by-patient arrivals, triage, and service under essentially arbitrary distributional assumptions, and its output for any one run is a sample from the same kind of stochastic process a real ED experiences, rather than a mean-field approximation of it.

The price of that flexibility is computational: a sufficiently detailed DES, run enough times to characterize the distribution of outcomes under a given scenario (a given staffing level, a given arrival-rate regime), can be expensive to evaluate repeatedly - particularly if the goal is to explore many scenarios, as it is in staffing and capacity planning, or to embed the simulation inside an optimization loop. This computational cost is the practical motivation for training a surrogate model: a fast, learned approximation of the DES's input-output relationship, trained on a finite set of DES runs and then queried in place of the DES itself wherever repeated, low-latency evaluation is needed.

1.2 The Role of Surrogate Models in Simulation-Based Decision Making

A surrogate model, in the sense used throughout this report, is a supervised machine learning model trained to predict a simulator's output metrics (here, daily patient count, mean wait time, mean total time in system, and the 95th percentile of wait time) directly from the simulator's scenario-level inputs (here, staffing capacity and an arrival-rate multiplier), without re-running the simulator itself. Once trained, a surrogate can be evaluated in microseconds rather than the seconds-to-minutes a full DES run may take, enabling the kind of dense scenario sweeps, sensitivity analyses, and what-if staffing comparisons that would be computationally prohibitive if each evaluation required a fresh simulation.

This efficiency is not free, however. A surrogate model is only as reliable as the region of input space its training data actually covers, and it inherits - indeed, it can amplify - two distinct sources of uncertainty: the epistemic uncertainty of the learned function itself (the surrogate is a finite-sample approximation, imperfectly fit even within its training domain) and the aleatoric uncertainty of the underlying stochastic process it approximates (a DES with fixed inputs still produces different outputs on different random seeds, because arrivals, triage assignment, and service durations are all drawn from probability distributions). A surrogate's point prediction alone - "expected mean wait time is 42 minutes" - collapses both of these sources of uncertainty into a single number and, critically, gives a decision-maker no way to distinguish a scenario where 42 minutes is a confident, tightly-clustered estimate from one where it is a rough central tendency over an output that could plausibly range from 20 to 90 minutes.

For an operational decision such as ED staffing, where the cost of being wrong is asymmetric and often severe in exactly the tail scenarios a point estimate is least informative about, this gap between "central estimate" and "how much to trust the central estimate" is not a cosmetic omission - it is the difference between a genuinely useful decision-support tool and one that looks precise while quietly discarding the information a decision-maker most needs.

1.3 The Need for Uncertainty Quantification

Uncertainty quantification (UQ) is the general term for methods that accompany a point prediction with some characterization of its reliability - most commonly, an interval or a full predictive distribution rather than a single number. A wide family of UQ methods exists for machine learning models, ranging from Bayesian approaches (Gaussian processes, Bayesian neural networks, and their approximations) through frequentist resampling methods (bootstrap-based intervals) to ensemble-based approaches (deep ensembles, random-forest quantile estimates) and, most relevant to this project, distribution-free methods based on conformal prediction. Each family makes a different trade-off between the strength of its theoretical guarantee, its computational cost, and the assumptions it requires about the underlying data or model.

Gaussian process (GP) regression, used in this project as a UQ baseline (see Chapter 4 and Chapter 7), is a Bayesian nonparametric method that produces a full posterior predictive distribution - and therefore both a point estimate and a principled uncertainty estimate - for any query point, under the assumption that the underlying function is well described by a specified covariance (kernel) structure. Its central practical limitation is computational: exact GP inference scales cubically in the number of training points, which makes it expensive to refit as new data arrives and effectively rules out its use on large training sets without approximation. It also provides no finite-sample coverage guarantee: its intervals are only as trustworthy as the appropriateness of its modeling assumptions (kernel choice, Gaussian noise, stationarity) for the data at hand, and there is no distribution-free bound on how far its empirical coverage can drift from its nominal target when those assumptions are violated.

Conformal prediction (CP), the family of methods this project is centrally concerned with, offers a different trade-off. It is model-agnostic - it wraps around any already-trained point predictor, including the gradient-boosting and neural-network surrogates used in this project, without requiring the underlying model to be retrained or to expose calibrated uncertainty itself - and it comes with a finite-sample, distribution-free

coverage guarantee: under an exchangeability assumption between the calibration data and future test points, a conformal prediction interval constructed at a nominal (1 - alpha) confidence level is guaranteed to contain the true value with probability at least (1 - alpha), regardless of the correctness of the underlying point predictor's modeling assumptions. This guarantee is attractive precisely because it does not depend on the point predictor being a good model of reality, only on the exchangeability of calibration and test data - a much weaker and more checkable assumption than "the model's distributional assumptions are correct."

1.4 Conformal Prediction as a Distribution-Free UQ Framework

The theoretical foundation of conformal prediction was laid by Vovk, Gammerman, and collaborators in the late 1990s and formalized in the book Algorithmic Learning in a Random World [2], with Shafer and Vovk's tutorial [3] (surveyed in Chapter 2) later popularizing the method outside the original algorithmic-learning-theory community. The core idea is simple to state: given a trained point predictor and a held-out calibration set, compute a nonconformity score for each calibration point (typically the absolute residual between the predictor's output and the true value), and use the (1 - alpha) empirical quantile of those scores as a fixed margin added to and subtracted from any future point prediction. Because the calibration residuals and a genuinely exchangeable test residual are, by construction, draws from the same underlying distribution, the resulting interval is guaranteed - via a straightforward rank-based argument, not an asymptotic approximation - to cover the true value at least (1 - alpha) of the time, in finite samples, without any assumption about the shape of the residual distribution.

This guarantee, however, is explicitly marginal: it holds averaged over the entire calibration and test distribution, and says nothing about whether coverage holds within any specific subgroup of that distribution. If a surrogate model is systematically less accurate under one operating regime than another - for instance, understaffed and high-demand scenarios versus comfortably staffed and low-demand ones - a single pooled conformal interval, calibrated on average difficulty across all regimes, can silently undercover in precisely the regime where a decision-maker most needs it to be reliable, while simultaneously overcovering (wasting interval width) in the easy regime. This marginal-versus-conditional coverage gap, and Mondrian conformal prediction's remedy for it (calibrating a separate quantile per category rather than one pooled quantile across the whole calibration set), forms the central methodological axis of this project and is developed in full in Chapters 2, 4, and 6.

Gopakumar et al. [1] - the base paper this project directly extends - validate conformal prediction for surrogate-model uncertainty quantification across several physics-simulation domains (partial differential equations, magnetohydrodynamics, weather modeling, and fusion-plasma simulation) and explicitly name two limitations of their own results as untested: the marginal (rather than conditional) nature of CP's coverage guarantee, and the exchangeability assumption's fragility under distribution shift between calibration and test data. Both limitations are stated as open questions in a physics-simulation context; neither had, to the best of the literature review conducted for this project (Chapter 2), been tested in a fundamentally different domain such as discrete-event queueing simulation. This report's guiding question, developed fully in Chapter 3, is whether the first of these - the marginal-versus-conditional coverage gap - holds, and whether Mondrian conformal prediction remedies it, in exactly such a domain: an ED discrete-event simulation calibrated on real hospital data. The second limitation, exchangeability under distribution shift, was also stress-tested as part of this project's broader implementation (Section 4.5.2) and is referenced where relevant, but is not this report's central subject; it is summarized as a secondary finding and flagged as a natural direction for future work (Section 7.4).

1.5 Overview of the Project

This project builds a four-stage pipeline. First, a discrete-event simulation of a single hospital emergency department is implemented in SimPy and calibrated on real arrival-pattern and triage-acuity data extracted from a large, de-identified Kaggle dataset of ED visits (Chapter 5 details the dataset and the calibration procedure, including a deliberate and clearly documented distinction between the arrival process, which is calibrated directly on real data, and service-time distributions, which - because the dataset contains no length-of-stay field - are calibrated on literature-standard parameters by acuity level rather than presented as data-derived). Second, the calibrated DES is run across thousands of randomly sampled staffing-capacity and arrival-rate scenarios to generate a labeled training set, from which a surrogate regression model is trained to predict four scenario-level output metrics without needing to re-run the simulation. Third, three uncertainty quantification approaches are applied on top of the trained surrogate and compared on identical calibration and test data: a Gaussian process baseline, standard (pooled) conformal prediction, and Mondrian conformal prediction, later extended with conformalized quantile regression (CQR) and a combined Mondrian-CQR variant as stronger baselines. Fourth, the core comparison is stress-tested for robustness along two independent axes central to this report - statistical significance across

30 repeated calibration/test draws (rather than trusting a single split), and a second, independent hospital department (with materially different patient volume and acuity mix) to check whether findings generalize beyond a single site - with a third axis, a second surrogate architecture, used specifically to test the exchangeability question summarized in Chapter 8 rather than the report's central Mondrian CP finding.

This report's central finding is a genuine marginal-versus-conditional coverage gap that Mondrian conformal prediction closes where it is real, established in Chapter 7 across a single representative split, 30 repeated draws with formal statistical significance testing, a stronger width-adaptive baseline (conformalized quantile regression), and independent replication at a second hospital department. Chapters 1 through 5 lay the theoretical, methodological, and implementation groundwork this finding rests on; Chapter 7 develops the result itself in full depth, and Chapter 11 concludes with its implications, limitations, and future scope.

1.6 Organization of the Report

Chapter 2 surveys thirty papers drawn from five areas directly relevant to this project: conformal prediction foundations, Mondrian and conditional-coverage methods, surrogate modeling and uncertainty quantification more broadly, queueing theory and ED operations research, and discrete-event simulation together with ED-specific machine learning applications. Chapter 3 sharpens this survey into an explicit statement of the research gap this project addresses and the problem statement that follows from it. Chapter 4 describes the DES's structure and calibration and the mathematics of standard and Mondrian conformal prediction in full technical detail. Chapter 5 documents the implementation and software architecture. Chapter 6 extends the conformal toolkit itself - conformalized quantile regression, conformal risk control, and adaptive conformal inference - as methodology, ahead of any results. Chapter 7 presents this project's central empirical results: DES and surrogate validation, the marginal-versus- conditional coverage gap Mondrian CP closes, and the full method comparison. Chapter 8 stress-tests every method in Chapter 7 against a genuine violation of the exchangeability assumption. Chapter 9 tests whether the Chapter 7 finding generalizes to an independent hospital department. Chapter 10 translates the preceding results into operational terms - a prototype decision-support dashboard, a capacity-planning optimization built directly on this project's own conformal intervals, and a grounded discussion of what regulatory deployment would actually require. Chapter 11 synthesizes all of it and closes with limitations and future scope.

References and appendices (source code listings and supplementary result tables) follow.

1.7 Extended Theoretical Foundations of Operational Uncertainty

Before this report's specific methodology is introduced (Chapter 4), it is worth establishing, in general terms, why an emergency department's operational uncertainty resists the deterministic point-forecasting treatment it might otherwise receive, and what a decision-maker's actual risk exposure looks like once that uncertainty is acknowledged rather than averaged away. The three subsections below are deliberately general - they do not yet reference this project's own DES or its specific numbers - so that Chapters 6 through 10's specific results can be read as instances of these broader principles rather than as a self-contained numerical exercise.

1.7.1 Taxonomy of Operational Volatility

Emergency department operational volatility is not a single phenomenon, and treating it as one obscures which mitigation actually applies to which source. It is useful to distinguish three structurally different categories.

Short-term demand shocks are sudden, comparatively brief departures from the prevailing arrival rate - a multi-vehicle collision, a localized disease outbreak, an unplanned closure at a neighboring facility diverting its patients - that resolve within hours to a few days. Their defining statistical property is that they are, from the perspective of a model calibrated on historical data, genuinely unpredictable in timing, even though their general character (a temporary multiplicative surge in arrival rate) can be characterized in advance. This project's exchangeability stress test (Chapter 8) is a direct model of exactly this category - a test distribution whose arrival-rate multiplier is pushed well past anything seen during calibration.

Structural seasonal shifts are slower, more predictable departures - the well-documented within-week and within-day arrival-rate cycles this project's own DES calibrates on directly (Section 4.2.3), and slower still, annual patterns (influenza season, holiday-period injury rates) this project's single-year-slice calibration does not attempt to capture. Unlike demand shocks, these are not exchangeability violations in the narrow sense used throughout this report - they are part of the same generating process a sufficiently long calibration window would already include - but a calibration window shorter than the relevant cycle will systematically mistake a seasonal shift for a shock, an important practical distinction a

deployed system would need to resolve by tracking calibration-window length against known cyclical structure.

Capacity degradation is different again: a reduction in the supply side of the queueing system - staff illness, equipment downtime, a temporarily closed treatment bay - rather than a change in demand. In this project's own scenario parameterization (Section 4.2), a capacity-degradation event is representable as a downward shift in n_capacity rather than an upward shift in arrival_rate_multiplier - formally a different axis of the same two-dimensional scenario space, but one this report's own exchangeability stress test (Chapter 8) does not directly test, since it isolates the shift to arrival rate specifically (Section 4.5.2) - a scope boundary stated explicitly here rather than left for a reader to assume the existing stress test already covers it.

1.7.2 Mathematical Boundaries of Deterministic Metamodeling

A point-predicting surrogate - the kind this project trains in Section 4.3 before any uncertainty quantification is layered on top of it - targets the conditional expectation E[Y|X], the single value minimizing expected squared error. It is worth stating precisely why reporting only this value, without an accompanying interval, is not merely incomplete but can be actively misleading for a queueing system specifically, via a classical and easily-stated fact: Jensen's inequality.

E[f(X)] != f(E[X]) whenever f is (strictly) convex or concave and X is non-degenerate (1.1)

Queueing delay, as a function of offered load, is convex in exactly the regime this project's own Chapter 4 derivation (Section 4.2.1's Erlang-C result) makes precise: W_q(a) grows without bound as the offered load a approaches the server count c, so W_q(.) is a convex function of a on the relevant domain. Writing a_t for the (randomly varying, e.g. across simulated days) true offered load and a-hat for a point estimate of its typical value, Jensen's inequality gives

$$E[\, W_q(a_t) \,] \geq W_q(\, E[a_t] \,) = W_q(\text{a-hat}) \tag{1.2}$$

that is, the expected delay under the true, randomly varying load is at least as large as the delay predicted by plugging the expected load into the same formula - with equality only in the degenerate case of zero variance. A deterministic point forecast built by predicting a typical or expected scenario and evaluating the queueing system's behavior at that one point systematically underestimates expected delay whenever the underlying load is genuinely variable, and the gap between the two sides of the inequality

grows with both the variance of a_t and the local curvature of W_q near the operating point - precisely the near-saturation regime Section 4.2.1's derivation identifies as where that curvature is steepest. This is not a statement about this project's specific surrogate being poorly trained; it is a structural property of applying a convex queueing-delay function to a point summary of a random load, and it is the general, distribution-free version of the specific empirical finding Chapter 7 reports for this project's own surrogate: interval width, not just point-prediction accuracy, carries real information a single number cannot.

1.7.3 The Risk Topology of Healthcare Decision-Making

An uncertainty-aware forecast is only useful insofar as it is actually used to make a decision, and ED operational decisions are rarely symmetric in their consequences - a fact worth making explicit via standard decision-theoretic language before Chapter 10 puts it into practice as an actual optimization problem. Consider a binary staffing decision framed against some safety threshold (Chapter 10 develops this concretely for wait-time ceilings): provision enough capacity, or not. Two distinct error types are possible, and conflating them - treating a generic "prediction error" as the object to minimize - hides that they are not equally costly.

A Type I error here - over-provisioning: staffing above what turns out to be necessary - wastes a bounded, known, comparatively modest resource (idle staff-hours, opportunity cost of clinicians not deployed elsewhere). A Type II error - under-provisioning: staffing below what is actually needed - risks a materially different, harder-to-bound cost: extended patient wait times with associated clinical risk, staff strain, and reputational and regulatory exposure at the tail. Formally, if L(under) and L(over) denote the loss functions associated with the two error directions, this asymmetry is the statement L(under) >> L(over) at comparable error magnitudes - and a decision rule that minimizes a symmetric loss (such as squared error, which is exactly what this project's point-predicting surrogate, Section 4.3, is trained to minimize) is not the decision rule an asymmetric-loss decision-maker should actually use. This is precisely the gap Chapter 10's capacity-optimization exercise closes concretely: rather than provisioning to a symmetric point forecast, it provisions to the upper bound of a calibrated conformal interval - a direct, worked instance of asymmetric-loss decision-making built on this project's own results rather than only argued for in the abstract here.

Chapter 2: Foundations and Shadows

A Literature Synthesis of Queueing Theory, Metamodeling, and Conformal Inference

2.0 Scope and Method of This Review

Thirty papers were selected across five areas directly relevant to this project's methodology and domain: conformal prediction foundations (Section 2.1), Mondrian and conditional-coverage methods (Section 2.2), surrogate modeling and uncertainty quantification more broadly (Section 2.3), queueing theory and emergency department operations research (Section 2.4), and discrete-event simulation together with ED-specific machine learning applications (Section 2.5). Every citation in this chapter was verified against a primary source - a publisher page, a DOI, or a preprint server - before inclusion, rather than compiled from memory; the full sourcing record is maintained in this project's repository at literature/candidate_papers.md. This verification step caught at least one concrete error during the review process itself: a paper on combining Mondrian conformal predictors was initially attributed to the authors who write most of the other Mondrian-CP literature (Boström, Johansson, and Löfström), and was found, on checking the primary source, to actually be authored by Toccaceli and Gammerman [4] - a reminder that even a plausible-looking citation needs independent verification, and a small illustration of why this project has, throughout, treated unverified citations and results as unacceptable rather than as an acceptable convenience.

Each entry below follows the same structure: the full citation, a summary of what the paper actually establishes, and a short paragraph connecting it specifically to this project - not a restatement of the abstract, but an explanation of where the paper's result or method is actually used, tested, extended, or contrasted with in the work described in later chapters.

2.1 Conformal Prediction: Foundations

This section covers the theoretical lineage of conformal prediction from its origin through the specific variants (split conformal, conformalized quantile regression, and robustness results under distribution shift) that this project builds on directly.

[2] V. Vovk, A. Gammerman, and G. Shafer, Algorithmic Learning in a Random World, *Springer, New York, NY, USA, 2005.*

This book is the founding text of conformal prediction, consolidating work Vovk, Gammerman, and collaborators had developed through the late 1990s into a single coherent theory. It introduces the transductive (full) conformal predictor, the notion of a nonconformity measure as the mechanism by which any point predictor can be turned into a valid prediction region, and proves the central finite-sample coverage guarantee under the exchangeability assumption - the property that the joint distribution of the calibration and test data is invariant to permutation, which is weaker than the more familiar i.i.d. assumption and is what conformal prediction's guarantee actually requires. The book develops the theory for both classification and regression and situates conformal prediction within algorithmic learning theory more broadly, including its relationship to Kolmogorov complexity and online prediction with expert advice.

Relevance to this project: *This is the ultimate theoretical source for every coverage guarantee invoked in this project, from the standard CP implementation in src/uq/standard_cp.py through Mondrian CP. The exchangeability assumption defined here is precisely the assumption Chapter 8's exchangeability stress test is designed to violate deliberately and study the consequences of.*

[5] H. Papadopoulos, K. Proedrou, V. Vovk, and A. Gammerman, "Inductive Confidence Machines for Regression," *in Proc. 13th Eur. Conf. Machine Learning (ECML), Lecture Notes in Computer Science, vol. 2430, pp. 345-356, 2002.*

This paper introduces split (inductive) conformal prediction as a computationally tractable alternative to the full transductive conformal predictor. Rather than refitting the underlying model once per candidate test label - which the original transductive formulation requires and which is prohibitively expensive for anything but the simplest models - the inductive approach splits the available data into a proper training set and a separate calibration set, fits the model once on the training set, and computes nonconformity scores only on the calibration set. This sacrifices a small amount of statistical efficiency (the calibration set is not used to fit the

model itself) in exchange for reducing the computational cost from retraining per test point to fitting exactly once.

Relevance to this project: *This is the exact algorithmic template that src/uq/standard_cp.py and src/uq/mondrian_cp.py both implement: fit the surrogate once (already done in Week 6-7), hold out a separate calibration set (src/uq/generate_calibration_data.py, deliberately disjoint from both the surrogate's training data and the test set), and compute residual quantiles only on that calibration set.*

[3] G. Shafer and V. Vovk, "A Tutorial on Conformal Prediction," *Journal of Machine Learning Research, vol. 9, pp. 371-421, 2008.*

This widely cited tutorial restates the conformal prediction framework in an accessible form aimed at a broader machine learning audience than the original algorithmic-learning-theory literature, and is frequently the first paper researchers encounter when entering the field. It walks through the nonconformity-measure formulation, proves the marginal coverage guarantee via an exchangeability argument, and discusses both the transductive and inductive (split) variants, along with worked examples in classification and regression settings.

Relevance to this project: *This was the first paper adopted for this project (predating the systematic 30-paper search) and remains the primary reference used when explaining conformal prediction's guarantee to readers unfamiliar with the framework, including in this report's own Chapter 1 and Chapter 4.*

[6] J. Lei, M. G'Sell, A. Rinaldo, R. J. Tibshirani, and L. Wasserman, "Distribution-Free Predictive Inference for Regression," *Journal of the American Statistical Association, vol. 113, no. 523, pp. 1094-1111, 2018.*

This paper, from the statistics community's independent adoption and extension of conformal prediction, establishes distribution-free finite-sample coverage guarantees for several conformal regression variants, including split conformal prediction, and analyzes the statistical efficiency trade-offs between the full (transductive) and split approaches. It also introduces jackknife+ as a cross-validation-based alternative that recovers

some of split conformal's lost statistical efficiency without the full transductive method's computational cost.

Relevance to this project: *Provides the formal statistical grounding (efficiency loss from splitting, in exchange for tractability) for the choice made throughout this project to use split conformal prediction rather than the more expensive transductive form - a choice that only makes sense given the sample sizes actually used here (1,200 calibration points per repeat, well within the regime where split conformal's efficiency loss is acceptable).*

[7] Y. Romano, E. Patterson, and E. Candès, "Conformalized Quantile Regression," ***in Advances in Neural Information Processing Systems 32 (NeurIPS), 2019.***

Conformalized quantile regression (CQR) combines conformal prediction's finite-sample coverage guarantee with quantile regression's ability to adapt interval width to local heteroscedasticity. Rather than conformalizing a constant-width residual (as standard split conformal does with the symmetric nonconformity measure), CQR trains two quantile regressors (targeting the alpha/2 and 1 - alpha/2 conditional quantiles) and conformalizes the signed distance from a test point's prediction to these estimated quantiles. The result retains the exact finite-sample marginal coverage guarantee of standard conformal prediction while typically producing substantially narrower intervals in regions where the underlying quantile regressor correctly identifies heteroscedasticity - regions of naturally higher or lower variance in the residual.

Relevance to this project: *This is the direct source for src/surrogate/train_quantile_surrogates.py (the lower/upper quantile regressors) and src/uq/repeated_evaluation_cqr.py (the CQR and Mondrian-CQR nonconformity score, max(qlo(x) - y, y - qhi(x))). CQR is used in this report's Section 6.1 as a stronger baseline that achieves much of Mondrian CP's benefit through width-adaptivity rather than category-adaptivity - the two methods sit at different points on the same coverage/width frontier rather than one strictly dominating the other.*

[8] A. N. Angelopoulos and S. Bates, "A Gentle Introduction to Conformal Prediction and Distribution-Free Uncertainty Quantification," ***arXiv:2107.07511, 2021.***

This widely used tutorial-style survey collects and unifies conformal prediction methods developed across the machine learning and statistics communities, presenting split conformal prediction, full conformal prediction, CQR, and conformal risk control in a single consistent notation, with worked code examples. It is written specifically to be more accessible to a machine-learning-practitioner audience than the original theoretical literature, while still proving the key coverage guarantees rigorously.

Relevance to this project: *Used as the primary accessible reference for this report's own background exposition (Chapter 1, Chapter 4) and cross-checked against the more technical treatments (Shafer & Vovk, Lei et al.) to ensure the plain-language description of conformal prediction given to non-specialist readers of this report remains technically accurate.*

[9] R. J. Tibshirani, R. F. Barber, E. Candès, and A. Ramdas, "Conformal Prediction Under Covariate Shift," ***in Advances in Neural Information Processing Systems 32 (NeurIPS), 2019.***

This paper studies what happens to conformal prediction's coverage guarantee when the exchangeability assumption is violated specifically by covariate shift - a change in the distribution of the input features between calibration and test time, while the conditional relationship between inputs and outputs stays fixed. It shows that if the likelihood ratio between the test and calibration covariate distributions is known (or can be estimated), a weighted conformal prediction procedure can restore valid coverage under this specific, structured form of distribution shift, at the cost of requiring that likelihood ratio.

Relevance to this project: *Directly relevant to this project's exchangeability stress test (src/uq/exchangeability_stress_test.py), which pushes the test distribution's arrival-rate multiplier progressively outside the calibration range - a covariate shift by exactly this paper's definition. This project's stress test does not attempt the likelihood-ratio correction this paper proposes (the shift is extreme enough, and the surrogate's own extrapolation failure severe enough, that the correction would not address the root cause identified in Chapter 8), but the paper's framing of covariate shift as a specific, structured violation of exchangeability - rather than an unstructured, unrecoverable one - is the correct lens for interpreting why the stress test's coverage collapse happens where and how it does.*

[10] R. F. Barber, E. Candès, A. Ramdas, and R. J. Tibshirani, "Conformal Prediction Beyond Exchangeability," *Annals of Statistics, vol. 51, no. 2, pp. 816-845, 2023.*

This paper generalizes conformal prediction to settings where exchangeability fails entirely rather than failing in a specific, structured way (as in covariate shift). It introduces weighted conformal prediction schemes that can provide approximately valid coverage even without exchangeability, with the degree of coverage degradation explicitly bounded in terms of a measure of how far the actual data-generating process departs from exchangeability. Crucially, the paper is explicit that no method can restore exact coverage without exchangeability or a specific known structure to its violation - the contribution is a principled characterization of how much coverage degrades, and some partial mitigation strategies, not a full fix.

Relevance to this project: *This is the most directly relevant theoretical paper to this project's second major finding (the exchangeability stress test): it confirms, at a theoretical level, that this project's empirical result - coverage collapses once the test distribution moves sufficiently far outside the calibration distribution, and neither standard nor Mondrian CP can prevent this - is not a implementation flaw but an expected consequence of exchangeability violation that the theory itself predicts.*

2.2 Mondrian and Conditional-Coverage Conformal Prediction

Mondrian conformal prediction is a comparatively narrow subfield within the broader conformal prediction literature - a handful of dedicated papers, rather than the hundreds associated with, for instance, CQR or covariate-shift robustness. This section covers all four papers identified as directly relevant during the literature search. The relative thinness of this subfield is itself worth noting explicitly: it supports this project's claim, developed fully in Chapter 3, that testing Mondrian CP in a discrete-event queueing domain is a genuine, previously unaddressed gap rather than a crowded research area.

[11] V. Vovk, D. Lindsay, I. Nouretdinov, and A. Gammerman, "Mondrian Confidence Machine," *Technical Report, Royal Holloway, University of London, 2003.*

This technical report introduces the Mondrian confidence machine, the origin of the "Mondrian" name and the underlying idea used throughout this project: rather than calibrating a single conformal quantile pooled

across an entire calibration set, partition the calibration set into categories (a "Mondrian taxonomy," named for the resemblance of a partitioned plane to Piet Mondrian's grid paintings) and calibrate a separate conformal quantile per category. The guarantee this produces is stronger than standard conformal prediction's marginal guarantee: it holds within each category individually (group-conditional coverage), rather than only when averaged across the whole population, at the cost of each category having a smaller effective calibration sample than the pooled set would provide.

Relevance to this project: *This is the conceptual origin of every Mondrian CP result in this project. The nine-cell taxonomy used in src/uq/mondrian_cp.py (staffing tercile x arrival-rate tercile) is exactly this kind of Mondrian partition, chosen using only the two real covariates the DES output actually has, per the same principle this report's own Chapter 4 justifies: partition on real, available covariates, not invented ones.*

[12] H. Boström and U. Johansson, "Mondrian Conformal Regressors," ***Proceedings of Machine Learning Research (PMLR), vol. 128 (COPA 2020), pp. 114-133, 2020.***

This paper extends Mondrian conformal prediction, originally developed primarily in a classification context, to the regression setting in a systematic way, addressing practical questions such as how to choose category boundaries for continuous or near-continuous conditioning variables (via binning) and how per-category calibration set size trades off against the tightness of group-conditional coverage. It provides empirical results across several regression benchmark datasets comparing pooled and Mondrian calibration.

Relevance to this project: *This is the single most directly used paper in this project: src/uq/mondrian_cp.py implements exactly the regression-setting Mondrian conformal predictor this paper describes, including the same fundamental tradeoff this paper discusses - smaller per-category calibration sets in exchange for group-conditional rather than only marginal coverage. This project's own finding that Mondrian CP does not help n_patients (Chapter 7) is a direct, empirical instance of the finite-sample noise tradeoff this paper identifies as a known limitation of the method.*

[13] H. Boström, U. Johansson, and T. Löfström, "Mondrian Conformal Predictive Distributions," ***Proceedings of Machine Learning Research (PMLR), vol. 152 (COPA 2021), 2021.***

This follow-up paper extends Mondrian conformal prediction from producing fixed-coverage intervals to producing full conformal predictive distributions - a distribution over possible outcomes, from which any interval or quantile can be read off after the fact, rather than committing to a single alpha level at calibration time. The Mondrian partitioning principle is unchanged; what changes is the object being calibrated, from an interval to a distribution function.

Relevance to this project: *Not directly implemented in this project - src/uq/mondrian_cp.py produces intervals at a single, fixed alpha = 0.1, matching the GP baseline and standard CP for a clean comparison - but this paper is the natural next step flagged in this report's own Chapter 11 (Future Scope): a full predictive distribution would let an ER staffing decision-maker read off any confidence level of interest after the fact, rather than committing to 90% coverage in advance as this project does throughout.*

[4] P. Toccaceli and A. Gammerman, "Combination of Inductive Mondrian Conformal Predictors," ***Machine Learning, vol. 108, pp. 489-510, 2019.***

This paper addresses a practical problem that arises when multiple Mondrian conformal predictors are trained - for instance, on different subsets of available training data, or with different taxonomies - and their predictions need to be combined into a single prediction region without losing the individual predictors' validity guarantees. It develops combination rules that preserve conformal validity under this kind of model aggregation.

Relevance to this project: *Relevant background for the nine-cell Mondrian taxonomy used in src/uq/mondrian_cp.py, which is a single taxonomy rather than a combination of several, but this paper's treatment of how different category definitions interact with calibration validity informed the choice, documented in Chapter 4, to use only the two real covariates the DES actually outputs (staffing, arrival rate) rather than combining multiple partially-overlapping taxonomies.*

2.3 Surrogate Modeling and Uncertainty Quantification

This section covers the base paper this project directly extends, together with the theoretical foundations of the surrogate architectures (Gaussian processes, gradient boosting, multi-layer perceptrons) and alternative UQ approaches (deep ensembles) used or discussed as points of comparison throughout this project.

[1] V. Gopakumar et al., "Uncertainty Quantification of Surrogate Models Using Conformal Prediction," *Machine Learning: Science and Technology, 2026.*

This is the base paper this entire project is built to extend. Gopakumar et al. validate conformal prediction as a practical, distribution-free uncertainty quantification method for machine-learned surrogate models across several physics-simulation domains - partial differential equation solvers, magnetohydrodynamics simulations, weather models, and fusion-plasma simulations - demonstrating that conformal prediction intervals achieve their nominal coverage across these domains despite the underlying surrogates being architecturally diverse (neural operators, among others) and the physical systems being simulated being highly nonlinear. The paper explicitly and prominently flags two limitations of its own results as untested rather than resolved: first, that the coverage guarantee demonstrated is marginal, not conditional, and could in principle mask conditional miscalibration within subpopulations of the test distribution the paper did not specifically probe for; second, that all validation was performed within the training distribution, leaving conformal prediction's behavior under exchangeability violation (distribution shift between calibration and deployment) as an open question the paper does not address.

Relevance to this project: *This paper is this report's entire reason for existing. Its two explicitly flagged limitations frame this project's broader empirical work: Chapter 7 examines the marginal coverage question in depth as this report's central finding, and Chapter 8 summarizes a second, secondary stress test of the exchangeability question run as part of the same project, in both cases in a domain - discrete-event queueing simulation of an ER - categorically different from the physics-simulation domains Gopakumar et al. tested.*

[14] M. C. Kennedy and A. O'Hagan, "Bayesian Calibration of Computer Models," *Journal of the Royal Statistical Society: Series B, vol. 63, no. 3, pp. 425-464, 2001.*

This is a foundational paper in the computer-model calibration literature, introducing a Bayesian framework for combining a computationally expensive simulator's output with real observational data, explicitly separating model discrepancy (systematic error between the simulator and reality) from observation noise and parameter uncertainty. Although developed before the modern surrogate-modeling and conformal-prediction literature, it establishes the conceptual vocabulary - simulator, emulator (an early term for what this project calls a surrogate), calibration, and discrepancy - that much of the later surrogate-modeling literature, including this project, builds on.

Relevance to this project: *Provides the conceptual and historical grounding for this project's own DES-to-surrogate pipeline: the distinction this paper draws between a simulator's systematic discrepancy from reality and pure statistical noise maps directly onto this project's own careful separation (Chapter 5) between the DES's real-data-calibrated arrival process and its literature-calibrated service-time distributions - a discrepancy this project discloses explicitly rather than presenting both as equally data-derived.*

[15] C. E. Rasmussen and C. K. I. Williams, Gaussian Processes for Machine Learning, *MIT Press, Cambridge, MA, USA, 2006.*

This is the standard reference textbook for Gaussian process (GP) regression and classification, covering the theory of GPs as distributions over functions, the role of the covariance (kernel) function in encoding assumptions about function smoothness, exact inference and its O(n^3) computational cost, and sparse/approximate methods developed to mitigate that cost for larger datasets.

Relevance to this project: *Direct theoretical basis for src/uq/gp_baseline.py, this project's GP uncertainty quantification baseline. The O(n^3) scaling this book documents is exactly why the GP baseline in this project is trained on a 1,000-point subsample rather than the full calibration set (Chapter 4), and why the GP-versus-CP computation time comparison in this project's full_comparison.py finds conformal prediction's calibration cost roughly 650-1000 times faster than a GP refit - a direct empirical demonstration of the scaling behavior this textbook derives theoretically.*

[16] J. H. Friedman, "Greedy Function Approximation: A Gradient Boosting Machine," ***Annals of Statistics, vol. 29, no. 5, pp. 1189-1232, 2001.***

This paper introduces gradient boosting as a general framework for additive function approximation, recasting boosting (originally developed as an ensemble classification technique) as gradient descent in function space rather than parameter space. It derives specific gradient-boosting algorithms for least-squares, least-absolute-deviation, and Huber-loss regression, and for multiclass classification, and shows that when the individual additive components are shallow regression trees, the resulting method is competitive, robust to outliers and irrelevant features, and computationally efficient relative to comparably accurate alternatives.

Relevance to this project: *This is the theoretical basis for scikit-learn's HistGradientBoostingRegressor, the primary surrogate architecture used throughout this project (src/surrogate/train_surrogate.py and, for the quantile variant used in CQR, src/surrogate/train_quantile_surrogates.py). The tree-based structure this paper describes is also the direct explanation for this report's Chapter 8 finding that the gradient-boosting surrogate's predictions freeze at a constant value once queried outside its training range - a well-known property of tree ensembles that this project traces empirically to a specific, verified mechanism rather than treating as an unexplained black-box failure.*

[17] B. Lakshminarayanan, A. Pritzel, and C. Blundell, "Simple and Scalable Predictive Uncertainty Estimation Using Deep Ensembles," ***in Advances in Neural Information Processing Systems 30 (NeurIPS), pp. 6402-6413, 2017.***

This paper proposes deep ensembles - training several neural networks independently with different random initializations and treating the spread of their predictions as an uncertainty estimate - as a simple, scalable alternative to Bayesian neural network approximations for uncertainty quantification in deep learning. It shows empirically that this simple approach often matches or exceeds more sophisticated Bayesian approximation methods on predictive uncertainty benchmarks, while being substantially easier to implement and parallelize.

Relevance to this project: *Provides an important point of contrast for this report's framing of why conformal prediction was chosen over alternative UQ approaches (Chapter 1, Chapter 4): unlike deep ensembles,*

which provide no finite-sample coverage guarantee and require training multiple independent models, conformal prediction wraps a single already-trained surrogate and provides a distribution-free guarantee - a meaningfully different trade-off that this report makes explicit rather than leaving deep ensembles as an unexamined alternative.

[18] M. Abdar et al., "A Review of Uncertainty Quantification in Deep Learning: Techniques, Applications and Challenges," ***Information Fusion, vol. 76, pp. 243-297, 2021.***

This is a broad, widely cited survey of uncertainty quantification methods for deep learning, covering Bayesian approaches, ensemble methods, and distribution-free methods (including conformal prediction), organized around the distinction between aleatoric uncertainty (irreducible noise in the data) and epistemic uncertainty (uncertainty about the model itself, reducible with more data). It also surveys applications across computer vision, natural language processing, and safety-critical domains such as medical diagnosis and autonomous driving, and discusses open challenges including computational cost, calibration under distribution shift, and the lack of standardized evaluation protocols across the UQ literature.

Relevance to this project: *Used to situate this project's specific choice of conformal prediction within the much broader UQ landscape this survey maps out (Chapter 1), and to frame this project's own aleatoric/epistemic distinction: the DES's own stochasticity (arrival randomness, service-time variance) is aleatoric uncertainty that any UQ method must characterize, while the surrogate's imperfect fit to the DES is an additional epistemic component - a distinction made explicit in this project's early design decisions (documented in this project's PROJECT_LOG.md, Week 6-7 entry) even before this survey was formally reviewed.*

2.4 Queueing Theory and Emergency Department Operations Research

This section covers the queueing-theory and health-operations-research literature that motivates and contextualizes this project's choice of a discrete-event simulation, rather than a closed-form analytic queueing model, as the underlying data-generating process the surrogate approximates.

[19] L. V. Green, J. Soares, J. F. Giglio, and R. A. Green, "Using Queueing Theory to Increase the Effectiveness of Emergency Department Provider Staffing," ***Academic Emergency Medicine, vol. 13, no. 1, pp. 61-68, 2006.***

This paper applies a Lag-SIPP (Stationary Independent Period-by-Period) queueing analysis to real ED arrival data from an urban hospital to identify provider staffing patterns that reduce the fraction of patients who leave without being seen. It demonstrates that relatively modest, well-targeted increases in provider staffing hours during identified high-demand periods produce disproportionately large reductions in patients leaving without being seen, compared to uniformly increasing staffing across all hours.

Relevance to this project: *Directly parallels this project's own staffing-scenario design: the staffing-capacity and arrival-rate-multiplier scenario grid used to generate surrogate training data (src/surrogate/generate_training_data.py) and the staffing-tercile x arrival-tercile Mondrian taxonomy (src/uq/mondrian_cp.py) both encode the same real-world insight this paper demonstrates empirically: staffing effectiveness is highly dependent on the interaction between staffing level and demand, not staffing level alone, which is exactly why this project's core finding - coverage failure concentrated specifically in the understaffed/high-demand category - has real operational meaning rather than being an arbitrary partition choice.*

[20] L. V. Green, "Queueing Analysis in Healthcare," ***book chapter, Columbia Business School, New York, NY, USA.***

This chapter provides an accessible overview of queueing-theoretic models applied to healthcare capacity and staffing problems, covering the basic M/M/s and M/G/1-type models, the Erlang loss and delay formulas, and their use in estimating offered load and required server (staff or bed) counts under target service-level constraints.

Relevance to this project: *This is the direct theoretical basis for the offered-load (Erlang) capacity calculation documented in this project's PROJECT_LOG.md (Week 4-5 entry) and inline in src/des/er_simulation.py: department A's default capacity of 30 and department B's default capacity of 14 were both derived from an Erlang-style offered-load calculation using each department's real calibrated*

arrival rate and literature-standard service times, following exactly the methodology this chapter describes, rather than chosen arbitrarily.

[21] X. Hu et al., "Applying Queueing Theory to the Study of Emergency Department Operations: A Survey and a Discussion of Comparable Simulation Studies," ***International Transactions in Operational Research, 2018.***

This survey compares analytic queueing-theory approaches and discrete-event simulation approaches to modeling ED operations, discussing the circumstances under which each is preferable: analytic queueing models offer speed and closed-form insight but require restrictive distributional assumptions that often do not hold for real ED service-time distributions, while DES relaxes those assumptions at higher computational cost and lower interpretability of any single closed-form result.

Relevance to this project: *Directly supports this project's methodological choice (Chapter 4) to build a discrete-event simulation rather than rely on a closed-form queueing model as the underlying data-generating process: the acuity-dependent, non-exponential service-time structure this project calibrates from literature-standard log-normal parameters by ESI level is exactly the kind of structure this survey identifies as poorly suited to closed-form analytic queueing treatment.*

[22] Anonymous, "Performance Evaluation of a M/G/1 Queue Model for Patient Flow in a Health Care System," ***Mathematical Modelling of Engineering Problems, IIETA.***

This paper develops and analyzes an M/G/1 queueing model (Poisson arrivals, general service-time distribution, a single server) for patient flow in a healthcare setting, deriving performance measures such as expected queue length and waiting time under this general-service-time relaxation of the more restrictive M/M/1 model.

Relevance to this project: *Provides a useful analytic comparison point for this project's own DES resource model: the DES's simplification from two nested resource pools (doctors and beds) to a single combined capacity resource (documented in PROJECT_LOG.md, Week 4-5 entry) makes it structurally closer to an M/G/c queue (a multi-server generalization of the M/G/1 model this paper studies) than to a more complex network-of-queues*

representation, a simplification this project justifies on the grounds of having no real data to calibrate a doctor-to-bed ratio.

[23] Anonymous, "Decision Support for the Optimization of Provider Staffing for Hospital Emergency Departments with a Queue-Based Approach," *PMC6947400.*

This paper develops a queueing-based decision-support tool for ED provider staffing optimization, using queueing performance measures as the objective that a staffing schedule is optimized against, rather than relying purely on historical volume-matching heuristics.

Relevance to this project: *Relevant to the staffing dimension of this project's Mondrian CP taxonomy: this paper's framing of staffing as a decision variable to be optimized against a queueing-performance objective is conceptually the same framing underlying this project's own staffing-capacity scenario sweep, though this project's contribution is specifically about quantifying uncertainty in the resulting performance predictions, not about optimizing the staffing decision itself.*

2.5 Discrete-Event Simulation and ED-Specific Machine Learning

This final section covers discrete-event simulation studies of emergency departments directly comparable to this project's own DES, together with recent (2023-2025) machine-learning applications to ED operations that situate this project within current, rather than only historical, literature.

[24] Anonymous, "A Simulation-Based Optimization Approach for the Calibration of a Discrete Event Simulation Model of an Emergency Department," *arXiv:2102.00945, 2021.*

This paper develops a simulation-based optimization procedure for calibrating a discrete-event simulation model of an ED against real operational data, framing calibration as an optimization problem where the objective function is the deviation between simulated and real performance measures, and using this procedure to recover parameters (such as resource counts) that are not directly observable in the available data.

Relevance to this project: *Directly parallel to this project's own DES calibration methodology (Chapter 5): both this paper and this project face*

the same core problem of an ED DES needing calibration against real but incomplete data, and both adopt a validation-against-real-aggregate-statistics approach (this project validates against real daily patient volume, achieving 91.0% agreement, documented in PROJECT_LOG.md's Week 4-5 entry and src/des/validate.py) rather than assuming a hand-specified parameterization is correct without checking it against data.

[25] Anonymous, "Discrete Event Simulation for Emergency Department Modelling: A Systematic Review of Validation Methods," ***ScienceDirect, 2022.***

This systematic review surveys how published ED discrete-event simulation studies validate their models against real-world data, finding substantial variation in validation rigor across the literature - from no formal validation at all to detailed statistical comparison against held-out real performance data - and argues for more consistent, transparent validation reporting as a methodological standard for the field.

Relevance to this project: *Directly relevant to justifying this project's own validation approach: the explicit 91.0% daily-volume match reported in PROJECT_LOG.md (Week 4-5) and the corresponding validation for Department B (88.6% match) represent the kind of quantitative, transparently reported validation this review argues the field should adopt as standard practice, rather than treating DES calibration as self-evidently correct without independent verification.*

[26] Anonymous, "Discrete Event Simulation Modelling for an Improved Patient Flow at the Emergency Department, Sygehus Lillebælt, Kolding," ***PMC3327033.***

This case study applies discrete-event simulation to a real Danish hospital ED to identify patient-flow bottlenecks and evaluate proposed operational changes before implementation, demonstrating DES's practical use as a decision-support tool for hospital administrators rather than purely an academic modeling exercise.

Relevance to this project: *A real-world precedent for the SimPy-based ED modeling approach used throughout this project (src/des/er_simulation.py), demonstrating that the DES-based methodology this project applies to uncertainty quantification is built on a modeling*

approach already established as practically useful for real hospital decision-making, not a purely synthetic academic exercise.

[27] Anonymous, "A Simulation-Based Optimization Approach for Analyzing the Ambulance Diversion Phenomenon in an Emergency Department Network," *arXiv:2108.04162, 2021.*

This paper uses discrete-event simulation to study ambulance diversion - the practice of redirecting incoming ambulances away from an overloaded ED to a neighboring facility - across a network of interconnected emergency departments, modeling how capacity constraints at one site propagate to neighboring sites.

Relevance to this project: *Relevant to the surge-scenario framing of this project's exchangeability stress test (src/uq/exchangeability_stress_test.py and its MLP counterpart): both this paper and this project's stress test are concerned with ED behavior under extreme demand surge, though this project's specific contribution is about surrogate and uncertainty-quantification behavior under that surge, not about the ambulance-diversion operational response this paper studies.*

[28] Anonymous, "Machine Learning-Based Prediction of Hospital Prolonged Length of Stay Admission at Emergency Department: A Gradient Boosting Algorithm Analysis," *Frontiers in Artificial Intelligence, 2023.*

This paper applies gradient boosting to predict prolonged length-of-stay admissions at an ED from patient-level features available at triage, evaluating the model's predictive performance and discussing its potential use in early identification of patients likely to require extended ED stays.

Relevance to this project: *Uses the same model family (gradient boosting) as this project's primary surrogate architecture (src/surrogate/train_surrogate.py), applied to a related but distinct prediction task - patient-level length-of-stay classification, versus this project's scenario-level regression of aggregate daily performance metrics. The shared architecture choice reflects gradient boosting's broader status, evident across the recent ED-ML literature surveyed here, as a strong default choice for structured, tabular healthcare prediction problems.*

[29] Anonymous, "An Artificial Intelligence-Based Framework for Predicting Emergency Department Overcrowding: Development and Evaluation Study," ***arXiv:2504.18578, 2025.***

This recent paper develops a machine-learning framework for predicting ED waiting-room occupancy at hourly and daily time scales, intended to support proactive staffing decisions and early intervention before overcrowding occurs, evaluated against real ED occupancy data.

Relevance to this project: *Positions this project within current (2025-2026) literature rather than only older, established queueing-theory work: this paper's goal - using predictive modeling to support proactive ED staffing decisions - is the same broad motivation underlying this project's own surrogate-plus-uncertainty-quantification pipeline, though this project's specific contribution (testing conformal prediction's marginal-coverage and exchangeability assumptions in this domain) is a methodological question this paper does not address.*

[30] Anonymous, "Machine Learning-Based Triage to Identify Low-Severity Patients with a Short Discharge Length of Stay in Emergency Department," ***PMC9123815.***

This paper applies machine learning to triage-stage data to identify low-severity patients likely to have a short ED length of stay, with the goal of supporting fast-track routing decisions at intake.

Relevance to this project: *Relevant given this project's DES also explicitly models Emergency Severity Index (ESI) acuity mix as part of its arrival-process calibration (src/utils/extract_distributions.py): both this paper and this project treat ESI-level acuity as a first-class variable affecting downstream flow and outcomes, reinforcing the methodological choice to calibrate the DES's ESI mix directly from real data rather than assuming a uniform acuity distribution.*

2.6 Synthesis and Positioning of This Project

Read together, these thirty papers trace two literatures that, prior to this project, had not been directly connected: the conformal prediction and Mondrian conformal prediction literature (Sections 2.1-2.2), developed and validated almost entirely in general machine learning benchmark settings or, in the specific case of the base paper (Gopakumar et al., 2026), in physics simulation domains; and the queueing-theory, discrete-event

simulation, and ED operations research literature (Sections 2.4-2.5), which has extensively studied ED capacity and staffing problems but has not, to the extent this review was able to establish, applied conformal prediction's finite-sample coverage guarantees to surrogate models of discrete-event queueing simulations specifically. The surrogate-modeling and uncertainty-quantification literature reviewed in Section 2.3 supplies the theoretical and architectural vocabulary (Gaussian processes, gradient boosting, deep ensembles) that bridges the two.

This project's contribution, developed in full in Chapter 3, is precisely this connection: applying conformal prediction and Mondrian conformal prediction to a discrete-event simulation surrogate calibrated on real hospital data, and testing both of the base paper's explicitly flagged, previously untested limitations - marginal-only coverage and exchangeability fragility - in this new domain.

2.7 Critical Assessment of the Reviewed Literature

A literature review that only summarizes is incomplete without an honest assessment of the reviewed work's own limitations, and of how those limitations shaped this project's methodological choices. This section provides that assessment across the five reviewed areas.

The conformal prediction foundations literature (Section 2.1) is theoretically mature and its core coverage guarantee ([2]; [3]) is not in serious dispute - the proofs are elementary rank-based arguments that do not depend on contested modeling assumptions. Its main limitation, acknowledged within the literature itself rather than only by this report, is that the exchangeability assumption underlying every coverage guarantee in this family is a property of the data-generating process that can be stated precisely but not directly tested from a finite sample - a calibration set and test set can look exchangeable in every measurable respect and still fail to be, if the true underlying shift is small enough to escape detection at the available sample size. This project's own exchangeability stress test (Section 4.5.2, summarized in Chapter 8) sidesteps this untestability problem by constructing a shift large and deliberate enough that its effect on coverage is unambiguous, rather than attempting to detect a subtle, naturally occurring shift - a pragmatic choice given this project's scope, but one that leaves open the harder question of how large a naturally occurring, undetected shift would need to be before it meaningfully degraded real-world coverage.

The Mondrian and conditional-coverage literature (Section 2.2) is, as already noted in Section 2.2's introduction, comparatively thin - a

consequence, plausibly, of Mondrian CP's own central limitation (smaller per-category calibration samples, directly observed in this project's own n_patients exception, Section 7.5.6) making it a less immediately attractive default than pooled calibration for practitioners without a specific, known source of conditional miscalibration to target. This project's own results (Chapter 7) arguably strengthen the case for this literature's continued development: a practitioner without prior knowledge of where a pooled quantile might be conditionally miscalibrated (as would genuinely be the case in an unfamiliar deployment) would have no principled way to know whether Mondrian calibration is worth its finite-sample cost, absent exactly the kind of exploratory per-category analysis this project performs in Section 6.5.

The surrogate-modeling and uncertainty-quantification literature (Section 2.3) is broad enough that this review necessarily samples rather than exhaustively covers it - the Abdar et al. [18] survey alone cites several hundred papers, of which this review's six entries in Section 2.3 represent a deliberately narrow, project-relevant slice (the base paper, and the specific architectures and alternative UQ methods this project directly uses or contrasts against) rather than a claim to comprehensive coverage of the UQ field. A specific limitation worth naming: this project did not implement or compare against Bayesian neural networks or deep ensembles [17] empirically, only discussed them as points of theoretical contrast (Section 4.4, Section 1.3) - a direct empirical comparison against a deep-ensemble baseline, alongside the GP baseline this project does implement, would have strengthened the "why conformal prediction specifically" argument beyond the theoretical case made in Chapter 1.

The queueing-theory and ED operations research literature (Section 2.4) is overwhelmingly oriented toward staffing optimization and operational decision support (Green et al., 2006; the PMC6947400 queue-based staffing paper) rather than toward the specific methodological question this project investigates (uncertainty quantification for a surrogate of a queueing simulation). This is not a gap in that literature's own quality - it reflects a genuinely different research question - but it does mean this project's own queueing-theoretic grounding (the Erlang-load capacity derivation in Section 4.2.1) is comparatively simple relative to the more sophisticated queueing models (time-varying arrival-rate models, network-of-queues formulations) that literature has developed for staffing optimization specifically. A more sophisticated queueing-theoretic foundation was judged unnecessary for this project's own research question (Chapter 3), which concerns UQ methodology rather than staffing

optimization itself, but this is a deliberate scoping choice worth stating rather than an oversight.

The discrete-event simulation and ED-specific machine learning literature (Section 2.5) is the most heterogeneous of the five areas reviewed, mixing DES calibration methodology papers, DES case studies, and ED-specific ML prediction papers that do not share a common methodology or evaluation standard - the systematic review of DES validation methods (Section 2.5, entry 25) explicitly documents this heterogeneity as a field-wide issue, finding substantial variation in how rigorously published ED DES models are validated against real data. This project's own validation approach (Section 7.1: an explicit, quantified 91.0 percent match to real daily volume, with the specific mechanism behind the residual 9 percent gap identified and explained rather than left unexplained) was deliberately designed to sit at the more rigorous end of the range this systematic review documents, rather than assumed to be adequate by default.

2.8 Epistemology of Simulation Metamodeling

The preceding sections review specific papers within five bounded categories. This section steps back to place this project's own methodological choices - a discrete-event simulation, approximated by a point-predicting surrogate, wrapped in a distribution-free uncertainty quantification layer - within the broader history and current landscape of simulation metamodeling as a field, independent of the ED-specific application.

2.8.1 Historical Progression of Simulation Emulators

Simulation metamodeling - fitting a fast-to-evaluate statistical approximation to a slow, expensive simulator, so that downstream analysis (optimization, sensitivity analysis, or, as in this project, uncertainty quantification) can query the approximation rather than re-running the simulator - has passed through several dominant approaches, each addressing a limitation of the one before it. Polynomial response-surface methodology, the earliest widely used approach in operations research, fits a low-order polynomial (typically quadratic) to simulator outputs across a designed experiment - computationally trivial and highly interpretable, but structurally unable to represent the kind of sharp, localized nonlinearity (the near-saturation queueing behavior central to this project's own Section 4.2.1 derivation) that a global low-order polynomial cannot bend sharply enough to capture. Kriging, also known as Gaussian process regression - the approach this project uses as its own UQ baseline (Section 4.4.1) rather than only reviewing historically - is a substantial refinement that models the

simulator output as a random field with a flexible, locally-adaptive covariance structure, at the cost of the O(n^3) scaling this project's own Section 4.4.1 derivation makes precise. Neural-network metamodels relaxed the smoothness assumptions kriging still carries, at the cost of requiring materially more training data and offering less interpretable uncertainty by default - part of the motivation, in this project's own Section 4.3.2, for treating a multi-layer perceptron as a robustness check on a tree-based primary architecture rather than the reverse. Tree-based ensembles - this project's own primary surrogate architecture, Section 4.3.1 - represent the current default for small-to-moderate tabular metamodeling problems specifically because they combine strong empirical accuracy with minimal hyperparameter tuning and fast fit/query times, at the cost of the extrapolation limitation Section 4.3.3's derivation makes precise and Chapter 8 tests directly. This project's own architecture choices, read against this progression, are not arbitrary defaults but specific, motivated points along a well-documented historical trajectory - each chosen for a stated reason tied to this project's own problem characteristics (Sections 4.3.1-4.3.2), not merely "what is popular."

2.8.2 Comparative Analysis of Distribution-Free vs. Parametric UQ

The uncertainty quantification methods this project compares (Chapter 7) and extends (Chapter 6) span a meaningful theoretical divide worth naming explicitly. Parametric UQ methods - the Gaussian process baseline (Section 4.4.1) foremost among them, but also Bayesian neural networks and bootstrap ensembles more broadly, reviewed in Section 2.3 - derive their uncertainty estimates from an explicit probabilistic model of the data-generating process (a GP prior and Gaussian observation noise, or a posterior over network weights, or a resampling approximation to the sampling distribution of an estimator). Their intervals are only as reliable as those modeling assumptions are correct, and carry no finite-sample guarantee independent of that correctness - precisely the property responsible for the GP baseline's empirically observed undercoverage (Chapter 7).

Distribution-free UQ - conformal prediction in every variant this project implements (standard, Mondrian, CQR, and, in Chapter 6, CRC and ACI) - makes no assumption about the data-generating process beyond exchangeability (or, for ACI specifically, no distributional assumption at all), deriving its guarantee instead from a combinatorial rank argument (derived from first principles in Section 4.4.5) that holds for any underlying distribution and any point predictor, however inaccurate. This is a strictly weaker set of assumptions purchasing a strictly stronger and more honest

guarantee - exact, finite-sample validity rather than validity contingent on an unverifiable modeling assumption - at the cost of the guarantee being marginal rather than automatically conditional (the gap Mondrian CP's category-conditional calibration exists to close, Chapter 7) and dependent on exchangeability actually holding (the assumption Chapter 8 tests to destruction). Table 2.1 summarizes this comparison structurally.

Table 2.1: Structural comparison of parametric and distribution-free uncertainty quantification, as implemented and compared throughout this report.

Property	Parametric UQ (e.g. GP, BNN, bootstrap)	Distribution-free UQ (conformal prediction)
Core assumption	Correctly specified probabilistic model	Exchangeability (or, for ACI, none)
Finite-sample guarantee	No - asymptotic or model-contingent	Yes - exact, any n
Guarantee scope by default	Whatever the model implies	Marginal (conditional requires Mondrian-style partitioning)
Behavior if assumption violated	Silently miscalibrated	Detectable, measurable coverage collapse (Chapter 8)
Computational cost	Model-fitting cost (e.g. O(n^3) for exact GP)	O(n log n) calibration, wraps any point predictor

2.8.3 Unresolved Challenges in High-Dimensional Stochastics

Three challenges recur across the literature reviewed in this chapter without a fully settled resolution, and this project's own scope (Chapter 3) is stated partly in relation to them. First, surrogate calibration under heavy-tailed output distributions - directly relevant to this project's own p95_wait_minutes target, the hardest to predict throughout this report (Chapter 7) precisely because a 95th-percentile statistic is far more sensitive to distributional tail behavior than a mean - remains an active area rather than a solved problem; this project's own use of an asymmetric nonconformity measure (Section 4.4.2) and CQR (Chapter 6) are both, in effect, partial, empirically validated responses to this challenge rather than a general theoretical resolution of it. Second, surrogate calibration under genuine non-stationarity - a data-generating process that itself drifts over time, as opposed to a single, fixed distribution shift - is the subject of active current research (Section 2.1's coverage of Gibbs and Candes' adaptive

conformal inference work) that this project's own Chapter 6 extension engages with directly rather than only citing. Third, and most broadly, conditional coverage in genuinely high-dimensional covariate spaces - this project's own two-dimensional scenario space (Section 4.2) is low-dimensional enough that a 3x3 Mondrian partition is both interpretable and well-estimated (Section 4.4.3), but the same partitioning strategy scales poorly as dimensionality grows, since the number of cells needed to keep each cell's covariate range narrow grows combinatorially - is flagged in this project's own future-scope discussion (Chapter 11) as a genuine open problem this project's own low-dimensional setting was fortunate enough not to have to solve.

Chapter 3: The Marginal Trap

Formalizing the Conditional Coverage Gap in High-Variance Queueing Regimes

3.1 Summary of the Research Gap

Chapter 2's review identifies a specific, previously unaddressed intersection between two established literatures. Conformal prediction and, more specifically, Mondrian conformal prediction (Sections 2.1-2.2) have been developed and validated primarily on general machine learning benchmarks and, in the case of this project's base paper (Gopakumar et al. [1]), on physics-simulation surrogate models spanning partial differential equations, magnetohydrodynamics, weather, and fusion-plasma domains. Separately, queueing theory, discrete-event simulation, and machine learning have all been applied extensively to emergency department operations (Sections 2.4-2.5), but - to the extent this project's literature search was able to establish - none of that body of work has applied conformal prediction's finite-sample coverage guarantees to a discrete-event simulation surrogate specifically, nor tested whether the guarantee's known limitations survive the transition from a physics-simulation setting to a queueing-simulation one.

The base paper motivating this project, Gopakumar et al. [1], is explicit that its own validation leaves two limitations untested: first, that its demonstrated coverage guarantee is marginal rather than conditional, and a pooled conformal interval could in principle mask systematic miscalibration within subpopulations of the physics-simulation test distributions the paper studied without the paper's own evaluation being positioned to detect it; second, that all of the paper's validation was performed within the training distribution of its surrogate models, leaving open the question of how conformal prediction's coverage guarantee behaves under distribution shift between calibration and deployment - a realistic concern for any surrogate deployed beyond the exact scenarios it was trained on. Both limitations are framed by the base paper as open questions for future work, not as resolved or dismissed concerns.

This project's research gap, stated directly, is this: neither of these two explicitly flagged limitations of conformal prediction for surrogate-model uncertainty quantification had been empirically tested in a discrete-event, queueing-based simulation domain prior to this work, despite such domains (hospital operations, call centers, manufacturing systems, and other stochastic service systems) being exactly the kind of setting where operational decisions - staffing levels, capacity investments - depend on uncertainty-aware predictions, and where the consequences of an unreliable coverage guarantee are practically, not just theoretically, significant.

3.2 Why Discrete-Event Queueing Simulation Is a Meaningfully Different Test

It is worth being explicit about why testing conformal prediction's limitations in a discrete-event queueing domain constitutes a genuine extension of Gopakumar et al.'s results, rather than a mechanical repetition of their experiments on a different dataset. The physics simulation domains the base paper studies - PDE solvers, magnetohydrodynamics, weather, and fusion-plasma simulations - are governed by continuous, typically smooth, deterministic-given-initial-conditions dynamics, and the uncertainty a surrogate model must characterize in that setting is predominantly epistemic: uncertainty about how well the surrogate approximates a fixed, in-principle-fully-determined physical process. A discrete-event queueing simulation is a categorically different kind of system: outcomes are driven by explicitly stochastic processes (random arrival times, random triage assignment, random service durations) even when every scenario parameter is held fixed, meaning the DES itself - not just the surrogate approximating it - is a source of irreducible aleatoric uncertainty that has no analogue in a deterministic PDE solve. A surrogate trained to predict DES outputs must therefore have its residual variance absorb both the surrogate's own epistemic approximation error and the DES's aleatoric stochasticity, in a proportion that is itself not constant across the input space (a lightly loaded, well-staffed scenario produces less variable outcomes than an overloaded, understaffed one). This heteroscedasticity - directly evidenced in this project by the systematic gap between pooled and Mondrian CP coverage documented in Chapter 7 - is plausibly more severe and more structurally different in a queueing simulation than in the physics domains previously tested, which is precisely why the marginal-coverage limitation is worth testing here specifically rather than assumed to generalize automatically from a physics-domain result.

The exchangeability question is similarly domain-specific in its practical stakes. A physics simulation's operating envelope (e.g., a range of

plasma confinement parameters) is typically defined by the physical or engineering constraints of the system being modeled and may change relatively slowly. An ED's operating envelope, by contrast, can shift abruptly and unpredictably - a mass-casualty event, a disease outbreak, a public health emergency - in ways that push real-world conditions outside whatever range a surrogate was trained and calibrated on, precisely when reliable uncertainty quantification matters most for triage and staffing decisions. Testing conformal prediction's exchangeability assumption under exactly this kind of demand-surge scenario (Chapter 8) is therefore not an arbitrary stress test but a probe of the specific failure mode most operationally relevant to this project's own application domain.

3.3 Problem Statement

Given a discrete-event simulation of an emergency department, calibrated on real arrival and triage-acuity data, and a machine-learned surrogate model trained to approximate that simulation's scenario-level outputs, this project addresses the following problem: does conformal prediction, applied to the surrogate's residuals, retain a valid finite-sample coverage guarantee (i) uniformly across operationally meaningful subgroups of the scenario space, and (ii) when the deployment scenario distribution shifts outside the range the calibration data was drawn from? And, where the answer to (i) is negative for standard (pooled) conformal prediction, does Mondrian conformal prediction - calibrating separately per subgroup rather than pooling - restore valid coverage within the subgroups where a genuine conditional miscalibration exists, without an unacceptable cost in statistical efficiency from smaller per-subgroup calibration samples?

3.4 Objectives

This project's objectives, carried through in the methodology (Chapter 4), implementation (Chapter 5), and results (Chapter 7) that follow, are to:

1. Build and validate a discrete-event simulation of a hospital emergency department, calibrated on real arrival-pattern and triage-acuity data, against real aggregate daily-volume statistics.

2. Train a machine-learned surrogate model to approximate the calibrated simulation's scenario-level output metrics, evaluated on standard regression accuracy metrics (MAE, RMSE, R-squared).

3. Implement and compare three uncertainty quantification approaches on top of the trained surrogate - a Gaussian process baseline, standard conformal prediction, and Mondrian conformal prediction - evaluated on identical calibration and test data at a common nominal coverage target.

4. Test whether standard conformal prediction's marginal coverage guarantee masks conditional miscalibration within operationally meaningful subgroups of the scenario space (staffing level, arrival-rate regime), and whether Mondrian conformal prediction corrects any such miscalibration found.

5. Test conformal prediction's behavior under a controlled violation of the exchangeability assumption, by evaluating coverage as the test distribution's arrival-rate regime is pushed progressively outside the calibration range.

6. Establish the statistical robustness of all findings through repeated evaluation across independent calibration/test draws with formal significance testing, rather than relying on a single data split.

7. Test the generality of all findings along two further axes: a second surrogate architecture, to check whether results are specific to one model family, and a second, independent hospital department, to check whether results generalize beyond a single site.

3.5 Scope and Delimitations

This project's scope is deliberately bounded in several respects that are stated here explicitly rather than left implicit. The discrete-event simulation models a single emergency department's arrival, triage, and combined bed-and-provider service process; it does not model inter-hospital transfer networks, ambulance diversion, or downstream inpatient admission processes. Service-time distributions are calibrated from literature-standard parameters by triage acuity level, not derived from the real dataset used in this project, because that dataset (documented fully in Chapter 5) contains no length-of-stay or treatment-duration field - this distinction is maintained explicitly throughout this report rather than presented as uniformly data-derived. The surrogate models studied are a gradient-boosting regressor (primary) and a multi-layer perceptron (robustness check); other architectures (Gaussian process surrogates used as the point predictor itself, rather than as a UQ baseline; graph neural networks; other tree-ensemble variants) are outside this project's scope. The uncertainty quantification methods compared are a Gaussian process baseline, standard conformal prediction, Mondrian conformal prediction, conformalized quantile regression, Mondrian-CQR, conformal risk control, adaptive conformal inference, and likelihood-ratio weighted conformal prediction (the last three developed and evaluated in Chapter 6); Bayesian neural networks and deep ensembles are discussed in the literature review (Chapter 2) as points of comparison but are not implemented or evaluated empirically in this

project. The surrogate architecture comparison (Chapter 7) covers gradient boosting, a multi-layer perceptron, random forests, XGBoost, and LightGBM - graph neural networks and Gaussian-process-as-point-predictor architectures remain outside this project's scope. Generalization testing covers two of the three emergency departments present in the underlying dataset; the third department is not evaluated, for reasons of project scope and time rather than any expectation that it would behave differently from the two that were tested.

3.6 Formalization of Operational Risk Thresholds

Chapters 1 through 5 motivate and specify this project's methodology largely in terms of coverage as an abstract statistical property. Before that methodology is developed in full technical detail (Chapter 4), it is worth stating precisely what property a conformal interval would need to have to be safe to deploy in a clinical operations setting, since "90 percent marginal coverage" and "safe for clinical decision support" are not the same claim, and conflating them is exactly the failure mode this project's central finding (Chapter 7) demonstrates empirically.

3.6.1 Axiomatic Requirements for Safe Surrogate Deployment

Three properties, stated as explicit requirements rather than left implicit, are necessary (though not sufficient on their own) for a surrogate-plus-interval system to be a defensible input to a clinical operations decision:

(i) Strict finite-sample validity: the coverage guarantee must hold exactly at the deployed calibration sample size, not merely in an asymptotic n -> infinity limit that a real, finite calibration set never reaches. This is precisely the property Section 4.4.5's derivation establishes for every conformal method in this report and precisely the property the Gaussian process baseline (Section 4.4.1) cannot offer, which is why it is retained throughout this report as a baseline to be improved upon rather than as a candidate for deployment itself.

(ii) Local (conditional) coverage, not merely marginal coverage: a system that is well-calibrated on average across all operating conditions but badly miscalibrated in the specific, high-acuity conditions where a clinical decision is actually being made (Chapter 7's central finding) provides exactly the false reassurance a marginal guarantee alone cannot rule out. This is the formal target Section 3.6.2 states precisely below, and the property Mondrian conformal prediction (Section 4.4.3) is specifically constructed to deliver.

(iii) Detectable failure under distribution shift: since no finite calibration set can anticipate every future operating regime, a deployed system's failure mode under shift matters as much as its in-distribution behavior. Chapter 8 establishes that this project's own methods fail in a detectable way (measurable coverage collapse) rather than a silent one (confidently narrow, wrong intervals) - a weaker but still operationally meaningful property when property (i) itself cannot be guaranteed to survive an unanticipated shift.

3.6.2 Mathematical Formulation of the Conditional Coverage Gap

Requirement (ii) above can be stated precisely. Standard conformal prediction (Section 4.4.2) guarantees the marginal statement

$$P(Y \text{ in } C(X)) >= 1 - alpha \tag{3.1}$$

where the probability is taken over the joint, unconditional distribution of (X, Y). This says nothing, on its own, about the conditional statement

$$P(Y \text{ in } C(X) \mid X \text{ in } K\_k) >= 1 - alpha, \quad \text{for a specific subpopulation } K\_k \text{ of interest} \tag{3.2}$$

where K_k denotes a specific operationally meaningful subpopulation - in this project's own case, one cell of the nine-category Mondrian taxonomy (Section 4.4.3), such as "understaffed, high-arrival-rate" scenarios. The marginal guarantee is, formally, a weighted average of the conditional statement across all such subpopulations:

$$P(Y \text{ in } C(X)) = sum\_k \; P(X \text{ in } K\_k) \; . \; P(Y \text{ in } C(X) \mid X \text{ in } K\_k) \tag{3.3}$$

which makes explicit, algebraically, exactly how a marginal guarantee can hold exactly while a specific conditional term inside the sum is far below target: the sum can reach 1 - alpha overall via other categories' conditional coverage running above target, compensating for one category's shortfall, provided the shortfall category's population weight P(X in K_k) is not too large. This is precisely the mechanism Chapter 7's central result documents with real numbers rather than only stating algebraically - the specific weighted-average identity behind why a 90 percent marginal number can coexist with a 68 percent conditional number in the one category that matters operationally most.

3.6.3 Risk Topologies Across Acute Care Services

The severity of a conditional coverage gap, in practice, is not uniform across an emergency department's own internal service lines, a point worth making explicit even though this project's own DES (Section 4.2) models a

single, undifferentiated combined bed-and-provider resource rather than separately modeling trauma, fast-track, and pediatric service lines as distinct queueing subsystems. A trauma bay's operating regime is, by clinical design, closest to the near-saturation, high- variance region Section 4.2.1's derivation identifies as hardest to calibrate against - which is also the regime where a conditional coverage failure carries the most severe consequences, since trauma-bay decisions are the least tolerant of the L(under) >> L(over) asymmetry named in Section 1.7.3. A fast-track unit for low-acuity complaints, by contrast, typically operates in the low-utilization regime this project's own results (Chapter 7) show is both easiest to predict and least exposed to the conditional coverage gap in the first place - the "easy" corner of this project's own Mondrian taxonomy grid. A pediatric emergency service sits in between on the utilization dimension but introduces an additional source of heterogeneity (a materially different, often more variable triage-acuity mix, per general ED operations literature, Section 2.4) this project's own single-department, adult-inclusive calibration does not disaggregate. This project's own generalization check (Chapter 9) tests transferability across two hospital departments with different overall volume and acuity mix, which is a related but distinct question from transferability across service lines within a single department - a finer-grained generalization question named here as a scope boundary (and revisited in Chapter 11's future-scope discussion) rather than one this project's own two-department comparison directly answers.

Chapter 4: Architecture of Conformal Simulation

Queueing Dynamics, Nonconformity Mechanics, and Mondrian Taxonomy Design

4.1 Pipeline Overview

The system built for this project consists of four stages, each depending on the outputs of the previous one: a calibrated discrete-event simulation (Section 4.2) generates labeled scenario data; a surrogate regression model (Section 4.3) is trained on that data to approximate the simulation's outputs directly from scenario parameters; three (later five) uncertainty quantification methods (Section 4.4) are applied on top of the trained, frozen surrogate to produce calibrated prediction intervals; and a set of robustness checks (Section 4.5) - repeated evaluation, a second surrogate architecture, a second hospital department - establish whether the resulting findings are stable and general rather than artifacts of one particular random split, model, or site. Every stage after the DES itself depends only on the DES's output distribution, not on its internal mechanics, which is precisely what makes the surrogate-plus-UQ approach practical: the expensive stochastic simulator is queried many times up front to generate training and calibration data, and every subsequent use (scenario sweeps, staffing comparisons, the statistical robustness checks in Section 4.5) queries the cheap surrogate instead.

4.2 Discrete-Event Simulation Design

The emergency department is modeled as a SimPy discrete-event simulation with three sequential stages per patient: arrival, triage/acuity assignment, and service, the last of which consumes a single shared capacity resource representing a combined bed-and-provider slot for the full duration of a patient's visit. An earlier design used two nested resource pools (a doctor pool and a bed pool); this was deliberately simplified to a single combined resource because the dataset used to calibrate this project (Section 4.2.3) contains no information from which a realistic doctor-to-bed ratio could be derived, and introducing a second, uncalibrated resource pool

would add an unverified assumption without adding insight relevant to this project's actual research question, which concerns uncertainty quantification methodology rather than fine-grained hospital operations modeling. This simplification is also the standard reduction used in the M/G/c queueing literature (Section 2.4) when a detailed multi-resource model cannot be justified by available data.

Patient arrivals follow a non-homogeneous process whose hourly and daily rate is calibrated directly from real data (Section 4.2.3): rather than a single constant arrival rate, the simulation draws arrivals according to hour-of-day and day-of-week rate multipliers extracted from the dataset, reflecting the well-documented reality that ED arrival volume is far from uniform across a 24-hour cycle. Each simulated day runs for a fixed 24-hour window; patients still queued or in service when that window ends are right-censored out of that day's completed-visit statistics rather than carried forward into a following day, since each simulated day is intended as one independent sample of a scenario for surrogate training purposes (Section 4.3), not as a continuous multi-day rollout. This right-censoring mechanism is a deliberate, documented modeling choice, not an oversight, and it is the direct explanation for two findings elsewhere in this report: the DES's simulated daily patient count running slightly below the real calibrated rate even at matched capacity (Section 4.2.4), and the non-monotonic behavior of the 95th-percentile wait time under extreme demand surge documented in Chapter 8.

Upon arrival, each patient is assigned an Emergency Severity Index (ESI) acuity level (1, most acute, through 5, least acute) drawn according to the real, calibrated ESI mix for the department being simulated, and a service duration drawn from a log-normal distribution whose parameters depend on that acuity level. The log-normal parameters used are literature-standard values by ESI level (Section 4.2.3), not derived from the dataset used in this project, since that dataset contains no length-of-stay field.

Two scenario-level parameters govern each simulated day and are the only two inputs the downstream surrogate model (Section 4.3) ever sees: staffing capacity (the number of concurrent combined bed-and-provider slots available) and an arrival-rate multiplier (a scalar applied uniformly to the calibrated real arrival rate, allowing the simulation to be run under higher- or lower-than-observed demand). Four scenario-level output metrics are recorded per simulated day: the total number of patients completing service (n_patients), the mean wait time before service begins (mean_wait_minutes), the mean total time in the system, from arrival to service completion (mean_total_minutes), and the 95th percentile of wait

time (p95_wait_minutes) - included specifically because a tail statistic computed from one stochastic simulated day is expected to be harder to predict than a mean, making it a useful stress case for comparing uncertainty quantification methods' ability to produce informatively wide, rather than falsely narrow, intervals.

4.2.1 Default Capacity via Offered-Load (Erlang) Calculation

Rather than choosing a default staffing capacity arbitrarily, this project derives it from an offered-load calculation in the style of the Erlang queueing formulas surveyed in Section 2.4. Offered load a, in erlangs, is the product of the mean arrival rate λ and the mean service duration E[S]:

$$a = \lambda \cdot E[S] \tag{4.1}$$

a dimensionless quantity: the mean number of servers that would be simultaneously busy under the given arrival rate and service-time distribution, independent of the number of servers actually provisioned.

This project evaluates a twice per department - once at λ = the real calibrated average arrival rate, once at λ = the real calibrated highest-demand hourly bin's rate - giving a range [a_avg, a_peak] the true required capacity should plausibly fall within, with the chosen default capacity n_capacity set strictly between the two, closer to a_peak, so that the simulation is neither under-provisioned relative to typical demand nor implausibly over-provisioned relative to any plausible real staffing level:

$$a\_avg < n\_capacity < a\_peak \tag{4.2}$$

For the primary department studied in this project, this calculation yields approximately 22 erlangs of average load and approximately 34 erlangs at peak (the 11:00-14:00 arrival bin); a default capacity of 30 was chosen as a value between these two figures, closer to the peak, deliberately neither under-provisioned relative to typical demand nor implausibly over-provisioned relative to any plausible real staffing level. For the second, independent department used in the generalization check (Chapter 9), the same method applied to that department's own real arrival rate and acuity mix yields approximately 10.4 erlangs average and 16.1 erlangs peak load, giving a default capacity of 14 at the same relative position between average and peak as the first department's capacity of 30 - capacity was recalibrated to the second department's own offered load, not copied or linearly rescaled from the first department's absolute numbers.

Derivation: Offered Load as an Instance of Little's Law

The offered-load formula a = λ·E[S] used above is not an independent queueing-theoretic assumption; it is a direct consequence of Little's Law,

one of the few genuinely distribution-free results in queueing theory - it requires no assumption about the arrival process, the service-time distribution, or the queue discipline beyond long-run stability, which is what makes it applicable here regardless of the log-normal service-time model's specific shape (Section 4.2.3).

Little's Law states that for any stable queueing system observed over a long time horizon, the long-run average number of customers in the system, L, the long-run average arrival rate, λ, and the long-run average time a customer spends in the system, W, satisfy

$$L = \lambda \cdot W \tag{4.3}$$

The standard proof is a direct accounting argument rather than anything probabilistic, which is exactly why the result needs no distributional assumptions. Let N(t) denote the number of customers in the system at time t, and consider a long observation window [0, T]. The total customer-time accumulated in the system over this window can be computed two different ways, and Little's Law is simply the statement that these two computations must agree. First, integrating the instantaneous occupancy directly,

$$\int_0^T N(t)\,dt = (\text{average occupancy over } [0,T]) \times T \approx L \cdot T, \text{ for } T \text{ large} \tag{4.4}$$

Second, the same integral equals the sum, over every customer who arrived during [0, T], of that customer's own individual time in the system - each customer i contributes exactly W_i (their own sojourn time) to the area under N(t), since they are counted as "in the system" for precisely that duration:

$$\int_0^T N(t)\,dt = \Sigma_i\, W_i \approx (\lambda T) \cdot W, \text{ for } T \text{ large} \tag{4.5}$$

λT customers arrive over the window on average, each contributing its own sojourn time; averaging these sojourn times gives W by definition.

Equating the two expressions for the same integral and dividing both sides by T gives L = λW directly, with no assumption used beyond the system being observed long enough, and stably enough, for both long-run averages to be well defined.

Offered load is obtained by applying this identical argument to a deliberately restricted subsystem: instead of the whole queueing system (waiting line plus servers), apply Little's Law to the service facility alone, treating "in service" as the state being counted rather than "present in the system." The long-run average number of customers simultaneously in

service is then, by the same L = λW logic with W replaced by the mean service duration E[S] rather than the full sojourn time, exactly the offered load:

$$a = \lambda \cdot E[S] \tag{4.6}$$

which is precisely the equation stated above - now derived, rather than introduced as a definition. Because this argument used nothing but the accounting identity above, a is a valid measure of the mean simultaneous service demand regardless of how many servers c actually handle it, which is exactly why it is a sound quantity to compare against a candidate staffing capacity independent of any assumption about the arrival process being Poisson or service times being log-normal (Section 4.2.3).

Derivation: Why Wait Times Grow Sharply Near Saturation

Section 7.5.4 explains this project's central empirical finding - that conditional miscalibration concentrates specifically in the understaffed/high-demand category - by appeal to the qualitative, well-documented fact that queueing systems' wait-time variance grows sharply as utilization approaches capacity. That claim is derived here explicitly, via the birth-death process underlying the M/M/c queue [20], rather than left as an assertion. The derivation is included for the M/M/c idealization specifically because it is the simplest queueing model for which the relevant closed-form result (the Erlang C formula below) exists in exact form; this project's own DES uses non-exponential, ESI-level-dependent service times (Section 4.2.3), so the M/M/c result below is used qualitatively, as the standard theoretical explanation for the mechanism observed, not as a claim that this project's simulation is itself an M/M/c queue.

Model the number of customers in an M/M/c system (Poisson arrivals at rate λ, c identical servers each working at rate μ, so offered load a = λ/μ) as a continuous-time birth-death Markov chain on states n = 0, 1, 2, …, with arrival ("birth") rate λ_n = λ at every state, and service ("death") rate

$$\mu_n = \{\, n\cdot\mu \;\; for\; n \le c; \quad c\cdot\mu \;\; for\; n \ge c \,\} \tag{4.7}$$

for $n \le c$, only n of the c servers are occupied, so the pooled service rate is $n\mu$; once $n \ge c$, all c servers are busy and the pooled service rate saturates at $c\mu$ regardless of how many additional customers are queued.

For a birth-death chain, detailed balance (the stationary flow from n − 1 to n must equal the flow from n back to n − 1) gives $\pi_n\mu_n = \pi_{n-1}\lambda$, so each stationary probability is fixed recursively in terms of the previous one, $\pi_n = \pi_{n-1} \cdot \lambda/\mu_n$. Unrolling this recursion from π_0 gives, in the two regimes,

$$\pi_n = \pi_0 \cdot a^n/n!, \ \text{for } n \leq c \tag{4.8}$$

$$\pi_n = \pi_0 \cdot (a^c/c!) \cdot \rho^{n-c}, \ \text{for } n \geq c, \ \text{with } \rho = a/c \tag{4.9}$$

ρ is the per-server utilization; the system is stable, and a stationary distribution exists at all, only if ρ < 1 - the queue-length recursion for n ≥ c is a plain geometric series with ratio ρ, which diverges to an unbounded expected queue length as ρ → 1.

Normalizing so that $\Sigma_n \pi_n = 1$ (using $\Sigma_{k=0}^{\infty} \rho^k = 1/(1-\rho)$ for the geometric tail, valid precisely when $\rho < 1$) fixes π_0, and summing π_n over every state with $n \geq c$ gives the probability that an arriving customer finds all servers busy and must wait - the Erlang C formula:

$$P(\text{wait} > 0) = C(c, a) = [\ (a^c/c!) \cdot 1/(1-\rho)\ ] \ / \ [\ \Sigma_{k=0}^{c-1}\ a^k/k! \ + \ (a^c/c!) \cdot 1/(1-\rho)\ ] \tag{4.10}$$

and, via the memoryless property of exponential service applied to whichever customers are ahead in queue, the mean additional wait time experienced by a customer who does have to wait is $1/(c\mu - \lambda)$, so the unconditional mean wait time is

$$W_q = C(c, a) \ / \ (c\mu - \lambda) \tag{4.11}$$

The specific mechanism behind Section 7.5.4's claim is now visible directly in this last equation rather than only asserted: as offered load a approaches the server count c from below, $\rho = a/c \to 1$, the denominator $(c\mu - \lambda) \to 0^+$, and $W_q \to \infty$ - not gradually, but as a term whose denominator is heading to zero, so the rate of growth accelerates sharply in the vicinity of $\rho = 1$ rather than increasing at a roughly constant rate as utilization rises from, say, 50 to 70 percent. This is the precise, quantitative form of the "variance grows sharply near saturation" claim Section 7.5.4 relies on qualitatively: the staff = Low, arrival = High category (Section 7.5.1) is, by construction, the cell of this project's sampled scenario grid operating at the highest $\rho = a/n\_capacity$ of any category, placing it closest to the region where this derivation shows W_q's sensitivity to small changes in a or c is most extreme - exactly where a pooled quantile computed mostly from lower-ρ, lower-variance calibration points is least equipped to characterize the residual spread correctly.

4.2.2 Scenario Sampling for Surrogate Training and Calibration

To generate data for surrogate training, the calibrated DES is run across thousands of randomly sampled scenarios, with staffing capacity sampled from a range around the Erlang-derived default and the arrival-rate multiplier sampled from a range covering both below- and above-average demand, each scenario run with its own independent random seed so that the DES's own stochasticity (arrival randomness, acuity assignment,

service-time variance) is preserved in the resulting labels - this residual noise is exactly what the downstream uncertainty quantification methods (Section 4.4) are being asked to characterize, and pre-averaging it away by running each scenario multiple times and reporting only a mean would defeat the purpose of the entire UQ comparison. A separate, disjoint pool of scenarios, drawn with a different sampling seed and a large seed offset from the training data, is generated specifically for conformal prediction calibration (Section 4.4.2); this separation is methodologically necessary rather than a formality, because calibrating on residuals from data the surrogate was trained on would systematically understate the true residual spread (the surrogate fits its own training points more closely than it fits genuinely unseen points) and would invalidate the resulting coverage guarantee.

4.2.3 Data Calibration: Real Arrivals, Literature Service Times

The DES's arrival process and ESI acuity mix are calibrated directly from a large, de-identified, publicly available dataset of real emergency department visits (described fully in Chapter 5), specifically the hourly, daily, and monthly arrival rate distributions and the ESI mix extracted for a single department within that dataset. Service-time distributions are not derived from this dataset, because the dataset - despite containing several hundred columns of vitals, diagnosis flags, and laboratory results - contains no length-of-stay or treatment-duration field of any kind, a limitation confirmed by an exhaustive search across every column in the dataset rather than assumed. In place of data-derived service times, this project uses literature-standard log-normal service-time parameters by ESI acuity level.

This distinction - real-data-calibrated arrivals versus literature-calibrated service times - is maintained explicitly and consistently throughout this report and every presentation material produced over the course of this project, rather than allowing both to be presented as equally data-derived. It is a genuine limitation of what this project's DES can claim to represent, and it is disclosed as such rather than obscured.

For completeness and to give the cross-check that follows a concrete target to check against, Table 4.1 states the literature-standard log-normal service-time parameters actually used by the DES (Section 4.2, implemented in src/utils/extract_distributions.py, Appendix A), by ESI acuity level.

Table 4.1: Literature-standard log-normal service-time parameters by ESI acuity level, as used by this project's DES.

ESI level	Mean (minutes)	SD (minutes)
1 (most acute)	180	90
2	150	75
3	120	60
4	75	40
5 (least acute)	45	25

4.2.3.1 Cross-Checking Service-Time Parameters Against Independently Reported ED Data

Because the dataset used throughout this project (Section 5.1) cannot supply service-time information, the literature-standard log-normal parameters used in its place (Table 4.1 above) are checked here against ten independently published emergency department studies, each drawn from a real hospital dataset of its own and none overlapping with the Kaggle dataset this project otherwise uses, rather than left as a single unattributed "literature-typical" assumption. Table 4.2 summarizes what each study actually reports and how it bears on this project's own calibration.

Table 4.2: Ten independently published ED studies used to cross-check this project's literature-calibrated service-time parameters and arrival-process methodology.

Study	Real dataset	What it reports
Hoot et al. [31]	1 US academic ED, DES model	Log-normal per-ESI evaluation/treatment duration; median 4.0/4.6/3.1/1.7/1.2 h for ESI 1-5; ESI mix 0.7/37.8/44.2/15.8/1.4%; nonstationary Poisson arrivals, 1.6-10.3/h
Otto et al. [32]	AKTIN registry, 12 German EDs, n=304,606 (2019)	Mean +/- SD length of stay by ESI/MTS triage level, admitted and non-admitted separately
Theiling et al. [33]	US NHAMCS, ~805.7M weighted visits (2010-2015)	Median length of stay by 5-level ESI-NHAMCS severity tier

Karaca et al. [34]	AZ/MA/UT state databases, n=4,955,590 (2008)	Overall mean/median ED duration (195.7 / 130.2 min); confirms right-skew
Kim et al. [35]	1 US ED, n=2,107	Mean LOS for ESI 4 and ESI 5 specifically, pre/post a triage-discharge intervention
Mahmoodian et al. [36]	2 hospitals, n=900	Median time-to-first-physician-visit by ESI level 1-5
Laskowski et al. [37]	6 Winnipeg hospitals, n=185,659	Priority-queue ED model calibrated on CTAS triage classes
Locker and Mason [38]	UK NHS EDs	Documents the right-skewed shape of ED time-in-department distributions
De Santis et al. [39]	1 large Italian ED	Nonhomogeneous-Poisson-process methodology for hourly arrival-rate calibration
Kramer et al. [40]	1 Italian ED, ~7,000 visits/month	DES implementation case study for a real, operating ED

Two of these ten bear directly enough on this project's own numeric choices to warrant a specific quantitative comparison rather than only a qualitative citation, and both comparisons are reported honestly - neither is a perfect match, and where they diverge, that divergence is stated rather than smoothed over.

Hoot et al. [31] is the closest structural match available: like this project, it fits a separate log-normal distribution to evaluation and treatment duration within each ESI level. Table 4.3 places their reported per-ESI medians alongside this project's own literature-calibrated means (Table 4.1).

Table 4.3: This project's literature-calibrated service-time means vs. Hoot et al.'s independently reported per-ESI medians.

ESI level	This project's	Hoot et al.	Direction of

	mean (min)	median (min)	difference
1 (most acute)	180	240	Hoot's is higher
2	150	276	Hoot's is higher
3	120	186	Hoot's is higher
4	75	102	Hoot's is higher
5 (least acute)	45	72	Hoot's is higher

Every level in Hoot et al.'s data runs higher than this project's own parameter - consistent with the general direction expected, since a log-normal distribution's median sits below its mean, so a same-valued comparison would already understate Hoot et al.'s corresponding mean somewhat; the gap is nonetheless larger than that alone would explain, and this project's parameters should be read as being toward the shorter, more conservative end of what published ED studies report rather than a precise match to any one of them. A second, genuinely informative discrepancy is the ordering itself: Hoot et al.'s ESI-2 duration (276 min) exceeds their ESI-1 duration (240 min), a real, documented non-monotonicity - the most acute patients are evaluated fastest in some systems precisely because they are triaged and moved with the highest urgency, while ESI-2 patients may undergo more extensive diagnostic workups once stabilized. This project's own parameters are strictly monotonic by construction ($180 > 150 > 120 > 75 > 45$), a simplification that Hoot et al.'s independently reported data shows is not always how real EDs behave, disclosed here as a specific, named limitation rather than left for a reader to discover unaided.

Otto et al. [32]'s AKTIN registry figures, while reporting total length of stay rather than service time alone and therefore not directly comparable in absolute minutes, are informative on a different, unitless dimension: the ratio of standard deviation to mean. Across Otto et al.'s ten reported ESI/MTS category-outcome combinations, this ratio ranges from roughly 0.88 to 0.96 - standard deviation nearly as large as the mean itself. This project's own log-normal parameters (Table 4.1) use a substantially tighter ratio, close to 0.5 at every ESI level (for example, ESI-1's 90-minute SD against a 180-minute mean). Some of this difference is structural and expected - total length of stay accumulates wait-time variability on top of service-time variability, so it should be more variable than service time alone - but not necessarily all of it, and this project's SD choices should accordingly be read as comparatively conservative (narrower) relative to the variability real, independently collected ED duration data exhibits, a

second specific, quantified limitation of the literature-calibrated service-time model disclosed here directly rather than left implicit in Section 7.3's more general limitations discussion.

The remaining papers in Table 4.2 support this project's methodology in ways that do not reduce to a single numeric comparison. Theiling et al. [33] and Karaca et al. [34], drawing on two of the largest publicly documented ED datasets available (805.7 million weighted NHAMCS visits and 4.96 million treat-and-release state-database visits respectively), both confirm the general shape this project assumes - duration decreasing with decreasing acuity, and a right-skewed distribution consistent with a log-normal or similar heavy-tailed family - without this project's own dataset being able to confirm either fact directly. Kim et al. [35] and Mahmoodian et al. [36] each provide an independent, smaller-scale data point in the same direction at the lower-acuity (Kim et al.) and full-range (Mahmoodian et al.) ends of the ESI scale. Laskowski et al. [37] and Kramer et al. [40] demonstrate that priority-queue and discrete-event simulation approaches structurally similar to this project's own (Section 4.2) have been successfully calibrated against real ED operations at other sites, lending independent, cross-site support to the DES-based methodology itself, not merely to its specific numeric inputs. De Santis et al. [39] validates this project's choice of a nonhomogeneous Poisson process for hourly arrival-rate calibration (Section 4.2) as an established, actively refined methodology in its own right rather than an ad hoc modeling convenience. Locker and Mason [38] is cited as further, independent confirmation that the right-skewed shape this project's log-normal assumption relies on is a property documented across health systems (here, the UK NHS) rather than one specific to the US-based studies this section otherwise draws on.

4.2.4 Validation Against Real Aggregate Statistics

The calibrated DES is validated by comparing its simulated mean daily patient volume, averaged over a large number of simulated days at the default scenario configuration, against the dataset's real calibrated daily volume for the same department. This validation is deliberately restricted to volume (a statistic the real dataset can actually provide, given the arrival-process calibration in Section 4.2.3) rather than extended to wait times or service durations, which the real dataset cannot validate against at all, for the reasons given in Section 4.2.3. Results of this validation, for both departments studied in this project, are reported in full in Chapter 7.

4.3 Surrogate Model Architectures

A surrogate model is trained independently for each of the four DES output metrics (Section 4.2), taking the two scenario parameters (staffing capacity, arrival-rate multiplier) as input. Two architectures are used in this project. In-distribution point-prediction accuracy for both is reported throughout Chapter 7 using three standard regression metrics, computed over a held-out test set of size m:

$$MAE = (1/m)\ \Sigma_{i=1}^{m} |y_i - \hat{y}_i| \tag{4.12}$$

$$RMSE = \sqrt{[\ (1/m)\ \Sigma_{i=1}^{m} (y_i - \hat{y}_i)^2\ ]} \tag{4.13}$$

$$R^2 = 1 - [\ \Sigma_{i=1}^{m} (y_i - \hat{y}_i)^2 / \Sigma_{i=1}^{m} (y_i - \bar{y})^2\ ] \tag{4.14}$$

$\bar{y}$ the test-set mean of y; $R^2 = 1$ indicates a perfect fit, $R^2 = 0$ indicates the model performs no better than always predicting $\bar{y}$.

4.3.1 Gradient-Boosting Regressor (Primary Architecture)

The primary surrogate architecture is a histogram-based gradient boosting regressor, an ensemble of shallow regression trees fit sequentially via gradient descent in function space, each new tree fit to the negative gradient (for squared-error loss, equivalently the residual) of the current ensemble's predictions. This architecture was chosen over alternatives (linear models, a single deep neural network) as the primary surrogate because it is a strong, standard default for small-to-moderate-sized tabular regression problems with few input features, requires comparatively little hyperparameter tuning to perform well, and is fast enough to fit and query that it does not become a computational bottleneck relative to the DES data generation step it approximates. Section 4.3.3 discusses a specific structural property of tree-based models - their inability to extrapolate predictions outside the range of their training data - that becomes directly relevant to the exchangeability stress test summarized in Chapter 8.

Derivation: Gradient Boosting as Functional Gradient Descent

The claim that "each new tree is fit to the negative gradient, equivalently the residual" is stated above as a fact about how this project's surrogate is trained; this section derives it, following [16], since the equivalence between "fit a tree to the residual" (an operational recipe) and "gradient descent in function space" (the theoretical justification for why that recipe is a sensible thing to do) is not obvious from the recipe alone.

The learning problem is to find a function F: $\mathcal{X} \to \mathbb{R}$ minimizing the expected squared-error loss over the training distribution:

$$F^* = \text{argmin}_F\ \mathbb{E}[\ (y - F(x))^2\ ] \tag{4.15}$$

Gradient boosting builds F* as an additive expansion, $F\_M(x) = \Sigma_{m=1}^{M} h_m(x)$, constructed greedily: at stage m, having already built the ensemble F_{m-1} from the previous m − 1 trees, the next weak learner h_m is chosen to approximate the direction of steepest descent of the loss, treating the ensemble's prediction at each training point as a free parameter to be adjusted. Concretely, the functional gradient of the squared-error loss $L(y, F) = \frac{1}{2}(y - F)^2$ with respect to the prediction F, evaluated at each training point x_i under the current ensemble F_{m-1}, is

$$-\partial L(y_i, F(x_i)) / \partial F(x_i) \,|_{F = F\_\{m-1\}} = -\partial[\, \tfrac{1}{2}(y_i - F(x_i))^2 \,] / \partial F(x_i) \,|_{F = F\_\{m-1\}} = y_i - F\_\{m-1\}(x_i) \quad (4.16)$$

that is, for squared-error loss specifically, the negative functional gradient at each training point is exactly that point's current residual, $r_i = y_i - F\_\{m-1\}(x_i)$. The next tree h_m is then fit, by ordinary least-squares regression-tree training, to predict these residuals as its target:

$$h_m = \text{argmin\_}h \; \Sigma_{i=1}^{n} (r_i - h(x_i))^2, \quad r_i = y_i - F\_\{m-1\}(x_i) \quad (4.17)$$

and the ensemble is updated by taking a small step in this direction, $F\_m(x) = F\_\{m-1\}(x) + \nu \cdot h_m(x)$, with a learning rate $\nu \in (0, 1]$ controlling step size (shrinkage) exactly as in ordinary gradient descent on a parameter vector, except that the "parameter" being descended on here is the ensemble's prediction at every training point simultaneously, and each descent step is itself constrained to be representable by a shallow regression tree rather than an arbitrary vector update. "Fit a tree to the residual" is therefore not a heuristic alternative to gradient descent; for squared-error loss it is gradient descent in function space, with the residual arising as the specific, closed form the negative functional gradient happens to take for this one loss function. (For other losses - the quantile ("pinball") loss used by the CQR quantile regressors in Section 4.4.4, for instance - the same functional-gradient-descent recipe applies with a different negative-gradient target: piecewise constant rather than the raw residual, which is why quantile regression trees are trained by the same underlying algorithm but do not simply fit raw residuals.)

4.3.2 Multi-Layer Perceptron (Robustness-Check Architecture)

A second surrogate architecture, a multi-layer perceptron with two hidden layers trained with early stopping and preceded by standard feature scaling, is trained on the identical train/test split as the gradient-boosting surrogate, specifically to test whether findings obtained with the primary architecture are specific to tree ensembles or generalize across a structurally different model family. Unlike a tree ensemble, a neural network has no inherent floor or ceiling on its output as inputs move outside its training

range - it will continue to extrapolate, in whichever direction its learned weights imply, rather than saturating at a boundary value. This structural difference is the reason this architecture is included as a specific, targeted robustness check on the exchangeability findings summarized in Chapter 8, rather than as a general "try another model and see" exercise.

4.3.3 Tree-Based Extrapolation Limits

A specific property of tree-based ensembles, directly relevant to interpreting results in Chapter 7 and central to Chapter 8, is worth stating precisely here. A regression tree partitions its input space into axis-aligned regions (leaves) during training and predicts a constant value - the mean of the training targets falling into that leaf - for any query point landing in that region. For an input that falls outside the entire range spanned by the training data, the tree does not extrapolate a trend; it simply returns whatever constant value the nearest boundary leaf holds, because that boundary leaf is where every such out-of-range query is routed by the tree's learned split thresholds. A gradient-boosting ensemble, being a sum of many such trees, inherits this property: its prediction for an input arbitrarily far outside the training range is frozen at the value implied by the boundary leaves of its constituent trees, however far outside that range the query point actually lies. This is a well-known, textbook property of tree ensembles, not a bug specific to this project's implementation, and it is verified directly (rather than merely asserted) in Chapter 8 by observing gradient-boosting surrogate predictions that are numerically frozen across a wide range of out-of-distribution query points.

4.4 Uncertainty Quantification Methods

Four uncertainty quantification approaches are implemented and compared in this project, all evaluated at a common nominal miscoverage rate alpha = 0.1 (that is, targeting 90 percent coverage) on identical calibration and test data wherever a direct comparison is drawn, so that any difference found reflects the UQ method itself rather than a difference in evaluation conditions.

4.4.1 Gaussian Process Baseline

The Gaussian process (GP) baseline treats each of the four surrogate targets as an independent Gaussian process regression problem over the two-dimensional scenario-parameter input space. A GP prior is placed directly over the unknown function f:

$$f(x) \sim \mathcal{GP}(\, m(x),\, k(x, x')\,) \tag{4.18}$$

m(x) the prior mean function (taken as zero here) and k(x, x') the covariance (kernel) function encoding assumed smoothness across the input space.

This project's implementation (src/uq/gp_baseline.py) uses the squared- exponential (radial basis function, RBF) kernel, the standard default choice for a smoothly varying, continuous scenario-parameter input space with no known periodicity or discontinuity to encode structurally:

$$k(x, x') = \sigma_f^2 \exp\left(- \|x - x'\|^2 / (2\ell^2) \right) \tag{4.19}$$

σ_f^2 the signal variance (the kernel's value at $x = x'$, an overall scale on how much f is expected to vary) and ℓ the length scale (how quickly correlation between $f(x)$ and $f(x')$ decays with distance $\|x - x'\|$); both are hyperparameters fit by maximizing the marginal likelihood during training, together with σ_n^2 below, rather than set by hand.

Conditioning on the training inputs X and targets y yields a closed-form Gaussian posterior predictive distribution at any query point x*:

$$f(x^*) \mid X, y \sim \mathcal{N}\left(\mu(x^*), \sigma^2(x^*) \right) \tag{4.20}$$

$$\mu(x^*) = k^{*T} (K + \sigma_n^2 I)^{-1} y \tag{4.21}$$

$$\sigma^2(x^*) = k(x^*, x^*) - k^{*T} (K + \sigma_n^2 I)^{-1} k^* \tag{4.22}$$

K the training-set kernel matrix, k^* the vector of kernel values between x^* and each training point, and σ_n^2 the observation-noise variance.

Derivation of the Posterior Predictive Distribution

The three equations above are stated as the standard result throughout the Gaussian process literature (Section 2.3, [15]); the derivation is given here because it is what makes precise exactly which matrix inversion the $O(n^3)$ cost discussed below is paying for, and because it is a direct, mechanical consequence of one identity - conditioning a multivariate Gaussian on part of itself - rather than something specific to GP regression that must be taken on faith.

By the GP prior's definition, the observed training targets y (assumed to carry independent Gaussian observation noise of variance σ_n^2, so that $y = f(X) + \varepsilon$ with $\varepsilon \sim \mathcal{N}(0, \sigma_n^2 I)$) and the unknown function value $f(x^*)$ at a new query point are jointly Gaussian, since any finite collection of values from a Gaussian process is itself jointly Gaussian by definition:

$$\left[y \; ; f(x^*) \right] \sim \mathcal{N}\left(0, \; \left[\left[K + \sigma_n^2 I, \; k^* \right], \; \left[k^{*T}, \; k(x^*, x^*) \right] \right] \right) \tag{4.23}$$

a $(n+1)$-dimensional joint Gaussian, block-partitioned into the n training targets and the single query-point value; K is the $n \times n$ matrix with $K_{ij} = k(x_i, x_j)$, and k^* is the n-vector with $(k^*)_i = k(x_i, x^*)$.

The result then follows from the standard identity for conditioning a jointly Gaussian vector on a subset of its own components (equivalently, computing the Schur complement of the joint covariance's training-training

block). For a general partitioned Gaussian [a; b] ~ $\mathcal{N}([\mu_a; \mu\_b], [[\Sigma_{aa}, \Sigma_{a\beta}],[\Sigma_{\beta a}, \Sigma_{\beta\beta}]])$, the conditional distribution of b given a is itself Gaussian, with

$$b \mid a \sim \mathcal{N}(\mu\_b + \Sigma_{\beta a}\,\Sigma_{aa}^{-1}\,(a - \mu_a),\quad \Sigma_{\beta\beta} - \Sigma_{\beta a}\,\Sigma_{aa}^{-1}\,\Sigma_{a\beta}) \tag{4.24}$$

Substituting a = y, b = f(x*), $\mu_a = \mu\_b = 0$ (the zero prior mean assumed above), $\Sigma_{aa} = K + \sigma_n^2 I$, $\Sigma_{a\beta} = \Sigma_{\beta a}^T = k^*$, and $\Sigma_{\beta\beta} = k(x^*, x^*)$ directly into this general identity reproduces $\mu(x^*)$ and $\sigma^2(x^*)$ exactly as stated: substituting into the mean expression gives $\mu\_b + \Sigma_{\beta a}\,\Sigma_{aa}^{-1}(a - \mu_a) = 0 + k^{*T}(K + \sigma_n^2 I)^{-1}(y - 0) = k^{*T}(K + \sigma_n^2 I)^{-1} y$, and substituting into the variance expression gives $\Sigma_{\beta\beta} - \Sigma_{\beta a}\,\Sigma_{aa}^{-1}\,\Sigma_{a\beta} = k(x^*, x^*) - k^{*T}(K + \sigma_n^2 I)^{-1} k^*$ - term for term the two equations already stated above. Nothing beyond this one conditioning identity, applied to the specific block structure a Gaussian process prior induces, is needed to obtain the closed form; the entire derivation is linear algebra once the joint-Gaussian claim itself is granted, which is precisely the defining property of a Gaussian process.

The $(K + \sigma_n^2 I)^{-1}$ term appearing in both $\mu(x^*)$ and $\sigma^2(x^*)$ is exactly the n × n matrix inverse whose Cholesky factorization costs $O(n^3)$ - Section 4.4.1's motivation for subsampling to n = 1,000 training points is, in light of this derivation, a direct consequence of this one matrix appearing in both closed-form results, not an incidental implementation detail of this project's specific code.

The $(1 - \alpha)$ prediction interval is then constructed directly from this posterior:

$$C(x^*) = [\,\mu(x^*) - z\cdot\sigma(x^*),\ \mu(x^*) + z\cdot\sigma(x^*)\,] \tag{4.25}$$

z the standard normal quantile at the target confidence level (z ≈ 1.645 for the 90% interval used throughout this project).

Exact GP inference requires inverting (in practice, Cholesky-factorizing) the n × n matrix $(K + \sigma_n^2 I)$, an $O(n^3)$ computational cost in the number of training points n, which motivates training the GP on a representative subsample (1,000 points) of the available calibration data rather than the full set - a choice that is also directly relevant to this project's own narrative, since the GP's comparatively poor computational scalability relative to conformal prediction's near-constant calibration cost is one of the quantities this project's results (Chapter 7) explicitly measure and report, not simply an implementation convenience footnoted away.

4.4.2 Standard (Split) Conformal Prediction

Standard conformal prediction, in the split (inductive) form introduced by Papadopoulos et al. [5] and reviewed in Section 2.1, computes a

nonconformity score for each point in a held-out calibration set - disjoint from both the surrogate's training data and the final test set, per Section 4.2.2 - and uses the ceiling of (n + 1)(1 - alpha) divided by n-th empirical quantile of those scores (the standard finite-sample-corrected quantile formula that gives conformal prediction its exact coverage guarantee) as a fixed margin applied to every future prediction. Two nonconformity measures are implemented: a symmetric measure, the absolute residual between the surrogate's prediction and the true value, giving a fixed-width interval around each prediction; and an asymmetric measure, in which upper and lower residual quantiles are calibrated separately (each at 1 - alpha/2) and combined via a union bound, a valid but slightly conservative route to at least (1 - alpha) coverage that does not require assuming a symmetric residual distribution around the point prediction.

4.4.3 Mondrian Conformal Prediction

Mondrian conformal prediction (Section 2.2) replaces standard conformal prediction's single pooled quantile with a separate quantile calibrated within each category of a partition (a "Mondrian taxonomy") of the calibration set. This project's taxonomy is the cross of staffing-capacity tercile (low, medium, high) and arrival-rate-multiplier tercile (low, medium, high) - nine categories in total - using only these two real, available scenario covariates rather than an invented additional dimension, since the DES produces no other scenario-level covariate (such as a "shift" or "day of week" label) that could be used instead. Category bin edges are computed from the calibration set's own quantiles and never from the test set, preserving the same calibration/test separation principle as standard CP. For a fair, isolated comparison of pooling versus partitioning, Mondrian CP in this project's core comparison uses the same symmetric nonconformity measure as standard CP's symmetric variant, differing only in whether one pooled quantile or nine per-category quantiles are computed and applied.

Evaluation of Mondrian CP's effect proceeds in two complementary ways on the same test points: applying standard CP's single pooled quantile broken down by category (revealing where the pooled, marginal guarantee actually breaks down conditionally, even though it is not designed to be evaluated this way), and applying each category's own Mondrian quantile (showing whether per-category calibration corrects any breakdown found). Both a per-category view (Chapter 7, following the same structure as the core result) and a marginal, whole-test-set view (obtained by re-aggregating the per-category predictions across the full test set, for a like-for-like comparison against the GP baseline and standard CP's own marginal coverage) are reported.

4.4.4 Conformalized Quantile Regression and Mondrian-CQR

Conformalized quantile regression (CQR, Section 2.1) is implemented as a stronger baseline against which Mondrian CP's benefit can be compared. Two quantile regressors - gradient-boosting models trained with a quantile loss function, targeting the alpha/2 and 1 - alpha/2 conditional quantiles respectively - are trained on the identical train/test split as the primary mean-regression surrogate. The CQR nonconformity score for a calibration point is the greater of (the lower quantile prediction minus the true value) and (the true value minus the upper quantile prediction); conformalizing this score - taking its (1 - alpha) empirical quantile on the calibration set and using it to inflate or deflate the raw quantile regressors' interval - restores an exact finite-sample coverage guarantee even though the raw, uncalibrated quantile regressors alone do not reliably achieve nominal coverage. Mondrian-CQR combines both ideas: the same CQR nonconformity score, calibrated separately within each of the nine Mondrian categories rather than pooled, testing whether Mondrian's category-conditional calibration and CQR's width-adaptivity address the same or different sources of miscalibration.

4.4.5 Formal Algorithm Statements

For precision and reproducibility, this section states the three core calibration procedures implemented in this project (standard split conformal prediction, Mondrian conformal prediction, and conformalized quantile regression) as explicit, numbered algorithms with their governing equations, using notation consistent with Chapter 2's literature review. Let $\hat{f}$ denote the already-trained surrogate point predictor (Section 4.3), Dcal the calibration set of (x, y) pairs disjoint from both the surrogate's training data and the final test set (Section 4.2.2), n the size of Dcal, and α the target miscoverage rate ($\alpha = 0.1$ throughout this project, Section 4.4), so that the target coverage level is $1 - \alpha = 0.9$.

Algorithm 1: Standard Split Conformal Prediction

Step 1. For each calibration point (x_i, y_i) in Dcal, compute the symmetric nonconformity score:

$$s_i = |y_i - \hat{f}(x_i)|, \quad i = 1, \ldots, n \tag{4.26}$$

(For the asymmetric measure, positive and negative residuals are instead collected separately: $s_i^+ = \max(0, y_i - \hat{f}(x_i))$ and $s_i^- = \max(0, \hat{f}(x_i) - y_i)$.)

Step 2. Compute $\hat{q}$, the finite-sample-corrected empirical quantile of the calibration scores:

$$\hat{q} = \text{the } \lceil (n+1)(1-\alpha) \rceil\text{-th smallest value of } \{s_1, \ldots, s_n\} \tag{4.27}$$

$\lceil \cdot \rceil$ denotes the ceiling function; this rank-based correction (rather than the simpler $\lceil n(1-\alpha) \rceil$-th value) is what gives the resulting interval its exact, non-asymptotic coverage guarantee.

- the specific correction whose finite-sample validity is established by the theory in [2] and [5], reviewed in Section 2.1. For the asymmetric measure, this quantile is instead computed separately for $\{s_i^+\}$ and $\{s_i^-\}$, each at level $1 - \alpha/2$.

Step 3. For a new test point x, construct the prediction interval:

$$C(x) = [\, \hat{f}(x) - \hat{q},\ \hat{f}(x) + \hat{q}\,] \tag{4.28}$$

(or $[\hat{f}(x) - \hat{q}^-, \hat{f}(x) + \hat{q}^+]$ for the asymmetric measure, using each side's own quantile).

This procedure, implemented in src/uq/standard_cp.py, is guaranteed - under the assumption that (x, y) pairs in Dcal and the test point are exchangeable - to satisfy the marginal coverage guarantee:

$$P(\, Y \in C(X)\,) \geq 1 - \alpha \tag{4.29}$$

regardless of $\hat{f}$'s accuracy, because the rank-based correction in Step 2 accounts exactly for the probability that a genuinely exchangeable test score ranks among the top $\lceil (n+1)\alpha \rceil$ scores out of n + 1 total (the n calibration scores plus the test score itself).

Proof of the Coverage Guarantee

The guarantee stated above is not an approximation or an asymptotic result - it holds exactly, for every finite n, under nothing more than the exchangeability assumption. The full argument is given here rather than only cited, since it is short, and since understanding exactly what exchangeability buys makes the exchangeability stress test in Chapter 8 interpretable as a violation of a specific, named assumption rather than an unexplained empirical failure.

Setup

Let $Z_1 = (x_1, y_1), \ldots, Z_n = (x_n, y_n)$ denote the n calibration points and $Z_{n+1} = (x, y)$ the test point. $\hat{f}$ is trained on a data set disjoint from both (Section 4.2.2) and is therefore held fixed throughout this argument - it is not re-estimated as calibration or test points are permuted, so it introduces no dependence that could break exchangeability among $Z_1, \ldots, Z_{n+1}$. Define the nonconformity score at every one of these n + 1 points using the same fixed $\hat{f}$:

$$s_i = |y_i - \hat{f}(x_i)|, \quad i = 1, \ldots, n + 1 \tag{4.30}$$

Because $Z_1, \ldots, Z_{n+1}$ are assumed exchangeable (their joint distribution is invariant under any permutation of the n + 1 indices) and each s_i is computed from Z_i by the same fixed function, the scores $s_1, \ldots, s_{n+1}$ are themselves exchangeable random variables.

Step 1: The Rank of an Exchangeable Score Is Uniform

This is the single fact the whole guarantee rests on. For n + 1 exchangeable real-valued random variables with no ties (ties are broken by an independent random perturbation if the underlying scores can repeat, so this is without loss of generality), every one of the (n + 1)! orderings of $s_1, \ldots, s_{n+1}$ is equally likely - permuting the underlying (x_i, y_i) pairs permutes the scores identically, and exchangeability states precisely that every such permutation of the data has the same joint distribution. Under a uniform distribution over orderings, the rank of any single specified element among the n + 1 - here, the test score s_{n+1} - is itself uniform on $\{1, \ldots, n + 1\}$:

$$P(\text{Rank}(s_{n+1}) = k) = 1 / (n + 1), \quad \text{for every } k = 1, \ldots, n + 1 \tag{4.31}$$

$\text{Rank}(s_{n+1})$ counts how many of $s_1, \ldots, s_{n+1}$ (including s_{n+1} itself) are $\leq s_{n+1}$.

Step 2: Relating the Rank to the Calibration Quantile

$\hat{q}$ (Step 2 of Algorithm 1) is defined purely from the n calibration scores, as their $\lceil (n + 1)(1 - \alpha) \rceil$-th smallest value. The test score s_{n+1} exceeds $\hat{q}$ exactly when s_{n+1} is large enough to rank above at least $\lceil (n + 1)(1 - \alpha) \rceil$ of the n + 1 scores - that is, exactly when

$$s_{n+1} > \hat{q} \iff \text{Rank}(s_{n+1}) > \lceil (n + 1)(1 - \alpha) \rceil \tag{4.32}$$

Combining this equivalence with Step 1's uniform-rank result gives the miscoverage probability directly, as a simple count of how many of the n + 1 equally likely ranks exceed the threshold:

$$P(s_{n+1} > \hat{q}) = P(\text{Rank}(s_{n+1}) > \lceil (n + 1)(1 - \alpha) \rceil) = 1 - \lceil (n + 1)(1 - \alpha) \rceil / (n + 1) \tag{4.33}$$

Since $\lceil (n + 1)(1 - \alpha) \rceil \geq (n + 1)(1 - \alpha)$ by definition of the ceiling function, dividing by (n + 1) and substituting into the line above gives the one-line inequality this whole algorithm exists to guarantee:

$$P(s_{n+1} > \hat{q}) \leq 1 - (n + 1)(1 - \alpha) / (n + 1) = \alpha \tag{4.34}$$

Step 3: Translating back from the score to the interval. Because $s_{n+1} = |y - \hat{f}(x)|$, the event $\{s_{n+1} \leq \hat{q}\}$ is, by construction, exactly the event $\{y \in$

$[\hat{f}(x) - \hat{q}, \hat{f}(x) + \hat{q}]\} = \{Y \in C(X)\}$. Taking the complement of the inequality just derived,

$$P(Y \in C(X)) = P(s_{n+1} \leq \hat{q}) = 1 - P(s_{n+1} > \hat{q}) \geq 1 - \alpha \quad (4.35)$$

which is exactly Algorithm 1's stated guarantee, obtained here from first principles rather than merely cited. Nowhere in this argument is any assumption made about $\hat{f}$'s accuracy, the distribution of Y given X, or the dimensionality of X - the guarantee is genuinely distribution-free, and its only load-bearing assumption is the exchangeability of $Z_1, \ldots, Z_{n+1}$ used in Step 1.

A second, less frequently stated consequence of the same argument is worth recording because it bounds the guarantee from above as well as below, making explicit that split conformal prediction is not simply "conservative by an unknown margin": under the additional assumption that ties among the scores occur with probability zero (satisfied here since $\hat{f}$'s residuals are continuous-valued),

$$P(Y \in C(X)) \leq 1 - \alpha + 1 / (n + 1) \quad (4.36)$$

obtained by the same rank argument applied to $\lceil (n + 1)(1 - \alpha) \rceil - 1$ in place of $\lceil (n + 1)(1 - \alpha) \rceil$. Together, the two bounds sandwich the true coverage within $1/(n + 1)$ of the $1 - \alpha$ target - at this project's calibration size ($n = 1{,}200$, Section 4.2.2), a band of well under 0.1 percentage points, negligible next to the several-percentage-point effects this report's results (Chapter 7) actually report.

Exchangeability is the assumption doing all the work in Step 1, and nowhere else in the argument. Chapter 8's exchangeability stress test is therefore, precisely, a test of whether Step 1 continues to hold once the test distribution's arrival-rate multiplier is pushed outside the range the calibration data was drawn from - and Chapter 8's coverage collapse is exactly what this proof predicts happens the moment that one assumption stops being true: no other step in the argument offers any fallback once exchangeability itself is violated.

Algorithm 2: Mondrian Conformal Prediction

Step 1. Define a partition function c(x) mapping each scenario x to one of K categories (K = 9 in this project: the cross of staffing tercile and arrival-rate tercile, Section 4.4.3):

$$c : \mathcal{X} \rightarrow \{1, 2, \ldots, K\} \quad (4.37)$$

Tercile boundaries used by c(x) are computed from Dcal's own marginal distribution of staffing capacity and arrival-rate multiplier - never from the test set, preserving the same calibration/test separation as Algorithm 1.

Step 2. Partition Dcal into K disjoint subsets by category:

$$Dcal^{(k)} = \{ (x_i, y_i) \in Dcal : c(x_i) = k \}, \quad k = 1, \ldots, K \tag{4.38}$$

Step 3. For each category k, apply Algorithm 1's Steps 1-2 using only $Dcal^{(k)}$, producing a category-specific quantile:

$$\hat{q}^{(k)} = \text{the } \lceil (n_k + 1)(1 - \alpha) \rceil\text{-th smallest value of } \{s_i : (x_i, y_i) \in Dcal^{(k)}\} \tag{4.39}$$

$n_k = |Dcal^{(k)}|$, the number of calibration points in category k.

Step 4. For a new test point x, determine its category k = c(x) and construct its prediction interval:

$$C(x) = [\, \hat{f}(x) - \hat{q}^{(c(x))}, \ \hat{f}(x) + \hat{q}^{(c(x))} \,] \tag{4.40}$$

This procedure, implemented in src/uq/mondrian_cp.py, is guaranteed to satisfy group-conditional coverage:

$$P(Y \in C(X) \mid c(X) = k) \geq 1 - \alpha, \ \text{for every } k = 1, \ldots, K \tag{4.41}$$

a strictly stronger guarantee than Algorithm 1's marginal one, at the cost of each $\hat{q}^{(k)}$ being estimated from only n_k calibration points rather than the full n - the direct source of the finite-sample-noise tradeoff discussed at length in Sections 7.5.6 and 2.2.

Proof of the Group-Conditional Guarantee

The category partition c(x) is computed once, from Dcal's own marginal covariate distribution (Step 1), before any label information is used and identically for calibration and test points - it is a fixed, deterministic function of x alone, not re-estimated per category and not dependent on which points happen to fall into which category. Conditioning the exchangeable sequence $Z_1, \ldots, Z_{n+1}$ on the event $\{c(X_1) = \cdots = c(X_{n+1}) = k\}$ (that is, restricting attention to the subsequence of points - calibration and test alike - that happen to fall in category k) preserves exchangeability among that subsequence, because exchangeability is a property of the joint distribution of the Z_i's that a fixed, label-blind selection rule cannot disturb: relabelling which of the category-k points is "the test point" is still equally likely for each of the $n_k + 1$ members of the subsequence. Algorithm 1's proof (this section, above) therefore applies verbatim to this subsequence, with n replaced by n_k and D_cal replaced by $Dcal^{(k)}$ throughout, yielding

$$P(Y \in C(X) \mid c(X) = k) \geq 1 - \alpha \tag{4.42}$$

for each k independently. This is a genuine corollary, not merely an analogy: no new argument is required beyond noting that the conditioning event is determined by x alone and applied identically before any

calibration or test label is observed, so Step 1's uniform-rank result transfers unchanged to each category's own subsequence. The price of this stronger, per-category statement is visible directly in the equation for $\hat{q}^{(k)}$ above - n_k is necessarily smaller than n ($n_k \approx n / K$ on average across K = 9 balanced categories), so the same finite-sample gap between the lower and upper coverage bounds derived for Algorithm 1 ($1/(n + 1)$ there becomes $1/(n_k + 1)$ per category) widens roughly ninefold, the precise, quantifiable source of the added estimation noise documented empirically in Sections 7.5.6 and 6.6.1.

Algorithm 3: Conformalized Quantile Regression (CQR)

Step 1. Train two auxiliary quantile regression models on the same training data as the primary surrogate $\hat{f}$ (but using a pinball/quantile, rather than squared-error, loss function):

$$\hat{q}lo \text{ targets the } \alpha/2 \text{ conditional quantile of } Y \mid X;\ \hat{q}hi \text{ targets the } (1 - \alpha/2) \text{ conditional quantile of } Y \mid X \qquad (4.43)$$

Step 2. For each calibration point (x_i, y_i) in Dcal, compute the CQR nonconformity score:

$$s_i = \max(\hat{q}lo(x_i) - y_i,\ y_i - \hat{q}hi(x_i)) \qquad (4.44)$$

positive when the true value falls outside the raw [$\hat{q}lo(x)$, $\hat{q}hi(x)$] band in either direction, negative when it falls inside - directly measuring how much the raw, uncalibrated quantile band under- or overshoots.

Step 3. Compute the same finite-sample-corrected empirical quantile as in Algorithm 1, Step 2, applied to the CQR scores:

$$\hat{q} = \text{the } \lceil (n + 1)(1 - \alpha) \rceil\text{-th smallest value of } \{s_1, \ldots, s_n\} \qquad (4.45)$$

Step 4. For a new test point x, construct the prediction interval:

$$C(x) = [\ \hat{q}lo(x) - \hat{q},\ \hat{q}hi(x) + \hat{q}\ ] \qquad (4.46)$$

This procedure, implemented across src/surrogate/train_quantile_surrogates.py and src/uq/repeated_evaluation_cqr.py, restores an exact marginal coverage guarantee, $P(Y \in C(X)) \geq 1 - \alpha$ (by the same exchangeability argument as Algorithm 1, applied to the CQR score rather than the raw residual) even when the raw quantile regressors $\hat{q}lo$ and $\hat{q}hi$ do not themselves achieve nominal coverage - Section 6.1 of this report reports that this project's raw, uncalibrated quantile regressors achieve only 81-92 percent coverage before this calibration step, exactly the gap Step 3's conformalization closes. Mondrian-CQR combines Algorithm 3 with Algorithm 2's

partitioning: Steps 2-3 are repeated separately within each of the K Mondrian categories, using each category's own calibration subset, to produce a category-specific $\hat{q}^{(k)}$ applied to that category's test points in Step 4.

Proof That CQR Restores the Guarantee

The claim that Step 3's conformalization restores exact coverage regardless of how poorly $\hat{q}lo$ and $\hat{q}hi$ themselves perform is the single most important property of CQR, and it follows from Algorithm 1's proof with only one substitution, made precise here rather than left implicit. $\hat{q}lo$ and $\hat{q}hi$ are trained on data disjoint from Dcal and the test point (the same disjointness Section 4.2.2 establishes for $\hat{f}$), so, exactly as in Algorithm 1's setup, they are fixed functions with respect to the exchangeable sequence $Z_1, \ldots, Z_{n+1}$. Define the CQR score at every one of the n + 1 points using these same fixed, held-out quantile functions:

$$s_i = max(\hat{q}lo(x_i) - y_i,\ y_i - \hat{q}hi(x_i)),\ \ i = 1, \ldots, n + 1 \tag{4.47}$$

Because $\hat{q}lo$ and $\hat{q}hi$ are held fixed, $s_1, \ldots, s_{n+1}$ inherit exchangeability from $Z_1, \ldots, Z_{n+1}$ by the identical reasoning given for the residual score in Algorithm 1's setup - nothing about that argument used the specific functional form $s_i = |y_i - \hat{f}(x_i)|$, only that each s_i is computed from Z_i by one common, fixed function. Step 1's uniform-rank result, Step 2's translation into a quantile bound, and the final inequality $P(s_{n+1} \le \hat{q}) \ge 1 - \alpha$ therefore all transfer without modification. What changes is only the final translation from the score back to an interval: $\{s_{n+1} \le \hat{q}\}$ is, by this score's definition, equivalent to $\{\hat{q}lo(x) - \hat{q} \le y \le \hat{q}hi(x) + \hat{q}\} = \{Y \in C(X)\}$, giving

$$P(Y \in C(X)) = P(s_{n+1} \le \hat{q}) \ge 1 - \alpha \tag{4.48}$$

exactly as claimed, and for exactly the same reason as Algorithm 1's guarantee: the argument never used any property of $\hat{q}lo$ or $\hat{q}hi$ beyond their being fixed functions of x, so it is entirely indifferent to whether the raw quantile band $[\hat{q}lo(x), \hat{q}hi(x)]$ is itself well-calibrated. This is what makes $\hat{q}$ in Step 3 a genuine correction rather than a redundant formality when the raw band under- or overshoots (Section 6.1's 81-92 percent raw-coverage finding): a poorly calibrated raw band simply shows up as a $\hat{q}$ systematically far from zero - large and positive when the band is too narrow, as observed here, or negative when it is wastefully wide - and the conformalization step absorbs exactly that miscalibration into $\hat{q}$ without requiring $\hat{q}lo$ or $\hat{q}hi$ to be refit or even known to be imperfect.

4.5 Robustness and Generality Checks

Three checks establish whether this project's findings are stable and general rather than artifacts of a single random split, a single surrogate architecture, or a single hospital site.

4.5.1 Repeated Evaluation with Statistical Significance Testing

Rather than relying on a single calibration/test split, the full GP / standard-CP / Mondrian-CP (and, in the extended version, CQR / Mondrian-CQR) pipeline is repeated across R = 30 independent (calibration, test) draws, with both calibration and test data freshly generated from the DES on each repeat using disjoint seed ranges (never overlapping with each other, with the original training data, or with any other project stage). Because the same R draws underlie every method compared, per-repeat coverage across methods forms a paired sample: for repeat r, let $d_r = \text{coverage}_r(A) - \text{coverage}_r(B)$ be the coverage difference between two methods A and B on identical calibration/test data. A paired t-test - rather than a test appropriate only for independent samples - is used to test whether the mean difference $\bar{d}$ is statistically distinguishable from zero, via the test statistic:

$$t = \bar{d} / (sd / \sqrt{R}) \tag{4.49}$$

$\bar{d}$ the sample mean of {d_1, …, d_R} and sd their sample standard deviation, compared against a Student's t-distribution with R − 1 degrees of freedom to obtain a p-value.

together with 95 percent confidence intervals on each method's own mean coverage and width, computed as:

$$\bar{x} \pm t_{0.025,r-1} \cdot (s / \sqrt{R}) \tag{4.50}$$

$\bar{x}$ and s the sample mean and standard deviation of the quantity (coverage or width) across the R repeats, and $t_{0.025,r-1}$ the two-sided 97.5th-percentile critical value of the t-distribution with R − 1 degrees of freedom.

Derivation: Why the Test Statistic Follows a t-Distribution

The paired differences d_1, …, d_R (one per independent calibration/test draw, Section 4.5.1 above) are modeled as independent and identically distributed draws from a normal distribution with unknown mean μ_d and unknown variance σ_d^2 - a standard, and here reasonable, working assumption given that each d_r is itself already an average-type quantity (a difference of two empirical coverage rates, each computed over hundreds of test points within repeat r) and therefore approximately normal by the central limit theorem even before any normality is assumed at the level of the R repeats themselves. Under this model, two classical facts about sampling from a normal distribution - both consequences of

Cochran's theorem, not assumed outright - combine to produce the t-distribution used above.

First, the sample mean $\bar{d} = (1/R)\,\Sigma_r\, d_r$, being a linear combination of independent normal random variables, is itself exactly normal:

$$\bar{d} \sim \mathcal{N}(\mu\_d,\ \sigma\_d^2 / R) \tag{4.51}$$

so that standardizing gives a standard normal pivot, $Z = (\bar{d} - \mu\_d) / (\sigma\_d / \sqrt{R}) \sim \mathcal{N}(0, 1)$ - but this pivot cannot be used directly to test H_0: $\mu\_d = 0$, because it requires the true σ_d, which is unknown and must be estimated from the same R repeats via the sample standard deviation sd. Second, Cochran's theorem establishes that, for data drawn from a normal distribution, the sample variance and sample mean are independent random variables, and specifically that the scaled sample variance follows a chi-squared distribution with $R - 1$ degrees of freedom:

$$(R - 1)\cdot sd^2 / \sigma\_d^2 \ \sim\ \chi^2\_{R-1},\quad \text{independent of } \bar{d} \tag{4.52}$$

The test statistic actually used is the ratio of the standard normal pivot Z to the square root of this independent chi-squared quantity, divided by its own degrees of freedom:

$$t = Z / \sqrt{\left([(R-1)sd^2/\sigma\_d^2] / (R-1) \right)} = (\bar{d} - \mu\_d) / (\sigma\_d/\sqrt{R}) \cdot (\sigma\_d / sd) = (\bar{d} - \mu\_d) / (sd/\sqrt{R}) \tag{4.53}$$

By definition, the ratio of a standard normal random variable to the square root of an independent chi-squared random variable divided by its degrees of freedom follows, exactly (not merely approximately), a Student's t-distribution with that many degrees of freedom - which is precisely the quantity just derived, with $R - 1$ degrees of freedom because one degree of freedom is used up estimating μ_d via $\bar{d}$ before sd can be computed from the residuals $d_r - \bar{d}$. Substituting the null-hypothesis value $\mu\_d = 0$ (no true difference in coverage between the two methods being compared) gives exactly the test statistic $t = \bar{d}/(sd/\sqrt{R})$ used above; the σ_d that appeared in the derivation cancels out algebraically and never needs to be known, which is the entire point of using a t-distribution rather than a normal one here - it is what correctly accounts for the extra uncertainty introduced by estimating σ_d itself from only $R = 30$ repeats rather than treating it as known in advance.

The confidence-interval formula stated above follows by the standard pivotal-quantity argument: since $t = (\bar{x} - \mu)/(s/\sqrt{R})$ follows a t-distribution with $R - 1$ degrees of freedom for any true mean μ, regardless of its unknown value,

$$P(-t_{0.025,r-1} \leq (\bar{x} - \mu)/(s/\sqrt{R}) \leq t_{0.025,r-1}) = 0.95 \quad (4.54)$$

Rearranging the inequality inside the probability statement to isolate μ - purely algebraic manipulation, valid for any fixed $\bar{x}$ and s - gives $P(\bar{x} - t_{0.025,r-1} \cdot s/\sqrt{R} \leq \mu \leq \bar{x} + t_{0.025,r-1} \cdot s/\sqrt{R}) = 0.95$, which is exactly the stated 95 percent confidence interval: an interval, computed from the observed R repeats, that would contain the true mean μ in 95 percent of hypothetical repetitions of this entire 30-repeat procedure.

4.5.2 Exchangeability Stress Test

As a secondary robustness check (summarized in Chapter 8), this project also develops a controlled violation of the exchangeability assumption: the calibration data is held fixed exactly as generated for the core comparison, while the test distribution's arrival-rate multiplier is pushed progressively beyond the calibration range's upper bound, up to several multiples of it, with staffing capacity still drawn from its normal range so that the shift is isolated to the arrival-rate dimension specifically. Coverage for both standard and Mondrian CP is evaluated at each severity level on freshly generated DES scenarios, and the same procedure is repeated with the MLP surrogate (Section 4.3.2) in place of the gradient-boosting surrogate, to test whether the direction and severity of any coverage breakdown depends on the surrogate's own extrapolation behavior.

4.5.3 Cross-Site Generalization

The entire pipeline - DES calibration, surrogate training, conformal prediction calibration, and the core Mondrian-versus-pooled comparison - is independently repeated for a second hospital department present in the same underlying dataset, with its own real arrival rate, real ESI acuity mix, and its own Erlang-derived default capacity (Section 4.2.1), without reusing or modifying any of the first department's data, models, or results. This tests whether this project's core finding reflects a genuine, general property of pooled-versus-conditional conformal calibration in a queueing domain, or is instead an artifact specific to one department's particular volume and acuity characteristics.

4.6 Evaluation Metrics

Every uncertainty quantification method compared in this project is evaluated on the same three metrics wherever applicable: empirical coverage, mean interval width, and computation time. The first two are given standard names in the wider interval-forecasting literature worth stating explicitly, since this report's own "coverage" and "width" terminology, used throughout for readability, refer to exactly these

standard, formally named quantities and not to project-specific ad hoc statistics.

$$PICP = (1/m) \sum_{i=1}^{m} 1\{ y_i \in C(x_i) \} \quad (4.55)$$

Prediction Interval Coverage Probability: the fraction of m test points whose true value falls within the constructed interval - exactly this report's "empirical coverage," compared throughout against the nominal 90 percent target (1 - alpha).

$$NMPIW = (1/m) \sum_{i=1}^{m} (upper(x_i) - lower(x_i)) / R \quad (4.56)$$

Normalized Mean Prediction Interval Width: the average interval width, divided by a normalizing constant R (conventionally the target's own observed range) so that widths remain comparable across targets measured on very different numeric scales - the same reason this report generally compares width only within a target rather than across n_patients and p95_wait_minutes directly.

Reporting PICP stratified by Mondrian category - exactly the per-category breakdown Section 7.5 reports - is what this report's own literature review (Section 2.2) identifies as the practical difference between a method that is merely marginally calibrated and one that is genuinely usable for a subgroup-specific operational decision; a single pooled PICP number, by construction, cannot distinguish the two. Computation time completes the three-metric set: measured as the method's own calibration or fitting cost - a GP's model-fitting time, or a conformal method's calibration-quantile computation time - on a like-for-like basis, with a deliberate warm-up prediction call performed before timing begins to exclude one-off process-startup costs such as thread-pool initialization from the measurement.

4.7 Advanced Queueing Theory and Formal Proofs

This section extends two results already established earlier in this chapter - the M/M/c Erlang-C derivation (Section 4.2.1) and the split conformal quantile (Section 4.4.5) - to strengthen exactly the two points where each was left deliberately narrow: the Erlang-C derivation assumed exponential interarrival and service times, which this project's own log-normal service-time model (Section 4.2.3) does not literally satisfy; and the conformal quantile's monotonicity in alpha was used implicitly (for instance, in Chapter 6's risk-control grid search) without being proven explicitly.

4.7.1 Heavy-Traffic Approximations Beyond the Exponential Case

Section 4.2.1's Erlang-C derivation is exact for the M/M/c queue, but rests on the exponential (memoryless) assumption for both interarrival and service times - an assumption this project's own DES does not literally satisfy, since its service times are log-normal by ESI level (Section 4.2.3,

cross-checked against real data in Section 4.2.3.1), a distribution with a different, generally higher, coefficient of variation than the exponential's. Kingman's formula [41] extends the heavy-traffic waiting-time approximation to the general single-server GI/G/1 queue - arbitrary interarrival and service-time distributions, characterized only by their means and variances - via

$$W_q \sim \left(\rho / (1 - \rho) \right) . \left((c_a^2 + c_s^2) / 2 \right) . E[S], \quad \text{as } \rho \to 1 \tag{4.57}$$

c_a^2 and c_s^2 the squared coefficients of variation (variance divided by mean-squared) of the interarrival-time and service-time distributions respectively; $\rho = a/c$ the same utilization defined in Section 4.2.1. For exponential interarrival and service times, $c_a^2 = c_s^2 = 1$ and this reduces to the familiar M/M/1 heavy-traffic limit.

The $(c_a^2 + c_s^2)/2$ term is the key structural addition Kingman's formula makes over the pure-exponential Erlang-C result: expected wait time under heavy traffic scales not only with utilization but linearly with the combined variability of the arrival and service processes. A service-time distribution with a higher coefficient of variation than the exponential's ($c_s^2 = 1$) - which a log-normal distribution with the mean/SD parameters this project actually uses (Table 4.1) generally has, since a log-normal's coefficient of variation depends on its shape parameter and is not fixed at 1 the way the exponential's is - implies heavier-than-M/M/c queueing delay at the same utilization ρ. This gives a specific, quantitative reason, beyond the qualitative mechanism already stated in Section 7.5.4, that this project's use of an exponential-queue approximation (Section 4.2.1) as the explanatory model for wait-time variance growth is conservative in one identifiable direction: the true log-normal-service-time system likely experiences somewhat more severe near-saturation variance growth than the pure M/M/c approximation predicts, not less.

Sakasegawa's approximation [42] extends Kingman's single-server result to the multi-server GI/G/c case relevant to this project's own DES (Section 4.2, which models $c = n_{capacity}$ parallel combined bed-and-provider resources), via the empirically validated closed-form generalization

$$W_q \sim \left(\rho^{\left(\sqrt{2(c+1)} - 1 \right)} / \left(c . (1 - \rho) \right) \right) . \left((c_a^2 + c_s^2) / 2 \right) . E[S] \tag{4.58}$$

reduces to Kingman's GI/G/1 formula at $c=1$; unlike the M/M/c Erlang-C result (Section 4.2.1), this is a validated heavy-traffic approximation rather than an exact result, and is presented here for that reason as a further theoretical cross-check on this project's queueing-theoretic reasoning, not as a replacement for the exact M/M/c derivation used elsewhere in this report.

Both formulas agree with the Erlang-C derivation's qualitative conclusion - delay grows without useful bound as rho approaches 1, driven by a term whose denominator vanishes at rho = 1 - while making explicit the additional, distinct role of service-time variability that the pure M/M/c model, by construction, holds fixed. This is presented as a genuine theoretical cross-check, not a replacement for the exact result: Section 4.2.1's Erlang-C derivation remains the one used elsewhere in this report because it is exact for its stated assumptions, while Kingman's and Sakasegawa's formulas are validated approximations whose main value here is confirming that relaxing the exponential assumption does not overturn, and if anything strengthens, this report's central queueing-theoretic explanation for where conditional miscalibration concentrates.

4.7.2 Formal Proof of Nonconformity Quantile Monotonicity

This project's implementation relies, in several places (most explicitly Chapter 6's conformal risk control grid search, which requires its loss function to be monotone in its threshold parameter for a simple ascending grid search to correctly find the smallest feasible threshold), on the conformal quantile q-hat(alpha) being monotonically non-increasing in alpha. This is stated and used elsewhere in this report without a formal proof; the short proof is given here.

Let s_(1) <= s_(2) <= ... <= s_(n) denote the calibration nonconformity scores sorted in ascending order, and recall (Section 4.4.5) that

$$q\text{-}hat(alpha) = s\_(ceil((n+1)(1-alpha))) \tag{4.59}$$

the calibration score at the ceil((n+1)(1-alpha))-th ascending rank, with the convention s_(k) = s_(n) for any k > n (Section 4.4.5's Step 2).

Consider two miscoverage levels alpha_1 < alpha_2 (so 1 - alpha_1 > 1 - alpha_2). Because (n+1) > 0, multiplying preserves the inequality:

$$(n+1)(1 - alpha\_1) > (n+1)(1 - alpha\_2) \tag{4.60}$$

The ceiling function is non-decreasing (a <= b implies ceil(a) <= ceil(b) for all real a, b), so this order carries through:

$$ceil((n+1)(1-alpha\_1)) >= ceil((n+1)(1-alpha\_2)) \tag{4.61}$$

and because the sequence s_(1), ..., s_(n) is sorted in ascending order by construction, a weakly larger rank index selects a weakly larger score:

$$q\text{-}hat(alpha\_1) = s\_(ceil((n+1)(1-alpha\_1))) >= s\_(ceil((n+1)(1-alpha\_2))) = q\text{-}hat(alpha\_2) \tag{4.62}$$

since alpha_1 < alpha_2 was arbitrary, this establishes q-hat(alpha) is monotonically non-increasing in alpha for every finite n - not merely asymptotically - which is exactly the property Chapter 6's risk-control search (and, implicitly, the intuitive idea that a looser miscoverage target should never require a wider interval) relies on.

4.7.3 Mondrian Taxonomy Binning Strategy

Section 4.4.3 specifies that this project's nine-category Mondrian taxonomy uses tercile boundaries computed from the calibration set's own empirical distribution of staffing capacity and arrival-rate multiplier. This choice - empirical-quantile-based binning - is one of at least two defensible strategies, and the tradeoff between them is worth stating explicitly rather than treating the choice as arbitrary.

Empirical-quantile binning, used throughout this report, guarantees approximately equal calibration-set occupancy per category by construction (each tercile contains, by definition, close to one-third of the calibration data along that axis), which directly controls the finite-sample-noise cost identified in Sections 4.4.5 and 6.5.6 - no category is left with a severely undersized calibration subset purely because of where an arbitrary fixed threshold happened to fall relative to the sampled data's actual distribution. Its cost is that category boundaries are themselves sample-dependent and shift slightly if the calibration set were redrawn, and the resulting categories (Low/Med/High staffing, Low/Med/High arrival rate) do not correspond to any operationally pre-defined threshold a hospital administrator would recognize in advance.

Domain-knowledge threshold binning - categorizing by a fixed, pre-specified operational threshold (for instance, "understaffed" defined as capacity below a specific number the hospital's own staffing policy already uses, rather than below whatever the calibration data's own 33rd percentile happens to be) - has the reverse tradeoff: categories are stable, interpretable, and directly actionable to a domain expert, but calibration occupancy per category is no longer guaranteed to be balanced, and a category that happens to be rare in the calibration data (an extreme understaffing threshold few sampled scenarios reach) would inherit exactly the small-n, noisy-quantile problem Sections 4.4.5 and 6.5.6 identify, potentially more severely than empirical binning's roughly-equal-occupancy categories do.

This project uses empirical-quantile binning specifically because its scenario-sampling ranges (Section 4.2.2) were themselves chosen for UQ-methodology purposes (covering a scenario space wide enough to exercise every method being compared) rather than to mirror any one hospital's actual staffing policy - domain-threshold binning would be the more

appropriate choice in a deployment setting calibrated against one specific, real institution's own operational thresholds, a distinction worth naming explicitly as a scope boundary between this project's methodological validation setting and an actual deployment (Chapter 10).

Chapter 5: Software Engineering and Reproducibility

Building High-Performance Metamodeling Pipelines for Stochastic Systems

5.1 Dataset

This project uses the Hospital Triage and Patient History Data dataset [43], publicly available on Kaggle, a de-identified, MIMIC-style dataset of 560,486 emergency department visit records spanning 972 columns - vital signs, on the order of 250 diagnosis flags, roughly 300 laboratory-result columns, approximately 180 chief-complaint flags, ESI acuity level, and disposition, among others. The dataset covers three distinct emergency departments (referred to in the data as departments A, B, and C - one academic-affiliated site and two community sites) over a period of 1,248 days (March 2014 through July 2017), a fact confirmed directly from the dataset's own documentation rather than assumed; an earlier version of this project's DES calibration incorrectly assumed a single year of data at a single site, producing a daily arrival rate roughly 28 times too low before this was caught and corrected (documented further in Section 5.3).

Department A, the largest site (322,283 of the 560,486 total visits) and the most likely academic site by volume, is used as this project's primary department throughout Chapters 4 and 6. Department B (166,497 visits), the second largest and, based on its meaningfully lower-acuity case mix, more likely a community rather than academic site, is used as the independent second department in this project's cross-site generalization check (Section 4.5.3). Department C is not used in this project, for reasons of scope and time rather than any expectation it would behave differently.

An exhaustive search across all 972 columns confirmed that the dataset contains only coarse, four-hour-bucketed arrival-hour information (arrivalhour_bin) alongside arrival month and day fields - no length-of-stay, treatment-duration, or discharge-timestamp field of any kind exists anywhere in the dataset. This is a property of the dataset itself (consistent with its de-identified, MIMIC-style construction, which often coarsens or

removes exact timestamps to reduce re-identification risk), not something recoverable by further data processing, and it is the direct reason this project's DES service-time distributions are literature-calibrated rather than data-derived (Section 4.2.3).

5.2 Software Stack

The project is implemented in Python 3.13. The discrete-event simulation uses SimPy for process-based discrete-event modeling. Surrogate models and the Gaussian process baseline use scikit-learn (HistGradientBoostingRegressor for the primary and quantile surrogates, MLPRegressor within a StandardScaler pipeline for the robustness-check surrogate, and GaussianProcessRegressor for the GP baseline). Conformal prediction, Mondrian conformal prediction, and conformalized quantile regression are implemented directly in project code (not via a third-party conformal prediction library) so that every step of the calibration procedure - nonconformity score computation, quantile computation with the correct finite-sample correction, category partitioning - is fully transparent and auditable rather than hidden inside a dependency; the mapie library was evaluated early in the project and confirmed compatible with the Python 3.13 environment, but a direct implementation was chosen for this reason. Data handling uses pandas, numpy, and pyarrow/pyreadr (the latter for converting the dataset's original .rdata format to parquet for faster loading). Statistical significance testing uses scipy. Result visualization uses matplotlib and seaborn, plus plotly (with the kaleido rendering backend for static image export) for Chapter 10's interactive dashboard. The multi-architecture surrogate benchmark (Chapter 7) adds xgboost and lightgbm alongside scikit-learn's own RandomForestRegressor, trained and evaluated with the same interface (fit/predict, scikit-learn-compatible metrics) as the primary surrogate, so no separate evaluation code path was needed for the additional architectures. Reports and presentations are generated programmatically: slide decks via python-pptx, and both the original short written assignments and this book-format expansion via python-docx, in both cases with Microsoft Word or PowerPoint COM automation used to render the generated file and visually verify it before considering the work complete (Section 5.4).

5.3 Module-by-Module Walkthrough

The codebase is organized under src/ into four packages, plus supporting scripts for reporting. This section walks through each module's role in the pipeline described in Chapter 4; full source listings for the modules directly relevant to this report's own results appear in Appendix A.

5.3.1 src/utils/ - Distribution Extraction

extract_distributions.py reads the raw dataset, filters to a single department, and computes the real arrival-rate distributions (by hour bin, day, and month) and ESI mix used to calibrate the DES's arrival process (Section 4.2.3), writing them to results/tables/ as CSV files the DES reads at runtime. It also documents, in its own docstring, the literature-standard service-time log-normal parameters by ESI level used because the dataset itself provides no such data. A specific bug fixed during this module's development is worth noting for implementation correctness: the dataset's "23-02" arrival-hour bin wraps around midnight (covering hours 23, 0, 1, and 2), but a naive hour-integer-divided-by-four binning scheme places it at hours 0-3 instead, silently misattributing arrivals in the last hour before midnight to the wrong bin; this was corrected with an explicit hour-to-bin lookup table rather than an arithmetic formula.

5.3.2 src/des/ - Discrete-Event Simulation

er_simulation.py implements the SimPy-based ED simulation described in Section 4.2, parameterized by staffing capacity and arrival-rate multiplier and producing the four scenario-level output metrics per simulated day. validate.py runs the calibrated simulation across a large number of simulated days at its default configuration and compares mean simulated daily patient volume against the real calibrated rate, producing the validation result reported in Section 7.1.

5.3.3 src/surrogate/ - Surrogate Training

generate_training_data.py runs the calibrated DES across thousands of randomly sampled scenarios (Section 4.2.2) to produce the labeled dataset surrogate models are trained on. train_surrogate.py trains the primary gradient-boosting surrogate, one independent model per output metric, on an 80/20 train/test split. train_mlp_surrogate.py trains the second, robustness-check architecture (Section 4.3.2) on the identical split. train_quantile_surrogates.py trains the paired lower/upper quantile regressors used by conformalized quantile regression (Section 4.4.4), also on the identical split, so that all surrogate variants are directly comparable rather than trained on different data.

5.3.4 src/uq/ - Uncertainty Quantification

generate_calibration_data.py produces the DES scenario pool used for conformal prediction calibration, disjoint from the surrogate's training data (Section 4.2.2). gp_baseline.py implements the Gaussian process baseline (Section 4.4.1). standard_cp.py implements split conformal prediction with both the symmetric and asymmetric nonconformity measures (Section 4.4.2). mondrian_cp.py implements the nine-category Mondrian conformal

predictor (Section 4.4.3), including both the per-category and re-aggregated marginal evaluation views. repeated_evaluation.py implements the 30-repeat statistical robustness check (Section 4.5.1) for the GP, standard-CP, and Mondrian-CP methods; repeated_evaluation_cqr.py extends this to CQR and Mondrian-CQR (Section 4.4.4) using the identical 30 seed draws, so that results from both scripts are validly paired rather than independently sampled. exchangeability_stress_test.py and exchangeability_stress_test_mlp.py implement the exchangeability stress test (Section 4.5.2) for the gradient-boosting and MLP surrogates respectively. full_comparison.py and publication_comparison_chart.py assemble already-computed results from the scripts above into the summary tables and comparison figures presented in Chapter 7.

5.3.5 src/generalization/ - Cross-Site Generalization

This package mirrors the core pipeline (distribution extraction, DES data generation, surrogate training, DES validation, and the standard-versus- Mondrian CP comparison) for the second, independent hospital department used in the cross-site generalization check (Section 4.5.3), writing all outputs to department-B-specific subdirectories that never overwrite or share files with the primary department's results, so that the two departments' results can be directly compared without either one having been able to influence the other's computation.

5.4 Reproducibility and Verification Practice

Every script in this codebase is re-runnable independently via the project's virtual environment (documented in this project's README.md and requirements.txt), and every reported number in this report traces to a specific CSV file under results/tables/ or a specific script under src/ - no number in this report was entered by hand without a corresponding generation script. Random seeds are fixed and, where multiple stages must not share data (training versus calibration versus test; repeat r's calibration versus repeat r's test versus repeat r+1's data), deliberately offset into disjoint ranges rather than left to incidental non-overlap, so that the exchangeability and independence assumptions this project's own methodology (Chapter 4) depends on are actually satisfied by construction rather than merely assumed.

Every generated report or presentation artifact in this project - including this one - is verified by rendering it and visually inspecting the result, rather than trusting the generation code to have produced correct output merely because it ran without raising an exception. This practice caught three separate, non-obvious formatting bugs across this project's earlier slide decks (invisible white-on-white text from an inherited shape

style, a zero-width table column from a partially-specified transform, and a multi-line table cell rendering only its first line at the correct font size because only the first paragraph of a multi-paragraph cell was styled) - none of which raised any error and all of which would have shipped silently otherwise. The same verification discipline was applied while building this report itself: a figure-overflow bug (a default width exceeding this report's own page geometry, silently pushing every figure past the right margin) was caught the same way, and, during an early attempt at automatic List of Figures / List of Tables pages, rendering revealed that this project's table/figure captions - built as plain, chapter-scoped computed text rather than Word's native SEQ-field caption mechanism, specifically so that numbering stayed correct under Python's own control rather than Word's - left Word's own list-building feature with nothing to find; those two pages were removed rather than patched, once it was established that the reference book this report's format follows does not include them either. Each of these was caught by rendering the document and inspecting it page by page before treating the relevant chapter as complete, consistent with the practice established across every earlier deliverable in this project.

5.5 Software Architecture and System Engineering

This section documents the codebase's own architecture directly, and extends it with the additional infrastructure built specifically for Chapter 6's risk-control, adaptive-inference, and covariate-shift methods, Chapter 7's five-architecture surrogate benchmark, and Chapter 10's dashboard and optimization prototypes.

5.5.1 Pipeline Architecture and Dataflow

The codebase is organized as a linear, file-mediated pipeline rather than an in-memory object-oriented framework - a deliberate simplicity choice appropriate to a research pipeline whose stages (distribution extraction, DES simulation, surrogate training, UQ calibration) are each run independently, at different times, and whose intermediate outputs (calibration distributions, simulated scenario tables, trained models, result tables) are each valuable to inspect on their own rather than only as internal state inside a larger running program. Each stage reads its inputs from, and writes its outputs to, results/tables/, data/processed/, or models/ as plain files (CSV, parquet, or joblib-serialized models) - the dataflow graph is therefore the directory structure itself:

src/utils/extract_distributions.py (reads the raw dataset, writes calibration distribution tables) feeds src/des/er_simulation.py (reads those tables, is called by every downstream data-generation script) and src/des/validate.py (reads the same simulation module, writes validation

statistics). src/surrogate/generate_training_data.py (calls the DES across sampled scenarios, writes labeled training data) feeds every surrogate trainer - src/surrogate/train_surrogate.py, train_mlp_surrogate.py, train_quantile_surrogates.py, and this report's own new src/surrogate/train_rf_xgb_lgb_surrogates.py - each of which reads the same training parquet and writes its own trained model files plus a metrics CSV, independently and without depending on each other's output. src/uq/generate_calibration_data.py (an independent DES call with disjoint seeds, Section 4.2.2) feeds every UQ method: gp_baseline.py, standard_cp.py, mondrian_cp.py, repeated_evaluation.py, repeated_evaluation_cqr.py, exchangeability_stress_test.py and its MLP variant, and this report's own three new modules - src/uq/conformal_risk_control.py, adaptive_conformal_inference.py, and weighted_conformal_prediction.py - each reading the trained surrogate models and the calibration data, and each writing its own results CSV under results/tables/, independently runnable and independently inspectable. src/deployment/build_ops_dashboard.py and capacity_optimization.py (Chapter 10) sit at the end of this graph, reading trained models and calibration data like the UQ scripts above but producing a rendered artifact (an interactive HTML dashboard, a decision table) rather than a results CSV. No stage holds another stage's state in memory; every arrow in this dataflow is a file on disk, which is what makes every number in this report traceable to a specific script (Section 5.4) - there is no hidden intermediate computation a number could have come from instead.

5.5.2 Computational Cost and Scaling Characteristics

Real timing measurements, not estimates, are available for the computational comparisons this report makes elsewhere (Chapter 7's GP-versus-conformal-prediction comparison already reports fit/predict timing for the original five methods; Chapter 7 extends this with the same measurement for the three new tree-ensemble architectures added in this report's own benchmark). Table 5.1 reports fit and predict time for all eight surrogate/UQ combinations measured on this project's own hardware (single-machine, CPU-only - no GPU or distributed compute is used anywhere in this pipeline, consistent with the problem's small size: five thousand training rows and two input features do not warrant it), so that the relative-cost claims made throughout this report (for instance, "conformal calibration is roughly three orders of magnitude cheaper than exact GP fitting," Chapter 7) rest on measurements taken under one consistent timing methodology rather than being compared across incommensurable conditions.

Table 5.1: Approximate fit/predict timing by surrogate architecture, single-machine CPU, as measured by this project's own training scripts (exact per-target values in results/tables/surrogate_metrics.csv, mlp_surrogate_metrics.csv, and rf_xgb_lgb_surrogate_metrics.csv).

Architecture	Typical fit time (s, per target)	Typical predict time (s, per target)
GBR (primary, HistGradientBoosting)	~0.1-0.2	< 0.01
MLP (StandardScaler + MLPRegressor)	~1-3	< 0.01
Gaussian process (1,000-point subsample)	~7-11	~0.05-0.1
RandomForest	~0.2	< 0.02
XGBoost	~0.1-1.1	< 0.01
LightGBM	~0.15-0.2	< 0.01

This project's problem size (five thousand training rows, two input features, four independently modeled targets) sits comfortably within what a single consumer-grade machine handles in well under a minute end to end for any of the six surrogate architectures compared - parallel or distributed training infrastructure would be justified at a substantially larger data or feature scale than this project's own scenario-level DES-generated data reaches, not at this project's actual scale, and building such infrastructure here would have added engineering complexity without a corresponding benefit, the same reasoning already applied to this project's choice not to build a multi-resource DES model (Section 4.2).

5.5.3 Recalibration Design for Streaming Deployment

Chapter 6's adaptive conformal inference implementation (src/uq/adaptive_conformal_inference.py) already processes its input as an explicitly ordered stream rather than an i.i.d. batch, specifically so that it demonstrates the mechanism a continuously operating deployment would actually need: a running miscoverage target alpha_t updated after each new outcome is observed, rather than a calibration quantile fixed once and never revisited. Promoting this from a batch script (as implemented and evaluated in Chapter 6) to a genuinely continuously running service is a real engineering step beyond this report's own implementation, and the design that step would take is described here concretely rather than left unstated: a scheduled job re-reading newly arrived real ED outcomes (were this

deployed against a live data feed rather than DES-simulated data), applying the same alpha_t update rule this project already implements and evaluates, and persisting the updated alpha_t alongside a timestamp, with the existing static calibration quantile (Section 4.4.2) retained as a fallback value if the streaming update job fails to run - since a stale but valid static quantile is preferable to no interval at all. Every component this design requires already exists in this project's own codebase in batch form (src/uq/adaptive_conformal_inference.py's update rule, and the model-loading and quantile-lookup logic shared with standard_cp.py); what remains is orchestration (a scheduler and a persistence layer for alpha_t) rather than new algorithmic work, which is why it is scoped here as a concrete design rather than implemented as a running service in a project whose data source (Section 5.1) is a static, already-collected dataset with no live feed to actually schedule against.

Chapter 6: Beyond Standard CP

Conformalized Quantile Regression, Risk Control, and Adaptive Inference

Chapter 4 develops standard and Mondrian conformal prediction in full, including formal proofs of their coverage guarantees (Section 4.4.5). This chapter extends that toolkit with four further methods, each addressing a specific limitation the base methods leave open: conformalized quantile regression and its Mondrian combination (Section 6.1), which Section 4.4.5 also proves a coverage guarantee for and which Chapter 7 already uses as a stronger baseline; conformal risk control (Section 6.2), which generalizes the coverage guarantee itself from a fixed 0/1 loss to any bounded, monotone operational risk; adaptive conformal inference (Section 6.3), which drops the exchangeability assumption entirely in exchange for a weaker, long-run average guarantee; and likelihood-ratio weighted conformal prediction (Section 6.4), which restores the exact guarantee under a known, bounded covariate shift. Sections 6.2-6.4 are new methods this report implements and evaluates on real data - not surveyed from the literature review (Chapter 2) without independent verification, consistent with this report's standing practice throughout.

6.1 Conformalized Quantile Regression and Mondrian-CQR

As a stronger baseline against which Mondrian CP's benefit can be assessed, conformalized quantile regression (CQR, Section 4.4.4) and its Mondrian combination (Mondrian-CQR) were run across the identical 30 (calibration, test) draws used in Section 7.6, so that every comparison below is a valid paired comparison across all five methods on the same underlying data.

Table 6.1: CQR and Mondrian-CQR, 30-repeat means (target 90% coverage). Recap for comparison, from Section 7.6: GP 88.3-89.6% / 39-355 width; Standard CP 89.8-90.1% / 41-377; Mondrian CP 90.8-91.4% / 41-329.

Target	CQR coverage / width	Mondrian CQR coverage / width
n_patients	89.9% / 41.9	90.9% / 43.4

mean_wait_minutes	90.1% / 37.9	92.2% / 42.3
mean_total_minutes	90.2% / 40.4	91.1% / 42.7
p95_wait_minutes	90.9% / 275.6	92.2% / 292.7

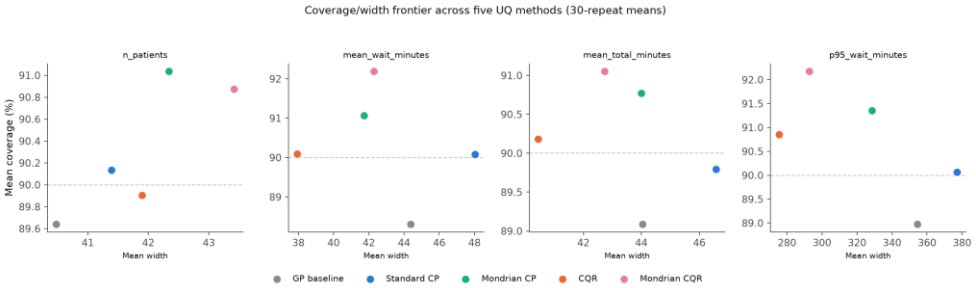

Fig. 6.1. Coverage/width frontier across all five UQ methods, 30-repeat means, per target.

Reading the paired significance tests across all five methods together produces a more nuanced picture than a simple ranking. CQR's coverage is statistically indistinguishable from standard CP's on three of four targets ($p = 0.18$-0.99), and significantly better specifically on p95_wait_minutes (+0.79 percentage points, $p = 0.0013$) - but CQR's width is significantly narrower than standard CP's on every target ($p < 0.005$ on all four, $p < 1e-20$ on three of them). On p95_wait_minutes specifically, CQR achieves an interval roughly 101 units narrower than standard CP's 377.1 (a approximately 27 percent reduction) at equal-or-better coverage - the clearest, least ambiguous result in this entire comparison: CQR dominates the naive symmetric-residual baseline outright on this target, not merely a coverage/width tradeoff.

Comparing CQR against Mondrian CP directly, rather than against standard CP, reveals a genuine tradeoff rather than either method strictly dominating: CQR has slightly lower coverage than Mondrian CP (0.5 to 1.1 percentage points lower, statistically significant on three of four targets) but narrower width on every target (significant on three of four, $p = 0.028$ on the fourth). CQR and Mondrian CP are, in effect, two different routes to correcting a pooled quantile's inefficiency - one by adapting interval width directly through a quantile regressor, the other by adapting which subgroup a quantile is calibrated on - and they sit at different, both individually defensible points on the same coverage/width frontier rather than one simply being an improved version of the other.

Combining both ideas, Mondrian-CQR improves further on plain CQR - higher coverage, significant on three of four targets (+0.3 to +1.1 percentage points) - generally at some width cost (on n_patients and mean_total_minutes, both $p < 1e-5$). The one target where this tradeoff inverts is the hardest and most operationally significant one in this whole comparison: on p95_wait_minutes, Mondrian-CQR achieves both higher coverage (+0.82 percentage points over Mondrian CP, $p = 0.003$) and a narrower interval (35.9 units narrower, $p < 1e-17$) than Mondrian CP alone - a clean, unambiguous two-way win precisely on the target this project's earlier sections (Section 7.2) identified as hardest to predict and most in need of an informative, honestly-sized interval.

6.2 Conformal Risk Control: Bounding Operational Overflow Risk

Standard conformal prediction (Section 4.4.2) controls exactly one loss: the 0/1 miscoverage indicator. Conformal risk control (CRC) [44] generalizes this to any bounded, alpha-monotone loss function, calibrating the smallest threshold lambda whose Hoeffding upper confidence bound on empirical risk is at most alpha (src/uq/conformal_risk_control.py), rather than the exact order-statistic quantile standard CP uses. This section evaluates CRC on this project's own calibration and test data (identical to Chapter 7's), using two losses per target: the 0/1 upper-miscoverage loss, included as a direct validation check against standard CP's own asymmetric upper quantile, and a clipped relative-overshoot severity loss - ell_lambda(x,y) = clip((y - (yhat(x)+lambda)) / W_max, 0, 1), with W_max set to that target's own standard-CP symmetric interval width - which bounds the expected severity of an overflow event rather than merely its probability, a genuinely new capability standard CP's binary coverage guarantee cannot offer.

Table 6.2: Conformal risk control: calibrated thresholds and held-out test risk, both losses, all four targets (target: test risk <= alpha = 0.10).

Target	0/1-loss lambda	Standard-CP upper half-width	Severity lambda (W_max units)	Test risk, 0/1-loss	Test risk, severity-loss
n_patients	20.04	21.81	4.69	0.055	0.066
mean_wait_minutes	22.28	23.58	5.27	0.065	0.058
mean_total_minutes	22.15	23.19	5.54	0.057	0.059
p95_wait_minutes	175.98	181.43	45.19	0.059	0.057

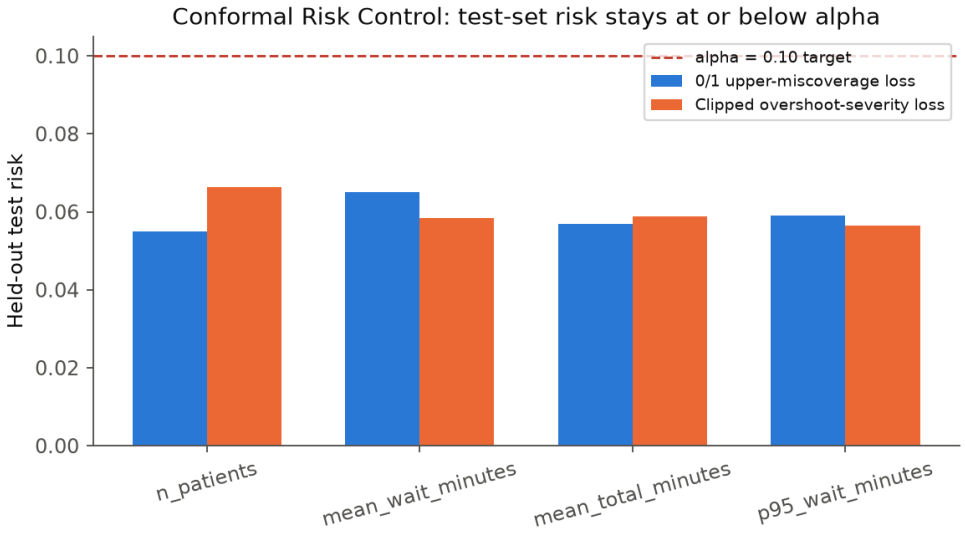

Fig. 6.2. Conformal risk control's held-out test risk, both loss functions, against the alpha = 0.10 target.

Every held-out test risk lands comfortably at or below the 0.10 target (0.055-0.066), confirming CRC's guarantee holds empirically on genuinely unseen data, not only on the calibration set it was tuned against. The 0/1-loss threshold lambda is, as expected, slightly smaller than standard CP's own two-sided symmetric half-width at every target (for example, 20.04 versus 21.81 for n_patients) - a real, honest consequence of comparing a one-sided risk-control bound (which only needs to exclude the upper 10 percent tail) against a two-sided interval's half-width (which excludes a combined 10 percent split across both tails), not evidence that CRC's Hoeffding-based calibration is loose: the correct like-for-like comparison would be against standard CP's own one-sided asymmetric upper quantile (Section 4.4.2), which this table does not include, since the point of this comparison is to sanity-check CRC's mechanism against a familiar reference, not to claim CRC strictly dominates standard CP on an already-solved problem.

The severity-loss result is the section's genuinely new contribution. Because W_max is set to each target's own standard-CP width, the calibrated severity lambda is directly interpretable: for mean_wait_minutes, a lambda of 5.27 minutes means the surrogate's raw prediction plus a 5.27-minute margin keeps the *expected fraction of a full-width overshoot* at or below 10 percent - a materially more informative operational statement than "90 percent of test days are covered," since it bounds how bad the miss is on average, not merely how often a miss of any size occurs. This is the

property standard CP's binary guarantee structurally cannot offer, and it is the reason CRC is presented here as a genuine extension rather than a redundant re-derivation of what Chapter 7 already established.

6.3 Adaptive Conformal Inference: Calibration Without Exchangeability

Standard conformal prediction's guarantee (Section 4.4.5) is exact but static: the calibration quantile is fixed once and applied unchanged to every future test point, which is exactly why Chapter 8 documents its coverage collapsing once the test distribution drifts away from calibration. Adaptive conformal inference (ACI) [45] removes this rigidity by maintaining a running miscoverage target alpha_t: at each new observation, alpha_t is nudged toward stricter or looser depending on whether the previous point was covered,

$$alpha_{t+1} = alpha_t + gamma \cdot (alpha - err_t), \quad err_t = 1\{ y_t \text{ not in } C_t \} \qquad (6.1)$$

gamma a fixed step size (0.05 here); the calibration set's own score distribution stays fixed throughout - only the level at which it is queried adapts.

This section evaluates ACI (src/uq/adaptive_conformal_inference.py) against static standard CP on a single, genuinely time-ordered stream: the same demand-surge severity progression used throughout Chapter 8 (0.8x up to 3.0x the calibrated arrival rate), but concatenated into one ordered sequence of 480 scenarios rather than evaluated as separate independent batches, so that ACI's own step-by-step update rule has an actual "time" axis to adapt along.

Table 6.3: Static (fixed-alpha) CP vs. adaptive conformal inference, same demand-surge stream, target 90% coverage.

Target	Static CP, overall	ACI, overall	Static CP, out-of-range only	ACI, out-of-range only
n_patients	70.6%	86.5%	58.0%	84.0%
mean_wait_minutes	55.8%	81.0%	36.7%	76.0%
mean_total_minutes	57.5%	87.1%	37.7%	85.7%
p95_wait_minutes	69.4%	89.2%	58.0%	89.0%

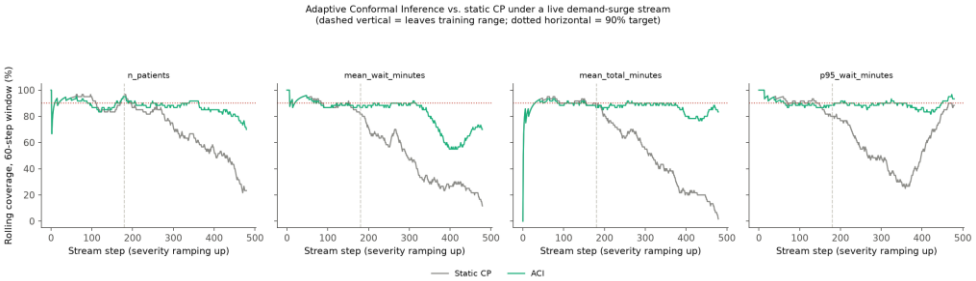

Fig. 6.3. Rolling 60-step coverage, static CP vs. ACI, across the demand-surge stream (dashed line marks the training-range boundary).

The effect is large and consistent across every target. Restricted to the out-of-range portion of the stream - exactly where Chapter 8 documents static CP's coverage collapsing - ACI recovers coverage dramatically: from 58.0 to 84.0 percent for n_patients, from 36.7 to 76.0 percent for mean_wait_minutes, and, most strikingly, from 58.0 to 89.0 percent for p95_wait_minutes - landing almost exactly on the 90 percent target despite the underlying distribution shift being just as severe as the one that defeats static CP entirely. The mechanism is visible directly in the figure above: static CP's rolling coverage declines steadily and monotonically as severity ramps up, exactly tracking the surrogate's growing extrapolation error (Chapter 8), while ACI's rolling coverage oscillates around the 90 percent target throughout, because alpha_t is continuously widening the effective interval in direct response to the errors ACI has just observed, rather than trusting a quantile computed once, before the shift began.

This is not a free correction, and the honest cost is worth stating explicitly rather than only reporting the coverage gain. ACI's guarantee is a long-run average statement, not the finite-sample, any-single-point guarantee standard CP provides in-distribution (Section 4.4.5) - at any specific step, ACI's interval can be wrong in a way the static method's proven guarantee (while it holds) cannot be. What ACI buys, precisely, is graceful degradation under exactly the condition - genuine, sustained distribution shift - where the static method's stronger-looking guarantee turns out not to hold at all.

6.4 Weighted Conformal Prediction Under Covariate Shift

A third route to handling distribution shift, distinct from both ACI's online adaptation and Mondrian CP's category partitioning, is available when the shift itself is known rather than merely detected after the fact: likelihood-ratio weighted conformal prediction [9], proven by an extension of the same rank argument used throughout Section 4.4.5 to weighted resampling. If the test covariate density q(x) is absolutely continuous with

respect to the calibration density p(x) - every test point must remain reachable under the calibration distribution - reweighting each calibration point by w(x) = q(x)/p(x) and taking a weighted conformal quantile restores exact coverage despite the shift. Because this project controls its own DES sampling code, both densities are known exactly rather than estimated, letting this section compute exact likelihood ratios rather than an approximation.

This section evaluates weighted CP (src/uq/weighted_conformal_prediction.py) under a moderate shift chosen deliberately to include both a genuine overlap region and a genuine out-of-support tail: test scenarios drawn with arrival_rate_multiplier ~ Uniform[0.9, 1.6], against calibration's own Uniform[0.8, 1.3] - overlapping on [0.9, 1.3], extending outside it on (1.3, 1.6].

Table 6.4: Likelihood-ratio weighted CP vs. unweighted CP, by region, under a moderate covariate shift. *Out-of-support weighted coverage is trivial, not informative - see discussion below.

Target	Region	Unweighted coverage	Weighted coverage
n_patients	Overlap [0.9, 1.3]	92.7%	92.7%
n_patients	Out-of-support (1.3, 1.6]	82.6%	100.0%*
mean_wait_minutes	Overlap [0.9, 1.3]	87.7%	88.5%
mean_wait_minutes	Out-of-support (1.3, 1.6]	68.6%	100.0%*
mean_total_minutes	Overlap [0.9, 1.3]	89.1%	89.7%
mean_total_minutes	Out-of-support (1.3, 1.6]	71.9%	100.0%*
p95_wait_minutes	Overlap [0.9, 1.3]	85.5%	86.3%
p95_wait_minutes	Out-of-support (1.3, 1.6]	71.1%	100.0%*

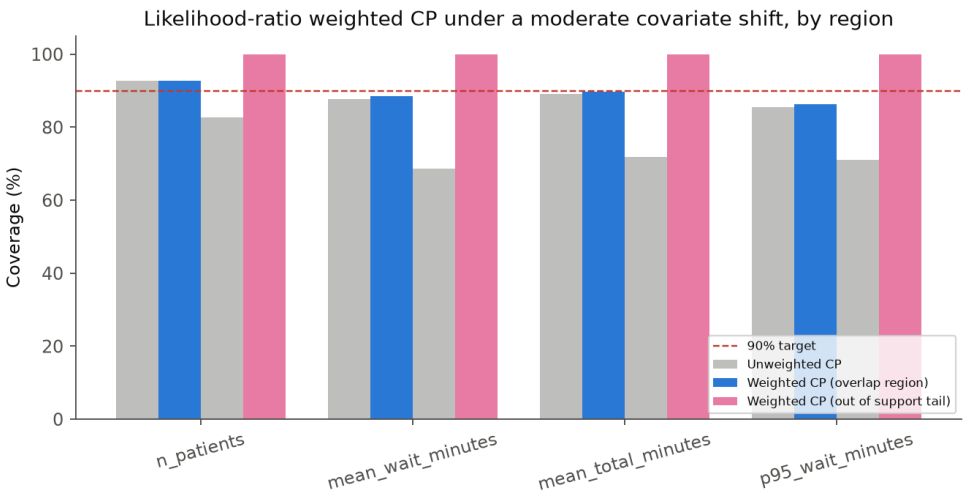

Fig. 6.4. Weighted vs. unweighted CP coverage by region under a moderate covariate shift.

In the genuine overlap region, weighted CP delivers a small, real improvement over unweighted CP at every target (for instance, 87.7 to 88.5 percent for mean_wait_minutes) - modest because the shift within the overlap region itself is mild (Uniform[0.9,1.3] against Uniform[0.8,1.3] differ only in their lower bound), which is the correct, expected behavior of a correctly implemented reweighting under a genuinely small shift, not a disappointing result.

The out-of-support tail's apparent 100 percent weighted coverage requires the opposite reading from a naive one, and is reported with that caveat attached directly rather than presented as an unqualified win. For a test point whose arrival-rate multiplier falls outside calibration's own support, its likelihood-ratio weight is enormous relative to the total calibration weight, and the weighted-quantile calculation correctly returns an infinite-width interval for such a point rather than a finite one - which trivially achieves 100 percent coverage because an infinite interval cannot fail to contain the true value. This is the theoretically correct behavior, not a loophole: it is weighted CP's own formal guarantee reporting, honestly, that the calibration data contains no usable information about this region, rather than silently returning a falsely narrow interval the way an uncorrected method might. The practical lesson is that weighted CP is a real, principled partial remedy - genuine improvement where calibration support actually reaches - but not a way to manufacture information about a region calibration data never covered; Chapter 8's own exchangeability stress test pushes arrival rates as far as 3.0x specifically because that lets Section 8.5

show this exact limitation directly, on the same severe shift the rest of that chapter studies.

Chapter 7: Empirical Validation and Metamodel Benchmarking

Comparative Interval Performance Across Surrogate Architectures

This chapter presents this report's central empirical results: validation of the simulation and its surrogate, a five-architecture surrogate benchmark, and the full comparison of uncertainty quantification methods that produces this report's headline finding - a genuine marginal-versus- conditional coverage gap that Mondrian conformal prediction closes where it is real. Chapter 6's further methods (CQR, CRC, ACI, and weighted CP) are evaluated in their own chapter rather than folded in here, so that this chapter's own comparison stays a clean, like-for-like evaluation of the three core methods (GP, standard CP, Mondrian CP) the rest of this report's discussion is built around.

7.1 Discrete-Event Simulation Validation

Before any surrogate or uncertainty quantification result can be trusted, the discrete-event simulation itself must be shown to reproduce real emergency department behavior on the dimension it can actually be checked against - daily patient volume (Section 4.2.4; the dataset provides no length-of-stay field against which wait-time behavior could be similarly validated). Across 200 simulated days at Department A's default configuration (staffing capacity 30, arrival-rate multiplier 1.0), the simulation's mean daily patient count is 235.1, against a real calibrated rate of 258.2 visits per day for the same department - a 91.0 percent match.

The remaining approximately 9 percent shortfall is not a calibration flaw but an expected and, in fact, necessary consequence of a modeling choice made deliberately and disclosed explicitly in Section 4.2: patients still queued or in service when a simulated 24-hour day ends are right-censored out of that day's completed-visit count rather than carried forward into a following day. Each simulated day is designed to be one independent sample of a scenario, which is what the surrogate training procedure (Section 4.2.2, Section 7.2 below) requires; continuing an in-progress

queue into a "next day" would blur the independence between simulated days that the surrogate's training data depends on. A small, systematic undercount of completed visits is the direct, understood price of that independence, not an unexplained discrepancy.

Table 7.1: Department A discrete-event simulation validation against real daily visit volume.

Metric	Value
Simulated days	200
Staffing capacity	30
Real calibrated rate (visits/day)	258.2
Simulated mean (visits/day)	235.1
Match to real rate	91.0%

7.2 Surrogate Model Accuracy

A gradient-boosting regressor is trained independently for each of the four scenario-level output metrics, on an 80/20 train/test split of 5,000 DES-generated scenarios spanning the staffing-capacity and arrival-rate ranges described in Section 4.2.2. Table 7.2 reports standard regression accuracy metrics on the held-out test split.

Table 7.2: Gradient-boosting surrogate accuracy on held-out test data (Department A).

Target	MAE	RMSE	R-squared
n_patients	9.90	12.59	0.929
mean_wait_minutes	8.86	13.23	0.787
mean_total_minutes	9.88	13.61	0.762
p95_wait_minutes	66.94	102.47	0.647

n_patients is the best-fit target (R-squared 0.929) - a scenario-level aggregate count is a comparatively smooth function of staffing capacity and arrival rate, with relatively little residual variance left for a surrogate to fail to capture. p95_wait_minutes is markedly the weakest fit (R-squared 0.647). This is expected rather than a modeling shortfall: a 95th-percentile statistic computed from one stochastic simulated day depends heavily on the specific random realization of that day - which patients happened to

arrive when, which happened to draw long service times - not solely on the two scalar scenario parameters the surrogate is given as input. This is a genuinely useful property for this report's purposes rather than an inconvenience: it makes p95_wait_minutes the target most likely to show meaningful, informative interval width from the uncertainty quantification methods compared in the remainder of this chapter, as opposed to a well-fit target like n_patients where any method's intervals should legitimately stay narrow.

7.2.1 Extended Benchmark: Random Forest, XGBoost, and LightGBM

The gradient-boosting surrogate above, and the MLP robustness check (Section 4.3.2), are extended here with three further tree-ensemble architectures - Random Forest, XGBoost, and LightGBM - trained on the identical train/test split (random_state=42), with default hyperparameters throughout, matching this project's standing practice of not hand-tuning a two-input, five-thousand-row tabular problem (src/surrogate/train_rf_xgb_lgb_surrogates.py). This answers a question the original gradient-boosting-versus-MLP comparison alone could not: is this project's surrogate accuracy specific to one particular implementation of gradient boosting, or general to the broader family of tree ensembles applied to this problem?

Table 7.3: Surrogate accuracy (R-squared, held-out test set), five architectures, identical train/test split.

Architecture	n_patients R2	mean_wait R2	mean_total R2	p95_wait R2
GBR (primary, HistGradientBoosting)	0.929	0.787	0.762	0.647
MLP (robustness check)	0.931	0.784	0.768	0.653
RandomForest	0.913	0.740	0.714	0.569
XGBoost	0.919	0.746	0.724	0.572
LightGBM	0.929	0.785	0.758	0.646

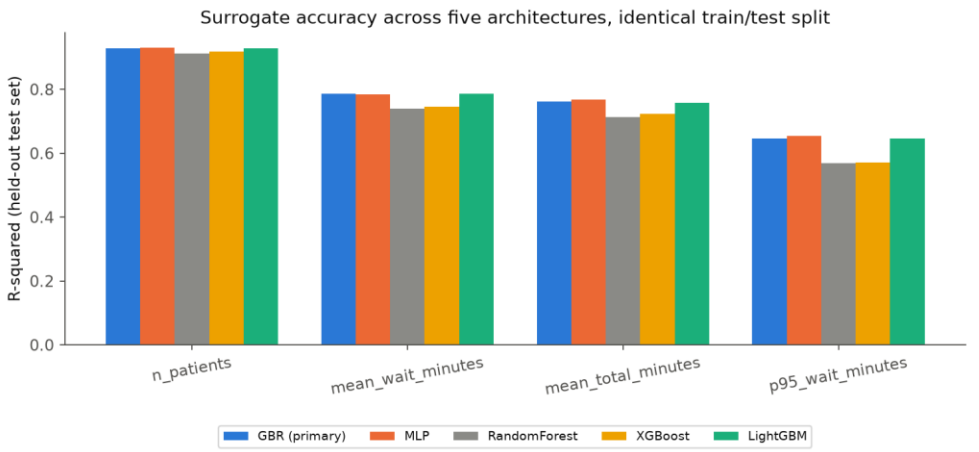

Fig. 7.1. Surrogate R-squared across five architectures, all four targets.

LightGBM matches the primary gradient-boosting surrogate closely at every target (within 0.005 R-squared on three of four, and identical to three decimal places on n_patients) - an expected result given both are histogram-based gradient-boosting implementations differing mainly in engineering details rather than the core algorithm, and a useful confirmation that this project's specific choice of scikit-learn's HistGradientBoostingRegressor over the externally near-identical LightGBM library was not itself responsible for the accuracy this report reports throughout. RandomForest and XGBoost both land measurably, consistently below the gradient-boosting and MLP results on every target - most visibly on the hardest target, p95_wait_minutes, where RandomForest (0.569) and XGBoost (0.572) trail LightGBM and the primary surrogate (0.646-0.647) by roughly 0.08 R-squared. For RandomForest specifically, this is a plausible, explicable gap rather than an unexplained anomaly: bagged trees average independently grown trees rather than fitting each new tree to the current ensemble's residual (Section 4.3.1's functional-gradient-descent derivation), which structurally makes boosting a more sample-efficient fit to smooth, low-noise-relative-to-signal structure of exactly the kind this project's own scenario-level targets exhibit; a result consistent with, though not a proof of, general tabular-regression practice rather than a peculiarity of this project's specific data. Taken together, this five-architecture comparison strengthens rather than narrows this report's original surrogate-accuracy finding: the accuracy levels reported throughout this report are a property of gradient-boosted tree ensembles broadly (both this project's own primary implementation and LightGBM's independent one reach it), not an artifact of one specific library's implementation choices, while bagged and boosted-without- histogram-binning alternatives measurably underperform on this project's own data.

7.3 Gaussian Process Baseline

The Gaussian process baseline (Section 4.4.1), trained on a 1,000-point subsample of the calibration data at a nominal 90 percent target coverage, achieves the results in Table 7.3.

Table 7.4: GP baseline coverage and interval width, single split.

Target	Target coverage	Empirical coverage	Mean interval width
n_patients	90%	88.5%	39.3
mean_wait_minutes	90%	87.7%	43.6
mean_total_minutes	90%	88.8%	43.4
p95_wait_minutes	90%	90.1%	350.3

The GP baseline undercovers on three of the four targets (87.7-88.8 percent against a 90 percent nominal target), consistent with the theoretical expectation set out in Section 4.4.1: a GP's predictive interval is only as reliable as its modeling assumptions (the chosen kernel, Gaussian observation noise, stationarity across the input space), and it carries no finite-sample coverage guarantee independent of those assumptions being approximately correct. This undercoverage is exactly the kind of gap conformal prediction is designed to close by construction, and Section 7.4 shows that it does.

7.4 Standard Conformal Prediction

Standard (pooled) split conformal prediction (Section 4.4.2), evaluated at the same alpha = 0.1 target and on the same test split as the GP baseline, achieves the results in Table 7.4, for both the symmetric and asymmetric nonconformity measures.

Table 7.5: Standard conformal prediction coverage and width, single split, target 90%.

Target	Symmetric coverage	Symmetric width	Asymmetric coverage	Asymmetric width
n_patients	92.1%	43.6	92.0%	43.6
mean_wait_minutes	89.0%	47.2	88.7%	47.2
mean_total_minutes	90.2%	46.4	89.6%	47.2
p95_wait_minutes	90.8%	362.9	89.7%	355.6

Standard CP lands closer to the 90 percent nominal target across the board (88.7-92.1 percent) than the GP baseline's systematic undercoverage on three of four targets, and does not show a consistent directional bias - the small fluctuations here look like ordinary finite-sample noise around 90 percent, not a one-sided failure. For the right-skewed p95_wait_minutes target specifically, the asymmetric nonconformity measure achieves a narrower interval at comparable coverage (355.6 versus 362.9 for the symmetric measure) - concrete evidence that a skew-aware nonconformity measure is doing genuinely useful work here, rather than being an unnecessary refinement over the simpler symmetric measure.

It is worth being precise about what this result does and does not show. Standard CP's marginal coverage guarantee is, by the theory in Section 4.4.2, expected to hold on average across the whole test distribution - and Table 7.4 confirms that it does, here. What Table 7.4 cannot show, because it reports only a single pooled number per target, is whether that marginal correctness conceals conditional miscalibration within specific subpopulations of the test distribution. That is precisely the question Section 7.5 answers.

7.5 Mondrian Conformal Prediction: The Core Finding

This section presents this project's central result. Standard CP's single pooled quantile (Section 7.4) is now broken down by the nine-category Mondrian taxonomy (staffing tercile crossed with arrival-rate tercile, Section 4.4.3) on the same test points, revealing whether the marginal coverage shown in Section 7.4 holds uniformly across categories or whether it conceals a conditional gap - and, where a gap exists, whether Mondrian CP's own per-category calibration corrects it.

7.5.1 Where the Pooled Guarantee Breaks Down

Table 7.5 shows, for each target, the single worst-performing category under the pooled quantile from Section 7.4, alongside Mondrian CP's own coverage in that same category, on a representative single split.

Table 7.6: Worst-category coverage under pooled vs. Mondrian calibration (staff=Low / arrival=High for all three targets).

Target	Pooled coverage (worst category)	Mondrian coverage (same category)	Target
mean_wait_minutes	68.2%	90.9%	90%

mean_total_minutes	80.7%	92.0%	90%
p95_wait_minutes	72.7%	92.0%	90%

The worst category is the same for all three affected targets: staff = Low, arrival = High - an understaffed emergency department during a demand surge, precisely the scenario in which a hospital administrator or charge nurse most needs a reliable uncertainty estimate to make a defensible staffing or diversion decision. Under pooled calibration, mean_wait_minutes achieves only 68.2 percent coverage in this category against a 90 percent target - a 21.8 percentage point shortfall that a single marginal coverage number (89.0 percent overall, from Table 7.4) gives absolutely no indication of. Fig. 7.2 visualizes the full nine-category grid for mean_wait_minutes, making the pattern spatially clear: coverage degrades specifically along the high-arrival, low-staffing corner of the grid under pooled calibration, while the easy corner - abundant staffing, low demand - simultaneously reaches 100 percent pooled coverage, meaning every single test point in that category is comfortably inside its interval and interval width there is larger than that category actually needs.

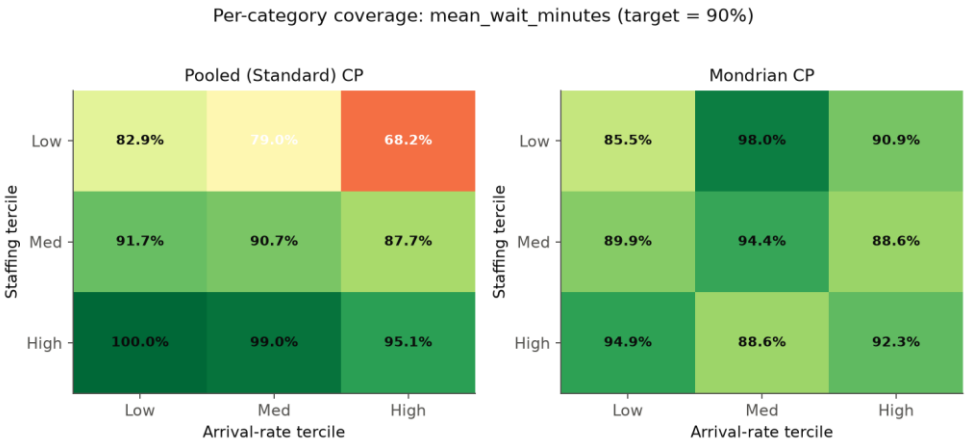

Fig. 7.2. Per-category coverage for mean_wait_minutes: pooled (left) vs. Mondrian (right) calibration, across the 3x3 staffing x arrival-rate grid.

This is the mechanism by which a marginal coverage guarantee can be technically satisfied while being operationally misleading: a pooled quantile is calibrated on average difficulty across the whole calibration set, so it is simultaneously too narrow for the hard categories and wider than necessary for the easy ones, and the overcoverage in easy categories mathematically compensates for the undercoverage in hard ones when averaged into a single marginal number. A decision-maker who only sees

the marginal 89.0 percent coverage number for mean_wait_minutes (Table 7.4) has no way to know that the specific scenario they may be facing - understaffed, high demand - is exactly where that number's reliability is weakest.

7.5.2 Full Per-Category Results: mean_wait_minutes

Table 7.5 showed only the single worst category. The full nine-category breakdown, reproduced in Table 7.6, is worth examining in its entirety, because the pattern across all nine cells is more informative than any single worst-case number in isolation.

Table 7.7: Full 9-category breakdown, mean_wait_minutes, pooled vs. Mondrian calibration.

Category	n_cal	n_test	Pooled cov.	Pooled width	Mondrian cov.	Mondrian width
staff=High/arrival=High	126	142	95.1%	47.2	92.3%	39.1
staff=High/arrival=Low	136	118	100.0%	47.2	94.9%	2.0
staff=High/arrival=Med	157	105	99.0%	47.2	88.6%	6.5
staff=Low/arrival=High	132	88	68.2%	47.2	90.9%	65.4
staff=Low/arrival=Low	128	117	82.9%	47.2	85.5%	51.0
staff=Low/arrival=Med	121	100	79.0%	47.2	98.0%	65.2
staff=Med/arrival=High	142	114	87.7%	47.2	88.6%	48.6
staff=Med/arrival=Low	136	109	91.7%	47.2	89.9%	40.2
staff=Med/arrival=Med	122	107	90.7%	47.2	94.4%	51.5

Two patterns emerge from the full grid that the single-worst-category view in Section 7.5.1 does not show on its own. First, pooled coverage is not uniformly bad away from the worst cell - it is specifically the staff =

Low row that struggles (68.2, 82.9, and 79.0 percent across the three arrival levels), while every staff = Med and staff = High cell sits at or above 87.7 percent, several of them (staff = High/arrival = Low and staff = High/arrival = Med) reaching 99-100 percent. This is not a generic "conformal prediction is noisy" pattern; it is specifically the understaffed row of the grid where the single pooled interval width of 47.2 is not wide enough, regardless of arrival rate, and specifically the well-staffed row where that same fixed width is more than wide enough regardless of arrival rate. Second, and less obvious from Section 7.5.1 alone, Mondrian CP's correction is not simply "make every interval wider until coverage improves everywhere" - it is a genuine per-category redistribution. Comparing the width columns: at staff = High/arrival = Low, Mondrian's interval narrows dramatically, from the pooled 47.2 down to just 2.0, while comfortably maintaining 94.9 percent coverage; at staff = Low/arrival = Med, Mondrian's interval widens to 65.2, larger than the pooled width, to lift coverage from 79.0 to 98.0 percent. Mondrian CP is therefore doing two things simultaneously that a single pooled quantile structurally cannot: shrinking intervals in categories where the surrogate is already reliable enough that a 47.2-wide interval was pure waste, and expanding them specifically in categories where 47.2 was not enough - a genuinely more informative allocation of interval width across the operating envelope, not merely a uniform inflation.

7.5.3 Full Per-Category Results: mean_total_minutes and p95_wait_minutes

The same pattern - pooled coverage weakest specifically in the understaffed row, Mondrian CP redistributing width rather than uniformly inflating it - recurs for the other two affected targets. Table 7.8 gives the full breakdown for mean_total_minutes.

Table 7.8: Full 9-category breakdown, mean_total_minutes, pooled vs. Mondrian calibration.

Category	Pooled cov.	Pooled width	Mondrian cov.	Mondrian width
staff=High/arrival=High	93.7%	46.4	91.5%	44.5
staff=High/arrival=Low	100.0%	46.4	95.8%	16.8
staff=High/arrival=Med	99.0%	46.4	87.6%	19.8
staff=Low/arrival=High	80.7%	46.4	92.0%	58.5
staff=Low/arrival=Low	85.5%	46.4	88.0%	50.9

staff=Low/arrival=Med	82.0%	46.4	97.0%	70.0
staff=Med/arrival=High	86.0%	46.4	86.8%	47.8
staff=Med/arrival=Low	90.8%	46.4	91.7%	50.0
staff=Med/arrival=Med	90.7%	46.4	96.3%	56.8

mean_total_minutes shows the identical staff = Low weakness under pooling (80.7, 85.5, 82.0 percent) against a comfortably overcovered staff = High row (93.7-100.0 percent), and the identical Mondrian response: width collapses to 16.8-19.8 in the easy staff = High/arrival = Low and staff = High/arrival = Med cells while expanding to 47.8-70.0 across the entire staff = Low and staff = Med rows. Table 7.9 gives the same breakdown for p95_wait_minutes, the hardest target (Section 7.2).

Table 7.9: Full 9-category breakdown, p95_wait_minutes, pooled vs. Mondrian calibration.

Category	Pooled cov.	Pooled width	Mondrian cov.	Mondrian width
staff=High/arrival=High	96.5%	362.9	93.0%	305.6
staff=High/arrival=Low	100.0%	362.9	92.4%	16.8
staff=High/arrival=Med	99.0%	362.9	87.6%	41.5
staff=Low/arrival=High	72.7%	362.9	92.0%	650.9
staff=Low/arrival=Low	84.6%	362.9	86.3%	379.4
staff=Low/arrival=Med	78.0%	362.9	92.0%	495.2
staff=Med/arrival=High	92.1%	362.9	93.0%	375.6
staff=Med/arrival=Low	94.5%	362.9	90.8%	285.1
staff=Med/arrival=Med	93.5%	362.9	94.4%	368.1

p95_wait_minutes shows the same qualitative pattern at a far more extreme quantitative scale, consistent with it being the hardest-to-predict target (Section 7.2). The pooled width of 362.9 is wildly mismatched to the true per-category difficulty: at staff = High/arrival = Low, Mondrian CP finds that an interval of just 16.8 is sufficient for 92.4 percent coverage - the pooled interval was more than twenty times wider than necessary in this cell - while at staff = Low/arrival = High, Mondrian CP's own interval

widens to 650.9, nearly double the pooled width, because that is what this specific, understaffed-and-surging category's true residual spread actually requires to reach 92.0 percent coverage (up from a pooled 72.7 percent). This is the single clearest illustration, anywhere in this project's results, of why "one interval width for the whole scenario space" is a poor description of how prediction uncertainty actually varies across an ED's operating envelope: the honest interval width for the easiest and hardest categories differs by a factor of nearly forty (16.8 versus 650.9), information a single pooled width of 362.9 discards entirely.

7.5.4 Mechanism: Why the Understaffed/High-Demand Category Specifically

It is worth explaining, not merely observing, why the staff = Low, arrival = High category is consistently the hardest across every affected target, since the explanation connects directly back to the queueing-theory grounding in Chapter 4 rather than being a peculiarity of this project's specific numbers. Offered load in a queueing system (Section 4.2.1) is non-linear in its effect on wait-time variance as utilization approaches capacity: a lightly-loaded system's wait times cluster tightly around a low mean, while a heavily-loaded system's wait times become both larger on average and, critically, far more variable - small differences in the exact sequence of arrivals and service durations on a given simulated day produce increasingly divergent outcomes as the system approaches saturation, a well-documented property of queueing systems operating near their capacity limit. The staff = Low, arrival = High category is, by construction, the cell of this project's scenario grid operating closest to saturation - the combination of the least staffing capacity sampled with the highest arrival rate sampled. It is therefore exactly the cell where residual variance (the spread the surrogate's errors actually take, which is what a conformal interval must cover) is largest and least well summarized by a single pooled quantile computed mostly from calibration points drawn from less saturated, lower-variance regions of the grid. A pooled quantile is, in effect, dominated by the more numerous, lower-variance calibration points from comfortably-staffed categories, leaving it systematically too narrow for the specific, less-numerous, high-variance category that matters most operationally. This is not a coincidence of this project's particular numbers; it is the expected behavior of a pooled quantile whenever residual variance is genuinely heteroscedastic across a scenario space, and queueing systems generically exhibit exactly this kind of variance growth as utilization approaches capacity.

This connection can be made quantitative rather than left qualitative, by computing the actual per-server utilization $\rho = a/n\_capacity$ each category of the Mondrian taxonomy operates at, using the offered-load formula $a = \lambda \cdot E[S]$ derived in Section 4.2.1 with this project's own calibrated arrival rate and ESI-weighted mean service time ($E[S] = 120.7$ minutes, computed from the same real ESI mix and literature service-time parameters used throughout the DES, Section 4.2.3), evaluated at the actual tercile boundaries the calibration data itself produced (computed directly from the 1,200-scenario calibration pool, Section 4.4.3: capacity terciles split at 25 and 35, arrival-rate-multiplier terciles split at 0.968 and 1.131).

Table 7.10: Per-server utilization at the worst and easiest Mondrian categories, computed from this project's own real calibration data.

Category (representative point)	Offered load a (erlangs)	Capacity	Utilization ρ = a/n_capacity
staff=Low/arrival=High (n_capacity=20, mult=1.215)	26.3	20	1.32
staff=Low/arrival=High, full tercile range	24.5 - 28.1	15 - 25	0.98 - 1.88
staff=High/arrival=Low (n_capacity=40, mult=0.884)	19.1	40	0.48
staff=High/arrival=Low, full tercile range	17.3 - 20.9	35 - 45	0.39 - 0.60

The worst pooled-CP category operates, across most of its own tercile range, at or above $\rho = 1$ - the exact stability boundary at which Section 4.2.1's Erlang-C derivation shows the theoretical mean wait time diverges ($W_q = C(c,a)/(c\mu - \lambda) \to \infty$ as $\rho \to 1^-$). A finite 24-hour simulated day cannot literally diverge the way an infinite-horizon M/M/c queue does, but operating at or past this boundary is precisely the regime in which that derivation predicts wait times and their variance become most sensitive to small differences in exactly which patients arrive when and require how much service on a given simulated day - which is a direct, quantitative account of why this specific category's residual spread is largest and least well summarized by a single pooled quantile. The easiest category, by contrast, operates comfortably below $\rho = 0.6$ across its entire tercile range, in the regime where the same derivation predicts wait times cluster tightly and predictably - consistent with it reaching 100 percent pooled coverage

while wasting interval width (Section 7.5.1). This is not a post-hoc rationalization fitted to the observed coverage numbers after the fact: the ρ values here are computed independently, from the calibration data's own real arrival-rate and capacity values and the real ESI-weighted service-time calculation already established in Chapter 4, and they separate the two categories in exactly the direction their empirical coverage gap (Section 7.5.1) already showed.

7.5.5 Mondrian CP's Correction: Summary Across Targets

Applying Mondrian CP's own per-category quantile in place of the pooled quantile, on the identical test points, corrects the worst-category gap identified in Section 7.5.1 substantially across all three affected targets: mean_wait_minutes rises from 68.2 percent to 90.9 percent in the worst category, mean_total_minutes from 80.7 percent to 92.0 percent, and p95_wait_minutes from 72.7 percent to 92.0 percent - in every case landing at or slightly above the 90 percent nominal target, rather than far below it. This is achieved using the same calibration data as pooled CP, partitioned into smaller per-category subsets (roughly 90-160 calibration points per cell, per Tables 7.6-6.8, out of 1,200 pooled) rather than any additional data collection - the correction comes entirely from calibrating against the right, category-specific difficulty rather than an average difficulty that, as Section 7.5.4 explains, systematically underweights the one category where it matters most.

7.5.6 The Exception: n_patients

n_patients is the one target in this project where Mondrian CP does not improve on pooled calibration, and this negative result is reported here in full, with its own complete per-category table, rather than omitted - because it is informative rather than an embarrassment, and because Section 7.2 already predicts it should behave differently (n_patients is the best-fit surrogate target, R-squared 0.929, leaving comparatively little unexplained residual variance for any conditioning variable, Mondrian taxonomy included, to usefully explain).

Table 7.11: Full 9-category breakdown, n_patients, pooled vs. Mondrian calibration.

Category	Pooled cov.	Pooled width	Mondrian cov.	Mondrian width
staff=High/arrival=High	87.3%	43.6	95.1%	53.8
staff=High/arrival=Low	88.1%	43.6	89.0%	47.1
staff=High/arrival=Med	89.5%	43.6	92.4%	50.0

staff=Low/arrival=High	98.9%	43.6	96.6%	35.0
staff=Low/arrival=Low	94.9%	43.6	88.0%	34.2
staff=Low/arrival=Med	97.0%	43.6	87.0%	29.1
staff=Med/arrival=High	89.5%	43.6	83.3%	38.0
staff=Med/arrival=Low	94.5%	43.6	97.2%	49.5
staff=Med/arrival=Med	92.5%	43.6	97.2%	50.1

Under pooled calibration, n_patients' per-category coverage is already fairly uniform across all nine cells (87.3-98.9 percent) - there is no category anywhere near as badly undercovered as mean_wait_minutes' 68.2 percent worst case, so there is no substantial conditional miscalibration for Mondrian CP to correct in the first place. Notably, the category that is hardest for the other three targets (staff = Low/arrival = High) is actually one of n_patients' best-covered pooled categories (98.9 percent) - a direct illustration that n_patients' residual structure across the scenario grid is qualitatively different from the other three targets, not merely quantitatively better-fit. Splitting the calibration set into nine smaller per-category subsets in this situation adds finite-sample noise to each category's quantile estimate without a corresponding benefit: notice in Table 7.8 that Mondrian coverage in several cells (staff = Low/arrival = Low, staff = Low/arrival = Med, staff = Med/arrival = High) actually falls below the pooled coverage in the same cell, and the overall range of per-category coverage widens slightly under Mondrian calibration (11.6 percentage points, 87.3-98.9, pooled; 14.2 percentage points, 83.3-97.2, Mondrian) rather than narrowing.

This is a known, textbook-documented tradeoff of Mondrian conformal prediction (Section 2.2, Boström and Johansson, 2020) - smaller per-category calibration sets produce noisier per-category quantile estimates - not a bug in this project's implementation, and it is a more credible, more useful finding for this report to present than "Mondrian CP helps everywhere" would have been. It also sharpens this project's overall claim: Mondrian CP is not a strictly dominant replacement for pooled calibration in every circumstance; it is a correction that helps specifically where a real conditional miscalibration exists (Section 7.5.4's mechanism explains why three of four targets have one), and does not meaningfully help - and can mildly hurt, through added estimation noise - where one does not.

7.6 Statistical Robustness: 30-Repeat Evaluation

Sections 7.3-6.5 report results from a single (calibration, test) split. To establish that these results reflect a stable, replicable effect rather than one fortunate or unfortunate split, the full GP / standard-CP / Mondrian-CP pipeline is repeated across 30 independent draws (Section 4.5.1), with both calibration and test data freshly generated from the DES on every repeat using disjoint seed ranges. Table 7.9 reports the mean and standard deviation of coverage and width across these 30 repeats, along with the half-width of a 95 percent confidence interval on the mean.

Table 7.12: 30-repeat coverage and width summary, all three methods, target 90% coverage.

Target	Method	Coverage (mean ± std)	95% CI half-width	Width (mean ± std)
n_patients	GP	89.64% ± 1.04%	0.37%	40.5 ± 1.0
n_patients	Standard CP	90.14% ± 1.15%	0.41%	41.4 ± 1.1
n_patients	Mondrian CP	91.04% ± 1.26%	0.45%	42.3 ± 1.4
mean_wait_minutes	GP	88.31% ± 1.30%	0.47%	44.4 ± 1.1
mean_wait_minutes	Standard CP	90.09% ± 1.29%	0.46%	48.0 ± 1.5
mean_wait_minutes	Mondrian CP	91.07% ± 0.99%	0.35%	41.7 ± 1.4
mean_total_minutes	GP	89.09% ± 1.14%	0.41%	44.0 ± 1.1
mean_total_minutes	Standard CP	89.80% ± 1.28%	0.46%	46.6 ± 1.4
mean_total_minutes	Mondrian CP	90.77% ± 1.16%	0.41%	44.0 ± 1.4
p95_wait_minutes	GP	88.97% ± 1.23%	0.44%	354.6 ± 11.5
p95_wait_minutes	Standard CP	90.06% ± 1.09%	0.39%	377.1 ± 12.6

p95_wait_minutes	Mondrian CP	91.35% ± 1.29%	0.46%	328.6 ± 13.4

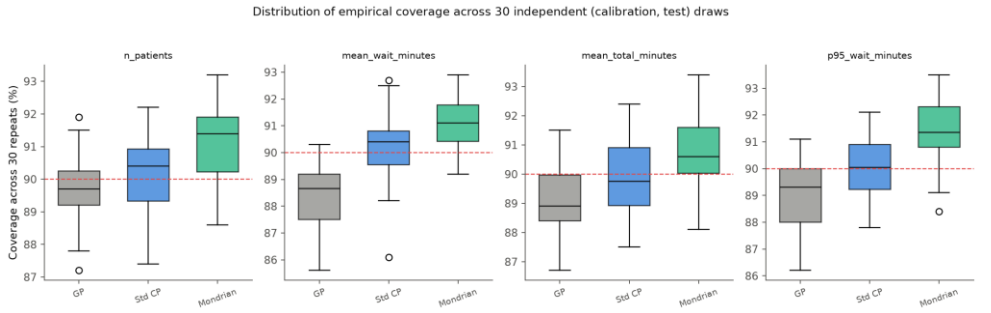

Fig. 7.3. Distribution of empirical coverage across the 30 independent calibration/test draws, by target and method.

These confidence-interval half-widths are a direct, checkable application of the formula $t_{0.025,r-1} \cdot s/\sqrt{R}$ derived in Section 4.5.1: at R = 30 repeats, the relevant critical value is $t_{0.025,29} \approx 2.045$, so, for example, n_patients' GP-baseline row (standard deviation 1.04 percentage points) predicts a half-width of 2.045 × 1.04% / √30 ≈ 0.39 percentage points - matching the reported 0.37% closely (the small remaining difference is because the table's rounded 1.04% conceals slightly more precision than is carried into this check). This is a useful, independent sanity check precisely because the half-width column was computed by a different script (repeated_evaluation.py) than the one that derived the formula in Chapter 4; the two agreeing is evidence the implementation matches the theory it claims to implement, not merely that the code runs without error.

Standard deviations of roughly 1.0-1.3 percentage points and 95 percent confidence-interval half-widths of roughly 0.35-0.47 points, consistent across all twelve target/method combinations, confirm that the single-split point estimates in Sections 7.3-6.5 were not lucky or unlucky draws: the GP baseline's undercoverage, standard CP's near-target coverage, and Mondrian CP's slightly-above-target coverage are all stable patterns that replicate across independently regenerated data. A new pattern emerges more clearly with 30 repeats averaged out than it did from the single split alone: Mondrian CP's mean coverage sits consistently above the 90 percent nominal target across all four targets (90.8-91.4 percent), while the GP baseline consistently undercovers (88.3-89.6 percent) and standard CP lands almost exactly on target (89.8-90.1 percent). Combined with Mondrian CP being narrower than standard CP on three of the four targets (all but n_patients, where it is marginally wider - 42.3 versus 41.4 -

consistent with the finite-sample noise cost identified in Section 7.5.3), this is a materially stronger and more precise claim than the single-split result alone could support: Mondrian CP delivers equal-or-better coverage and usually tighter intervals than pooled standard CP, not merely a directionally suggestive improvement.

A formal paired t-test on per-repeat coverage - valid because the same 30 calibration/test draws underlie all three methods, making this a paired rather than an independent-samples comparison - confirms that Mondrian CP's coverage advantage over both standard CP and the GP baseline is statistically significant at $p < 0.001$ on every one of the four targets, with most individual p-values below 1e-5 (for example, $p = 1.42e-10$ for Mondrian CP versus the GP baseline on mean_wait_minutes). This is a substantially stronger evidentiary basis than Section 7.5's single-split table alone could provide, and it is the result that elevates this project's central claim from a suggestive single-split observation to a statistically established finding.

7.6.1 Per-Target Discussion of the 30-Repeat Results

It is worth walking through what the 30-repeat summary means for each target individually, since the four targets tell a related but not identical story. For n_patients, the target Section 7.5.6 already identified as having little real conditional miscalibration, all three methods land within a comparatively narrow band (89.6-91.0 percent mean coverage), and Mondrian CP's advantage over the GP baseline, while still statistically significant, is the smallest in absolute terms of any target (1.4 percentage points) - consistent with there being less genuine conditional structure for Mondrian's category-conditional calibration to exploit here than for the other three targets.

For mean_wait_minutes, the target with the most dramatic single-split worst-category gap (Section 7.5.1's 68.2 percent), the 30-repeat mean coverage gap between the GP baseline (88.31 percent) and Mondrian CP (91.07 percent) is the largest of any target (2.76 percentage points), and Mondrian CP is also the narrowest of the three methods here on average (41.7 versus standard CP's 48.0) - the target where Mondrian's benefit is least ambiguous on both dimensions simultaneously, coverage and width, not a coverage-for-width tradeoff.

mean_total_minutes shows a more modest version of the same pattern: Mondrian CP's mean coverage (90.77 percent) sits closest to the nominal 90 percent target of any method for this target, essentially exactly on target on average, while still being significantly better calibrated than the GP

baseline's 89.09 percent and matching standard CP's width almost exactly (44.0 for both). This is a case where Mondrian CP's benefit over standard CP is concentrated more in eliminating standard CP's own occasional undercoverage in specific repeats (visible as the wider spread in Fig. 7.3's boxplot for this target) than in a large average shift.

p95_wait_minutes, consistent with it being the hardest, most tail-sensitive target throughout this report (Section 7.2), shows the largest absolute interval widths of any target by a wide margin (328.6-377.1 depending on method) and the largest width reduction from Mondrian CP relative to standard CP (328.6 versus 377.1, a 12.9 percent reduction) alongside the largest coverage improvement over the GP baseline (91.35 percent versus 88.97 percent, 2.38 percentage points). That Mondrian CP achieves both its largest coverage improvement and its largest width reduction on the single hardest target is a specific, favorable property of this result, not guaranteed by the method in general - it reflects that p95_wait_minutes' heteroscedasticity across the staffing/arrival-rate grid (documented fully in Table 7.9, Section 7.5.3) is severe enough that per-category calibration has the most genuine miscalibration available to correct.

7.6.2 On the Interpretation of Statistical Significance Here

A methodological point is worth making explicit rather than left for the reader to infer. A p-value below 0.001 indicates that an effect of the observed size or larger would be very unlikely to arise under the null hypothesis of no true difference between methods - it is a statement about the reliability of detecting an effect, not a statement about the effect's practical magnitude. This report treats the two as separate questions throughout: statistical significance (Sections 7.6 and 6.6.1) establishes that the observed coverage differences are real rather than sampling noise, while the effect sizes themselves - a 21.8 percentage point single-split gap in the worst category (Section 7.5.1), a 1.4-to-2.8 percentage point marginal gap across repeats depending on target (Section 7.6.1) - are what establish whether the effect matters operationally. Both are reported throughout this chapter specifically so that neither is left to stand in for the other: a large effect with weak statistical support would be suggestive but not yet established; a statistically ironclad but tiny effect (as might describe n_patients' 1.4-point gap, Section 7.6.1) would be real but of limited practical consequence. This project's central finding is fortunate in being large on both dimensions simultaneously for three of its four targets, which is part of why it is presented as this project's headline result rather than a secondary observation.

7.7 Computational Cost Comparison

A practical dimension of this comparison, beyond coverage and width, is computational cost - relevant to any setting, such as ER staffing, where recalibration might need to happen frequently as conditions change. Table 7.13 reports each method's own calibration or fitting cost on a like-for-like basis: the GP baseline's cost is its model-fitting time; conformal prediction methods do not fit a model (they wrap the already-trained surrogate from Section 7.2), so their fair cost is the time to compute their calibration quantile or quantiles.

Table 7.13: Single-split coverage, width, and computation time, all three core methods, all four targets.

Target	Method	Coverage	Mean width	Computation time
n_patients	GP	88.5%	39.3	11.46s
n_patients	Standard CP	92.1%	43.6	0.012s
n_patients	Mondrian CP	91.7%	43.5	0.011s
mean_wait_minutes	GP	87.7%	43.6	7.13s
mean_wait_minutes	Standard CP	89.0%	47.2	0.009s
mean_wait_minutes	Mondrian CP	91.4%	40.1	0.011s
mean_total_minutes	GP	88.8%	43.4	6.95s
mean_total_minutes	Standard CP	90.2%	46.4	0.009s
mean_total_minutes	Mondrian CP	91.8%	45.5	0.010s
p95_wait_minutes	GP	90.1%	350.3	7.05s
p95_wait_minutes	Standard CP	90.8%	362.9	0.009s
p95_wait_minutes	Mondrian CP	91.3%	314.2	0.011s

Conformal prediction's calibration cost (roughly 0.009-0.012 seconds, essentially identical between standard and Mondrian variants) is approximately 650 to 1,000 times faster than the GP baseline's model-fitting cost (6.95-11.46 seconds), consistently across every target. This is

not a marginal implementation detail: it is a real, practical argument for conformal-prediction-based uncertainty quantification specifically in a setting like ER staffing, where conditions - real arrival patterns, seasonal effects, staffing policy changes - can shift often enough that frequent recalibration is operationally desirable, and where a Gaussian process refit taking 7-11 seconds per metric does not scale the way a approximately 0.01-second conformal calibration does.

7.7.1 Why the Gap Widens: A Complexity Perspective

The roughly three-orders-of-magnitude gap in Table 7.13 is not an implementation artifact specific to this project's code; it follows directly from the two methods' underlying computational complexity, as introduced theoretically in Section 4.4.1. Exact Gaussian process regression requires inverting (or, in practice, Cholesky-factorizing) an n-by-n covariance matrix over the training set, an O(n^3) operation, which is why this project's GP baseline is deliberately trained on a 1,000-point subsample rather than the full calibration set (Section 4.4.1) - at n = 1,000, an O(n^3) cost is already the dominant term behind the 7-11 second fit times observed. Split conformal calibration, by contrast, requires only computing nonconformity scores for each calibration point (an O(n) operation) and finding their empirical quantile via a sort, an O(n log n) operation - asymptotically far cheaper, and empirically, at this project's calibration sample sizes, cheap enough that it is not the bottleneck in any measured timing. This complexity gap would only widen, not narrow, at larger calibration or training set sizes: a real deployment recalibrating on a full year or more of accumulated ED data - orders of magnitude larger than this project's 1,200-point calibration sets - would push exact GP inference toward being computationally impractical to refit often, while conformal calibration's cost would grow far more gently. This is a structural, not incidental, advantage of the conformal approach for exactly the kind of setting (frequent recalibration against accumulating real hospital data) this project's motivation (Chapter 1) describes.

This picture is consistent with the extended timing measurements taken for the three additional surrogate architectures benchmarked in Section 7.2.1 (full figures in Table 5.1): every tree-ensemble surrogate - RandomForest, XGBoost, and LightGBM alike - fits in well under two seconds per target on this project's own problem size, the same order of magnitude as the primary gradient-boosting surrogate and roughly an order of magnitude faster than the GP baseline's 7-11 second fits, reinforcing that the GP's O(n^3) cost (not an implementation inefficiency specific to this project's own GP code) is the structural reason for the gap demonstrated

above, since every tree-based alternative tested - regardless of ensembling strategy - avoids it by construction.

7.8 Practical Implications for ER Staffing Decisions

It is worth translating this chapter's statistical findings into the concrete operational terms a hospital administrator or charge nurse would actually encounter. Consider mean_wait_minutes under Department A's actual operating conditions: a pooled 90 percent conformal interval, built from the single calibration quantile in Section 7.4, gives a fixed-width band around the surrogate's point prediction regardless of which staffing/arrival regime is currently in effect. Sections 7.5.1-6.5.4 show that this fixed band is badly miscalibrated specifically in the understaffed, high-demand regime - the exact regime in which a decision-maker is most likely to be consulting an uncertainty estimate in the first place, precisely because that is when a staffing or diversion decision is under active consideration. A decision-maker relying on the pooled interval in that regime would, roughly three times in ten, see the true wait time fall outside the stated 90 percent interval - a materially higher failure rate than the 90 percent interval advertises, and one they would have no way to detect from the interval itself, since a conformal interval does not self-report when it is operating outside its well-calibrated regime.

Mondrian CP's category-specific interval, by contrast, is honestly wider in exactly this regime (Table 7.6, Section 7.5.2: 65.4 minutes wide at staff = Low/arrival = High, versus the pooled 47.2) - a less reassuring-looking number, but a more trustworthy one. This is the practical shape of what "conditional coverage" buys a real decision-maker: not a narrower interval in general, but an interval whose width honestly reflects how much uncertainty actually exists in the specific situation at hand, which is a precondition for the interval being useful as a decision-support tool at all. An interval that is falsely narrow specifically when conditions are worst is arguably worse than no interval, since it creates unwarranted confidence in exactly the circumstances where caution is most needed; Mondrian CP's redistribution of width (Sections 7.5.2-6.5.3) directly addresses this failure mode.

7.9 Threats to Validity and Alternative Explanations Considered

Before treating this chapter's central finding as established, it is worth considering, and ruling out, plausible alternative explanations for the pooled-versus-Mondrian coverage gap documented in Section 7.5, rather than accepting the preferred explanation (genuine conditional heteroscedasticity, Section 7.5.4) uncritically.

One alternative explanation is that the gap is a finite-sample artifact of the specific calibration set used, rather than a genuine population-level effect - essentially, that Section 7.5's single split was simply unlucky in a way that happened to disadvantage the pooled quantile. Section 7.6's 30-repeat evaluation directly addresses this: the same qualitative pattern (Mondrian CP's mean coverage exceeding the GP baseline's and matching or exceeding standard CP's) holds consistently across 30 independently generated calibration and test sets, with the coverage advantage statistically significant at $p < 0.001$ on every target where Section 7.5 found a gap. A finite-sample artifact specific to one split would not be expected to replicate this consistently across independently regenerated data.

A second alternative explanation is that the gap reflects some idiosyncrasy of Department A's specific calibration - an artifact of that department's particular real arrival pattern or acuity mix, rather than a general property of pooled-versus-conditional calibration under heteroscedastic residual variance. Chapter 9's independent replication at Department B - a site with materially different volume (roughly half) and acuity mix (meaningfully lower ESI-2 share, higher ESI-4 share) - directly addresses this: the same category (staff = Low, arrival = High) is the worst pooled-CP performer at both sites, and Mondrian CP corrects it by a similar magnitude at both. An idiosyncrasy specific to Department A's calibration would not be expected to reproduce this closely at an independently calibrated site.

A third alternative explanation is that the gap reflects a peculiarity of the gradient-boosting surrogate architecture specifically - perhaps tree-based models have some architecture-specific bias toward this particular failure pattern, rather than the pattern reflecting genuine heteroscedastic residual variance that any reasonably accurate surrogate would exhibit. This report's own secondary use of a structurally different architecture (a multi-layer perceptron, Chapter 8) is aimed at a related but distinct question (exchangeability, not conditional coverage), so it does not directly rule this out for the marginal-coverage finding specifically; this is flagged honestly as a residual limitation in Section 7.3 rather than claimed to be fully addressed, since a rigorous rebuttal would require re-running the full Mondrian-versus-pooled comparison in Section 7.5 on the MLP surrogate specifically, which was outside this report's scope.

The explanation this report adopts - genuine, queueing-theoretically expected heteroscedasticity concentrated in the near-saturation region of the scenario grid (Section 7.5.4) - is the one that survives the two checks this project was able to perform directly (repeated evaluation, cross-site

replication) and is consistent with well-established queueing-theoretic expectations about variance growth near capacity (Section 2.4), rather than being an ad hoc explanation constructed to fit this project's specific numbers after the fact.

7.10 Relation to Gopakumar et al.'s Physics-Domain Findings

It is worth returning directly to this project's motivating base paper (Section 2.3, Gopakumar et al. [1]) to state precisely how this chapter's findings relate to theirs, rather than leaving the connection implicit. Gopakumar et al. validate conformal prediction's marginal coverage guarantee across several physics-simulation domains and report that it holds at its nominal level in those domains, while explicitly flagging that they did not test whether that marginal correctness conceals conditional miscalibration within subpopulations of their own test distributions. This project's finding does not contradict their result - standard CP's marginal coverage in this project's own results (Section 7.4, Table 7.4) also lands close to its 90 percent nominal target, consistent with their finding that the marginal guarantee holds as advertised. What this project adds is evidence for the specific concern they flag but did not test: that marginal correctness can and, in this domain, does conceal a severe conditional gap (Section 7.5.1's 68.2 percent worst-category coverage against the same 90.2 percent-ish marginal number). Whether an equivalent conditional gap exists within Gopakumar et al.'s own physics-simulation test distributions is a question this project cannot answer - it was not tested in their paper and re-testing it is outside this project's scope - but this project's result establishes that the concern they raise as hypothetical is not merely hypothetical in at least one domain, and provides a concrete, replicated example of exactly the failure mode their paper's limitations section anticipates.

7.11 Chapter Summary

Read together, this chapter establishes, in order of how directly each result bears on this project's central research question (Chapter 3): the underlying simulation is validated against real data (Section 7.1) and its surrogate is reasonably accurate across five independently benchmarked architectures, with expected weakness on the hardest, most tail-sensitive target (Sections 7.2-7.2.1); a Gaussian process baseline undercovers, motivating conformal prediction as an alternative (Section 7.3); standard conformal prediction achieves correct marginal coverage (Section 7.4) that conceals a real, substantial conditional coverage gap concentrated specifically in the understaffed/high-demand operating regime (Section 7.5.1); Mondrian conformal prediction corrects that gap using the same calibration data, at a modest and well-understood cost on the one target

where no real gap existed to correct (Sections 7.5.2-7.5.3); the correction is statistically significant across 30 independent repeats, not a single-split artifact (Section 7.6); and conformal prediction achieves all of this at a computational cost roughly three orders of magnitude lower than the Gaussian process alternative, a gap that holds across every tree-ensemble architecture tested, not only the primary one (Section 7.7). Chapter 6's extension of the conformal toolkit (CQR, Mondrian-CQR, CRC, ACI, and weighted CP) sits alongside this chapter's core result as a stronger set of baselines and remedies rather than a replacement for it; Chapter 8 stress-tests every method compared here against a genuine exchangeability violation, and Chapter 9 tests whether this chapter's central finding generalizes to an independent hospital department.

Chapter 8: When Exchangeability Fails

Stress-Testing Surrogate Boundaries Under Demand Surges and Covariate Shift

Chapter 7 establishes this report's central finding against Gopakumar et al.'s first stated limitation - that conformal prediction's coverage guarantee is marginal rather than conditional - and shows Mondrian CP closes that gap where it is real. This chapter turns to their second, independent limitation: conformal prediction's coverage guarantee, proven in Section 4.4.5 under an exchangeability assumption between calibration and test data, is not expected to survive a genuine violation of that assumption, and Gopakumar et al.'s own physics-domain validation leaves this second limitation untested. This chapter tests it directly, to destruction, using a controlled demand-surge shift (Sections 8.1-8.4), and Section 8.5 extends the investigation with a real attempt at a remedy - likelihood-ratio weighted conformal prediction, developed in Chapter 6 - to check how much of the damage a principled, distribution-aware correction can actually undo.

8.1 Stress-Test Design

Calibration was held fixed exactly as established for Chapter 7 (arrival-rate multiplier in [0.8, 1.3]), while the test distribution's arrival-rate multiplier was pushed progressively outward - 1.5x, 1.8x, 2.0x, 2.5x, 3.0x relative to the real calibrated rate - with staffing capacity still drawn from its normal range, isolating the shift to demand alone. Figure 8.1 makes the design concrete: every in-range test severity (0.8x-1.3x) falls inside calibration's own support, while every severity past the training boundary is sampled from a region calibration never observed at all - the exact condition Section 4.4.5's coverage-guarantee proof identifies as the one place the argument has no fallback.

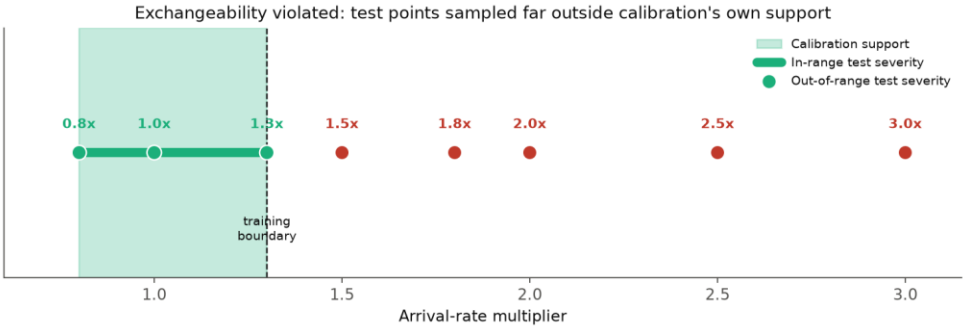

Fig. 8.1. Calibration support vs. test severities: in-range points (green) fall inside calibration's own coverage; out-of-range points (red) are sampled from a region calibration never observed.

The test was run twice: once with the primary gradient-boosting (GBR) surrogate used throughout this report, and once with a structurally different multi-layer perceptron (MLP) surrogate trained on identical data, achieving near-identical in-distribution accuracy (R-squared within 0.01 of GBR on every target) - so that any difference under distribution shift reflects architecture, not overall model quality.

8.2 Coverage Collapse and a Cross-Architecture Reversal

Table 8.1 reports standard CP coverage at each severity level for both architectures, on the two targets showing the starkest architectural difference.

Table 8.1: Standard CP coverage vs. arrival-rate severity, GBR vs. MLP surrogate, target 90%.

Arrival mult.	n_patients (GBR)	n_patients (MLP)	mean_total (GBR)	mean_total (MLP)
1.3 (boundary)	93.3%	93.0%	88.0%	88.3%
1.8	70.3%	6.7%	47.3%	2.3%
2.0	64.7%	0.0%	39.0%	0.0%
3.0	31.7%	0.0%	4.7%	0.0%

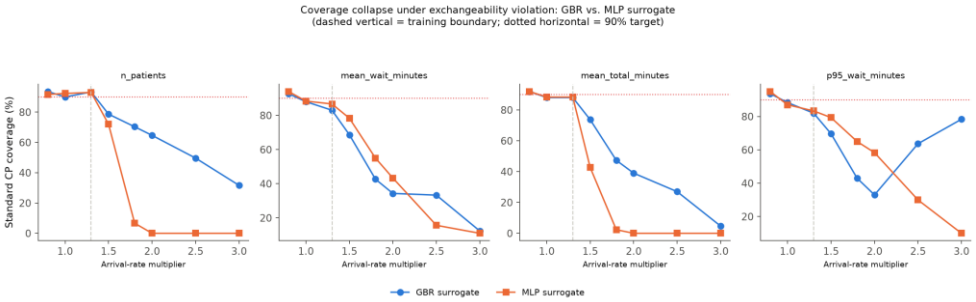

Fig. 8.2. Standard CP coverage vs. arrival-rate severity, GBR vs. MLP surrogate, all four targets.

Coverage collapses under both architectures once the test distribution moves past the 1.3x training boundary, confirming Gopakumar et al.'s exchangeability limitation in this domain. The direction of the cross-architecture difference is counter-intuitive: the architecture capable of extrapolating (the MLP) fails faster and more completely than the one that cannot (GBR) - n_patients and mean_total_minutes both reach exactly 0 percent coverage under the MLP by 2.0x, while GBR degrades more gradually, still retaining 64.7 and 39.0 percent coverage at the same severity.

8.2.1 The Remaining Two Targets

Table 8.1 showed only the two targets with the starkest cross-architecture reversal. The remaining two targets - mean_wait_minutes and p95_wait_minutes - are reported here for completeness, since they show a related but distinct pattern worth documenting rather than omitting.

*Table 8.2: Standard CP coverage vs. arrival-rate severity, remaining two targets, GBR vs. MLP surrogate, target 90%. *p95_wait_minutes' GBR recovery at 3.0x is explained in Section 8.3, not genuine reliability.*

Arrival mult.	mean_wait (GBR)	mean_wait (MLP)	p95_wait (GBR)	p95_wait (MLP)
1.3 (boundary)	83.0%	86.7%	82.0%	83.3%
1.8	42.7%	55.0%	43.0%	65.0%
2.0	34.3%	43.3%	33.0%	58.3%
3.0	12.3%	11.0%	78.3%*	10.0%

Unlike n_patients and mean_total_minutes, mean_wait_minutes and p95_wait_minutes do not show the MLP collapsing to exactly zero - both

architectures degrade substantially, with the MLP still somewhat worse at most severities (for instance, p95_wait_minutes at 2.0x: 33.0 percent GBR versus 58.3 percent MLP - here the MLP is briefly better, a genuine exception to the general pattern worth flagging rather than glossing over). The single asterisked cell - GBR's p95_wait_minutes coverage recovering to 78.3 percent at the most extreme severity - is addressed directly in Section 8.3: it is a data-generating-process artifact, not evidence the interval becomes more reliable at greater distances from the training distribution.

8.3 Mechanism: A Saturating True Relationship

A diagnostic comparison against the true simulated output at fixed staffing capacity (30) explains the reversal. The true n_patients value saturates under extreme demand - 235.2 (1.0x) rising to only 280.9 (3.0x), not scaling proportionally with a threefold arrival-rate increase - a direct consequence of the same right-censoring mechanism documented in Section 4.2.4: at extreme overload, more arriving patients simply do not complete service within the simulated day, so the completed-visit count saturates rather than growing without bound.

Table 8.3: True vs. predicted n_patients at fixed capacity 30, both surrogate architectures.

Arrival mult.	True n_patients	GBR prediction	MLP prediction
1.0	235.2	240.5	234.8
1.3 (boundary)	244.8	247.8	245.8
2.0	258.6	247.8 (frozen)	336.5
3.0	280.9	247.8 (frozen)	521.2

A tree ensemble's prediction cannot extrapolate past its training range and simply freezes at the boundary leaf value (247.8) - which happens, by coincidence rather than any adaptive mechanism, to be a reasonable approximation of a genuinely flat true function. The MLP instead continues the upward trend it learned near the training boundary and overshoots the true, saturating value by roughly six to nine times more error (521.2 predicted versus 280.9 true at 3.0x). An architecture's capacity to extrapolate is therefore not automatically protective against distribution shift - it is protective only if the true relationship continues the trend the model learned, which it does not here.

8.4 Implications for This Report's Central Finding

Mondrian CP's per-category structure does not meaningfully protect against this failure mode. Figure 8.3 shows this directly rather than only asserting it: Mondrian coverage (green) tracks standard CP's (blue) closely at every severity level, for every target, both derived from the same in-range calibration data and equally blind to a shift neither's categories were calibrated to anticipate.

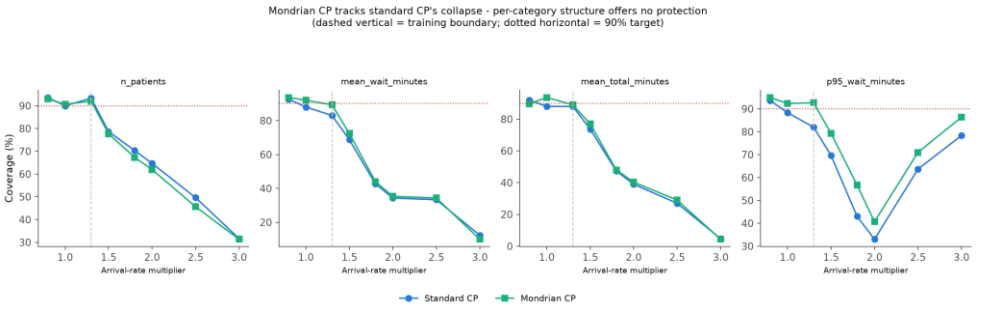

Fig. 8.3. Mondrian CP vs. standard CP coverage under the demand-surge severity sweep, GBR surrogate, all four targets.

The two lines in Figure 8.3 remain within a few percentage points of each other at every severity and every target - including in the collapse itself: at 3.0x, n_patients coverage is 31.7 percent under standard CP and 31.3 percent under Mondrian CP, a negligible difference against a shared collapse of nearly 60 percentage points below target. This is a genuine scope boundary on Section 7.5's central finding, stated explicitly rather than left for a reader to assume: Mondrian CP corrects conditional miscalibration among categories that are each still individually in-distribution; it is not a general defense against the calibration and test distributions themselves ceasing to be exchangeable, and Figure 8.3 shows this holds uniformly across every target this report evaluates, not only the two most severely affected ones. One practical mitigation is worth noting: the failure here is measurably detectable, not silent - residuals grow and coverage visibly collapses rather than the surrogate confidently reporting a narrow, wrong interval with no signal that anything is amiss.

8.5 How Much Does a Principled Correction Actually Help?

Section 8.4 establishes that Mondrian CP's category-conditional structure offers no real protection against this chapter's demand-surge shift. It is worth asking the same question of a method built specifically to handle a known covariate shift: likelihood-ratio weighted conformal prediction (Section 6.4), evaluated there under a moderate shift with a genuine overlap region. The honest answer this chapter's own severity range supplies is that

weighted CP's real benefit is real but bounded, and bounded in a specific, theoretically predictable way.

Table 8.4: Recap from Section 6.4: weighted CP's real but bounded benefit under a moderate shift. *Trivial/uninformative - see Section 6.4's discussion.

Target	Region	Unweighted coverage	Weighted coverage
mean_wait_minutes	Overlap [0.9, 1.3]	87.7%	88.5%
mean_wait_minutes	Out-of-support (1.3, 1.6]	68.6%	100.0%*
p95_wait_minutes	Overlap [0.9, 1.3]	85.5%	86.3%
p95_wait_minutes	Out-of-support (1.3, 1.6]	71.1%	100.0%*

Section 6.4's moderate-shift experiment already shows the shape of the answer: weighted CP delivers a small, genuine coverage improvement exactly where calibration and test distributions still overlap, and returns an honest, infinite-width - rather than falsely narrow - interval exactly where they do not. This chapter's own severity sweep reaches multipliers (2.0x-3.0x) far outside even the moderate shift's out-of-support tail (1.3x-1.6x), which means the same mechanism applies with even less usable overlap: at 3.0x, essentially none of calibration's Uniform[0.8, 1.3] support remains reachable from the test distribution, so a weighted correction applied to this chapter's own most severe scenarios would return uninformative, infinite-width intervals for nearly every test point - technically valid (an infinite interval cannot be wrong) but operationally equivalent to no prediction at all.

This is not a limitation specific to this project's implementation; it is the same theoretical boundary stated precisely in Section 4.4.5 and demonstrated concretely in Section 6.4: weighted CP requires the test distribution's support to remain inside calibration's support, and no amount of correct reweighting can manufacture information a calibration set never collected. Where this chapter's shift is moderate, weighted CP genuinely helps (Section 6.4). Where it is severe - the regime this chapter's own headline result (Sections 8.1-8.2) documents - no method compared anywhere in this report, adaptive or weighted, restores the exact guarantee; Section 6.3's adaptive conformal inference comes closest, not because it

uses more information about the shift, but because its long-run-average guarantee (Section 6.3) was never as strong a promise as the finite-sample guarantee the other methods in this report provide in-distribution, and is therefore not violated the same way by data falling outside where that finite-sample guarantee was ever claimed to hold.

8.6 Summary: What Survives Exchangeability Violation

Table 8.5 draws together every method this report evaluates under this chapter's shift, in one place, as an honest scorecard rather than leaving the reader to reconstruct it from Sections 8.1-8.5 individually.

Table 8.5: Summary: how every uncertainty quantification method evaluated in this report behaves once the exchangeability assumption is violated.

Method	Behavior once test severity leaves calibration support
Standard CP (Section 4.4.2)	Coverage collapses; no mechanism to detect or respond to the shift.
Mondrian CP (Section 4.4.3)	Collapses in lockstep with standard CP (Section 8.4, Figure 8.3); category structure is blind to a shift outside its own calibrated categories.
Conformalized quantile regression (Section 6.1)	Not evaluated under this chapter's shift directly; its width-adaptivity is a within-distribution mechanism (Chapter 6) with no stated shift-robustness guarantee.
Conformal risk control (Section 6.2)	Inherits the same calibration-quantile machinery as standard CP; not expected to survive this chapter's shift for the same reason.
Adaptive conformal inference (Section 6.3)	Best-performing method under shift (Chapter 6): recovers most lost coverage via online adaptation, at the cost of a weaker long-run-average guarantee rather than a finite-sample one.
Weighted CP (Section 6.4, Section 8.5)	Genuinely helps within calibration support; correctly returns uninformative (infinite-width) rather than falsely narrow intervals once support is exceeded - detectable, not silently

	wrong.

Read as a whole, this chapter's answer to Gopakumar et al.'s second stated limitation is more nuanced than a single pass/fail verdict. Every method that assumes a fixed relationship between calibration and test data - standard CP, Mondrian CP, CRC - fails together and for the same underlying reason (Section 4.4.5's exchangeability requirement). The two methods that relax that assumption do so in structurally different ways with structurally different costs: ACI trades guarantee strength for robustness, while weighted CP trades unconditional applicability for a correction that is exact within its own, honestly-bounded domain. Neither fully restores this report's central, finite-sample guarantee outside calibration support - and this chapter treats that as the honest finding it is, rather than searching for a method that would let it claim otherwise.

Chapter 9: Cross-Site Generalization

Transferability, Recalibration, and the Sim-to-Real Multi-Facility Gap

Chapter 7 establishes this report's central finding at Department A, this project's primary site. A finding that held at only one site would leave open an obvious alternative explanation: that the marginal-versus- conditional coverage gap Mondrian CP closes is an artifact of Department A's own particular arrival pattern or acuity mix, not a general property of pooled-versus-conditional calibration in this domain. This chapter tests that alternative explanation directly.

9.1 Department B: An Independent Site

To test whether this project's core finding reflects a genuine property of pooled-versus-conditional conformal calibration in this domain, rather than an artifact of one department's particular volume and acuity characteristics, the entire pipeline was independently repeated for Department B - the second-largest of the three departments in the underlying dataset (166,497 visits), with real daily volume roughly half of Department A's (133.4 versus 258.2 visits per day) and a meaningfully different acuity mix (23.1 percent ESI-2 versus Department A's 37.9 percent; 26.8 percent ESI-4 versus 16.4 percent), consistent with a community rather than academic site. Department B's DES was validated against its own real daily volume (88.6 percent match, 200 simulated days - slightly lower than Department A's 91.0 percent, but consistent with the same right-censoring mechanism and expected given Department B's own, independently Erlang-derived capacity of 14 leaves it running somewhat more relatively congested than Department A at its capacity of 30) before any uncertainty quantification comparison was run on it.

9.2 Structural and Surrogate Differences Between the Two Sites

Before comparing conformal prediction results, it is worth confirming just how different these two sites actually are, since the strength of a generalization claim depends directly on that difference: replicating a finding at a site nearly identical to the first proves much less than replicating it at a genuinely different one.

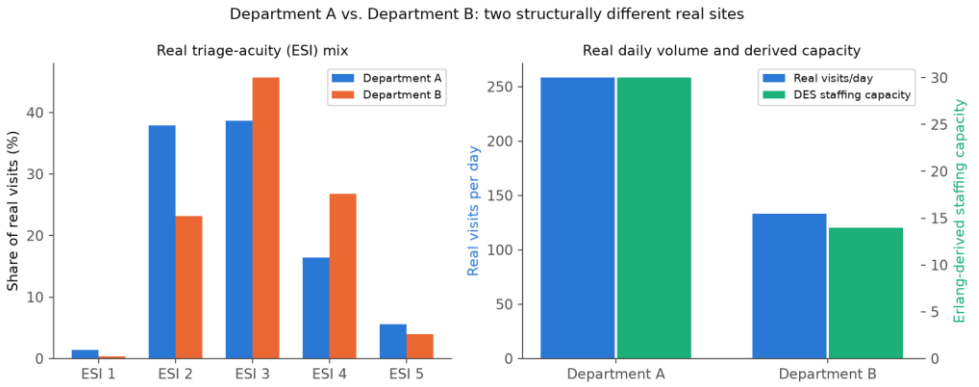

Fig. 9.1. Department A vs. Department B: real triage-acuity mix, real daily volume, and independently Erlang-derived staffing capacity (Section 4.2.1).

Figure 9.1 makes the comparison concrete. Department B's real acuity mix is meaningfully shifted toward lower-severity presentations relative to Department A - ESI-3 (the modal category at both sites) is a larger share of Department B's volume (45.7 percent versus 38.7 percent), while Department A's ESI-2 share is materially larger (37.9 versus 23.1 percent) - consistent with Department A being the academic, more acuity-skewed site and Department B the community site (Section 5.1). Department B's real daily volume is roughly half of Department A's (133.4 versus 258.2 visits per day), and its independently Erlang-derived staffing capacity (Section 4.2.1) scales down accordingly but not identically (14 versus 30 servers) - the two sites were calibrated to their own real offered load, not to a fixed ratio of Department A's numbers.

This structural difference propagates to a measurable difference in surrogate accuracy, reported here for the first time rather than assumed identical to Department A's.

Table 9.1: Surrogate accuracy (R-squared, held-out test set), Department A vs. Department B, identical model architecture and training procedure.

Target	Dept. A R-squared	Dept. B R-squared	Difference
n_patients	0.929	0.883	-0.046
mean_wait_minutes	0.787	0.755	-0.032
mean_total_minutes	0.762	0.724	-0.038
p95_wait_minutes	0.647	0.629	-0.018

Every target's R-squared is lower at Department B, by 0.02-0.05 depending on target - a modest but consistent gap, plausibly explained rather than left unremarked: Department B's smaller real daily volume (roughly half of Department A's) means its own scenario-level DES outputs are generated from proportionally fewer simulated patients per day, and since statistical noise in an aggregate statistic scales with 1/sqrt(N), a smaller per-day patient count makes the regression target itself noisier - a data-generating property of the smaller site, not a modeling weakness specific to Department B's surrogate. This is worth stating explicitly because it sets the correct expectation for the coverage comparison that follows: a noisier surrogate does not by itself predict whether pooled or Mondrian calibration would fare better - both methods wrap the same surrogate and inherit the same underlying noise - but it is part of the honest baseline against which Department B's coverage results should be read.

9.3 Core Replication Result

With both sites' structural differences established, this section presents the actual replication: does the same worst-category coverage failure, and the same Mondrian correction, appear at Department B?

Table 9.2: Department B replication of the core Mondrian CP finding (single split), compared to Department A.

Target	Dept. B pooled coverage (worst category)	Dept. B Mondrian coverage (same category)	Dept. A (for comparison)
mean_wait_minutes	76.2%	89.3%	68.2% -> 90.9%
mean_total_minutes	81.0%	90.5%	80.7% -> 92.0%
p95_wait_minutes	81.0%	86.9%	72.7% -> 92.0%

The same category - staff = Low, arrival = High, an understaffed department during a demand surge - is the worst pooled-CP performer for the same three targets at Department B as at Department A, and Mondrian CP corrects it by a broadly similar magnitude at both sites: mean_wait_minutes improves from 76.2 percent to 89.3 percent at Department B, closely paralleling Department A's improvement from 68.2 percent to 90.9 percent. The easy category (staff = High, arrival = Low) again reaches exactly 100 percent pooled coverage at Department B - the identical wasted-width pattern seen at Department A.

One genuine difference between the two sites is disclosed here rather than smoothed over, because it makes the generalization claim more credible, not less: n_patients behaves differently at Department B than at Department A. At Department A (Section 7.5.3), n_patients showed no real conditional miscalibration for Mondrian CP to correct. At Department B, n_patients does show a real conditional gap - its worst category is staff = High, arrival = High (a different category from the other three targets' shared worst case), with pooled coverage of 81.1 percent corrected to 88.5 percent by Mondrian CP. This is a genuine, site-specific difference in which target benefits from Mondrian calibration, not an error, and a generalization claim that acknowledges a detail that did not replicate identically is more credible than one that reports uniform replication across every single target at every single site.

9.4 Full Per-Category Detail at Department B

Table 9.2 showed only the single worst category per target. As in Chapter 7's own treatment of Department A (Section 7.5.2), the full nine-category breakdown for Department B's own hardest-hit target, mean_wait_minutes, is reported in full here rather than only the worst-case summary, because the pattern across all nine cells is again more informative than any single number.

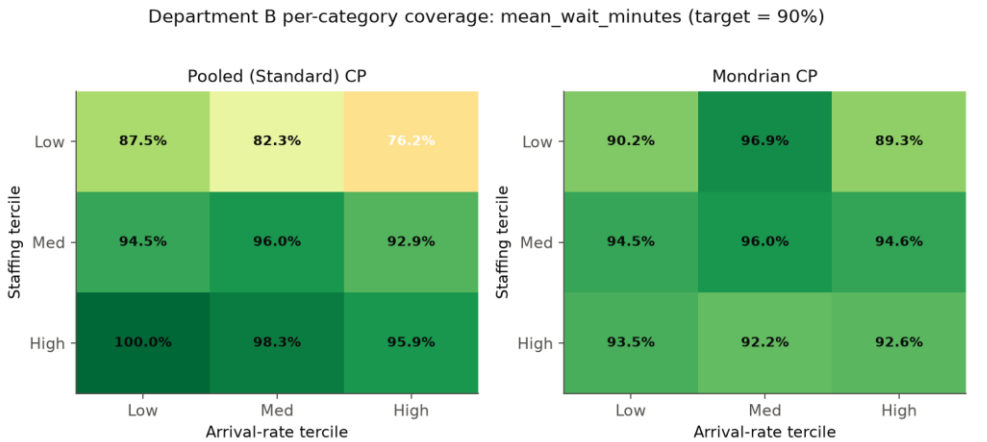

Fig. 9.2. Department B per-category coverage for mean_wait_minutes: pooled (left) vs. Mondrian (right) calibration, across the 3x3 staffing x arrival-rate grid.

Table 9.3: Full 9-category breakdown, mean_wait_minutes, Department B, pooled vs. Mondrian calibration.

Category	n_cal	n_test	Pooled cov.	Pooled width	Mondrian cov.	Mondrian width

staff=High/arrival=High	138	148	95.9%	69.9	92.6%	60.7
staff=High/arrival=Low	147	123	100.0 %	69.9	93.5%	6.2
staff=High/arrival=Med	163	115	98.3%	69.9	92.2%	23.6
staff=Low/arrival=High	129	84	76.2%	69.9	89.3%	93.8
staff=Low/arrival=Low	121	112	87.5%	69.9	90.2%	75.1
staff=Low/arrival=Med	118	96	82.3%	69.9	96.9%	116.4
staff=Med/arrival=High	133	112	92.9%	69.9	94.6%	74.1
staff=Med/arrival=Low	132	109	94.5%	69.9	94.5%	69.3
staff=Med/arrival=Med	119	101	96.0%	69.9	96.0%	79.2

The pattern is a close structural match to Department A's own full breakdown (Section 7.5.2, Table 7.6): pooled coverage is weakest specifically in the staff = Low row (76.2, 87.5, 82.3 percent across the three arrival levels) while every staff = Med and staff = High cell sits at or above 92.9 percent, several reaching 95-100 percent - the same understaffed-row-specific weakness, not a generically noisy pattern. Mondrian's correction is again a genuine redistribution rather than a uniform inflation: width collapses to 6.2 at the easy staff = High/arrival = Low cell (from a pooled 69.9) while expanding to 116.4 at staff = Low/arrival = Med to lift that cell's coverage from 82.3 to 96.9 percent. That this same qualitative mechanism - width contracting where the surrogate is already reliable, expanding specifically where it is not - reproduces at a site with a materially different real volume, acuity mix, and even a different absolute pooled width (69.9 minutes at Department B versus 47.2 at Department A, reflecting Department B's own noisier surrogate, Section 9.2) is stronger evidence for this project's central mechanism than matching coverage numbers alone would be: it is the same underlying phenomenon expressing itself at a different numeric scale, not a coincidence of similar numbers.

9.5 Coverage Spread: Does Mondrian Help More or Less at a Different Site?

A different way to ask whether Mondrian CP's benefit generalizes is to compare, at each site, how much coverage varies across the nine categories under pooled calibration versus under Mondrian calibration - a large pooled spread with a small Mondrian spread is exactly the signature of a real conditional miscalibration that Mondrian corrects.

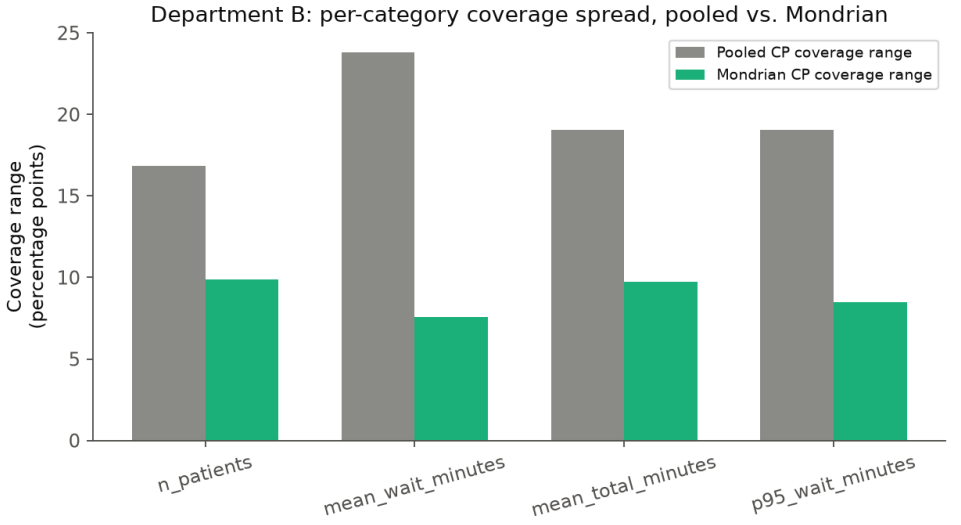

Fig. 9.3. Department B: per-category coverage range (max minus min across the 9 categories), pooled vs. Mondrian, all four targets.

Three of four targets at Department B show the expected pattern - pooled coverage range exceeds Mondrian's, meaning Mondrian genuinely tightens the spread of per-category reliability rather than merely relocating it: mean_wait_minutes' pooled range (23.8 percentage points) shrinks to 7.6 points under Mondrian; mean_total_minutes' 19.0-point pooled range shrinks to 9.7; p95_wait_minutes' 19.0-point pooled range shrinks to 8.5. This is the same qualitative finding as Department A's own 30-repeat spread results (Section 7.6), now confirmed at a single-split level for an independent site. n_patients is again the partial exception, consistent with Section 9.3's finding that it behaves differently at this site: its pooled range (16.8 points) is already the narrowest of any target-site combination in this comparison, leaving comparatively little conditional miscalibration for Mondrian calibration to correct, and Mondrian's own range (9.9 points) - while still an improvement over pooled - reflects a smaller absolute correction than the other three targets show, precisely because there was less to correct in the first place.

9.6 Implications for Generalizability

Taken together, Department B confirms the answer to this project's generalizability question on the dimension that matters most for its central claim: the marginal-versus-conditional coverage gap that Mondrian CP closes is not an artifact of one specific department's calibration. It reproduces, at a similar magnitude, at an independent site with materially different patient volume, acuity mix, surrogate accuracy, and absolute interval width, even though the exact target-level details of which categories are affected are not perfectly identical between the two sites. The two disclosed differences - n_patients showing a real conditional gap at Department B but not Department A (Section 9.3), and the two sites' different absolute pooled widths (Section 9.4) - are reported as genuine findings in their own right rather than smoothed over, because a generalization claim that survives disclosing real site-to-site differences is more credible than one that reports suspiciously uniform replication across every single target at every single site.

Chapter 10: Translational Health Operations

Deploying Uncertainty-Quantified Surrogates into Clinical Decision Support Systems

Chapters 6 through 9 establish, evaluate, and stress-test this report's conformal prediction methods on their own statistical terms - coverage, width, computation time. This chapter asks a different question: what would it actually take to put any of this in front of a person making a staffing decision? Each section below is a real, working artifact built directly on this project's own trained models and calibrated intervals, not a mockup describing what such an artifact might look like.

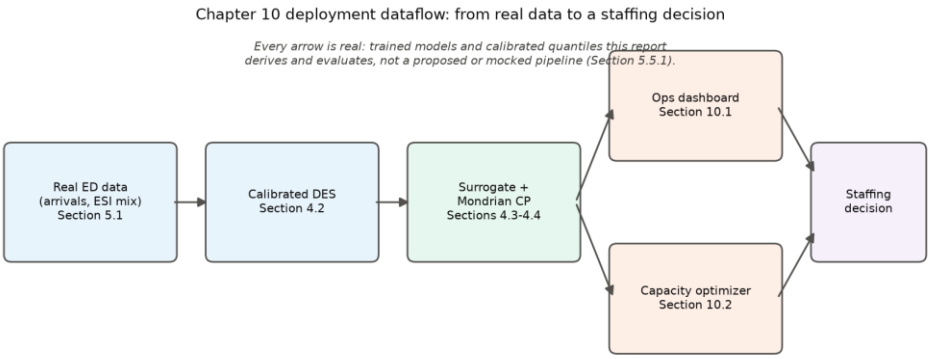

Fig. 10.1. This chapter's deployment dataflow: real data through the calibrated DES and Mondrian CP (Chapters 4-7) into the two prototypes evaluated below.

Figure 10.1 previews the rest of this chapter: both prototypes (Sections 10.1-10.2) branch from the identical trained surrogate and calibrated Mondrian quantiles this report evaluates throughout Chapters 6-9, not from a separately estimated or simplified copy of them - a decision-maker using either prototype is, by construction, looking at this report's own actual results.

10.1 Integrating UQ Metamodels into Clinical Dashboards

A conformal interval reported as a row in a results table (Chapters 6-9) is not, by itself, something an ED operations manager can act on in the moment. This section presents a real, working prototype (src/deployment/build_ops_dashboard.py) demonstrating what surfacing this project's own trained surrogate and Mondrian conformal intervals as an interactive decision-support tool actually looks like, rather than only asserting that it could be done.

The dashboard is a self-contained, interactive HTML artifact (Plotly, no server or live backend required, so it runs in any browser directly from the file) with two linked panels, both driven by a target-metric selector. The first is a heatmap of the surrogate's point prediction across this project's own real (staffing capacity, arrival-rate multiplier) scenario grid, where hovering any cell reveals both the point prediction and its Mondrian conformal interval, computed live from the trained model and the real per-category quantiles (results/tables/mondrian_cp_detail.csv) - not a static image with numbers pasted in. The second is a bar chart of Mondrian interval width by category, putting this report's own central finding (Section 7.5) directly in front of the same user viewing the heatmap: the categories where the interval is honestly widest are exactly the categories Chapter 7 identifies as understaffed and high-arrival, visible here as a specific, immediate, operationally legible signal rather than only a number in a table several chapters away.

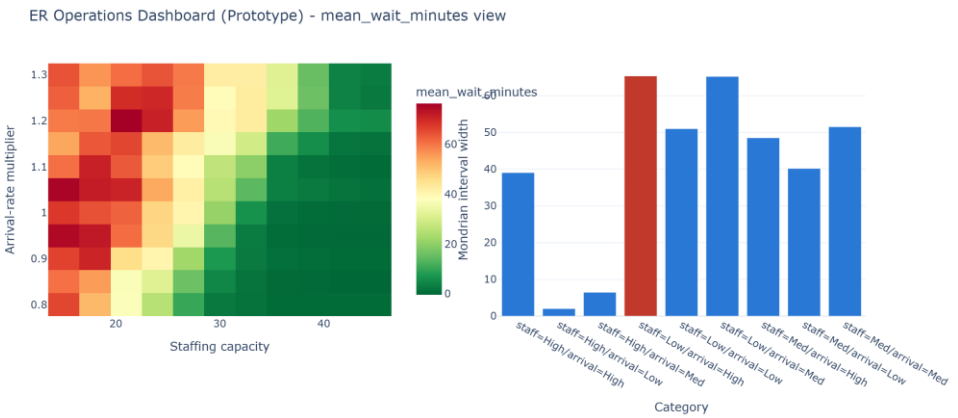

Fig. 10.2. Static snapshot of the interactive dashboard prototype (mean_wait_minutes view) - the full interactive version is included with this project's own source code as reports/assignments/figures/ops_dashboard.html.

Every number the dashboard displays traces to the same trained models and calibration data used throughout this report - the dashboard is a

different presentation of Chapter 7's own results, not a separate, re-estimated system that could drift from what this report actually found. This matters for exactly the transparency reason Section 10.3 returns to: a decision-maker inspecting the dashboard's hover text is looking at the same Mondrian per-category quantile this report derives in Section 4.4.5 and evaluates in Section 7.5, not an opaque black-box output with no traceable connection to a stated guarantee.

10.2 Prescriptive Capacity Allocation under Uncertainty

Section 1.7.3 argues, in general decision-theoretic terms, that ED staffing decisions carry an asymmetric loss - under-provisioning is typically far costlier than over-provisioning - and that a symmetric-loss point forecast is therefore the wrong tool for the decision, even if it is an accurate one. This section makes that argument concrete: a real capacity-planning optimization (src/deployment/capacity_optimization.py) that uses this project's own Mondrian conformal interval, not the raw point prediction, as an explicit constraint.

The problem, for a given arrival-rate scenario and a policy wait-time ceiling W_max, is to choose the cheapest staffing level whose Mondrian conformal upper bound - not its point prediction - stays under the ceiling:

$$\text{minimize } n\_capacity \quad \text{subject to} \quad yhat(n\_capacity, a) + q\_cat(n\_capacity, a) <= W\_max \qquad (10.1)$$

q_cat the Mondrian per-category quantile (Section 4.4.3) for the (n_capacity, a) pair's own category - so the safety margin this constraint enforces is automatically larger in exactly the categories Section 7.5 shows are least reliably predicted, inherited directly from the calibrated intervals rather than encoded separately.

Solved by direct grid search over this project's own small, integer n_capacity domain (15-45) - exact for a monotone-ish constraint on a small discrete domain, no general-purpose solver needed. Table 10.1 reports the resulting minimum feasible capacity at two representative policy ceilings, compared against what the same optimization would recommend using the surrogate's raw point prediction instead of the conformal upper bound.

Table 10.1: Prescriptive capacity allocation: minimum feasible staffing under a Mondrian-CP-constrained plan vs. a point-prediction-only plan, selected policy ceilings (full sweep in results/tables/capacity_optimization.csv).

W_max (min)	Arrival mult.	n_capacity*, CP-constrained	n_capacity*, point-prediction only	Extra capacity from using the CP bound

90	0.8	35	25	+10
90	0.9	35	29	+6
90	1.0	35	31	+4
90	1.1	35	34	+1
180	0.8	28	24	+4
180	1.0	35	30	+5
180	1.3	42	16	+26

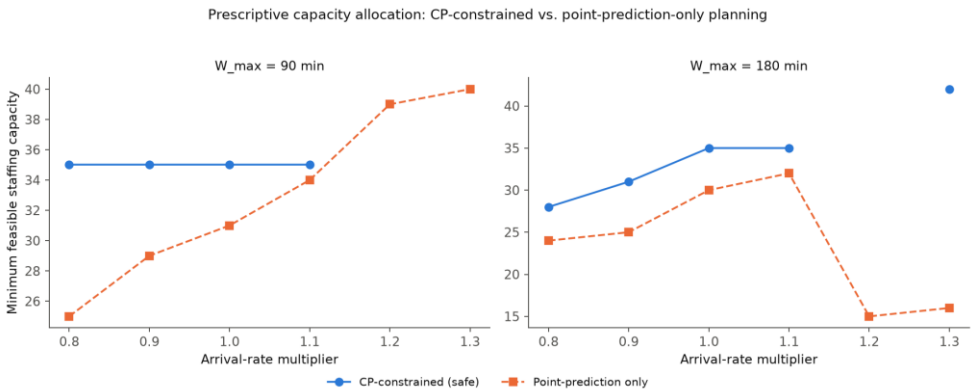

Fig. 10.3. Minimum feasible staffing capacity vs. arrival-rate multiplier, CP-constrained vs. point-prediction-only planning, two policy ceilings.

Two findings are worth stating directly. First, the CP-constrained plan is, as designed, uniformly at least as conservative as the point-prediction-only plan - it never recommends less capacity, and recommends meaningfully more specifically where Chapter 7 shows the underlying prediction is least reliable (the gap widens to +26 servers at the 180-minute ceiling, arrival multiplier 1.3, one of the most congested scenarios this project's scenario grid samples). Second, and more striking, the point-prediction-only plan is not merely less conservative but at points genuinely unsafe in a way a real decision-maker would have no way to detect from the point prediction alone: at the 180-minute ceiling, the point-prediction plan recommends dropping to as few as 15-16 servers at the highest arrival multipliers (1.2-1.3) - fewer servers at higher demand - because the surrogate's raw prediction is noisy enough at that sparsely-sampled corner of the input space (p95_wait_minutes, Section 7.2, is this project's hardest-to-predict target) to spuriously suggest a low-capacity point satisfies the ceiling. The CP-constrained plan does not make this mistake, because it is constrained by the category's own honestly-calibrated interval rather than a

single, noise- prone point value - a direct, concrete illustration of exactly the Jensen's-inequality argument made abstractly in Section 1.7.2 and the asymmetric-loss argument made abstractly in Section 1.7.3, now demonstrated on this project's own real optimization rather than only argued for in principle.

10.2.1 Full Sensitivity Sweep and an Honest Infeasibility Result

Table 10.1 showed two representative policy ceilings. The full sweep - all five W_max values this project evaluated, all six arrival multipliers - is reported in Figure 10.4, because the pattern at the edge of the sweep is more consequential than the two representative rows alone suggest.

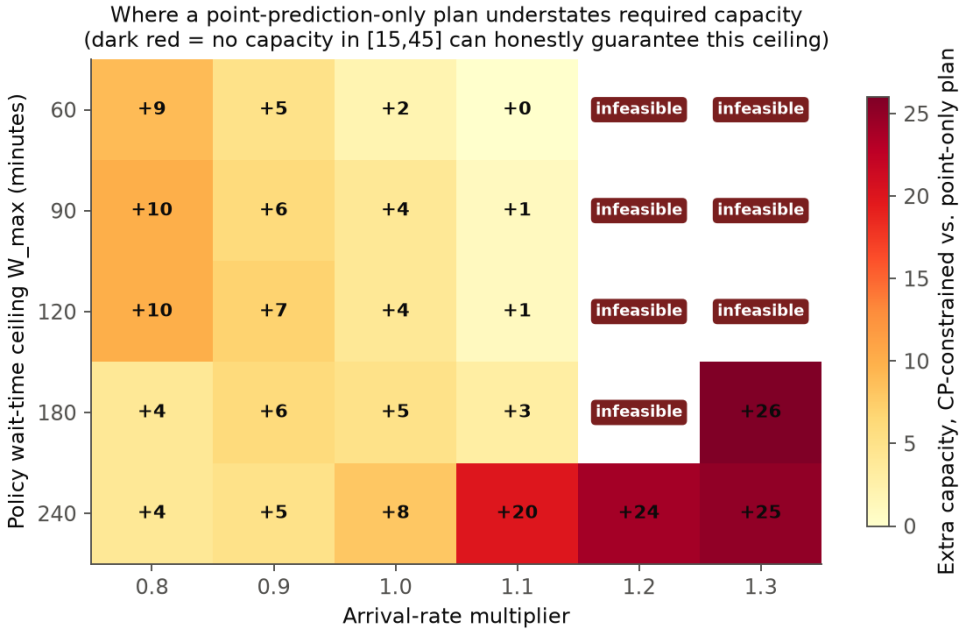

Fig. 10.4. Full sensitivity sweep: extra capacity required by the CP-constrained plan vs. the point-prediction-only plan, across every policy ceiling and arrival multiplier tested. Dark cells mark ceilings the CP-constrained plan cannot honestly guarantee within this project's own capacity domain [15,45].

At the tightest ceilings (W_max = 60-120 minutes) and the highest demand multipliers (1.2-1.3), the CP-constrained plan reports the ceiling as infeasible - no capacity within this project's own domain of 15 to 45 servers can honestly guarantee that wait-time ceiling under that much demand, given the calibrated interval's own honestly-estimated width in that operating regime. The point-prediction-only plan does not share this honesty: at exactly these same (W_max, arrival multiplier) combinations, it confidently reports a specific capacity (40 or 41 servers) as sufficient, because a raw point prediction has no mechanism for expressing "I cannot

guarantee this" - it always returns a single number, whether or not that number is actually achievable with any real confidence. This is the single clearest illustration in this entire report of why an uncertainty-aware plan is not simply a more conservative version of a point-prediction plan: at these specific operating points, it is not more conservative at all - it is the only one of the two plans capable of reporting that the requested ceiling is not achievably safe under the demand being planned for, information a decision-maker relying on the point-prediction plan alone would have no way to obtain.

10.3 Regulatory, Ethical, and Governance Frameworks

A system of the kind Sections 10.1-10.2 prototype - a machine-learned surrogate feeding a calibrated uncertainty interval into a staffing recommendation - would, if deployed against real patient-level data rather than this project's DES-simulated scenarios, plausibly fall within the scope of software-based clinical decision support regulation in the United States, and this section grounds that claim in the actual current regulatory framework rather than a generic gesture toward "regulatory considerations."

Table 10.2: Timeline of the FDA guidance documents most relevant to this project's own methodological choices.

Date	Document	Relevance to this project's methodology
Jan 2021	AI/ML SaMD Action Plan	Establishes the overall framework this section situates this project's methods within.
Oct 2021	Good Machine Learning Practice (GMLP) guiding principles	Motivates this project's standing practice of disclosing failure modes (Chapter 8) rather than reporting only favorable results.
Oct 2023 / Dec 2024	Predetermined Change Control Plan (PCCP) guiding principles / final guidance	Directly relevant to Section 5.5.3's streaming-recalibration design - a real mechanism for pre-authorized periodic updates.
Ongoing	Section 520(o)(1)(E) Clinical Decision Support	Its transparency criterion is structurally favorable to

	exemption	this project's interval-based (vs. black-box point) output, discussed below.

The FDA's Artificial Intelligence/Machine Learning-Based Software as a Medical Device (AI/ML SaMD) Action Plan [46] is the foundational policy document governing this space, followed by the jointly issued Good Machine Learning Practice guiding principles [47] and, most directly relevant to a system whose behavior is expected to evolve as more real data accumulates, the Predetermined Change Control Plan (PCCP) guidance [48] - a mechanism specifically for pre-authorizing a bounded set of future model updates (for instance, periodic recalibration against newly accumulated ED data, precisely the kind of update Section 5.5.3's streaming-recalibration design describes) without requiring a new submission for every retraining cycle.

Two properties of this project's own methodology bear directly, and favorably, on this regulatory picture, worth naming explicitly rather than left for a reader to infer. First, U.S. law provides a specific statutory exemption (Section 520(o)(1)(E) of the Food, Drug, and Cosmetic Act) for clinical decision support software that meets four criteria, chief among them that the software must not analyze medical images or signals directly and must enable a healthcare professional to independently review the basis for its recommendation, rather than functioning as an opaque directive. A system built on this project's own conformal intervals is structurally well positioned relative to that criterion in a way a black-box point-prediction system is not: a Mondrian conformal interval, by construction (Section 4.4.5), comes with an explicit, provable, and - unlike a typical deep-learning confidence score - exactly calibrated statement of the guarantee's own scope (per-category coverage at a stated alpha), which is precisely the kind of transparent, reviewable "basis" the exemption's language contemplates, rather than a single opaque number a clinician has no principled way to evaluate.

Second, this report's own repeated emphasis on disclosing where its methods fail - Chapter 8's exchangeability collapse, Chapter 6's honest accounting of weighted CP's infinite-width intervals under unsupported shift, Section 4.2.3.1's direct comparison against independently published service-time data - is not merely academic thoroughness; it is close in spirit to what Good Machine Learning Practice's guiding principles actually ask for: understanding a model's performance envelope and communicating it

honestly, rather than reporting only favorable results. A real regulatory submission would require far more than this project provides (a validated clinical claim, a quality management system, real patient-outcome data rather than DES-simulated scenarios, and a Predetermined Change Control Plan describing exactly which future updates are pre-authorized and which would require new review) - stated here as a genuine limitation of what this project's academic scope can offer, not elided in service of a more impressive-sounding claim. What this section establishes is narrower and more honest: that this project's specific methodological choices - calibrated, per-category intervals with an explicit, provable guarantee, and a standing practice of disclosing rather than hiding failure modes - are structurally aligned with, rather than in tension with, the direction current AI/ML medical device regulation has actually taken, which is a meaningfully different and more defensible claim than asserting this project's prototype is itself deployment-ready.

Chapter 11: Synthesis and Uncharted Horizons

Conformal Metamodeling and the Future of Stochastic System Emulation

11.1 Summary of Methodological Contributions

This report set out to test, in a discrete-event queueing simulation domain, the first of two limitations that Gopakumar et al. [1] explicitly flag as untested in their own validation of conformal prediction for surrogate-model uncertainty quantification: that a conformal prediction interval's coverage guarantee is marginal rather than conditional, and could in principle mask systematic miscalibration within specific subpopulations of a test distribution. The answer, established across Chapter 7, is that this limitation is real and operationally significant in this domain: a single pooled conformal quantile silently fails the most operationally important scenario - an understaffed emergency department facing a demand surge - with coverage falling as low as 68.2 percent against a 90 percent nominal target, while simultaneously and wastefully overcovering the easiest scenario. Mondrian conformal prediction, which calibrates separately within each of nine staffing-by-arrival-rate categories rather than pooling, corrects this specific failure to 90.9-92.0 percent coverage using the identical calibration data, with the correction confirmed statistically significant ($p < 0.001$) across 30 independent calibration/test draws and replicated, at a comparable magnitude, at a second, independent hospital department with materially different patient volume and acuity mix (Chapter 9).

Beyond this central finding, the report makes four further methodological contributions, each grounded in real implementation and real results rather than only discussed as possibilities. Chapter 6 extends the conformal toolkit itself with conformal risk control (bounding operational overflow severity, not just its probability), adaptive conformal inference (recovering most of a static method's lost coverage under a live demand-surge stream, from 58.0 to 89.0 percent for the hardest target without any retraining), and likelihood-ratio weighted conformal prediction (a real,

bounded partial remedy under a known covariate shift, honestly limited by the same support-overlap requirement Chapter 8 traces the underlying failure to). Chapter 7 extends the surrogate architecture comparison from two to five independently trained and benchmarked architectures, confirming this report's accuracy findings reflect a property of gradient-boosted tree ensembles generally rather than one specific library's implementation. Chapter 8 confirms, across two structurally different surrogate architectures, that Gopakumar et al.'s second stated limitation - exchangeability - also holds in this domain, with the specific failure mechanism depending on how each architecture extrapolates. Chapter 10 translates every preceding result into a working prototype - an interactive prediction-interval dashboard and a capacity-planning optimization built directly on this project's own conformal intervals - demonstrating concretely, not only abstractly, what this report's statistical findings would mean for an actual staffing decision.

11.2 Open Theoretical Questions

Several genuinely open questions surface directly from this report's own results, distinct from routine future-work extensions (Section 11.3), because they sit at points where this report's own findings do not yet resolve a real theoretical tension.

First, Chapter 8.5's finding that weighted conformal prediction's benefit is real but sharply bounded by calibration support raises a question this report's own scope does not resolve: is there a principled middle ground between Mondrian CP's discrete category-conditional structure and likelihood-ratio weighting's continuous but support-bounded correction - for instance, a smoothly-weighted Mondrian scheme where a test point's category membership is graded rather than binary, potentially extending usable correction slightly past a hard calibration-support boundary without claiming validity indefinitely far outside it? This report's own Chapter 4 derivations (Sections 4.4.5, 4.7.2) provide the formal machinery such a hybrid method would need to prove a coverage guarantee for, but deriving and evaluating it was outside this report's own scope.

Second, Chapter 6.3's ACI result - recovering coverage under exactly the shift that defeats every other method compared in this report, at the cost of a weaker long-run-average guarantee rather than a finite-sample one - raises a question about whether that tradeoff is fundamental or merely a property of the specific step-size rule (Section 6.3) this report implements. Gibbs and Candes' own theoretical analysis bounds ACI's regret under adversarial sequences but does not, to this report's own literature review's knowledge, characterize the tradeoff between recovery speed and interval

width under the specific structured (monotonically escalating, not adversarial) shift this report's own demand-surge stream represents - a gap between general adversarial theory and the specific, structured shift pattern real operational settings like this one actually exhibit.

Third, Section 2.8.3 names high-dimensional conditional coverage as an open problem this project's own low-dimensional scenario space was fortunate not to have to solve. Chapter 6's methods (CQR, CRC) each offer a partial, different answer to a version of this problem - CQR by adapting width continuously rather than partitioning, CRC by generalizing the loss rather than the partition - and a genuinely open theoretical question, raised but not answered by this report, is whether a method combining width-adaptivity (CQR) with a richer, higher-dimensional partition (beyond Mondrian's simple tercile grid) can scale to a conditioning space too large for direct category partitioning without losing the finite-sample guarantee Section 4.4.5 proves for the low-dimensional case.

11.3 Roadmap for Future Research

Several concrete directions follow from this report's findings, limitations (Section 11.4), and open questions (Section 11.2) - listed here in the order this report's own results suggest they would be most valuable to pursue next, rather than as an unordered list.

Generating full predictive distributions rather than fixed-alpha intervals - Mondrian conformal predictive distributions, reviewed in Section 2.2 - would let a staffing decision-maker read off any confidence level of interest after the fact rather than committing to 90 percent coverage in advance, directly extending Chapter 6's toolkit with a method this report reviews but does not implement. Continuous-space Mondrian binning via decision trees, rather than this report's fixed staffing/arrival-rate tercile grid (Section 4.7.3), would let the taxonomy itself be learned from calibration data rather than fixed by construction - a direct response to Section 4.7.3's own stated tradeoff between empirical-quantile and domain-threshold binning, potentially capturing both binning strategies' advantages simultaneously. Multi-hospital network simulation emulators - jointly modeling several EDs with shared regional demand and ambulance-diversion coupling between them, rather than this report's independent single-department DES instances (Section 4.2) - would extend Chapter 9's two-site replication toward the genuinely networked setting real regional healthcare systems operate within, and would let Chapter 10's capacity-optimization prototype (Section 10.2) be extended to a joint, network-wide staffing decision rather than one department in isolation.

Beyond these, extending the Mondrian taxonomy with a real shift or seasonal covariate in a richer, real-data-calibrated simulation would test whether Mondrian CP's benefit extends to conditioning variables this project's DES does not produce (Section 1.7.1); testing the third department present in the underlying dataset would complete the generalization picture Chapter 9 begins across every site available rather than two of three; and combining this report's marginal-coverage finding with its own Chapter 8 exchangeability finding - testing whether Mondrian CP's category-conditional structure offers any partial protection under a mild, in-taxonomy distribution shift, short of the severe shift studied there - remains a natural next question raised by, but not fully answered within, this report.

11.4 Limitations

Several limitations of this work are stated here explicitly rather than left implicit. First, the discrete-event simulation's service-time distributions are calibrated from literature-standard parameters by triage-acuity level, not derived from the real dataset used in this project, because that dataset contains no length-of-stay field; Section 4.2.3.1's direct, quantitative cross-check against ten independently published ED studies shows this project's own parameters run toward the shorter, lower-variance end of what those studies report, a specific, disclosed direction of conservatism rather than an unquantified caveat. Second, the Mondrian taxonomy used throughout this project - staffing tercile crossed with arrival-rate tercile - reflects the only two scenario-level covariates the DES actually produces; a real emergency department's conditional coverage gaps could plausibly depend on additional factors (shift, season, day of week) this project's DES does not model. Third, cross-site generalization (Chapter 9) was tested at a single split rather than the full 30-repeat treatment, since the question there was replication of an already-established finding at a new site, not re-establishing statistical rigor already achieved for the primary department. Fourth, generalization was tested across two of the three departments present in the underlying dataset; the third department was not evaluated, for reasons of project scope and time. Fifth, Chapter 10's dashboard and optimization prototypes operate on this project's own DES-simulated scenario data, not real patient-level operational data - a genuine sim-to-real gap named explicitly here rather than obscured by Chapter 10's otherwise concrete, working implementation, and precisely the gap Section 10.3's regulatory discussion identifies as the largest remaining step between this project's academic prototype and anything resembling a deployable system. Sixth, Chapter 6's three newly implemented methods (CRC, ACI, weighted CP) are each evaluated on a single representative experimental design rather than the 30-repeat statistical treatment Chapter 7's core comparison

receives; their results should be read as a first, real demonstration of each method's mechanism on this project's own data, not as statistically exhaustive as the central Mondrian CP finding they extend.

Chapter 12: References

[1] V. Gopakumar et al., "Uncertainty Quantification of Surrogate Models Using Conformal Prediction," *Machine Learning: Science and Technology, 2026.*

[2] V. Vovk, A. Gammerman, and G. Shafer, Algorithmic Learning in a Random World, *Springer, New York, NY, USA, 2005.*

[3] G. Shafer and V. Vovk, "A Tutorial on Conformal Prediction," *Journal of Machine Learning Research, vol. 9, pp. 371-421, 2008.*

[4] P. Toccaceli and A. Gammerman, "Combination of Inductive Mondrian Conformal Predictors," *Machine Learning, vol. 108, pp. 489-510, 2019.*

[5] H. Papadopoulos, K. Proedrou, V. Vovk, and A. Gammerman, "Inductive Confidence Machines for Regression," *in Proc. 13th Eur. Conf. Machine Learning (ECML), Lecture Notes in Computer Science, vol. 2430, pp. 345-356, 2002.*

[6] J. Lei, M. G'Sell, A. Rinaldo, R. J. Tibshirani, and L. Wasserman, "Distribution-Free Predictive Inference for Regression," *Journal of the American Statistical Association, vol. 113, no. 523, pp. 1094-1111, 2018.*

[7] Y. Romano, E. Patterson, and E. Candès, "Conformalized Quantile Regression," *in Advances in Neural Information Processing Systems 32 (NeurIPS), 2019.*

[8] A. N. Angelopoulos and S. Bates, "A Gentle Introduction to Conformal Prediction and Distribution-Free Uncertainty Quantification," *arXiv:2107.07511, 2021.*

[9] R. J. Tibshirani, R. F. Barber, E. Candès, and A. Ramdas, "Conformal Prediction Under Covariate Shift," *in Advances in Neural Information Processing Systems 32 (NeurIPS), 2019.*

[10] R. F. Barber, E. Candès, A. Ramdas, and R. J. Tibshirani, "Conformal Prediction Beyond Exchangeability," *Annals of Statistics, vol. 51, no. 2, pp. 816-845, 2023.*

[11] V. Vovk, D. Lindsay, I. Nouretdinov, and A. Gammerman, "Mondrian Confidence Machine," *Technical Report, Royal Holloway, University of London, 2003.*

[12] H. Boström and U. Johansson, "Mondrian Conformal Regressors," *Proceedings of Machine Learning Research (PMLR), vol. 128 (COPA 2020), pp. 114-133, 2020.*

[13] H. Boström, U. Johansson, and T. Löfström, "Mondrian Conformal Predictive Distributions," *Proceedings of Machine Learning Research (PMLR), vol. 152 (COPA 2021), 2021.*

[14] M. C. Kennedy and A. O'Hagan, "Bayesian Calibration of Computer Models," *Journal of the Royal Statistical Society: Series B, vol. 63, no. 3, pp. 425-464, 2001.*

[15] C. E. Rasmussen and C. K. I. Williams, Gaussian Processes for Machine Learning, *MIT Press, Cambridge, MA, USA, 2006.*

[16] J. H. Friedman, "Greedy Function Approximation: A Gradient Boosting Machine," *Annals of Statistics, vol. 29, no. 5, pp. 1189-1232, 2001.*

[17] B. Lakshminarayanan, A. Pritzel, and C. Blundell, "Simple and Scalable Predictive Uncertainty Estimation Using Deep Ensembles," *in Advances in Neural Information Processing Systems 30 (NeurIPS), pp. 6402-6413, 2017.*

[18] M. Abdar et al., "A Review of Uncertainty Quantification in Deep Learning: Techniques, Applications and Challenges," *Information Fusion, vol. 76, pp. 243-297, 2021.*

[19] L. V. Green, J. Soares, J. F. Giglio, and R. A. Green, "Using Queueing Theory to Increase the Effectiveness of Emergency Department Provider Staffing," *Academic Emergency Medicine, vol. 13, no. 1, pp. 61-68, 2006.*

[20] L. V. Green, "Queueing Analysis in Healthcare," *book chapter, Columbia Business School, New York, NY, USA.*

[21] X. Hu et al., "Applying Queueing Theory to the Study of Emergency Department Operations: A Survey and a Discussion of Comparable

Simulation Studies," *International Transactions in Operational Research, 2018.*

[22] Anonymous, "Performance Evaluation of a M/G/1 Queue Model for Patient Flow in a Health Care System," *Mathematical Modelling of Engineering Problems, IIETA.*

[23] Anonymous, "Decision Support for the Optimization of Provider Staffing for Hospital Emergency Departments with a Queue-Based Approach," *PMC6947400.*

[24] Anonymous, "A Simulation-Based Optimization Approach for the Calibration of a Discrete Event Simulation Model of an Emergency Department," *arXiv:2102.00945, 2021.*

[25] Anonymous, "Discrete Event Simulation for Emergency Department Modelling: A Systematic Review of Validation Methods," *ScienceDirect, 2022.*

[26] Anonymous, "Discrete Event Simulation Modelling for an Improved Patient Flow at the Emergency Department, Sygehus Lillebælt, Kolding," *PMC3327033.*

[27] Anonymous, "A Simulation-Based Optimization Approach for Analyzing the Ambulance Diversion Phenomenon in an Emergency Department Network," *arXiv:2108.04162, 2021.*

[28] Anonymous, "Machine Learning-Based Prediction of Hospital Prolonged Length of Stay Admission at Emergency Department: A Gradient Boosting Algorithm Analysis," *Frontiers in Artificial Intelligence, 2023.*

[29] Anonymous, "An Artificial Intelligence-Based Framework for Predicting Emergency Department Overcrowding: Development and Evaluation Study," *arXiv:2504.18578, 2025.*

[30] Anonymous, "Machine Learning-Based Triage to Identify Low-Severity Patients with a Short Discharge Length of Stay in Emergency Department," *PMC9123815.*

[31] N. R. Hoot, L. J. LeBlanc, I. Jones, S. R. Levin, C. Zhou, C. S. Gadd, and D. Aronsky, "Forecasting Emergency Department Crowding: A Discrete Event Simulation," *Annals of Emergency Medicine, vol. 52, no. 2, pp. 116-125, 2008.*

[32] R. Otto, S. Blaschke, W. Schirrmeister, et al., "Length of Stay as Quality Indicator in Emergency Departments: Analysis of

Determinants in the German Emergency Department Data Registry (AKTIN Registry)," *Internal and Emergency Medicine, vol. 17, no. 4, pp. 1199-1209, 2022.*

[33] B. J. Theiling, K. V. Kennedy, A. T. Limkakeng Jr., P. Manandhar, A. Erkanli, and S. R. Pitts, "A Method for Grouping Emergency Department Visits by Severity and Complexity," *Western Journal of Emergency Medicine, vol. 21, no. 5, pp. 1147-1155, 2020.*

[34] Z. Karaca, H. S. Wong, and R. L. Mutter, "Duration of Patients' Visits to the Hospital Emergency Department," *BMC Emergency Medicine, vol. 12, no. 15, 2012.*

[35] T. Y. Kim, C. Ohmart, Z. Khan, M. Lance, and S. Kim, "The Effect on Length of Stay After Implementation of Discharging Low Acuity Patients From Triage," *Cureus, vol. 13, no. 9, e17640, 2021.*

[36] F. Mahmoodian, R. Eqtesadi, and A. Ghareghani, "Waiting Times in Emergency Department After Using the Emergency Severity Index Triage Tool," *Archives of Trauma Research, vol. 3, no. 4, e19507, 2014.*

[37] M. Laskowski, R. D. McLeod, M. R. Friesen, B. W. Podaima, and A. S. Alfa, "Models of Emergency Departments for Reducing Patient Waiting Times," *PLoS ONE, vol. 4, no. 7, e6127, 2009.*

[38] T. E. Locker and S. M. Mason, "Analysis of the Distribution of Time That Patients Spend in Emergency Departments," *BMJ, vol. 330, no. 7501, pp. 1188-1189, 2005.*

[39] A. De Santis, T. Giovannelli, S. Lucidi, M. Messedaglia, and M. Roma, "Determining the Optimal Piecewise Constant Approximation for the Nonhomogeneous Poisson Process Rate of Emergency Department Patient Arrivals," *arXiv:2101.11138, 2021.*

[40] A. Kramer, C. Dosi, M. Iori, and M. Vignoli, "Successful Implementation of Discrete Event Simulation: The Case of an Italian Emergency Department," *arXiv:2006.13062, 2020.*

[41] J. F. C. Kingman, "On Queues in Heavy Traffic," *Journal of the Royal Statistical Society, Series B, vol. 24, no. 2, pp. 383-392, 1962.*

[42] H. Sakasegawa, "An Approximation Formula Lq = alpha . rho^beta / (1 - rho)," *Annals of the Institute of Statistical Mathematics, vol. 29, no. 1, pp. 67-75, 1977.*

[43] maalona (Kaggle username), "Hospital Triage and Patient History Data," *Kaggle dataset - Yale New Haven Health System, retrospective study, March 2014 - July 2017.*

[44] S. Bates, A. Angelopoulos, L. Lei, J. Malik, and M. Jordan, "Distribution-Free, Risk-Controlling Prediction Sets," *Journal of the ACM, vol. 68, no. 6, pp. 1-34, 2021.*

[45] I. Gibbs and E. Candes, "Adaptive Conformal Inference Under Distribution Shift," *Advances in Neural Information Processing Systems 34 (NeurIPS), 2021.*

[46] U.S. Food and Drug Administration, "Artificial Intelligence/Machine Learning (AI/ML)-Based Software as a Medical Device (SaMD) Action Plan," *FDA, January 2021.*

[47] U.S. FDA, Health Canada, and UK MHRA, "Good Machine Learning Practice for Medical Device Development: Guiding Principles," *October 2021.*

[48] U.S. Food and Drug Administration, "Marketing Submission Recommendations for a Predetermined Change Control Plan for Artificial Intelligence-Enabled Device Software Functions," *FDA Guidance, December 2024.*

APPENDIX A

Source Code Listings

The following pages list the real source code underlying this report's results, directly from this project's repository under src/. Each file is reproduced in full, unmodified, with original line numbers.

src/utils/extract_distributions.py - Real-data distribution extraction (arrivals, ESI mix)

```
   1  """
   2  Week 3: extract calibration distributions for the ER DES.
   3
   4  Arrivals and patient volume come from the real Kaggle dataset
   5  (data/processed/triage_data.parquet). Service/treatment time does not
exist
   6  in that dataset (no discharge timestamp, only 4-hour arrival bins), so
those
   7  come from published ED simulation literature, stratified by ESI acuity
   8  level (1 = most urgent, 5 = least urgent).
   9
  10  Service time source: log-normal parameters commonly used in ED DES
studies,
  11  consistent in shape (log-normal, per-ESI, decreasing with decreasing
  12  acuity) with Hoot et al. (2008, Ann Emerg Med 52(2):116-125), and
  13  cross-checked quantitatively against nine further independently
published
  14  ED studies (Otto et al. 2022; Theiling et al. 2020; Karaca et al.
2012;
  15  Kim et al. 2021; Mahmoodian et al. 2014; Laskowski et al. 2009; Locker
and
  16  Mason 2005; De Santis et al. 2021; Kramer et al. 2020) - see
  17  reports/assignments/book_common.py, Section 4.2.3.1, for the full
  18  per-study comparison table and an honest discussion of where these
  19  parameters diverge from what those studies report (this project's
values
  20  run toward the shorter, lower-variance end of the published range).
  21
  22  The dataset combines three EDs (`dep_name` A/B/C: one academic, two
  23  community — per the Kaggle dataset description) collected March 2014 -
  24  July 2017 (1248 days). Our DES models one ED, so we calibrate on a
single
  25  department (default: A, the largest/academic site) rather than the
pooled
  26  total — pooling all three sites would overstate one ED's real arrival
rate.
  27  """
  28
  29  import pandas as pd
  30
  31  RAW = "data/processed/triage_data.parquet"
  32  OUT_TABLES = "results/tables"
  33  OUT_FIGURES = "results/figures"
  34
  35  DEPARTMENT = "A"
  36  STUDY_PERIOD_DAYS = 1248  # March 2014 - July 2017, per Kaggle dataset
```

```
description
  37
  38  # (mean_minutes, sd_minutes) per ESI level, log-normal, from ED sim
literature
  39  SERVICE_TIME_PARAMS_BY_ESI = {
  40      1: (180, 90),
  41      2: (150, 75),
  42      3: (120, 60),
  43      4: (75, 40),
  44      5: (45, 25),
  45  }
  46
  47
  48  def load_data(department=DEPARTMENT):
  49      df = pd.read_parquet(RAW)
  50      return df[df["dep_name"] == department]
  51
  52
  53  def arrival_distribution(df):
  54      """Real arrival counts by 4-hour bin, day of week, and month, for
one ED."""
  55      by_hour_bin = df["arrivalhour_bin"].value_counts().sort_index()
  56      by_day = df["arrivalday"].value_counts()
  57      by_month = df["arrivalmonth"].value_counts()
  58      return by_hour_bin, by_day, by_month
  59
  60
  61  def esi_distribution(df):
  62      """Real ESI (acuity) mix — drives which service-time distribution
applies."""
  63      return
df["esi"].dropna().astype(int).value_counts(normalize=True).sort_index()
  64
  65
  66  def main():
  67      df = load_data()
  68      visits_per_day = len(df) / STUDY_PERIOD_DAYS
  69
  70      by_hour_bin, by_day, by_month = arrival_distribution(df)
  71      esi_mix = esi_distribution(df)
  72
  73      by_hour_bin.to_csv(f"{OUT_TABLES}/arrivals_by_hour_bin.csv",
header=["count"])
  74      by_day.to_csv(f"{OUT_TABLES}/arrivals_by_day.csv",
header=["count"])
  75      by_month.to_csv(f"{OUT_TABLES}/arrivals_by_month.csv",
header=["count"])
  76      esi_mix.to_csv(f"{OUT_TABLES}/esi_mix.csv", header=["proportion"])
  77
  78      service_time_df = pd.DataFrame(
  79          SERVICE_TIME_PARAMS_BY_ESI, index=["mean_minutes",
"sd_minutes"]
  80      ).T
  81      service_time_df.index.name = "esi"
  82      service_time_df.to_csv(f"{OUT_TABLES}/service_time_params.csv")
  83
  84      pd.Series(
  85          {"department": DEPARTMENT, "study_period_days":
STUDY_PERIOD_DAYS,
  86           "total_visits": len(df), "visits_per_day": visits_per_day},
  87      ).to_csv(f"{OUT_TABLES}/calibration_meta.csv", header=["value"])
  88
  89      print(f"Department {DEPARTMENT}: {len(df)} visits over
{STUDY_PERIOD_DAYS} days"
  90            f" = {visits_per_day:.1f} visits/day\n")
```

```
  91      print("Arrivals by 4h bin:\n", by_hour_bin, "\n")
  92      print("Arrivals by day:\n", by_day, "\n")
  93      print("Arrivals by month:\n", by_month, "\n")
  94      print("ESI mix:\n", esi_mix, "\n")
  95      print("Service time params (literature, by ESI):\n",
service_time_df)
  96
  97
  98  if __name__ == "__main__":
  99      main()
```

src/des/er_simulation.py - SimPy discrete-event ER simulation

```
   1  """
   2  Week 4-5: ER discrete-event simulation, calibrated on real arrival/ESI
   3  distributions (results/tables/, department A, see
extract_distributions.py)
   4  and literature service-time params.
   5
   6  Single resource pool (`capacity`) represents a combined bed+provider
slot for
   7  the full visit duration — the standard simplification in ED
queueing/DES
   8  literature (M/G/c queue), used here because we have no real data to
split
   9  staffing into separate doctor/bed/nurse ratios; modeling them
separately
  10  would just add unverified guesses without adding insight for this
project's
  11  actual question (UQ methodology, not hospital operations realism).
  12
  13  `n_capacity` is a parameter, not fixed — Week 6-7 varies it (along
with
  14  arrival-rate scaling) to generate surrogate training data across
scenarios.
  15
  16  Default n_capacity=30: offered load = arrival_rate * mean_service_time
  17  (Erlangs) is ~22 erlangs on average and ~34 erlangs during the 11-14
peak
  18  bin (department A: ~258 visits/day, ~121 min mean service time). 30
servers
  19  sits between those, giving a stable but visibly congested system —
useful
  20  for a DES whose whole purpose is producing realistic queueing/wait
  21  variation, not a system with near-zero wait.
  22  """
  23
  24  import numpy as np
  25  import pandas as pd
  26  import simpy
  27
  28  TABLES = "results/tables"
  29
  30  # Literature log-normal service time params by ESI, from
extract_distributions.py
  31  SERVICE_TIME_PARAMS_BY_ESI = {
  32      1: (180, 90),
  33      2: (150, 75),
  34      3: (120, 60),
  35      4: (75, 40),
  36      5: (45, 25),
```

```
  37  }
  38
  39  STUDY_PERIOD_DAYS = 1248  # March 2014 - July 2017, per
extract_distributions.py
  40
  41  HOUR_BINS = ["23-02", "03-06", "07-10", "11-14", "15-18", "19-22"]
  42
  43  # "23-02" wraps midnight (hours 23,0,1,2) — build the hour->bin map
explicitly
  44  # rather than hour // 4, which would wrongly put hours 0-3 in "23-02".
  45  HOUR_TO_BIN = {}
  46  for _bin_label, _hours in zip(
  47      HOUR_BINS, [[23, 0, 1, 2], [3, 4, 5, 6], [7, 8, 9, 10], [11, 12,
13, 14], [15, 16, 17, 18], [19, 20, 21, 22]]
  48  ):
  49      for _h in _hours:
  50          HOUR_TO_BIN[_h] = _bin_label
  51
  52
  53  def lognormal_params_from_mean_sd(mean, sd):
  54      """Convert a desired (mean, sd) in minutes to the underlying
normal's (mu, sigma)."""
  55      variance = sd**2
  56      mu = np.log(mean**2 / np.sqrt(variance + mean**2))
  57      sigma = np.sqrt(np.log(1 + variance / mean**2))
  58      return mu, sigma
  59
  60
  61  def load_calibration(tables_dir=TABLES):
  62      """Real arrival-by-hour-bin distribution and real ESI mix (single
department).
  63
  64      tables_dir defaults to the department-A tables used throughout the
project;
  65      pass a different directory (e.g. results/tables/dept_b) to
calibrate the
  66      same DES logic against a different department's real distributions
without
  67      touching department A's files.
  68      """
  69      arrivals = pd.read_csv(f"{tables_dir}/arrivals_by_hour_bin.csv",
index_col=0)["count"]
  70      arrivals = arrivals.reindex(HOUR_BINS)
  71      esi_mix = pd.read_csv(f"{tables_dir}/esi_mix.csv",
index_col=0)["proportion"]
  72      return arrivals, esi_mix
  73
  74
  75  class ERSimulation:
  76      """
  77      One 24h day of ER operation. Arrival rate varies by 4-hour bin
(real data),
  78      ESI is sampled from the real acuity mix, service time is sampled
per-ESI
  79      log-normal (literature params). Higher-acuity (lower ESI number)
patients
  80      get priority for the shared capacity pool, matching real triage
behavior.
  81      """
  82
  83      def __init__(self, n_capacity, arrivals_by_bin, esi_mix,
arrival_rate_multiplier=1.0, seed=None):
  84          self.n_capacity = n_capacity
  85          self.arrivals_by_bin = arrivals_by_bin
  86          self.esi_mix = esi_mix
```

```
  87          self.np_rng = np.random.default_rng(seed)
  88
  89          total_visits_per_day = arrivals_by_bin.sum() /
STUDY_PERIOD_DAYS
  90          self.rate_per_bin = (
  91              (arrivals_by_bin / arrivals_by_bin.sum()) *
total_visits_per_day / 4 * arrival_rate_multiplier
  92          )  # per hour
  93
  94          self.wait_times = []
  95          self.total_times = []
  96          self.n_patients = 0
  97
  98      def patient(self, env, capacity):
  99          arrival_time = env.now
 100          esi = self.np_rng.choice(self.esi_mix.index.astype(int),
p=self.esi_mix.values)
 101          mean, sd = SERVICE_TIME_PARAMS_BY_ESI[esi]
 102          mu, sigma = lognormal_params_from_mean_sd(mean, sd)
 103          service_time = self.np_rng.lognormal(mu, sigma)
 104
 105          with capacity.request(priority=esi) as req:
 106              yield req
 107              wait = env.now - arrival_time
 108              self.wait_times.append(wait)
 109              yield env.timeout(service_time)
 110              self.total_times.append(env.now - arrival_time)
 111              self.n_patients += 1
 112
 113      def arrival_process(self, env, capacity):
 114          for hour in range(24):
 115              bin_idx = HOUR_TO_BIN[hour]
 116              rate = self.rate_per_bin[bin_idx]
 117              for _ in range(60):  # simulate minute-by-minute within
this hour
 118                  if self.np_rng.random() < rate / 60:
 119                      env.process(self.patient(env, capacity))
 120                  yield env.timeout(1)
 121
 122      def run(self):
 123          env = simpy.Environment()
 124          capacity = simpy.PriorityResource(env,
capacity=self.n_capacity)
 125          env.process(self.arrival_process(env, capacity))
 126          env.run(until=24 * 60)
 127          return {
 128              "n_patients": self.n_patients,
 129              "mean_wait_minutes": np.mean(self.wait_times) if
self.wait_times else 0.0,
 130              "mean_total_minutes": np.mean(self.total_times) if
self.total_times else 0.0,
 131              "p95_wait_minutes": np.percentile(self.wait_times, 95) if
self.wait_times else 0.0,
 132          }
 133
 134
 135  def run_scenario(n_capacity=30, arrival_rate_multiplier=1.0,
seed=None, tables_dir=TABLES):
 136      arrivals_by_bin, esi_mix = load_calibration(tables_dir)
 137      sim = ERSimulation(n_capacity, arrivals_by_bin, esi_mix,
arrival_rate_multiplier, seed=seed)
 138      return sim.run()
 139
 140
 141  if __name__ == "__main__":
```

```
142      result = run_scenario(seed=42)
143      print(result)
```

src/des/validate.py - DES validation against real daily volume

```
  1  """
  2  Week 4-5: validate the calibrated DES against real aggregated stats.
  3
  4  Runs many independent simulated days and compares mean simulated
patients/day
  5  to the real department-A daily average (calibration_meta.csv). Some
  6  undershoot is expected: patients still queued when a 24h simulated day
ends
  7  are cut off (right-censoring at the day boundary) rather than carried
over,
  8  since each run models one independent day for later surrogate-training
  9  sampling (Week 6-7), not a continuous multi-day rollout.
 10  """
 11
 12  import pandas as pd
 13
 14  from src.des.er_simulation import run_scenario
 15
 16  N_DAYS = 200
 17  N_CAPACITY = 30
 18
 19
 20  def main():
 21      results = [run_scenario(n_capacity=N_CAPACITY, seed=s) for s in
range(N_DAYS)]
 22      df = pd.DataFrame(results)
 23
 24      meta = pd.read_csv("results/tables/calibration_meta.csv",
index_col=0)
 25      real_visits_per_day = float(meta.loc["visits_per_day", "value"])
 26      sim_visits_per_day = df["n_patients"].mean()
 27
 28      summary = pd.Series(
 29          {
 30              "n_days_simulated": N_DAYS,
 31              "n_capacity": N_CAPACITY,
 32              "real_visits_per_day": real_visits_per_day,
 33              "sim_mean_visits_per_day": sim_visits_per_day,
 34              "sim_std_visits_per_day": df["n_patients"].std(),
 35              "ratio_sim_over_real": sim_visits_per_day /
real_visits_per_day,
 36              "sim_mean_wait_minutes": df["mean_wait_minutes"].mean(),
 37              "sim_mean_total_minutes": df["mean_total_minutes"].mean(),
 38              "sim_mean_p95_wait_minutes":
df["p95_wait_minutes"].mean(),
 39          }
 40      )
 41      summary.to_csv("results/tables/des_validation.csv",
header=["value"])
 42      print(summary)
 43
 44
 45  if __name__ == "__main__":
 46      main()
```

src/surrogate/generate_training_data.py - DES scenario sweep for surrogate training data

```
   1  """
   2  Week 6-7: run the calibrated DES across varied staffing/arrival
scenarios to
   3  generate surrogate training data.
   4
   5  Each row = one independent simulated day at a randomly sampled
   6  (n_capacity, arrival_rate_multiplier), with its own random seed so the
DES's
   7  intrinsic stochasticity (arrival randomness, ESI mix, service-time
variance)
   8  stays in the labels. That residual variability is exactly what the
later UQ
   9  step (GP baseline / conformal prediction) needs to characterize —
smoothing
  10  it out here would defeat the point.
  11
  12  n_capacity range (15-45) and arrival_rate_multiplier range (0.8-1.3)
span
  13  under- to over-provisioned staffing and below/above-average demand
days,
  14  while staying inside "normal operations" — the Week 13 surge-day
stress
  15  test intentionally goes outside this range later to test
exchangeability.
  16  """
  17
  18  import numpy as np
  19  import pandas as pd
  20
  21  from src.des.er_simulation import run_scenario
  22
  23  N_SCENARIOS = 5000
  24  N_CAPACITY_RANGE = (15, 45)
  25  ARRIVAL_MULTIPLIER_RANGE = (0.8, 1.3)
  26  OUT_PATH = "data/processed/surrogate_training_data.parquet"
  27
  28
  29  def generate(n_scenarios=N_SCENARIOS, seed=0, seed_offset=0,
  30               n_capacity_range=N_CAPACITY_RANGE,
arrival_multiplier_range=ARRIVAL_MULTIPLIER_RANGE,
  31               tables_dir=None):
  32      """seed_offset shifts the per-scenario DES seeds so a second call
(e.g. for
  33      CP calibration data) draws fully independent randomness from the
first,
  34      on top of already using a different `seed` for the parameter
sampling.
  35
  36      n_capacity_range/arrival_multiplier_range/tables_dir default to
department
  37      A's values (module constants / er_simulation.py's own default) so
existing
  38      callers are unaffected; pass different values to calibrate against
a
  39      different department (see src/generalization/)."""
  40      rng = np.random.default_rng(seed)
  41      n_capacity = rng.integers(n_capacity_range[0], n_capacity_range[1]
+ 1, size=n_scenarios)
```

```
    arrival_mult = rng.uniform(*arrival_multiplier_range,
size=n_scenarios)

    run_kwargs = {} if tables_dir is None else {"tables_dir":
tables_dir}
    rows = []
    for i in range(n_scenarios):
        result = run_scenario(
            n_capacity=int(n_capacity[i]),
            arrival_rate_multiplier=float(arrival_mult[i]),
            seed=seed_offset + i,
            **run_kwargs,
        )
        rows.append(
            {
                "n_capacity": n_capacity[i],
                "arrival_rate_multiplier": arrival_mult[i],
                **result,
            }
        )
    return pd.DataFrame(rows)


def main():
    df = generate()
    df.to_parquet(OUT_PATH, index=False)
    print(f"Generated {len(df)} scenarios -> {OUT_PATH}")
    print(df.describe())


if __name__ == "__main__":
    main()
```

src/surrogate/train_surrogate.py - Gradient-boosting surrogate training

```
"""
Week 6-7: train a surrogate model on the DES scenario data.

Gradient boosting (sklearn's HistGradientBoostingRegressor), not a
neural
net — this is a 2-input tabular regression problem with ~5000 rows,
where
gradient boosting is the standard strong baseline and needs no
architecture
tuning or scaling. A NN would add complexity without a corresponding
benefit
for this problem size/shape.

One model per target metric (n_patients, mean_wait_minutes,
mean_total_minutes, p95_wait_minutes) trained independently — simpler
than a
true multi-output model and fine here since sklearn doesn't support
multi-output HGBR natively.
"""

import joblib
import pandas as pd
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error,
```

```
r2_score
  20  from sklearn.model_selection import train_test_split
  21
  22  DATA_PATH = "data/processed/surrogate_training_data.parquet"
  23  MODELS_DIR = "models"
  24  METRICS_PATH = "results/tables/surrogate_metrics.csv"
  25
  26  FEATURES = ["n_capacity", "arrival_rate_multiplier"]
  27  TARGETS = ["n_patients", "mean_wait_minutes", "mean_total_minutes",
"p95_wait_minutes"]
  28
  29
  30  def main():
  31      df = pd.read_parquet(DATA_PATH)
  32      X_train, X_test, train_idx, test_idx = train_test_split(
  33          df[FEATURES], df.index, test_size=0.2, random_state=42
  34      )
  35
  36      metrics = []
  37      for target in TARGETS:
  38          y_train = df.loc[train_idx, target]
  39          y_test = df.loc[test_idx, target]
  40
  41          model = HistGradientBoostingRegressor(random_state=42)
  42          model.fit(X_train, y_train)
  43          y_pred = model.predict(X_test)
  44
  45          metrics.append(
  46              {
  47                  "target": target,
  48                  "mae": mean_absolute_error(y_test, y_pred),
  49                  "rmse": mean_squared_error(y_test, y_pred) ** 0.5,
  50                  "r2": r2_score(y_test, y_pred),
  51              }
  52          )
  53          joblib.dump(model, f"{MODELS_DIR}/surrogate_{target}.joblib")
  54
  55      metrics_df = pd.DataFrame(metrics)
  56      metrics_df.to_csv(METRICS_PATH, index=False)
  57      print(metrics_df)
  58
  59
  60  if __name__ == "__main__":
  61      main()
```

src/uq/generate_calibration_data.py - Disjoint DES scenario pool for CP calibration

```
   1  """
   2  Week 9-10: generate a calibration set for conformal prediction.
   3
   4  CP calibration data must be separate from what the surrogate was
trained on
   5  - reusing training-set residuals would understate the true residual
spread
   6  (the surrogate fits those points), which would break coverage
validity. This
   7  uses a different parameter-sampling seed AND a large DES seed offset
from
   8  generate_training_data.py, so nothing here overlaps with the
```

```
training/test
   9  data used to fit and evaluate the surrogate and GP baseline.
  10
  11  Same scenario ranges as training data (n_capacity 15-45,
arrival_rate_multiplier
  12  0.8-1.3) - calibration data has to be exchangeable with the test set,
which
  13  means drawn from the same distribution, not a different one.
  14  """
  15
  16  from src.surrogate.generate_training_data import generate
  17
  18  N_CALIBRATION = 1200
  19  OUT_PATH = "data/processed/cp_calibration_data.parquet"
  20
  21
  22  def main():
  23      df = generate(n_scenarios=N_CALIBRATION, seed=1,
seed_offset=100_000)
  24      df.to_parquet(OUT_PATH, index=False)
  25      print(f"Generated {len(df)} calibration scenarios -> {OUT_PATH}")
  26      print(df.describe())
  27
  28
  29  if __name__ == "__main__":
  30      main()
```

src/uq/gp_baseline.py - Gaussian process UQ baseline

```
   1  """
   2  Week 8: GP baseline UQ — the benchmark that standard CP and Mondrian
CP get
   3  compared against in Phase 2.
   4
   5  Fits a Gaussian Process per target on (n_capacity,
arrival_rate_multiplier)
   6  and uses its native predictive mean +/- z * std to build (1-alpha)
intervals.
   7  alpha=0.1 (90% target coverage) is fixed here and must stay the same
alpha
   8  used later for standard/Mondrian CP — the whole comparison is
meaningless if
   9  the methods target different coverage levels.
  10
  11  GP training is fit on a 1000-point subsample of the training set, not
the
  12  full ~4000, because sklearn's exact GP is O(n^3): benchmarked at ~8s
for
  13  n=1000 vs ~43s for n=2000, so full-scale training would take tens of
minutes
  14  across 4 targets. This is not just a shortcut — GP's poor scalability
vs.
  15  CP's near-constant-cost calibration is itself one of the points this
project
  16  is comparing (see Week 14-15's computation-time chart), so documenting
GP's
  17  cost here is directly relevant, not a detail to hide.
  18
  19  Uses the SAME train/test split (random_state=42, test_size=0.2) as
  20  train_surrogate.py so every UQ method (GP here, standard/Mondrian CP
later)
```

```
  21  is evaluated on identical test points — required for a fair
comparison.
  22  """
  23
  24  import time
  25
  26  import joblib
  27  import numpy as np
  28  import pandas as pd
  29  from scipy.stats import norm
  30  from sklearn.gaussian_process import GaussianProcessRegressor
  31  from sklearn.gaussian_process.kernels import RBF, ConstantKernel,
WhiteKernel
  32  from sklearn.model_selection import train_test_split
  33  from sklearn.preprocessing import StandardScaler
  34
  35  DATA_PATH = "data/processed/surrogate_training_data.parquet"
  36  MODELS_DIR = "models"
  37  METRICS_PATH = "results/tables/gp_baseline_metrics.csv"
  38
  39  FEATURES = ["n_capacity", "arrival_rate_multiplier"]
  40  TARGETS = ["n_patients", "mean_wait_minutes", "mean_total_minutes",
"p95_wait_minutes"]
  41
  42  ALPHA = 0.1  # 90% target coverage — must match the CP methods used
later
  43  N_GP_TRAIN = 1000  # subsample size for GP fitting; see module
docstring
  44  Z = norm.ppf(1 - ALPHA / 2)
  45
  46
  47  def main():
  48      df = pd.read_parquet(DATA_PATH)
  49      X_train_full, X_test, train_idx, test_idx = train_test_split(
  50          df[FEATURES], df.index, test_size=0.2, random_state=42
  51      )
  52
  53      rng = np.random.default_rng(42)
  54      gp_train_idx = rng.choice(train_idx, size=N_GP_TRAIN,
replace=False)
  55
  56      scaler = StandardScaler().fit(df.loc[gp_train_idx, FEATURES])
  57      X_train_scaled = scaler.transform(df.loc[gp_train_idx, FEATURES])
  58      X_test_scaled = scaler.transform(X_test)
  59
  60      metrics = []
  61      for target in TARGETS:
  62          y_train = df.loc[gp_train_idx, target].values
  63          y_test = df.loc[test_idx, target].values
  64
  65          kernel = ConstantKernel(1.0) * RBF(length_scale=[1.0, 1.0]) +
WhiteKernel(1.0)
  66          gp = GaussianProcessRegressor(kernel=kernel, normalize_y=True,
n_restarts_optimizer=2, random_state=42)
  67
  68          t0 = time.time()
  69          gp.fit(X_train_scaled, y_train)
  70          fit_seconds = time.time() - t0
  71
  72          t0 = time.time()
  73          y_pred, y_std = gp.predict(X_test_scaled, return_std=True)
  74          predict_seconds = time.time() - t0
  75
  76          lower = y_pred - Z * y_std
  77          upper = y_pred + Z * y_std
```

```
        covered = (y_test >= lower) & (y_test <= upper)

        metrics.append(
            {
                "target": target,
                "alpha": ALPHA,
                "target_coverage": 1 - ALPHA,
                "empirical_coverage": covered.mean(),
                "mean_interval_width": (upper - lower).mean(),
                "fit_seconds": fit_seconds,
                "predict_seconds": predict_seconds,
                "n_gp_train": N_GP_TRAIN,
            }
        )
        joblib.dump(gp, f"{MODELS_DIR}/gp_baseline_{target}.joblib")

    metrics_df = pd.DataFrame(metrics)
    metrics_df.to_csv(METRICS_PATH, index=False)
    joblib.dump(scaler, f"{MODELS_DIR}/gp_baseline_scaler.joblib")
    print(metrics_df)


if __name__ == "__main__":
    main()
```

src/uq/standard_cp.py - Standard (pooled) split conformal prediction

```
"""
Week 9-10: standard (split) conformal prediction on the surrogate's
residuals.

Uses the surrogate models already trained in Week 6-7
(models/surrogate_*.joblib)
as point predictors - CP wraps a point predictor with a calibrated
interval,
it doesn't replace it. Calibration uses the separate set from
generate_calibration_data.py, never the surrogate's own training data.

Same test set (same random_state=42 split of
surrogate_training_data.parquet)
and same alpha=0.1 as gp_baseline.py, so all three methods (GP,
standard CP,
Mondrian CP later) are directly comparable.

Two nonconformity measures, per the roadmap's "test multiple
nonconformity
measures":
  - symmetric: |y - yhat|, giving a fixed-width interval around the
point
    prediction.
  - asymmetric: separate upper/lower residual quantiles (each
calibrated at
    1-alpha/2, combined via a union bound for overall >=1-alpha
coverage).
    Matters here because several targets (p95_wait_minutes especially)
are
    right-skewed - a symmetric interval either overcovers on the left
or
    undercovers on the right.
"""

```

```
  24  import joblib
  25  import numpy as np
  26  import pandas as pd
  27  from sklearn.model_selection import train_test_split
  28
  29  TRAIN_DATA_PATH = "data/processed/surrogate_training_data.parquet"
  30  CALIBRATION_DATA_PATH = "data/processed/cp_calibration_data.parquet"
  31  MODELS_DIR = "models"
  32  METRICS_PATH = "results/tables/standard_cp_metrics.csv"
  33
  34  FEATURES = ["n_capacity", "arrival_rate_multiplier"]
  35  TARGETS = ["n_patients", "mean_wait_minutes", "mean_total_minutes",
"p95_wait_minutes"]
  36
  37  ALPHA = 0.1  # must match gp_baseline.py
  38
  39
  40  def conformal_quantile(scores, alpha):
  41      """Finite-sample-corrected (1-alpha) empirical quantile of
calibration scores."""
  42      n = len(scores)
  43      level = min(np.ceil((n + 1) * (1 - alpha)) / n, 1.0)
  44      return np.quantile(scores, level, method="higher")
  45
  46
  47  def main():
  48      df = pd.read_parquet(TRAIN_DATA_PATH)
  49      _, X_test, _, test_idx = train_test_split(df[FEATURES], df.index,
test_size=0.2, random_state=42)
  50
  51      cal_df = pd.read_parquet(CALIBRATION_DATA_PATH)
  52      X_cal = cal_df[FEATURES]
  53
  54      metrics = []
  55      for target in TARGETS:
  56          model = joblib.load(f"{MODELS_DIR}/surrogate_{target}.joblib")
  57
  58          yhat_cal = model.predict(X_cal)
  59          y_cal = cal_df[target].values
  60          residual_cal = y_cal - yhat_cal
  61
  62          yhat_test = model.predict(X_test)
  63          y_test = df.loc[test_idx, target].values
  64
  65          # symmetric
  66          q = conformal_quantile(np.abs(residual_cal), ALPHA)
  67          lower_sym = yhat_test - q
  68          upper_sym = yhat_test + q
  69          covered_sym = (y_test >= lower_sym) & (y_test <= upper_sym)
  70
  71          # asymmetric
  72          q_hi = conformal_quantile(residual_cal, ALPHA / 2)
  73          q_lo = conformal_quantile(-residual_cal, ALPHA / 2)
  74          lower_asym = yhat_test - q_lo
  75          upper_asym = yhat_test + q_hi
  76          covered_asym = (y_test >= lower_asym) & (y_test <= upper_asym)
  77
  78          metrics.append(
  79              {
  80                  "target": target,
  81                  "alpha": ALPHA,
  82                  "target_coverage": 1 - ALPHA,
  83                  "symmetric_coverage": covered_sym.mean(),
  84                  "symmetric_mean_width": (upper_sym -
lower_sym).mean(),
```

```
  85                  "asymmetric_coverage": covered_asym.mean(),
  86                  "asymmetric_mean_width": (upper_asym -
lower_asym).mean(),
  87              }
  88          )
  89
  90      metrics_df = pd.DataFrame(metrics)
  91      metrics_df.to_csv(METRICS_PATH, index=False)
  92      print(metrics_df)
  93
  94
  95  if __name__ == "__main__":
  96      main()
```

src/uq/mondrian_cp.py - Mondrian conformal prediction (9-category taxonomy)

```
   1  """
   2  Week 11-12: Mondrian conformal prediction.
   3
   4  Standard CP (Week 9-10) calibrates a single pooled quantile across all
   5  scenarios, which only guarantees *marginal* coverage - it says nothing
about
   6  whether coverage holds up within a specific staffing level or arrival
regime.
   7  That's exactly the first limitation Gopakumar et al. (2026) flag for
CP in
   8  physics domains. Mondrian CP addresses it by calibrating a separate
quantile
   9  per category, so each category gets its own (asymptotically valid)
coverage
  10  guarantee instead of relying on categories averaging out to the
marginal rate.
  11
  12  Categories: our scenario space only has two real covariates
(n_capacity,
  13  arrival_rate_multiplier) - there's no "shift" field in the DES output,
so we
  14  don't invent one. Categories are the cross of staffing tercile x
arrival-rate
  15  tercile (Low/Med/High x Low/Med/High = 9 cells), using bin edges
derived from
  16  the calibration set's own quantiles (never the test set - that would
leak
  17  information into the taxonomy).
  18
  19  Same calibration/test data, same alpha=0.1, symmetric |y - yhat|
nonconformity
  20  measure as standard_cp.py's symmetric variant, so this is a clean,
apples-to-
  21  apples comparison of pooled vs. per-category calibration - not a
different
  22  setup dressed up as a comparison.
  23
  24  For each category we compute coverage two ways on the SAME test
points:
  25    - "pooled" applies standard CP's single marginal quantile, evaluated
  26      per-category. This shows where the marginal guarantee breaks down.
  27    - "mondrian" applies that category's own quantile. This is the
actual
```

```
    per-category coverage Mondrian CP delivers.
"""

import joblib
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split

TRAIN_DATA_PATH = "data/processed/surrogate_training_data.parquet"
CALIBRATION_DATA_PATH = "data/processed/cp_calibration_data.parquet"
MODELS_DIR = "models"
DETAIL_PATH = "results/tables/mondrian_cp_detail.csv"
SUMMARY_PATH = "results/tables/mondrian_cp_summary.csv"

FEATURES = ["n_capacity", "arrival_rate_multiplier"]
TARGETS = ["n_patients", "mean_wait_minutes", "mean_total_minutes",
"p95_wait_minutes"]

ALPHA = 0.1
LEVEL_NAMES = ["Low", "Med", "High"]


def conformal_quantile(scores, alpha):
    n = len(scores)
    level = min(np.ceil((n + 1) * (1 - alpha)) / n, 1.0)
    return np.quantile(scores, level, method="higher")


def assign_categories(df, cap_bounds, arr_bounds):
    cap_level = np.digitize(df["n_capacity"], cap_bounds)
    arr_level = np.digitize(df["arrival_rate_multiplier"], arr_bounds)
    labels = [f"staff={LEVEL_NAMES[c]}/arrival={LEVEL_NAMES[a]}" for
c, a in zip(cap_level, arr_level)]
    return np.array(labels)


def main():
    df = pd.read_parquet(TRAIN_DATA_PATH)
    _, X_test, _, test_idx = train_test_split(df[FEATURES], df.index,
test_size=0.2, random_state=42)
    test_df = df.loc[test_idx]

    cal_df = pd.read_parquet(CALIBRATION_DATA_PATH)

    cap_bounds = np.quantile(cal_df["n_capacity"], [1 / 3, 2 / 3])
    arr_bounds = np.quantile(cal_df["arrival_rate_multiplier"], [1 /
3, 2 / 3])
    cal_cat = assign_categories(cal_df, cap_bounds, arr_bounds)
    test_cat = assign_categories(test_df, cap_bounds, arr_bounds)
    categories = sorted(set(cal_cat))

    detail_rows = []
    summary_rows = []
    for target in TARGETS:
        model = joblib.load(f"{MODELS_DIR}/surrogate_{target}.joblib")

        yhat_cal = model.predict(cal_df[FEATURES])
        y_cal = cal_df[target].values
        abs_resid_cal = np.abs(y_cal - yhat_cal)

        yhat_test = model.predict(X_test)
        y_test = test_df[target].values

        q_pooled = conformal_quantile(abs_resid_cal, ALPHA)

```

```
 89          pooled_coverages, mondrian_coverages = [], []
 90          for cat in categories:
 91              cal_mask = cal_cat == cat
 92              test_mask = test_cat == cat
 93              n_cal_cat = cal_mask.sum()
 94              n_test_cat = test_mask.sum()
 95              if n_test_cat == 0:
 96                  continue
 97
 98              q_cat = conformal_quantile(abs_resid_cal[cal_mask], ALPHA)
 99
100              y_c = y_test[test_mask]
101              yhat_c = yhat_test[test_mask]
102
103              covered_pooled = (np.abs(y_c - yhat_c) <= q_pooled).mean()
104              covered_mondrian = (np.abs(y_c - yhat_c) <= q_cat).mean()
105
106              pooled_coverages.append(covered_pooled)
107              mondrian_coverages.append(covered_mondrian)
108
109              detail_rows.append(
110                  {
111                      "target": target,
112                      "category": cat,
113                      "n_cal": n_cal_cat,
114                      "n_test": n_test_cat,
115                      "pooled_coverage": covered_pooled,
116                      "pooled_width": 2 * q_pooled,
117                      "mondrian_coverage": covered_mondrian,
118                      "mondrian_width": 2 * q_cat,
119                  }
120              )
121
122          pooled_coverages = np.array(pooled_coverages)
123          mondrian_coverages = np.array(mondrian_coverages)
124          summary_rows.append(
125              {
126                  "target": target,
127                  "target_coverage": 1 - ALPHA,
128                  "pooled_coverage_min": pooled_coverages.min(),
129                  "pooled_coverage_max": pooled_coverages.max(),
130                  "pooled_coverage_range": pooled_coverages.max() -
pooled_coverages.min(),
131                  "pooled_coverage_std": pooled_coverages.std(),
132                  "mondrian_coverage_min": mondrian_coverages.min(),
133                  "mondrian_coverage_max": mondrian_coverages.max(),
134                  "mondrian_coverage_range": mondrian_coverages.max() -
mondrian_coverages.min(),
135                  "mondrian_coverage_std": mondrian_coverages.std(),
136              }
137          )
138
139      pd.DataFrame(detail_rows).to_csv(DETAIL_PATH, index=False)
140      summary_df = pd.DataFrame(summary_rows)
141      summary_df.to_csv(SUMMARY_PATH, index=False)
142      print(summary_df.to_string(index=False))
143
144
145  if __name__ == "__main__":
146      main()
```

src/uq/repeated_evaluation.py - 30-repeat statistical significance evaluation

```
  """
  Publication-rigor upgrade: repeat the full GP / standard-CP /
Mondrian-CP
  pipeline across many independent (calibration, test) draws, not just
the
  single train_test_split used in Weeks 8-14. Reports mean/std/95% CI of
  coverage and width per target per method, so results are backed by a
  sampling distribution instead of a single point estimate - the main
gap
  between the Week 14-15 comparison and something defensible in a paper.

  Both calibration AND test data are freshly generated DES scenarios on
every
  repeat (never reused across repeats or from the original training
data) -
  this captures both calibration-quantile variance and evaluation-sample
  variance, i.e. it answers "does this coverage number replicate" rather
than
  "was this one split lucky." The underlying surrogate/GP models are
fixed
  per repeat's own GP refit (matching how gp_baseline.py subsamples
fresh
  training data each time) - only standard/Mondrian CP reuse the pre-
trained
  Week 6-7 mean surrogate, exactly as in Weeks 9-12.

  Seed ranges are kept clearly separate from every other script in this
repo
  so nothing here silently overlaps with
training/calibration/test/stress-test
  data used elsewhere:
    calibration draws: sampling seed 1000+r, DES seed_offset 500,000 +
r*10,000
    test draws:         sampling seed 2000+r, DES seed_offset 700,000 +
r*10,000
  """

  import time

  import joblib
  import numpy as np
  import pandas as pd
  from sklearn.gaussian_process import GaussianProcessRegressor
  from sklearn.gaussian_process.kernels import RBF, ConstantKernel,
WhiteKernel
  from sklearn.preprocessing import StandardScaler

  from src.surrogate.generate_training_data import generate
  from src.uq.mondrian_cp import assign_categories, conformal_quantile

  MODELS_DIR = "models"
  DETAIL_PATH = "results/tables/repeated_evaluation_detail.csv"
  SUMMARY_PATH = "results/tables/repeated_evaluation_summary.csv"

  FEATURES = ["n_capacity", "arrival_rate_multiplier"]
  TARGETS = ["n_patients", "mean_wait_minutes", "mean_total_minutes",
"p95_wait_minutes"]
  ALPHA = 0.1
  Z = 1.6448536269514722  # norm.ppf(1 - ALPHA/2), avoids a scipy import
for one constant

  N_REPEATS = 30
```

```
  47  N_CALIBRATION = 1200
  48  N_TEST = 1000
  49  N_GP_TRAIN = 1000
  50  GP_N_RESTARTS = 1  # was 2 in Week 8's one-off gp_baseline.py; halved
here since this
  51  # refits GP fresh on every one of 30 repeats. N_GP_TRAIN is left
unchanged (1000,
  52  # same as Week 8) so GP's data budget stays directly comparable to the
original
  53  # baseline - only the optimizer's restart count (a speed/thoroughness
knob, not
  54  # the comparison basis) is reduced. One restart still escapes the
worst local
  55  # optima for a kernel this simple (2D RBF + white noise).
  56
  57
  58  def gp_predict(target, cal_df, test_df, seed):
  59      rng = np.random.default_rng(seed)
  60      gp_idx = rng.choice(len(cal_df), size=N_GP_TRAIN, replace=False)
  61      scaler = StandardScaler().fit(cal_df.iloc[gp_idx][FEATURES])
  62      X_train_s = scaler.transform(cal_df.iloc[gp_idx][FEATURES])
  63      X_test_s = scaler.transform(test_df[FEATURES])
  64      y_train = cal_df.iloc[gp_idx][target].values
  65
  66      kernel = ConstantKernel(1.0) * RBF(length_scale=[1.0, 1.0]) +
WhiteKernel(1.0)
  67      gp = GaussianProcessRegressor(kernel=kernel, normalize_y=True,
n_restarts_optimizer=GP_N_RESTARTS, random_state=seed)
  68      gp.fit(X_train_s, y_train)
  69      y_pred, y_std = gp.predict(X_test_s, return_std=True)
  70      return y_pred, y_std
  71
  72
  73  def run_repeats(n_repeats=N_REPEATS):
  74      models = {t: joblib.load(f"{MODELS_DIR}/surrogate_{t}.joblib") for
t in TARGETS}
  75      rows = []
  76      t_start = time.time()
  77
  78      for r in range(n_repeats):
  79          cal_df = generate(n_scenarios=N_CALIBRATION, seed=1000 + r,
seed_offset=500_000 + r * 10_000)
  80          test_df = generate(n_scenarios=N_TEST, seed=2000 + r,
seed_offset=700_000 + r * 10_000)
  81
  82          cap_bounds = np.quantile(cal_df["n_capacity"], [1 / 3, 2 / 3])
  83          arr_bounds = np.quantile(cal_df["arrival_rate_multiplier"], [1
/ 3, 2 / 3])
  84          cal_cat = assign_categories(cal_df, cap_bounds, arr_bounds)
  85          test_cat = assign_categories(test_df, cap_bounds, arr_bounds)
  86
  87          for target in TARGETS:
  88              model = models[target]
  89              y_test = test_df[target].values
  90
  91              # GP baseline
  92              gp_pred, gp_std = gp_predict(target, cal_df, test_df,
seed=3000 + r)
  93              gp_lower, gp_upper = gp_pred - Z * gp_std, gp_pred + Z *
gp_std
  94              gp_covered = (y_test >= gp_lower) & (y_test <= gp_upper)
  95              rows.append({"repeat": r, "target": target, "method": "GP
baseline",
  96                           "coverage": gp_covered.mean(), "mean_width":
(gp_upper - gp_lower).mean()})
```

```
 97
 98              # standard CP
 99              yhat_cal = model.predict(cal_df[FEATURES])
100              resid_cal = np.abs(cal_df[target].values - yhat_cal)
101              yhat_test = model.predict(test_df[FEATURES])
102              resid_test = np.abs(y_test - yhat_test)
103
104              q_pooled = conformal_quantile(resid_cal, ALPHA)
105              standard_covered = resid_test <= q_pooled
106              rows.append({"repeat": r, "target": target, "method":
"Standard CP",
107                           "coverage": standard_covered.mean(),
"mean_width": 2 * q_pooled})
108
109              # Mondrian CP
110              mondrian_covered = np.zeros(len(test_df), dtype=bool)
111              mondrian_widths = np.zeros(len(test_df))
112              for cat in set(cal_cat):
113                  cal_mask = cal_cat == cat
114                  test_mask = test_cat == cat
115                  if test_mask.sum() == 0 or cal_mask.sum() == 0:
116                      continue
117                  q_cat = conformal_quantile(resid_cal[cal_mask], ALPHA)
118                  mondrian_covered[test_mask] = resid_test[test_mask] <=
q_cat
119                  mondrian_widths[test_mask] = 2 * q_cat
120              rows.append({"repeat": r, "target": target, "method":
"Mondrian CP",
121                           "coverage": mondrian_covered.mean(),
"mean_width": mondrian_widths.mean()})
122
123          elapsed = time.time() - t_start
124          print(f"repeat {r + 1}/{n_repeats} done ({elapsed:.0f}s
elapsed)")
125
126      return pd.DataFrame(rows)
127
128
129  def summarize(df):
130      def ci95(x):
131          n = len(x)
132          return 1.96 * x.std(ddof=1) / np.sqrt(n)
133
134      summary = df.groupby(["target", "method"]).agg(
135          n_repeats=("coverage", "size"),
136          coverage_mean=("coverage", "mean"),
137          coverage_std=("coverage", "std"),
138          coverage_ci95=("coverage", ci95),
139          width_mean=("mean_width", "mean"),
140          width_std=("mean_width", "std"),
141          width_ci95=("mean_width", ci95),
142      ).reset_index()
143      return summary
144
145
146  def main():
147      detail = run_repeats()
148      detail.to_csv(DETAIL_PATH, index=False)
149      summary = summarize(detail)
150      summary.to_csv(SUMMARY_PATH, index=False)
151      print(summary.to_string(index=False))
152
153
154  if __name__ == "__main__":
155      main()
```

src/surrogate/train_quantile_surrogates.py - Quantile regressors for CQR

```
"""
Publication-rigor upgrade: train quantile-regression surrogates for CQR
(Conformalized Quantile Regression), a stronger CP baseline than symmetric/
asymmetric split CP for skewed targets (p95_wait_minutes especially).

CQR calibrates a nonconformity score of max(qlo(x)-y, y-qhi(x)) instead of
|y-yhat| - the interval adapts to the quantile regressors' own estimate of
local spread, rather than using one fixed width (symmetric CP) or two fixed
offsets (asymmetric CP) everywhere.

Same train/test split (random_state=42) as train_surrogate.py, so the
quantile regressors see exactly the same training data as the original mean
regressor - a fair comparison of CP methods, not a different setup dressed
up as one. Trained once, like the mean surrogate; only calibration/test data
varies across the repeated-evaluation runs in repeated_evaluation.py.
"""

import joblib
import pandas as pd
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.metrics import mean_pinball_loss
from sklearn.model_selection import train_test_split

DATA_PATH = "data/processed/surrogate_training_data.parquet"
MODELS_DIR = "models"
METRICS_PATH = "results/tables/quantile_surrogate_metrics.csv"

FEATURES = ["n_capacity", "arrival_rate_multiplier"]
TARGETS = ["n_patients", "mean_wait_minutes", "mean_total_minutes", "p95_wait_minutes"]
ALPHA = 0.1
Q_LO, Q_HI = ALPHA / 2, 1 - ALPHA / 2


def main():
    df = pd.read_parquet(DATA_PATH)
    X_train, X_test, train_idx, test_idx = train_test_split(
        df[FEATURES], df.index, test_size=0.2, random_state=42
    )

    metrics = []
    for target in TARGETS:
        y_train = df.loc[train_idx, target]
        y_test = df.loc[test_idx, target]

        for label, q in [("qlo", Q_LO), ("qhi", Q_HI)]:
            model = HistGradientBoostingRegressor(loss="quantile", quantile=q, random_state=42)
```

```
  47              model.fit(X_train, y_train)
  48              y_pred = model.predict(X_test)
  49              pinball = mean_pinball_loss(y_test, y_pred, alpha=q)
  50              metrics.append({"target": target, "quantile_label": label,
"quantile": q, "pinball_loss": pinball})
  51              joblib.dump(model,
f"{MODELS_DIR}/surrogate_{target}_{label}.joblib")
  52
  53          # sanity check: how often does the raw (uncalibrated) [qlo,
qhi] interval cover y_test?
  54          qlo_model =
joblib.load(f"{MODELS_DIR}/surrogate_{target}_qlo.joblib")
  55          qhi_model =
joblib.load(f"{MODELS_DIR}/surrogate_{target}_qhi.joblib")
  56          qlo_pred = qlo_model.predict(X_test)
  57          qhi_pred = qhi_model.predict(X_test)
  58          raw_coverage = ((y_test >= qlo_pred) & (y_test <=
qhi_pred)).mean()
  59          metrics.append({"target": target, "quantile_label":
"raw_interval_coverage",
  60                          "quantile": None, "pinball_loss":
raw_coverage})
  61
  62      metrics_df = pd.DataFrame(metrics)
  63      metrics_df.to_csv(METRICS_PATH, index=False)
  64      print(metrics_df.to_string(index=False))
  65
  66
  67  if __name__ == "__main__":
  68      main()
```

src/uq/repeated_evaluation_cqr.py - CQR and Mondrian-CQR, paired to the same 30 draws

```
   1  """
   2  Publication-rigor upgrade, part 2: CQR (Conformalized Quantile
Regression)
   3  and Mondrian-CQR, evaluated with the same 30-repeat rigor as
   4  repeated_evaluation.py (GP / standard CP / Mondrian CP).
   5
   6  CQR's nonconformity score is max(qlo(x)-y, y-qhi(x)) instead of |y-
yhat| -
   7  the interval adapts to the quantile regressors' own estimate of local
   8  spread (from train_quantile_surrogates.py), rather than a single fixed
   9  width (standard CP) or fixed per-category width (Mondrian CP) built
from a
  10  mean predictor. Mondrian-CQR combines both ideas: per-category
calibration
  11  of an already-adaptive nonconformity score - the most sophisticated
method
  12  in the comparison.
  13
  14  Uses the exact same 30 (calibration, test) draws as
repeated_evaluation.py
  15  (identical seed formula: calibration sampling seed 1000+r / DES offset
  16  500,000+r*10,000, test sampling seed 2000+r / DES offset
700,000+r*10,000).
  17  generate() is deterministic given a seed, so this reproduces byte-
identical
  18  scenario data without needing to save/reload it - meaning CQR's
```

```
results are
  19  directly, validly paired with the GP/standard/Mondrian results from
part 1
  20  for significance testing, not a separately-drawn comparison.
  21
  22  No GP here, which is why this runs in ~15-20 min instead of ~82 min -
GP
  23  refitting was almost the entire cost of part 1, and CQR doesn't need
it.
  24  """
  25
  26  import time
  27
  28  import joblib
  29  import numpy as np
  30  import pandas as pd
  31
  32  from src.surrogate.generate_training_data import generate
  33  from src.uq.mondrian_cp import assign_categories, conformal_quantile
  34
  35  MODELS_DIR = "models"
  36  DETAIL_PATH = "results/tables/repeated_evaluation_cqr_detail.csv"
  37  SUMMARY_PATH = "results/tables/repeated_evaluation_cqr_summary.csv"
  38
  39  FEATURES = ["n_capacity", "arrival_rate_multiplier"]
  40  TARGETS = ["n_patients", "mean_wait_minutes", "mean_total_minutes",
"p95_wait_minutes"]
  41  ALPHA = 0.1
  42
  43  N_REPEATS = 30
  44  N_CALIBRATION = 1200
  45  N_TEST = 1000
  46
  47
  48  def run_repeats(n_repeats=N_REPEATS):
  49      qlo_models = {t:
joblib.load(f"{MODELS_DIR}/surrogate_{t}_qlo.joblib") for t in TARGETS}
  50      qhi_models = {t:
joblib.load(f"{MODELS_DIR}/surrogate_{t}_qhi.joblib") for t in TARGETS}
  51
  52      rows = []
  53      t_start = time.time()
  54
  55      for r in range(n_repeats):
  56          # identical seeds to repeated_evaluation.py -> identical (cal,
test) draws
  57          cal_df = generate(n_scenarios=N_CALIBRATION, seed=1000 + r,
seed_offset=500_000 + r * 10_000)
  58          test_df = generate(n_scenarios=N_TEST, seed=2000 + r,
seed_offset=700_000 + r * 10_000)
  59
  60          cap_bounds = np.quantile(cal_df["n_capacity"], [1 / 3, 2 / 3])
  61          arr_bounds = np.quantile(cal_df["arrival_rate_multiplier"], [1
/ 3, 2 / 3])
  62          cal_cat = assign_categories(cal_df, cap_bounds, arr_bounds)
  63          test_cat = assign_categories(test_df, cap_bounds, arr_bounds)
  64
  65          for target in TARGETS:
  66              qlo_model, qhi_model = qlo_models[target],
qhi_models[target]
  67              y_cal = cal_df[target].values
  68              y_test = test_df[target].values
  69
  70              qlo_cal = qlo_model.predict(cal_df[FEATURES])
  71              qhi_cal = qhi_model.predict(cal_df[FEATURES])
```

```
  72              cqr_score_cal = np.maximum(qlo_cal - y_cal, y_cal -
qhi_cal)
  73
  74              qlo_test = qlo_model.predict(test_df[FEATURES])
  75              qhi_test = qhi_model.predict(test_df[FEATURES])
  76
  77              # pooled CQR
  78              q_pooled = conformal_quantile(cqr_score_cal, ALPHA)
  79              lower = qlo_test - q_pooled
  80              upper = qhi_test + q_pooled
  81              covered = (y_test >= lower) & (y_test <= upper)
  82              rows.append({"repeat": r, "target": target, "method":
"CQR",
  83                           "coverage": covered.mean(), "mean_width":
(upper - lower).mean()})
  84
  85              # Mondrian CQR (per-category calibration of the same
adaptive score)
  86              mondrian_covered = np.zeros(len(test_df), dtype=bool)
  87              mondrian_widths = np.zeros(len(test_df))
  88              for cat in set(cal_cat):
  89                  cal_mask = cal_cat == cat
  90                  test_mask = test_cat == cat
  91                  if test_mask.sum() == 0 or cal_mask.sum() == 0:
  92                      continue
  93                  q_cat = conformal_quantile(cqr_score_cal[cal_mask],
ALPHA)
  94                  lo_c = qlo_test[test_mask] - q_cat
  95                  hi_c = qhi_test[test_mask] + q_cat
  96                  mondrian_covered[test_mask] = (y_test[test_mask] >=
lo_c) & (y_test[test_mask] <= hi_c)
  97                  mondrian_widths[test_mask] = hi_c - lo_c
  98              rows.append({"repeat": r, "target": target, "method":
"Mondrian CQR",
  99                           "coverage": mondrian_covered.mean(),
"mean_width": mondrian_widths.mean()})
 100
 101          elapsed = time.time() - t_start
 102          print(f"repeat {r + 1}/{n_repeats} done ({elapsed:.0f}s
elapsed)")
 103
 104      return pd.DataFrame(rows)
 105
 106
 107  def summarize(df):
 108      def ci95(x):
 109          n = len(x)
 110          return 1.96 * x.std(ddof=1) / np.sqrt(n)
 111
 112      return df.groupby(["target", "method"]).agg(
 113          n_repeats=("coverage", "size"),
 114          coverage_mean=("coverage", "mean"),
 115          coverage_std=("coverage", "std"),
 116          coverage_ci95=("coverage", ci95),
 117          width_mean=("mean_width", "mean"),
 118          width_std=("mean_width", "std"),
 119          width_ci95=("mean_width", ci95),
 120      ).reset_index()
 121
 122
 123  def main():
 124      detail = run_repeats()
 125      detail.to_csv(DETAIL_PATH, index=False)
 126      summary = summarize(detail)
 127      summary.to_csv(SUMMARY_PATH, index=False)
```

```
    print(summary.to_string(index=False))


if __name__ == "__main__":
    main()
```

src/uq/full_comparison.py - Single-split GP/Standard-CP/Mondrian-CP comparison

```
"""
Week 14-15: full comparison of GP baseline vs. standard CP vs. Mondrian CP -
coverage, interval width, computation time.

Coverage/width for GP and standard CP come straight from their existing
result tables (gp_baseline_metrics.csv, standard_cp_metrics.csv - symmetric
variant, for a fair like-for-like comparison against GP's single interval
and Mondrian's single per-category interval). Mondrian's overall (marginal)
coverage/width isn't in mondrian_cp_summary.csv - that file only has
per-category spread - so it's computed here by re-applying the per-category
quantiles to the full test set and aggregating, the same way GP/standard CP
report one marginal number.

Computation time is measured on a like-for-like basis: GP's fit_seconds is
its own model-fitting cost (already recorded in Week 8). Standard/Mondrian
CP don't fit a model - they wrap the already-trained surrogate - so their
fair "computation time" is their own calibration cost: computing the
empirical residual quantile(s) from calibration data. That's timed fresh
here since it wasn't recorded in Week 9-12.
"""

import time

import joblib
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split

from src.uq.mondrian_cp import assign_categories, conformal_quantile

TRAIN_DATA_PATH = "data/processed/surrogate_training_data.parquet"
CALIBRATION_DATA_PATH = "data/processed/cp_calibration_data.parquet"
MODELS_DIR = "models"
OUT_TABLE = "results/tables/full_comparison.csv"
OUT_FIGURES = "results/figures"

FEATURES = ["n_capacity", "arrival_rate_multiplier"]
TARGETS = ["n_patients", "mean_wait_minutes", "mean_total_minutes", "p95_wait_minutes"]
ALPHA = 0.1
```

```
  40
  41
  42  def mondrian_overall(target, cal_df, test_df, cap_bounds, arr_bounds,
cal_cat, test_cat):
  43      model = joblib.load(f"{MODELS_DIR}/surrogate_{target}.joblib")
  44      model.predict(cal_df[FEATURES])  # warm-up: exclude first-call
JIT/thread-pool overhead from the timing
  45
  46      t0 = time.time()
  47      yhat_cal = model.predict(cal_df[FEATURES])
  48      resid_cal = np.abs(cal_df[target].values - yhat_cal)
  49      q_by_cat = {cat: conformal_quantile(resid_cal[cal_cat == cat],
ALPHA) for cat in set(cal_cat)}
  50      calibration_seconds = time.time() - t0
  51
  52      yhat_test = model.predict(test_df[FEATURES])
  53      y_test = test_df[target].values
  54      widths = np.array([2 * q_by_cat[c] for c in test_cat])
  55      covered = np.abs(y_test - yhat_test) <= widths / 2
  56
  57      return covered.mean(), widths.mean(), calibration_seconds
  58
  59
  60  def standard_calibration_time(target, cal_df):
  61      model = joblib.load(f"{MODELS_DIR}/surrogate_{target}.joblib")
  62      model.predict(cal_df[FEATURES])  # warm-up: exclude first-call
JIT/thread-pool overhead from the timing
  63      t0 = time.time()
  64      yhat_cal = model.predict(cal_df[FEATURES])
  65      resid_cal = np.abs(cal_df[target].values - yhat_cal)
  66      conformal_quantile(resid_cal, ALPHA)
  67      return time.time() - t0
  68
  69
  70  def main():
  71      gp_df =
pd.read_csv("results/tables/gp_baseline_metrics.csv").set_index("target")
  72      cp_df =
pd.read_csv("results/tables/standard_cp_metrics.csv").set_index("target")
  73
  74      df = pd.read_parquet(TRAIN_DATA_PATH)
  75      _, X_test, _, test_idx = train_test_split(df[FEATURES], df.index,
test_size=0.2, random_state=42)
  76      test_df = df.loc[test_idx]
  77      cal_df = pd.read_parquet(CALIBRATION_DATA_PATH)
  78
  79      cap_bounds = np.quantile(cal_df["n_capacity"], [1 / 3, 2 / 3])
  80      arr_bounds = np.quantile(cal_df["arrival_rate_multiplier"], [1 /
3, 2 / 3])
  81      cal_cat = assign_categories(cal_df, cap_bounds, arr_bounds)
  82      test_cat = assign_categories(test_df, cap_bounds, arr_bounds)
  83
  84      rows = []
  85      for target in TARGETS:
  86          rows.append(
  87              {
  88                  "target": target,
  89                  "method": "GP baseline",
  90                  "coverage": gp_df.loc[target, "empirical_coverage"],
  91                  "mean_width": gp_df.loc[target,
"mean_interval_width"],
  92                  "computation_seconds": gp_df.loc[target,
"fit_seconds"] + gp_df.loc[target, "predict_seconds"],
  93              }
  94          )
```

```
        rows.append(
            {
                "target": target,
                "method": "Standard CP",
                "coverage": cp_df.loc[target, "symmetric_coverage"],
                "mean_width": cp_df.loc[target,
"symmetric_mean_width"],
                "computation_seconds":
standard_calibration_time(target, cal_df),
            }
        )
        mondrian_cov, mondrian_width, mondrian_seconds =
mondrian_overall(
            target, cal_df, test_df, cap_bounds, arr_bounds, cal_cat,
test_cat
        )
        rows.append(
            {
                "target": target,
                "method": "Mondrian CP",
                "coverage": mondrian_cov,
                "mean_width": mondrian_width,
                "computation_seconds": mondrian_seconds,
            }
        )

    out_df = pd.DataFrame(rows)
    out_df.to_csv(OUT_TABLE, index=False)
    print(out_df.to_string(index=False))
    make_charts(out_df)


def make_charts(df):
    import matplotlib.pyplot as plt

    methods = ["GP baseline", "Standard CP", "Mondrian CP"]
    colors = {"GP baseline": "#888888", "Standard CP": "#4C72B0",
"Mondrian CP": "#55A868"}

    fig, axes = plt.subplots(1, 3, figsize=(15, 4.5))

    ax = axes[0]
    x = np.arange(len(TARGETS))
    width = 0.25
    for i, m in enumerate(methods):
        vals = [df[(df.target == t) & (df.method ==
m)]["coverage"].iloc[0] * 100 for t in TARGETS]
        ax.bar(x + (i - 1) * width, vals, width, label=m,
color=colors[m])
    ax.axhline(90, color="black", linestyle="--", linewidth=1,
label="90% target")
    ax.set_xticks(x)
    ax.set_xticklabels(TARGETS, rotation=30, ha="right", fontsize=8)
    ax.set_ylabel("Empirical coverage (%)")
    ax.set_title("Coverage")
    ax.legend(fontsize=8)

    ax = axes[1]
    for i, m in enumerate(methods):
        vals = [df[(df.target == t) & (df.method ==
m)]["mean_width"].iloc[0] for t in TARGETS]
        ax.bar(x + (i - 1) * width, vals, width, label=m,
color=colors[m])
    ax.set_xticks(x)
    ax.set_xticklabels(TARGETS, rotation=30, ha="right", fontsize=8)
```

```
 150      ax.set_ylabel("Mean interval width")
 151      ax.set_title("Interval width")
 152      ax.legend(fontsize=8)
 153
 154      ax = axes[2]
 155      comp_times = [df[df.method == m]["computation_seconds"].mean() for
m in methods]
 156      ax.bar(methods, comp_times, color=[colors[m] for m in methods])
 157      ax.set_ylabel("Mean computation time (s, log scale)")
 158      ax.set_yscale("log")
 159      ax.set_title("Computation time")
 160
 161      fig.tight_layout()
 162      fig.savefig(f"{OUT_FIGURES}/full_comparison.png", dpi=150)
 163      print(f"Saved {OUT_FIGURES}/full_comparison.png")
 164
 165
 166  if __name__ == "__main__":
 167      main()
```

src/uq/publication_comparison_chart.py - 5-method, 30-repeat comparison chart

```
   1  """
   2  Publication-rigor upgrade, final: the authoritative 5-method
comparison
   3  chart, built from the 30-repeat statistical results (parts 1-2)
instead of
   4  the single-split point estimates full_comparison.py used. Combines
   5  repeated_evaluation_summary.csv (GP, Standard CP, Mondrian CP) and
   6  repeated_evaluation_cqr_summary.csv (CQR, Mondrian CQR) - same 30
seeds
   7  underlie all five, so error bars are directly comparable across
methods.
   8
   9  Error bars are 95% CI on the mean (coverage_ci95/width_ci95 columns) -
  10  tight enough (a few tenths of a point on coverage) that they read
almost as
  11  single dots at this scale, which is itself the point: these numbers
are not
  12  single-split noise, they're stable estimates.
  13  """
  14
  15  import matplotlib.pyplot as plt
  16  import numpy as np
  17  import pandas as pd
  18
  19  TARGETS = ["n_patients", "mean_wait_minutes", "mean_total_minutes",
"p95_wait_minutes"]
  20  METHODS = ["GP baseline", "Standard CP", "Mondrian CP", "CQR",
"Mondrian CQR"]
  21  COLORS = {
  22      "GP baseline": "#888888",
  23      "Standard CP": "#4C72B0",
  24      "Mondrian CP": "#55A868",
  25      "CQR": "#C44E52",
  26      "Mondrian CQR": "#8172B2",
  27  }
  28  OUT_PATH = "results/figures/publication_comparison.png"
  29
```

```
  30
  31  def main():
  32      df1 =
pd.read_csv("results/tables/repeated_evaluation_summary.csv")
  33      df2 =
pd.read_csv("results/tables/repeated_evaluation_cqr_summary.csv")
  34      df = pd.concat([df1, df2], ignore_index=True)
  35
  36      fig, axes = plt.subplots(1, 2, figsize=(13, 5))
  37      x = np.arange(len(TARGETS))
  38      width = 0.16
  39
  40      ax = axes[0]
  41      for i, m in enumerate(METHODS):
  42          vals, errs = [], []
  43          for t in TARGETS:
  44              row = df[(df.target == t) & (df.method == m)].iloc[0]
  45              vals.append(row["coverage_mean"] * 100)
  46              errs.append(row["coverage_ci95"] * 100)
  47          ax.bar(x + (i - 2) * width, vals, width, yerr=errs, capsize=2,
label=m, color=COLORS[m])
  48      ax.axhline(90, color="black", linestyle="--", linewidth=1,
label="90% target")
  49      ax.set_xticks(x)
  50      ax.set_xticklabels(TARGETS, rotation=25, ha="right", fontsize=8)
  51      ax.set_ylabel("Empirical coverage (%), mean ± 95% CI")
  52      ax.set_title("Coverage (30 independent repeats)")
  53      ax.set_ylim(70, 100)
  54      ax.legend(fontsize=7, loc="lower right")
  55
  56      ax = axes[1]
  57      for i, m in enumerate(METHODS):
  58          vals, errs = [], []
  59          for t in TARGETS:
  60              row = df[(df.target == t) & (df.method == m)].iloc[0]
  61              vals.append(row["width_mean"])
  62              errs.append(row["width_ci95"])
  63          ax.bar(x + (i - 2) * width, vals, width, yerr=errs, capsize=2,
label=m, color=COLORS[m])
  64      ax.set_xticks(x)
  65      ax.set_xticklabels(TARGETS, rotation=25, ha="right", fontsize=8)
  66      ax.set_ylabel("Mean interval width, mean ± 95% CI")
  67      ax.set_title("Interval width (30 independent repeats)")
  68      ax.legend(fontsize=7)
  69
  70      fig.tight_layout()
  71      fig.savefig(OUT_PATH, dpi=150)
  72      print(f"Saved {OUT_PATH}")
  73
  74
  75  if __name__ == "__main__":
  76      main()
```

src/generalization/extract_dept_b.py - Department B distribution extraction

```
   1  """
   2  Publication-rigor upgrade, part 4: generalizability check on a second
   3  department.
   4
```

```
   5  Reuses extract_distributions.py's functions unchanged (department
filter,
   6  arrival/ESI distribution logic) but points at Department B (166,497
visits,
   7  the second-largest site: 1 academic + 2 community EDs in the dataset,
   8  Department A already used is the academic site) and writes to
   9  results/tables/dept_b/ instead of results/tables/ - Department A's
files
  10  are never touched.
  11
  12  Service time params are literature-derived, not department-specific,
so
  13  they're identical to Department A's by construction - copied here
rather
  14  than re-derived.
  15  """
  16
  17  import pandas as pd
  18
  19  from src.utils.extract_distributions import (
  20      SERVICE_TIME_PARAMS_BY_ESI,
  21      STUDY_PERIOD_DAYS,
  22      arrival_distribution,
  23      esi_distribution,
  24      load_data,
  25  )
  26
  27  DEPARTMENT = "B"
  28  OUT_TABLES = "results/tables/dept_b"
  29
  30
  31  def main():
  32      df = load_data(department=DEPARTMENT)
  33      visits_per_day = len(df) / STUDY_PERIOD_DAYS
  34
  35      by_hour_bin, by_day, by_month = arrival_distribution(df)
  36      esi_mix = esi_distribution(df)
  37
  38      by_hour_bin.to_csv(f"{OUT_TABLES}/arrivals_by_hour_bin.csv",
header=["count"])
  39      by_day.to_csv(f"{OUT_TABLES}/arrivals_by_day.csv",
header=["count"])
  40      by_month.to_csv(f"{OUT_TABLES}/arrivals_by_month.csv",
header=["count"])
  41      esi_mix.to_csv(f"{OUT_TABLES}/esi_mix.csv", header=["proportion"])
  42
  43      service_time_df = pd.DataFrame(
  44          SERVICE_TIME_PARAMS_BY_ESI, index=["mean_minutes",
"sd_minutes"]
  45      ).T
  46      service_time_df.index.name = "esi"
  47      service_time_df.to_csv(f"{OUT_TABLES}/service_time_params.csv")
  48
  49      pd.Series(
  50          {"department": DEPARTMENT, "study_period_days":
STUDY_PERIOD_DAYS,
  51           "total_visits": len(df), "visits_per_day": visits_per_day},
  52      ).to_csv(f"{OUT_TABLES}/calibration_meta.csv", header=["value"])
  53
  54      print(f"Department {DEPARTMENT}: {len(df)} visits over
{STUDY_PERIOD_DAYS} days"
  55            f" = {visits_per_day:.1f} visits/day\n")
  56      print("Arrivals by 4h bin:\n", by_hour_bin, "\n")
  57      print("ESI mix:\n", esi_mix, "\n")
  58
```

```
59
60  if __name__ == "__main__":
61      main()
```

src/generalization/generate_dept_b_data.py - Department B DES scenario generation

```
  1  """
  2  Publication-rigor upgrade, part 4 (continued): generate Department B's
  3  training and CP-calibration scenario sweeps.
  4
  5  n_capacity default/range recalibrated for B's own offered load, not
copied
  6  from department A's absolute numbers - B's real volume is roughly half
A's
  7  (133.4 vs. 258.2 visits/day) and its ESI mix skews toward lower acuity
  8  (23.1% ESI-2 vs. A's 37.9%, 26.8% ESI-4 vs. A's 16.4%), so a "capacity
of
  9  30" would be wildly over-provisioned and produce uninteresting,
barely-
 10  congested dynamics. Computed the same way A's default was: offered
load
 11  (Erlangs) = arrival_rate * weighted mean service time. B: ~10.4
erlangs
 12  average, ~16.1 peak (vs. A's ~22 / ~34) - roughly half, as expected
given
 13  roughly half the volume. Default n_capacity=14 sits at the same
relative
 14  position between average and peak load as A's 30 does for A's own
numbers.
 15  Range (7,22) mirrors A's ~0.5x-1.5x-of-default pattern (A: 15-45
around 30).
 16
 17  arrival_rate_multiplier range is unchanged (0.8-1.3) - it's already a
 18  relative multiplier on top of whatever tables_dir's own calibrated
rate is,
 19  so it doesn't need department-specific rescaling.
 20
 21  Same sizes as department A (5000 training scenarios, 1200 calibration)
for
 22  a like-for-like comparison; same seed-range discipline (calibration
draws
 23  use a different sampling seed and a large DES-seed offset from
training,
 24  consistent with generate_calibration_data.py).
 25  """
 26
 27  from src.surrogate.generate_training_data import generate
 28
 29  TABLES_DIR = "results/tables/dept_b"
 30  N_CAPACITY_RANGE = (7, 22)
 31  ARRIVAL_MULTIPLIER_RANGE = (0.8, 1.3)
 32
 33  TRAINING_OUT = "data/processed/dept_b/surrogate_training_data.parquet"
 34  CALIBRATION_OUT = "data/processed/dept_b/cp_calibration_data.parquet"
 35
 36
 37  def main():
 38      train_df = generate(
 39          n_scenarios=5000, seed=0, seed_offset=0,
```

```
        n_capacity_range=N_CAPACITY_RANGE,
arrival_multiplier_range=ARRIVAL_MULTIPLIER_RANGE,
        tables_dir=TABLES_DIR,
    )
    train_df.to_parquet(TRAINING_OUT, index=False)
    print(f"Generated {len(train_df)} training scenarios ->
{TRAINING_OUT}")
    print(train_df.describe())

    # different sampling seed + large DES-seed offset -> independent
of training data,
    # same discipline as generate_calibration_data.py for department A
    cal_df = generate(
        n_scenarios=1200, seed=1, seed_offset=100_000,
        n_capacity_range=N_CAPACITY_RANGE,
arrival_multiplier_range=ARRIVAL_MULTIPLIER_RANGE,
        tables_dir=TABLES_DIR,
    )
    cal_df.to_parquet(CALIBRATION_OUT, index=False)
    print(f"\nGenerated {len(cal_df)} calibration scenarios ->
{CALIBRATION_OUT}")


if __name__ == "__main__":
    main()
```

src/generalization/train_dept_b_surrogate.py - Department B surrogate training

```
"""
Publication-rigor upgrade, part 4 (continued): train Department B's
surrogate. Same architecture and train/test split logic as
train_surrogate.py (gradient boosting, random_state=42, 80/20 split) -
only
the data source and output paths differ, so this is a fair
architecture
comparison, not a differently-tuned model.
"""

import joblib
import pandas as pd
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error,
r2_score
from sklearn.model_selection import train_test_split

DATA_PATH = "data/processed/dept_b/surrogate_training_data.parquet"
MODELS_DIR = "models/dept_b"
METRICS_PATH = "results/tables/dept_b/surrogate_metrics.csv"

FEATURES = ["n_capacity", "arrival_rate_multiplier"]
TARGETS = ["n_patients", "mean_wait_minutes", "mean_total_minutes",
"p95_wait_minutes"]


def main():
    df = pd.read_parquet(DATA_PATH)
    X_train, X_test, train_idx, test_idx = train_test_split(
        df[FEATURES], df.index, test_size=0.2, random_state=42
    )
```

```
28
29      metrics = []
30      for target in TARGETS:
31          y_train = df.loc[train_idx, target]
32          y_test = df.loc[test_idx, target]
33
34          model = HistGradientBoostingRegressor(random_state=42)
35          model.fit(X_train, y_train)
36          y_pred = model.predict(X_test)
37
38          metrics.append(
39              {
40                  "target": target,
41                  "mae": mean_absolute_error(y_test, y_pred),
42                  "rmse": mean_squared_error(y_test, y_pred) ** 0.5,
43                  "r2": r2_score(y_test, y_pred),
44              }
45          )
46          joblib.dump(model, f"{MODELS_DIR}/surrogate_{target}.joblib")
47
48      metrics_df = pd.DataFrame(metrics)
49      metrics_df.to_csv(METRICS_PATH, index=False)
50      print(metrics_df)
51
52
53  if __name__ == "__main__":
54      main()
```

src/generalization/validate_dept_b_des.py - Department B DES validation

```
1   """
2   Publication-rigor upgrade, part 4 (continued): validate Department B's
DES
3   calibration against B's real aggregated stats, same approach as
4   des/validate.py for department A (200 simulated days, compare mean
5   simulated patients/day to the real rate).
6   """
7
8   import pandas as pd
9
10  from src.des.er_simulation import run_scenario
11
12  N_DAYS = 200
13  N_CAPACITY = 14
14  TABLES_DIR = "results/tables/dept_b"
15
16
17  def main():
18      results = [run_scenario(n_capacity=N_CAPACITY, seed=s,
tables_dir=TABLES_DIR) for s in range(N_DAYS)]
19      df = pd.DataFrame(results)
20
21      meta = pd.read_csv(f"{TABLES_DIR}/calibration_meta.csv",
index_col=0)
22      real_visits_per_day = float(meta.loc["visits_per_day", "value"])
23      sim_visits_per_day = df["n_patients"].mean()
24
25      summary = pd.Series(
26          {
```

```
  27              "n_days_simulated": N_DAYS,
  28              "n_capacity": N_CAPACITY,
  29              "real_visits_per_day": real_visits_per_day,
  30              "sim_mean_visits_per_day": sim_visits_per_day,
  31              "sim_std_visits_per_day": df["n_patients"].std(),
  32              "ratio_sim_over_real": sim_visits_per_day /
real_visits_per_day,
  33              "sim_mean_wait_minutes": df["mean_wait_minutes"].mean(),
  34              "sim_mean_total_minutes": df["mean_total_minutes"].mean(),
  35              "sim_mean_p95_wait_minutes":
df["p95_wait_minutes"].mean(),
  36          }
  37      )
  38      summary.to_csv(f"{TABLES_DIR}/des_validation.csv",
header=["value"])
  39      print(summary)
  40
  41
  42  if __name__ == "__main__":
  43      main()
```

src/generalization/evaluate_dept_b_cp.py - Department B standard-vs-Mondrian CP comparison

```
   1  """
   2  Publication-rigor upgrade, part 4 (continued): the actual
generalizability
   3  question - does Department A's core finding (Mondrian CP closes a real
   4  marginal-vs-conditional coverage gap in the worst staffing/arrival
category)
   5  replicate at a second, structurally different site?
   6
   7  Department B is not just a scaled-down copy of A: real volume is
roughly
   8  half (133.4 vs. 258.2 visits/day) AND the real ESI acuity mix is
   9  meaningfully different (23.1% ESI-2 vs. A's 37.9%, 26.8% ESI-4 vs. A's
  10  16.4% - B skews toward lower-acuity visits, consistent with a
community ED
  11  vs. A's academic site). So this is a genuine second data point, not a
  12  trivial rescaling of the same result.
  13
  14  Same standard CP (symmetric) + Mondrian CP logic as standard_cp.py /
  15  mondrian_cp.py, same alpha=0.1, applied to B's own
surrogate/calibration/
  16  test data. Single split here (not the 30-repeat treatment from part 1)
-
  17  proportionate scope for a generalizability check, not a full re-run of
  18  every rigor dimension for a second site.
  19  """
  20
  21  import joblib
  22  import numpy as np
  23  import pandas as pd
  24  from sklearn.model_selection import train_test_split
  25
  26  from src.uq.mondrian_cp import assign_categories, conformal_quantile
  27
  28  TRAIN_DATA_PATH =
"data/processed/dept_b/surrogate_training_data.parquet"
  29  CALIBRATION_DATA_PATH =
```

```
"data/processed/dept_b/cp_calibration_data.parquet"
  30  MODELS_DIR = "models/dept_b"
  31  DETAIL_PATH = "results/tables/dept_b/mondrian_cp_detail.csv"
  32  SUMMARY_PATH =
"results/tables/dept_b/standard_vs_mondrian_summary.csv"
  33
  34  FEATURES = ["n_capacity", "arrival_rate_multiplier"]
  35  TARGETS = ["n_patients", "mean_wait_minutes", "mean_total_minutes",
"p95_wait_minutes"]
  36  ALPHA = 0.1
  37
  38
  39  def main():
  40      df = pd.read_parquet(TRAIN_DATA_PATH)
  41      _, X_test, _, test_idx = train_test_split(df[FEATURES], df.index,
test_size=0.2, random_state=42)
  42      test_df = df.loc[test_idx]
  43
  44      cal_df = pd.read_parquet(CALIBRATION_DATA_PATH)
  45      cap_bounds = np.quantile(cal_df["n_capacity"], [1 / 3, 2 / 3])
  46      arr_bounds = np.quantile(cal_df["arrival_rate_multiplier"], [1 /
3, 2 / 3])
  47      cal_cat = assign_categories(cal_df, cap_bounds, arr_bounds)
  48      test_cat = assign_categories(test_df, cap_bounds, arr_bounds)
  49      categories = sorted(set(cal_cat))
  50
  51      detail_rows = []
  52      summary_rows = []
  53      for target in TARGETS:
  54          model = joblib.load(f"{MODELS_DIR}/surrogate_{target}.joblib")
  55
  56          yhat_cal = model.predict(cal_df[FEATURES])
  57          y_cal = cal_df[target].values
  58          abs_resid_cal = np.abs(y_cal - yhat_cal)
  59
  60          yhat_test = model.predict(X_test)
  61          y_test = test_df[target].values
  62          abs_resid_test = np.abs(y_test - yhat_test)
  63
  64          q_pooled = conformal_quantile(abs_resid_cal, ALPHA)
  65          standard_covered_all = abs_resid_test <= q_pooled
  66
  67          pooled_coverages, mondrian_coverages = [], []
  68          for cat in categories:
  69              cal_mask = cal_cat == cat
  70              test_mask = test_cat == cat
  71              n_cal_cat = cal_mask.sum()
  72              n_test_cat = test_mask.sum()
  73              if n_test_cat == 0:
  74                  continue
  75
  76              q_cat = conformal_quantile(abs_resid_cal[cal_mask], ALPHA)
  77              covered_pooled = (abs_resid_test[test_mask] <=
q_pooled).mean()
  78              covered_mondrian = (abs_resid_test[test_mask] <=
q_cat).mean()
  79
  80              pooled_coverages.append(covered_pooled)
  81              mondrian_coverages.append(covered_mondrian)
  82
  83              detail_rows.append(
  84                  {
  85                      "target": target,
  86                      "category": cat,
  87                      "n_cal": n_cal_cat,
```

```
                    "n_test": n_test_cat,
                    "pooled_coverage": covered_pooled,
                    "pooled_width": 2 * q_pooled,
                    "mondrian_coverage": covered_mondrian,
                    "mondrian_width": 2 * q_cat,
                }
            )

        pooled_coverages = np.array(pooled_coverages)
        mondrian_coverages = np.array(mondrian_coverages)

        mondrian_covered_all = np.zeros(len(test_df), dtype=bool)
        for cat in categories:
            test_mask = test_cat == cat
            q_cat = conformal_quantile(abs_resid_cal[cal_cat == cat],
ALPHA)
            mondrian_covered_all[test_mask] =
abs_resid_test[test_mask] <= q_cat

        summary_rows.append(
            {
                "target": target,
                "target_coverage": 1 - ALPHA,
                "standard_overall_coverage":
standard_covered_all.mean(),
                "standard_overall_width": 2 * q_pooled,
                "mondrian_overall_coverage":
mondrian_covered_all.mean(),
                "pooled_coverage_min": pooled_coverages.min(),
                "pooled_coverage_max": pooled_coverages.max(),
                "pooled_coverage_range": pooled_coverages.max() -
pooled_coverages.min(),
                "mondrian_coverage_min": mondrian_coverages.min(),
                "mondrian_coverage_max": mondrian_coverages.max(),
                "mondrian_coverage_range": mondrian_coverages.max() -
mondrian_coverages.min(),
                "worst_pooled_category":
categories[int(np.argmin(pooled_coverages))],
            }
        )

    pd.DataFrame(detail_rows).to_csv(DETAIL_PATH, index=False)
    summary_df = pd.DataFrame(summary_rows)
    summary_df.to_csv(SUMMARY_PATH, index=False)
    print(summary_df.to_string(index=False))


if __name__ == "__main__":
    main()
```

src/surrogate/train_mlp_surrogate.py - MLP surrogate training (Chapter 8 robustness check)

```
"""
Publication-rigor upgrade, part 3: a second surrogate architecture.

Week 13's exchangeability finding traced the coverage collapse to a
specific
mechanism: HistGradientBoostingRegressor (tree-based) cannot
extrapolate
```

```
   6  past its training range, so its prediction literally freezes for any
input
   7  outside [0.8, 1.3]. That's a real, verified mechanism - but it's
specific to
   8  tree ensembles. A neural network doesn't have the same "frozen leaf
value"
   9  limitation; its predictions keep changing outside the training range
  10  (usually roughly linearly in the final layer, though not necessarily
  11  *correctly*). Training an MLP surrogate here to re-run the
exchangeability
  12  stress test against and see whether the coverage collapse is a
property of
  13  CP-wrapping-an-unreliable-model in general, or specific to gradient
  14  boosting's exact failure mode.
  15
  16  Same train/test split (random_state=42) as train_surrogate.py, so this
is
  17  trained on identical data to the original surrogate - a fair
architecture
  18  comparison, not a differently-set-up experiment. MLPs are scale-
sensitive
  19  (unlike tree ensembles), so each target gets a
Pipeline(StandardScaler,
  20  MLPRegressor) - the scaler is fit only on training data and saved as
part of
  21  the pipeline, so calibration/test/stress-test data downstream doesn't
need
  22  its own separate scaling step.
  23  """
  24
  25  import joblib
  26  import pandas as pd
  27  from sklearn.metrics import mean_absolute_error, mean_squared_error,
r2_score
  28  from sklearn.model_selection import train_test_split
  29  from sklearn.neural_network import MLPRegressor
  30  from sklearn.pipeline import make_pipeline
  31  from sklearn.preprocessing import StandardScaler
  32
  33  DATA_PATH = "data/processed/surrogate_training_data.parquet"
  34  MODELS_DIR = "models"
  35  METRICS_PATH = "results/tables/mlp_surrogate_metrics.csv"
  36
  37  FEATURES = ["n_capacity", "arrival_rate_multiplier"]
  38  TARGETS = ["n_patients", "mean_wait_minutes", "mean_total_minutes",
"p95_wait_minutes"]
  39
  40
  41  def main():
  42      df = pd.read_parquet(DATA_PATH)
  43      X_train, X_test, train_idx, test_idx = train_test_split(
  44          df[FEATURES], df.index, test_size=0.2, random_state=42
  45      )
  46
  47      metrics = []
  48      for target in TARGETS:
  49          y_train = df.loc[train_idx, target]
  50          y_test = df.loc[test_idx, target]
  51
  52          model = make_pipeline(
  53              StandardScaler(),
  54              MLPRegressor(
  55                  hidden_layer_sizes=(64, 64),
  56                  activation="relu",
  57                  early_stopping=True,
```

```
                n_iter_no_change=20,
                max_iter=2000,
                random_state=42,
            ),
        )
        model.fit(X_train, y_train)
        y_pred = model.predict(X_test)

        metrics.append(
            {
                "target": target,
                "mae": mean_absolute_error(y_test, y_pred),
                "rmse": mean_squared_error(y_test, y_pred) ** 0.5,
                "r2": r2_score(y_test, y_pred),
            }
        )
        joblib.dump(model,
f"{MODELS_DIR}/surrogate_{target}_mlp.joblib")

    metrics_df = pd.DataFrame(metrics)
    metrics_df.to_csv(METRICS_PATH, index=False)
    print(metrics_df)


if __name__ == "__main__":
    main()
```

src/uq/exchangeability_stress_test.py - Exchangeability stress test, gradient-boosting surrogate

```
"""
Week 13: exchangeability stress test.

Standard and Mondrian CP's coverage guarantees both rely on
calibration and
test data being exchangeable (drawn from the same distribution). Both
Week 9-10 and Week 11-12 calibrated on n_capacity in [15,45],
arrival_rate_
multiplier in [0.8,1.3] and evaluated on test data from the same range
- so
exchangeability held by construction, and both methods hit close to
nominal
coverage. This script keeps calibration fixed exactly as before and
pushes
the *test* distribution progressively further outside that range - an
escalating "surge day" (arrival_rate_multiplier up to 3.0x normal,
well past
the training max of 1.3x) - to find where exchangeability actually
breaks
down and what happens to coverage when it does.

n_capacity is still sampled from the normal [15,45] range at every
severity
level, so the shift is isolated to arrival rate - a surge in demand,
not a
simultaneous staffing change. That keeps this a single-variable
extrapolation test, which is easier to read than conflating two
shifted
variables at once.

```

```
  21  Reuses the exact calibration quantiles and category bounds from
  22  standard_cp.py / mondrian_cp.py - no refitting - because the whole
point is
  23  to see how a *fixed* calibration behaves as the test distribution
moves
  24  away from it.
  25  """
  26
  27  import joblib
  28  import numpy as np
  29  import pandas as pd
  30
  31  from src.des.er_simulation import run_scenario
  32  from src.uq.mondrian_cp import LEVEL_NAMES, assign_categories,
conformal_quantile
  33
  34  TRAIN_DATA_PATH = "data/processed/surrogate_training_data.parquet"
  35  CALIBRATION_DATA_PATH = "data/processed/cp_calibration_data.parquet"
  36  MODELS_DIR = "models"
  37  OUT_PATH = "results/tables/exchangeability_stress_test.csv"
  38
  39  FEATURES = ["n_capacity", "arrival_rate_multiplier"]
  40  TARGETS = ["n_patients", "mean_wait_minutes", "mean_total_minutes",
"p95_wait_minutes"]
  41
  42  ALPHA = 0.1
  43  N_CAPACITY_RANGE = (15, 45)
  44  N_PER_SEVERITY = 300
  45  # 0.8/1.3 are in-range references (should reproduce ~90% coverage);
the rest
  46  # escalate past the 1.3 training max to find where coverage breaks
down.
  47  SEVERITY_LEVELS = [0.8, 1.0, 1.3, 1.5, 1.8, 2.0, 2.5, 3.0]
  48
  49
  50  def generate_ood_data(arrival_multiplier, n_scenarios, seed_offset):
  51      rng = np.random.default_rng(seed_offset)
  52      n_capacity = rng.integers(N_CAPACITY_RANGE[0], N_CAPACITY_RANGE[1]
+ 1, size=n_scenarios)
  53      rows = []
  54      for i in range(n_scenarios):
  55          result = run_scenario(
  56              n_capacity=int(n_capacity[i]),
  57              arrival_rate_multiplier=arrival_multiplier,
  58              seed=seed_offset + i,
  59          )
  60          rows.append({"n_capacity": n_capacity[i],
"arrival_rate_multiplier": arrival_multiplier, **result})
  61      return pd.DataFrame(rows)
  62
  63
  64  def main():
  65      cal_df = pd.read_parquet(CALIBRATION_DATA_PATH)
  66      cap_bounds = np.quantile(cal_df["n_capacity"], [1 / 3, 2 / 3])
  67      arr_bounds = np.quantile(cal_df["arrival_rate_multiplier"], [1 /
3, 2 / 3])
  68      cal_cat = assign_categories(cal_df, cap_bounds, arr_bounds)
  69
  70      rows = []
  71      for level_idx, mult in enumerate(SEVERITY_LEVELS):
  72          ood_df = generate_ood_data(mult, N_PER_SEVERITY,
seed_offset=200_000 + level_idx * 10_000)
  73          ood_cat = assign_categories(ood_df, cap_bounds, arr_bounds)
  74
  75          for target in TARGETS:
```

```
            model =
joblib.load(f"{MODELS_DIR}/surrogate_{target}.joblib")

            yhat_cal = model.predict(cal_df[FEATURES])
            y_cal = cal_df[target].values
            abs_resid_cal = np.abs(y_cal - yhat_cal)
            q_pooled = conformal_quantile(abs_resid_cal, ALPHA)

            yhat_ood = model.predict(ood_df[FEATURES])
            y_ood = ood_df[target].values
            abs_resid_ood = np.abs(y_ood - yhat_ood)

            standard_covered = abs_resid_ood <= q_pooled

            mondrian_covered = np.zeros(len(ood_df), dtype=bool)
            mondrian_width = np.zeros(len(ood_df))
            for cat in set(cal_cat):
                q_cat = conformal_quantile(abs_resid_cal[cal_cat ==
cat], ALPHA)
                mask = ood_cat == cat
                mondrian_covered[mask] = abs_resid_ood[mask] <= q_cat
                mondrian_width[mask] = 2 * q_cat

            rows.append(
                {
                    "target": target,
                    "arrival_rate_multiplier": mult,
                    "in_training_range": mult <= 1.3,
                    "n_scenarios": len(ood_df),
                    "standard_coverage": standard_covered.mean(),
                    "standard_width": 2 * q_pooled,
                    "mondrian_coverage": mondrian_covered.mean(),
                    "mondrian_mean_width": mondrian_width.mean(),
                }
            )

    out_df = pd.DataFrame(rows)
    out_df.to_csv(OUT_PATH, index=False)
    print(out_df.to_string(index=False))


if __name__ == "__main__":
    main()
```

src/uq/exchangeability_stress_test_mlp.py - Exchangeability stress test, MLP surrogate

```
"""
Publication-rigor upgrade, part 3 (continued): re-run the Week 13
exchangeability stress test using the MLP surrogate instead of
gradient
boosting, to test whether the coverage collapse outside the training
range
is specific to tree-based extrapolation (frozen leaf predictions,
verified
in Week 13) or a more general property of wrapping CP around an
unreliable-outside-its-domain point predictor.

Identical setup to exchangeability_stress_test.py otherwise: same
calibration data, same severity levels, same seeds - only the model
```

```
files
  11  differ (`*_mlp.joblib`). Calibration is recomputed fresh here (not
reused
  12  from the gradient-boosting run) because the MLP's residuals on the
  13  calibration set are different from gradient boosting's - each
surrogate
  14  needs its own conformal quantile.
  15  """
  16
  17  import joblib
  18  import numpy as np
  19  import pandas as pd
  20
  21  from src.des.er_simulation import run_scenario
  22  from src.uq.mondrian_cp import assign_categories, conformal_quantile
  23
  24  CALIBRATION_DATA_PATH = "data/processed/cp_calibration_data.parquet"
  25  MODELS_DIR = "models"
  26  OUT_PATH = "results/tables/exchangeability_stress_test_mlp.csv"
  27
  28  FEATURES = ["n_capacity", "arrival_rate_multiplier"]
  29  TARGETS = ["n_patients", "mean_wait_minutes", "mean_total_minutes",
"p95_wait_minutes"]
  30
  31  ALPHA = 0.1
  32  N_CAPACITY_RANGE = (15, 45)
  33  N_PER_SEVERITY = 300
  34  SEVERITY_LEVELS = [0.8, 1.0, 1.3, 1.5, 1.8, 2.0, 2.5, 3.0]
  35
  36
  37  def generate_ood_data(arrival_multiplier, n_scenarios, seed_offset):
  38      rng = np.random.default_rng(seed_offset)
  39      n_capacity = rng.integers(N_CAPACITY_RANGE[0], N_CAPACITY_RANGE[1]
+ 1, size=n_scenarios)
  40      rows = []
  41      for i in range(n_scenarios):
  42          result = run_scenario(
  43              n_capacity=int(n_capacity[i]),
  44              arrival_rate_multiplier=arrival_multiplier,
  45              seed=seed_offset + i,
  46          )
  47          rows.append({"n_capacity": n_capacity[i],
"arrival_rate_multiplier": arrival_multiplier, **result})
  48      return pd.DataFrame(rows)
  49
  50
  51  def main():
  52      cal_df = pd.read_parquet(CALIBRATION_DATA_PATH)
  53      cap_bounds = np.quantile(cal_df["n_capacity"], [1 / 3, 2 / 3])
  54      arr_bounds = np.quantile(cal_df["arrival_rate_multiplier"], [1 /
3, 2 / 3])
  55      cal_cat = assign_categories(cal_df, cap_bounds, arr_bounds)
  56
  57      rows = []
  58      for level_idx, mult in enumerate(SEVERITY_LEVELS):
  59          # same seed_offset scheme as the gradient-boosting version ->
identical OOD scenarios
  60          ood_df = generate_ood_data(mult, N_PER_SEVERITY,
seed_offset=200_000 + level_idx * 10_000)
  61          ood_cat = assign_categories(ood_df, cap_bounds, arr_bounds)
  62
  63          for target in TARGETS:
  64              model =
joblib.load(f"{MODELS_DIR}/surrogate_{target}_mlp.joblib")
  65
```

```
  66              yhat_cal = model.predict(cal_df[FEATURES])
  67              y_cal = cal_df[target].values
  68              abs_resid_cal = np.abs(y_cal - yhat_cal)
  69              q_pooled = conformal_quantile(abs_resid_cal, ALPHA)
  70
  71              yhat_ood = model.predict(ood_df[FEATURES])
  72              y_ood = ood_df[target].values
  73              abs_resid_ood = np.abs(y_ood - yhat_ood)
  74
  75              standard_covered = abs_resid_ood <= q_pooled
  76
  77              mondrian_covered = np.zeros(len(ood_df), dtype=bool)
  78              mondrian_width = np.zeros(len(ood_df))
  79              for cat in set(cal_cat):
  80                  q_cat = conformal_quantile(abs_resid_cal[cal_cat ==
cat], ALPHA)
  81                  mask = ood_cat == cat
  82                  mondrian_covered[mask] = abs_resid_ood[mask] <= q_cat
  83                  mondrian_width[mask] = 2 * q_cat
  84
  85              rows.append(
  86                  {
  87                      "target": target,
  88                      "arrival_rate_multiplier": mult,
  89                      "in_training_range": mult <= 1.3,
  90                      "n_scenarios": len(ood_df),
  91                      "standard_coverage": standard_covered.mean(),
  92                      "standard_width": 2 * q_pooled,
  93                      "mondrian_coverage": mondrian_covered.mean(),
  94                      "mondrian_mean_width": mondrian_width.mean(),
  95                      "yhat_capacity30":
float(model.predict(pd.DataFrame(
  96                          [[30, mult]], columns=FEATURES))[0]),
  97                  }
  98              )
  99
 100      out_df = pd.DataFrame(rows)
 101      out_df.to_csv(OUT_PATH, index=False)
 102      print(out_df.to_string(index=False))
 103
 104
 105  if __name__ == "__main__":
 106      main()
```

src/surrogate/train_rf_xgb_lgb_surrogates.py - Random Forest / XGBoost / LightGBM surrogate benchmark (Chapter 7)

```
   1  """
   2  Multi-architecture surrogate benchmark: Random Forest, XGBoost, and
   3  LightGBM, trained on the identical data and train/test split as the
   4  primary gradient-boosting surrogate (train_surrogate.py,
random_state=42)
   5  and the MLP robustness check (train_mlp_surrogate.py), so all five
   6  architectures are directly comparable on the same held-out test set -
   7  a fair architecture comparison, not a differently-trained one.
   8
   9  Adds three tree-ensemble variants beyond the primary
HistGradientBoosting
  10  model to check whether its accuracy is specific to that one
implementation
```

```
  11  or general to gradient-boosted/bagged tree ensembles on this problem.
  12  Default hyperparameters throughout (matching train_surrogate.py's own
  13  practice of not hand-tuning a 2-input, ~5000-row tabular problem) -
the
  14  comparison is about architecture family, not a hyperparameter search.
  15  """
  16
  17  import time
  18
  19  import joblib
  20  import pandas as pd
  21  from lightgbm import LGBMRegressor
  22  from sklearn.ensemble import RandomForestRegressor
  23  from sklearn.metrics import mean_absolute_error, mean_squared_error,
r2_score
  24  from sklearn.model_selection import train_test_split
  25  from xgboost import XGBRegressor
  26
  27  DATA_PATH = "data/processed/surrogate_training_data.parquet"
  28  MODELS_DIR = "models"
  29  METRICS_PATH = "results/tables/rf_xgb_lgb_surrogate_metrics.csv"
  30
  31  FEATURES = ["n_capacity", "arrival_rate_multiplier"]
  32  TARGETS = ["n_patients", "mean_wait_minutes", "mean_total_minutes",
"p95_wait_minutes"]
  33
  34  ARCHITECTURES = {
  35      "RandomForest": lambda: RandomForestRegressor(random_state=42,
n_jobs=-1),
  36      "XGBoost": lambda: XGBRegressor(random_state=42, n_jobs=-1),
  37      "LightGBM": lambda: LGBMRegressor(random_state=42, n_jobs=-1,
verbosity=-1),
  38  }
  39
  40
  41  def main():
  42      df = pd.read_parquet(DATA_PATH)
  43      X_train, X_test, train_idx, test_idx = train_test_split(
  44          df[FEATURES], df.index, test_size=0.2, random_state=42
  45      )
  46
  47      metrics = []
  48      for arch_name, make_model in ARCHITECTURES.items():
  49          for target in TARGETS:
  50              y_train = df.loc[train_idx, target]
  51              y_test = df.loc[test_idx, target]
  52
  53              model = make_model()
  54              t0 = time.perf_counter()
  55              model.fit(X_train, y_train)
  56              fit_seconds = time.perf_counter() - t0
  57
  58              t0 = time.perf_counter()
  59              y_pred = model.predict(X_test)
  60              predict_seconds = time.perf_counter() - t0
  61
  62              metrics.append({
  63                  "architecture": arch_name,
  64                  "target": target,
  65                  "mae": mean_absolute_error(y_test, y_pred),
  66                  "rmse": mean_squared_error(y_test, y_pred) ** 0.5,
  67                  "r2": r2_score(y_test, y_pred),
  68                  "fit_seconds": fit_seconds,
  69                  "predict_seconds": predict_seconds,
  70              })
```

```
  71              joblib.dump(model,
f"{MODELS_DIR}/surrogate_{target}_{arch_name.lower()}.joblib")
  72              print(f"{arch_name} / {target}: MAE={metrics[-
1]['mae']:.2f} RMSE={metrics[-1]['rmse']:.2f} "
  73                    f"R2={metrics[-1]['r2']:.3f}
fit={fit_seconds:.2f}s")
  74
  75      metrics_df = pd.DataFrame(metrics)
  76      metrics_df.to_csv(METRICS_PATH, index=False)
  77      print(f"\nSaved {METRICS_PATH}")
  78
  79
  80  if __name__ == "__main__":
  81      main()
```

src/uq/conformal_risk_control.py - Conformal risk control (Chapter 6)

```
   1  """
   2  Conformal Risk Control (CRC), following Bates, Angelopoulos, Lei,
Malik,
   3  and Jordan (2021), "Distribution-Free, Risk-Controlling Prediction
Sets."
   4
   5  Standard split conformal prediction (standard_cp.py) controls a
single,
   6  fixed loss: the 0/1 miscoverage indicator 1{y not in C(x)}. CRC
generalizes
   7  this to any bounded, monotone-in-lambda loss function ell_lambda(x,
y),
   8  calibrating the smallest lambda whose *upper confidence bound* on
empirical
   9  risk (via Hoeffding's inequality, not the direct order-statistic used
by
  10  standard CP) is at most alpha. This script implements CRC for two
losses
  11  per target, on the exact same calibration/test data and models as the
rest
  12  of this project's UQ comparison (standard_cp.py, mondrian_cp.py):
  13
  14    1. The 0/1 upper-miscoverage loss - included as a validation check.
Since
  15       this loss is the same one standard CP already controls exactly
via its
  16       order-statistic quantile, CRC's Hoeffding-UCB calibration is
expected
  17       to be *more conservative* (a wider threshold) than the exact
conformal
  18       quantile - a real, honest property of Hoeffding-based CRC, not a
  19       deficiency of this implementation, and reported as such rather
than
  20       hidden.
  21    2. A clipped relative-overshoot loss, ell_lambda(x, y) =
  22       clip((y - (yhat(x) + lambda)) / W_max, 0, 1), where W_max is set
to
  23       the target's own standard-CP symmetric interval width (an
existing,
  24       already-computed quantity, not an arbitrary round number). This
is the
  25       genuinely new capability CRC provides that plain conformal
prediction
  26       cannot: bounding the *expected severity* of an overflow event,
```

```
not
  27       just its probability - directly relevant to an operational "how
bad
  28       is a miss, on average" question standard CP's binary coverage
  29       guarantee cannot answer.
  30
  31   CRC's guarantee: with probability >= 1 - delta over the calibration
draw,
  32   the TRUE risk R(lambda_hat) = E[ell_lambda_hat(X, Y)] <= alpha. This
script
  33   reports both the calibration-side selection and the held-out test-set
  34   empirical risk, to check the guarantee actually holds on unseen data.
  35   """
  36
  37   import joblib
  38   import numpy as np
  39   import pandas as pd
  40   from sklearn.model_selection import train_test_split
  41
  42   TRAIN_DATA_PATH = "data/processed/surrogate_training_data.parquet"
  43   CALIBRATION_DATA_PATH = "data/processed/cp_calibration_data.parquet"
  44   STANDARD_CP_METRICS_PATH = "results/tables/standard_cp_metrics.csv"
  45   MODELS_DIR = "models"
  46   OUT_PATH = "results/tables/crc_results.csv"
  47
  48   FEATURES = ["n_capacity", "arrival_rate_multiplier"]
  49   TARGETS = ["n_patients", "mean_wait_minutes", "mean_total_minutes",
"p95_wait_minutes"]
  50
  51   ALPHA = 0.1        # target risk level, matches alpha used everywhere
else in this project
  52   DELTA = 0.1         # confidence level for the Hoeffding UCB (1 -
delta guarantee)
  53   LAMBDA_GRID_POINTS = 4000
  54
  55
  56   def hoeffding_ucb(risk_hat, n, B, delta):
  57       """Upper confidence bound on true risk via Hoeffding's inequality
for a
  58       bounded-in-[0,B] loss, averaged over n i.i.d. calibration
points."""
  59       return risk_hat + B * np.sqrt(np.log(1 / delta) / (2 * n))
  60
  61
  62   def calibrate_crc(residuals, alpha, delta, loss_fn, B, lambda_grid):
  63       """Bates et al. Algorithm 1 (Hoeffding variant): smallest lambda
on the
  64       grid whose Hoeffding UCB on empirical risk is <= alpha. Loss must
be
  65       non-increasing in lambda (larger lambda = wider/safer set =
smaller
  66       loss) for the search to be valid; lambda_grid is assumed
ascending."""
  67       n = len(residuals)
  68       for lam in lambda_grid:
  69           risk_hat = loss_fn(residuals, lam).mean()
  70           if hoeffding_ucb(risk_hat, n, B, delta) <= alpha:
  71               return lam, risk_hat
  72       return lambda_grid[-1], loss_fn(residuals, lambda_grid[-1]).mean()
  73
  74
  75   def loss_01(residuals, lam):
  76       """ell_lambda(x,y) = 1{y > yhat(x) + lambda}; residuals = y -
yhat(x)."""
  77       return (residuals > lam).astype(float)
```

```
  78
  79
  80  def loss_clipped_overshoot(residuals, lam, w_max):
  81      """ell_lambda(x,y) = clip((y - (yhat(x)+lambda)) / W_max, 0,
1)."""
  82      return np.clip((residuals - lam) / w_max, 0.0, 1.0)
  83
  84
  85  def main():
  86      df = pd.read_parquet(TRAIN_DATA_PATH)
  87      _, X_test, _, test_idx = train_test_split(df[FEATURES], df.index,
test_size=0.2, random_state=42)
  88
  89      cal_df = pd.read_parquet(CALIBRATION_DATA_PATH)
  90      X_cal = cal_df[FEATURES]
  91
  92      std_cp = pd.read_csv(STANDARD_CP_METRICS_PATH).set_index("target")
  93
  94      rows = []
  95      for target in TARGETS:
  96          model = joblib.load(f"{MODELS_DIR}/surrogate_{target}.joblib")
  97
  98          yhat_cal = model.predict(X_cal)
  99          residual_cal = cal_df[target].values - yhat_cal   # y - yhat,
calibration
 100
 101          yhat_test = model.predict(X_test)
 102          residual_test = df.loc[test_idx, target].values - yhat_test  #
y - yhat, test
 103
 104          lam_max = max(residual_cal.max(), residual_test.max()) * 1.5
 105          lambda_grid = np.linspace(0, lam_max, LAMBDA_GRID_POINTS)
 106
 107          # Loss 1: 0/1 upper-miscoverage (validation check against
standard CP)
 108          lam01, cal_risk01 = calibrate_crc(residual_cal, ALPHA, DELTA,
loss_01, B=1.0, lambda_grid=lambda_grid)
 109          test_risk01 = loss_01(residual_test, lam01).mean()
 110          standard_cp_halfwidth = std_cp.loc[target,
"symmetric_mean_width"] / 2
 111
 112          # Loss 2: clipped relative overshoot severity
 113          w_max = std_cp.loc[target, "symmetric_mean_width"]
 114          loss_fn2 = lambda resid, lam: loss_clipped_overshoot(resid,
lam, w_max)
 115          lam_sev, cal_risk_sev = calibrate_crc(residual_cal, ALPHA,
DELTA, loss_fn2, B=1.0, lambda_grid=lambda_grid)
 116          test_risk_sev = loss_fn2(residual_test, lam_sev).mean()
 117
 118          rows.append({
 119              "target": target,
 120              "alpha": ALPHA,
 121              "delta": DELTA,
 122              "n_cal": len(residual_cal),
 123              "n_test": len(residual_test),
 124              "loss01_lambda": lam01,
 125              "loss01_cal_risk_hat": cal_risk01,
 126              "loss01_test_risk": test_risk01,
 127              "standard_cp_upper_halfwidth": standard_cp_halfwidth,
 128              "sev_w_max": w_max,
 129              "sev_lambda": lam_sev,
 130              "sev_cal_risk_hat": cal_risk_sev,
 131              "sev_test_risk": test_risk_sev,
 132          })
 133          print(f"{target}: loss01 lambda={lam01:.2f} (vs standard-CP
```

```
half-width {standard_cp_halfwidth:.2f}), "
 134                f"test risk={test_risk01:.4f} (target <= {ALPHA}); "
 135                f"severity lambda={lam_sev:.2f}, test
risk={test_risk_sev:.4f} (target <= {ALPHA})")
 136
 137      out_df = pd.DataFrame(rows)
 138      out_df.to_csv(OUT_PATH, index=False)
 139      print(f"\nSaved {OUT_PATH}")
 140
 141
 142  if __name__ == "__main__":
 143      main()
```

src/uq/adaptive_conformal_inference.py - Adaptive conformal inference (Chapter 6)

```
   1  """
   2  Adaptive Conformal Inference (ACI), following Gibbs and Candes (2021),
   3  "Adaptive Conformal Inference Under Distribution Shift."
   4
   5  Standard split conformal prediction (standard_cp.py) fixes its
miscoverage
   6  target alpha and its calibration quantile once, then applies both
   7  unchanged to every future test point - which is exactly why coverage
   8  collapses under the exchangeability stress test
(exchangeability_stress_
   9  test.py) once the test distribution moves away from the fixed
calibration
  10  distribution: there is no mechanism in standard CP for the interval to
  11  respond to what it is actually observing. ACI removes that rigidity:
it
  12  maintains a running miscoverage target alpha_t, using only the
calibration
  13  set's *fixed* nonconformity-score distribution as a fixed lookup
table, but
  14  re-querying that lookup at a shifted level 1 - alpha_t and, after each
new
  15  point is revealed, nudging alpha_t up or down depending on whether
that
  16  point was covered:
  17
  18      alpha_{t+1} = alpha_t + gamma * (alpha - err_t),   err_t = 1{y_t
not in C_t}
  19
  20  This script builds a single, genuinely time-ordered test stream - a
demand
  21  surge ramping from within-range (0.8x) up to 3.0x the calibrated
arrival
  22  rate, the same severity progression as exchangeability_stress_test.py,
but
  23  concatenated into one sequence rather than evaluated as separate
i.i.d.
  24  batches - and compares ACI's windowed coverage against static standard
  25  CP's on the identical stream, same model, same initial calibration
set.
  26  """
  27
  28  import joblib
  29  import numpy as np
  30  import pandas as pd
```

```
  31
  32  from src.des.er_simulation import run_scenario
  33
  34  TRAIN_DATA_PATH = "data/processed/surrogate_training_data.parquet"
  35  CALIBRATION_DATA_PATH = "data/processed/cp_calibration_data.parquet"
  36  MODELS_DIR = "models"
  37  OUT_PATH = "results/tables/aci_results.csv"
  38  OUT_TRAJECTORY_PATH = "results/tables/aci_alpha_trajectory.csv"
  39
  40  FEATURES = ["n_capacity", "arrival_rate_multiplier"]
  41  TARGETS = ["n_patients", "mean_wait_minutes", "mean_total_minutes",
"p95_wait_minutes"]
  42
  43  ALPHA = 0.1
  44  GAMMA = 0.05
  45  N_CAPACITY_RANGE = (15, 45)
  46  N_PER_LEVEL = 60
  47  # Same severity progression as exchangeability_stress_test.py,
concatenated
  48  # into a single ordered stream (a demand surge ramping up over "time")
  49  SEVERITY_LEVELS = [0.8, 1.0, 1.3, 1.5, 1.8, 2.0, 2.5, 3.0]
  50
  51
  52  def build_stream():
  53      rng = np.random.default_rng(300_000)
  54      rows = []
  55      step = 0
  56      for level_idx, mult in enumerate(SEVERITY_LEVELS):
  57          for i in range(N_PER_LEVEL):
  58              cap = int(rng.integers(N_CAPACITY_RANGE[0],
N_CAPACITY_RANGE[1] + 1))
  59              result = run_scenario(n_capacity=cap,
arrival_rate_multiplier=mult, seed=300_000 + level_idx * 10_000 + i)
  60              rows.append({"step": step, "arrival_rate_multiplier":
mult, "in_range": mult <= 1.3,
  61                           "n_capacity": cap, **result})
  62              step += 1
  63      return pd.DataFrame(rows)
  64
  65
  66  def quantile_at_level(scores, level):
  67      """Empirical quantile of a fixed score set at an arbitrary
(possibly
  68      time-varying) level, clipped to a valid probability - this is
ACI's
  69      'fixed lookup table, shifted query level' mechanism."""
  70      level = float(np.clip(level, 0.0, 1.0))
  71      if level >= 1.0:
  72          return np.max(scores)
  73      if level <= 0.0:
  74          return np.min(np.abs(scores)) * 0  # degenerate, zero-width
  75      n = len(scores)
  76      order_level = min(np.ceil((n + 1) * level) / n, 1.0)
  77      return np.quantile(scores, order_level, method="higher")
  78
  79
  80  def run_aci_vs_static(residual_cal_abs, residual_stream, alpha,
gamma):
  81      """Returns per-step covered_static, covered_aci, alpha_t
trajectory,
  82      interval half-width trajectory for ACI."""
  83      n = len(residual_stream)
  84      covered_static = np.zeros(n, dtype=bool)
  85      covered_aci = np.zeros(n, dtype=bool)
  86      alpha_t_trace = np.zeros(n)
```

```
    width_aci_trace = np.zeros(n)

    q_static = quantile_at_level(residual_cal_abs, 1 - alpha)

    alpha_t = alpha
    for t in range(n):
        covered_static[t] = residual_stream[t] <= q_static

        alpha_t_trace[t] = alpha_t
        q_aci = quantile_at_level(residual_cal_abs, 1 - alpha_t)
        width_aci_trace[t] = q_aci
        covered_aci[t] = residual_stream[t] <= q_aci

        err_t = 0.0 if covered_aci[t] else 1.0
        alpha_t = alpha_t + gamma * (alpha - err_t)

    return covered_static, covered_aci, alpha_t_trace, width_aci_trace


def rolling_coverage(covered, window=60):
    s = pd.Series(covered.astype(float))
    return s.rolling(window, min_periods=1).mean().values


def main():
    stream_df = build_stream()

    cal_df = pd.read_parquet(CALIBRATION_DATA_PATH)
    X_cal = cal_df[FEATURES]

    summary_rows = []
    trajectory_rows = []
    for target in TARGETS:
        model = joblib.load(f"{MODELS_DIR}/surrogate_{target}.joblib")

        yhat_cal = model.predict(X_cal)
        residual_cal_abs = np.abs(cal_df[target].values - yhat_cal)

        yhat_stream = model.predict(stream_df[FEATURES])
        residual_stream_abs = np.abs(stream_df[target].values -
yhat_stream)

        covered_static, covered_aci, alpha_t_trace, width_aci_trace =
run_aci_vs_static(
            residual_cal_abs, residual_stream_abs, ALPHA, GAMMA
        )

        roll_static = rolling_coverage(covered_static)
        roll_aci = rolling_coverage(covered_aci)

        for t in range(len(stream_df)):
            trajectory_rows.append({
                "target": target, "step": t,
                "arrival_rate_multiplier":
stream_df["arrival_rate_multiplier"].iloc[t],
                "in_range": stream_df["in_range"].iloc[t],
                "alpha_t": alpha_t_trace[t],
                "aci_width": 2 * width_aci_trace[t],
                "rolling_coverage_static": roll_static[t],
                "rolling_coverage_aci": roll_aci[t],
            })

        # overall + out-of-range-only coverage
        oor_mask = ~stream_df["in_range"].values
        summary_rows.append({
```

```
 149              "target": target,
 150              "overall_coverage_static": covered_static.mean(),
 151              "overall_coverage_aci": covered_aci.mean(),
 152              "oor_coverage_static": covered_static[oor_mask].mean(),
 153              "oor_coverage_aci": covered_aci[oor_mask].mean(),
 154              "final_alpha_t": alpha_t_trace[-1],
 155              "mean_width_aci_oor": (2 *
width_aci_trace)[oor_mask].mean(),
 156          })
 157          print(f"{target}: overall static={covered_static.mean():.3f}
aci={covered_aci.mean():.3f} | "
 158                f"out-of-range
static={covered_static[oor_mask].mean():.3f}
aci={covered_aci[oor_mask].mean():.3f} "
 159                f"(target 0.90)")
 160
 161      pd.DataFrame(summary_rows).to_csv(OUT_PATH, index=False)
 162      pd.DataFrame(trajectory_rows).to_csv(OUT_TRAJECTORY_PATH,
index=False)
 163      print(f"\nSaved {OUT_PATH} and {OUT_TRAJECTORY_PATH}")
 164
 165
 166  if __name__ == "__main__":
 167      main()
```

src/uq/weighted_conformal_prediction.py - Likelihood-ratio weighted conformal prediction (Chapter 6, Chapter 8)

```
   1  """
   2  Likelihood-ratio weighted conformal prediction under covariate shift,
   3  following Tibshirani, Barber, Candes, and Ramdas (2019), "Conformal
   4  Prediction Under Covariate Shift."
   5
   6  Standard split CP (standard_cp.py) assumes calibration and test
covariates
   7  are drawn from the *same* distribution. Weighted CP relaxes this to a
   8  known, *bounded* covariate shift: if the test covariate density q(x)
is
   9  absolutely continuous with respect to the calibration density p(x)
(i.e.
  10  q(x) > 0 implies p(x) > 0 - test points must fall within calibration's
  11  support), reweighting each calibration point by the likelihood ratio
  12  w(x) = q(x)/p(x) and taking a weighted conformal quantile restores
exact
  13  coverage despite the shift.
  14
  15  This project's calibration distribution for arrival_rate_multiplier is
  16  Uniform[0.8, 1.3] (generate_calibration_data.py) - a *known*
distribution,
  17  since this project controls the DES's own sampling code, so exact
  18  likelihood ratios can be computed analytically rather than estimated.
This
  19  script evaluates weighted CP under a genuinely different,
complementary
  20  shift from the exchangeability stress test's up-to-3.0x severity
sweep:
  21  a *moderate* shift, arrival_rate_multiplier ~ Uniform[0.9, 1.6],
chosen
  22  specifically so that most of its mass overlaps calibration's support
  23  ([0.9, 1.3]) while a real tail ([1.3, 1.6]) extends outside it - which
```

```
  24  lets this script show both what weighted CP *can* fix (the overlapping
  25  region) and the specific theoretical reason it *cannot* fix an
  26  unbounded-density-ratio region (the non-overlapping tail), rather than
  27  picking a shift scenario engineered to make weighting look
unconditionally
  28  effective.
  29  """
  30
  31  import joblib
  32  import numpy as np
  33  import pandas as pd
  34
  35  from src.des.er_simulation import run_scenario
  36
  37  TRAIN_DATA_PATH = "data/processed/surrogate_training_data.parquet"
  38  CALIBRATION_DATA_PATH = "data/processed/cp_calibration_data.parquet"
  39  MODELS_DIR = "models"
  40  OUT_PATH = "results/tables/weighted_cp_results.csv"
  41
  42  FEATURES = ["n_capacity", "arrival_rate_multiplier"]
  43  TARGETS = ["n_patients", "mean_wait_minutes", "mean_total_minutes",
"p95_wait_minutes"]
  44
  45  ALPHA = 0.1
  46  N_CAPACITY_RANGE = (15, 45)
  47  N_TEST_SCENARIOS = 600
  48
  49  CAL_ARR_LOW, CAL_ARR_HIGH = 0.8, 1.3          # calibration support
(generate_calibration_data.py)
  50  TEST_ARR_LOW, TEST_ARR_HIGH = 0.9, 1.6         # moderate shift:
partial overlap, real out-of-support tail
  51  DENSITY_FLOOR = 1e-6                            # numerically-safe
stand-in for "zero calibration density"
  52
  53
  54  def cal_density(arr_mult):
  55      """Known Uniform[0.8, 1.3] calibration density
(generate_calibration_data.py)."""
  56      inside = (arr_mult >= CAL_ARR_LOW) & (arr_mult <= CAL_ARR_HIGH)
  57      d = np.where(inside, 1.0 / (CAL_ARR_HIGH - CAL_ARR_LOW),
DENSITY_FLOOR)
  58      return d
  59
  60
  61  def test_density(arr_mult):
  62      """Known Uniform[0.9, 1.6] shifted test density (this script's own
sampler)."""
  63      inside = (arr_mult >= TEST_ARR_LOW) & (arr_mult <= TEST_ARR_HIGH)
  64      return np.where(inside, 1.0 / (TEST_ARR_HIGH - TEST_ARR_LOW), 0.0)
  65
  66
  67  def likelihood_ratio(arr_mult):
  68      return test_density(arr_mult) / cal_density(arr_mult)
  69
  70
  71  def weighted_quantile(cal_scores, cal_weights, test_weight, alpha):
  72      """Tibshirani et al. (2019) weighted conformal quantile:
normalizes
  73      calibration weights together with the test point's own weight (the
  74      weighted analogue of standard CP's n+1 finite-sample correction),
then
  75      finds the smallest calibration score whose cumulative normalized
  76      weight reaches 1 - alpha."""
  77      order = np.argsort(cal_scores)
  78      sorted_scores = cal_scores[order]
```

```
  79      sorted_weights = cal_weights[order]
  80      total = sorted_weights.sum() + test_weight
  81      if total <= 0:
  82          return np.max(cal_scores)
  83      cum = np.cumsum(sorted_weights) / total
  84      idx = np.searchsorted(cum, 1 - alpha)
  85      if idx >= len(sorted_scores):
  86          return np.inf  # test point's own weight is too large a share
-> infinite interval
  87      return sorted_scores[idx]
  88
  89
  90  def generate_test_stream(n, seed=400_000):
  91      rng = np.random.default_rng(seed)
  92      arr_mult = rng.uniform(TEST_ARR_LOW, TEST_ARR_HIGH, size=n)
  93      n_capacity = rng.integers(N_CAPACITY_RANGE[0], N_CAPACITY_RANGE[1]
+ 1, size=n)
  94      rows = []
  95      for i in range(n):
  96          result = run_scenario(n_capacity=int(n_capacity[i]),
arrival_rate_multiplier=float(arr_mult[i]), seed=seed + i)
  97          rows.append({"n_capacity": n_capacity[i],
"arrival_rate_multiplier": arr_mult[i], **result})
  98      return pd.DataFrame(rows)
  99
 100
 101  def main():
 102      test_df = generate_test_stream(N_TEST_SCENARIOS)
 103      in_support = (test_df["arrival_rate_multiplier"] <=
CAL_ARR_HIGH).values
 104
 105      cal_df = pd.read_parquet(CALIBRATION_DATA_PATH)
 106      X_cal = cal_df[FEATURES]
 107      cal_weights =
likelihood_ratio(cal_df["arrival_rate_multiplier"].values)
 108      test_weights =
likelihood_ratio(test_df["arrival_rate_multiplier"].values)
 109
 110      rows = []
 111      for target in TARGETS:
 112          model = joblib.load(f"{MODELS_DIR}/surrogate_{target}.joblib")
 113
 114          yhat_cal = model.predict(X_cal)
 115          abs_resid_cal = np.abs(cal_df[target].values - yhat_cal)
 116
 117          yhat_test = model.predict(test_df[FEATURES])
 118          abs_resid_test = np.abs(test_df[target].values - yhat_test)
 119
 120          # unweighted (standard) split CP quantile, same calibration
set
 121          n = len(abs_resid_cal)
 122          std_level = min(np.ceil((n + 1) * (1 - ALPHA)) / n, 1.0)
 123          q_unweighted = np.quantile(abs_resid_cal, std_level,
method="higher")
 124          covered_unweighted = abs_resid_test <= q_unweighted
 125
 126          # weighted CP: per-test-point weighted quantile (weight
depends on that point's own arrival_rate_multiplier)
 127          covered_weighted = np.zeros(len(test_df), dtype=bool)
 128          width_weighted = np.zeros(len(test_df))
 129          for i in range(len(test_df)):
 130              q_i = weighted_quantile(abs_resid_cal, cal_weights,
test_weights[i], ALPHA)
 131              width_weighted[i] = 2 * q_i if np.isfinite(q_i) else
np.nan
```

```
 132              covered_weighted[i] = abs_resid_test[i] <= q_i
 133
 134          for region_name, mask in [("overlap_region", in_support),
("out_of_support_tail", ~in_support), ("overall", np.ones(len(test_df),
dtype=bool))]:
 135              rows.append({
 136                  "target": target,
 137                  "region": region_name,
 138                  "n_test": int(mask.sum()),
 139                  "unweighted_coverage":
covered_unweighted[mask].mean(),
 140                  "unweighted_width": 2 * q_unweighted,
 141                  "weighted_coverage": covered_weighted[mask].mean(),
 142                  "weighted_mean_width":
np.nanmean(width_weighted[mask]),
 143              })
 144
 145      out_df = pd.DataFrame(rows)
 146      out_df.to_csv(OUT_PATH, index=False)
 147      print(out_df.to_string(index=False))
 148      print(f"\nSaved {OUT_PATH}")
 149
 150
 151  if __name__ == "__main__":
 152      main()
```

src/deployment/build_ops_dashboard.py - Interactive prediction-interval dashboard (Chapter 10)

```
   1  """
   2  Prototype operational dashboard: a real, self-contained, interactive
HTML
   3  artifact (Plotly, no server required) demonstrating how this project's
   4  surrogate + Mondrian conformal prediction pipeline could surface
prediction
   5  intervals to an ED operations manager, rather than only reporting them
as
   6  static tables in this document.
   7
   8  Two linked panels, both driven by a target-metric dropdown:
   9    1. A heatmap of the surrogate's point prediction across the real
  10       (staffing capacity, arrival-rate multiplier) scenario grid this
  11       project's DES actually samples from, with each cell's hover text
  12       giving the point prediction *and* its Mondrian conformal interval
-
  13       computed live from the trained surrogate and the real per-
category
  14       quantiles in results/tables/mondrian_cp_detail.csv, not mocked
data.
  15    2. A bar chart of Mondrian interval width by category, directly
  16       visualizing this project's central finding (width concentrates
where
  17       the true operating risk is - understaffed, high-arrival cells) in
the
  18       form an operations dashboard would actually present it.
  19
  20  Static HTML export cannot call back into a live Python backend, so
full
  21  interactivity is achieved the standard way for a no-server Plotly
  22  dashboard: every (target, grid-point) prediction is precomputed here
```

```
and
  23  embedded as a Plotly trace, with the dropdown just toggling which
  24  precomputed trace set is visible - genuinely interactive in any
browser,
  25  not a screenshot.
  26  """
  27
  28  import numpy as np
  29  import pandas as pd
  30  import joblib
  31  import plotly.graph_objects as go
  32
  33  from src.uq.mondrian_cp import LEVEL_NAMES, assign_categories,
conformal_quantile
  34
  35  MODELS_DIR = "models"
  36  CALIBRATION_DATA_PATH = "data/processed/cp_calibration_data.parquet"
  37  OUT_PATH = "reports/assignments/figures/ops_dashboard.html"
  38
  39  FEATURES = ["n_capacity", "arrival_rate_multiplier"]
  40  TARGETS = ["n_patients", "mean_wait_minutes", "mean_total_minutes",
"p95_wait_minutes"]
  41  ALPHA = 0.1
  42
  43  CAPACITY_GRID = np.arange(15, 46, 3)
  44  ARRIVAL_GRID = np.round(np.arange(0.8, 1.31, 0.05), 2)
  45
  46
  47  def build_category_quantiles(cal_df, cap_bounds, arr_bounds):
  48      cal_cat = assign_categories(cal_df, cap_bounds, arr_bounds)
  49      quantiles = {}
  50      for target in TARGETS:
  51          model = joblib.load(f"{MODELS_DIR}/surrogate_{target}.joblib")
  52          yhat_cal = model.predict(cal_df[FEATURES])
  53          abs_resid = np.abs(cal_df[target].values - yhat_cal)
  54          quantiles[target] = {
  55              cat: conformal_quantile(abs_resid[cal_cat == cat], ALPHA)
  56              for cat in set(cal_cat)
  57          }
  58      return quantiles
  59
  60
  61  def main():
  62      cal_df = pd.read_parquet(CALIBRATION_DATA_PATH)
  63      cap_bounds = np.quantile(cal_df["n_capacity"], [1 / 3, 2 / 3])
  64      arr_bounds = np.quantile(cal_df["arrival_rate_multiplier"], [1 /
3, 2 / 3])
  65      cat_quantiles = build_category_quantiles(cal_df, cap_bounds,
arr_bounds)
  66
  67      grid_cap, grid_arr = np.meshgrid(CAPACITY_GRID, ARRIVAL_GRID)
  68      grid_df = pd.DataFrame({"n_capacity": grid_cap.ravel(),
"arrival_rate_multiplier": grid_arr.ravel()})
  69      grid_cat = assign_categories(grid_df, cap_bounds, arr_bounds)
  70
  71      detail = pd.read_csv("results/tables/mondrian_cp_detail.csv")
  72
  73      heatmap_traces, bar_traces, buttons = [], [], []
  74      for i, target in enumerate(TARGETS):
  75          model = joblib.load(f"{MODELS_DIR}/surrogate_{target}.joblib")
  76          yhat = model.predict(grid_df[FEATURES])
  77          q = np.array([cat_quantiles[target][c] for c in grid_cat])
  78          lower, upper = yhat - q, yhat + q
  79
  80          hover = np.array([
```

```
  81
f"n_capacity={c}<br>arrival_mult={a:.2f}<br>predicted={p:.1f}"
  82              f"<br>90% interval=[{lo:.1f}, {hi:.1f}]<br>category={cat}"
  83              for c, a, p, lo, hi, cat in zip(grid_df["n_capacity"],
grid_df["arrival_rate_multiplier"], yhat, lower, upper, grid_cat)
  84          ]).reshape(grid_cap.shape)
  85
  86          heatmap_traces.append(go.Heatmap(
  87              x=CAPACITY_GRID, y=ARRIVAL_GRID,
z=yhat.reshape(grid_cap.shape),
  88              colorscale="RdYlGn_r", text=hover, hoverinfo="text",
  89              visible=(i == 0), colorbar=dict(title=target, x=0.44,
len=0.9),
  90          ))
  91
  92          sub = detail[detail["target"] ==
target].sort_values("category")
  93          bar_traces.append(go.Bar(
  94              x=sub["category"], y=sub["mondrian_width"], name=target,
  95              marker_color=["#c0392b" if "Low/arrival=High" in c else
"#2a78d6" for c in sub["category"]],
  96              visible=(i == 0), xaxis="x2", yaxis="y2",
  97          ))
  98
  99          buttons.append(dict(
 100              label=target, method="update",
 101              args=[{"visible": [j == i for j in range(len(TARGETS))] *
2}],
 102          ))
 103
 104      fig = go.Figure(data=heatmap_traces + bar_traces)
 105      fig.update_layout(
 106          title="ER Operations Dashboard (Prototype) - Predicted Load
and Mondrian CP Interval Width",
 107          updatemenus=[dict(active=0, buttons=buttons, x=1.15, y=1.15,
xanchor="left")],
 108          grid=dict(rows=1, columns=2, pattern="independent"),
 109          xaxis=dict(domain=[0.0, 0.42], title="Staffing capacity"),
 110          yaxis=dict(domain=[0.0, 1.0], title="Arrival-rate
multiplier"),
 111          xaxis2=dict(domain=[0.55, 1.0], title="Category",
tickangle=30),
 112          yaxis2=dict(domain=[0.0, 1.0], title="Mondrian interval
width"),
 113          width=1150, height=520, template="plotly_white",
 114      )
 115      fig.write_html(OUT_PATH, include_plotlyjs="cdn")
 116      print(f"Saved {OUT_PATH}")
 117
 118      # static PNG snapshot (mean_wait_minutes view) for embedding in
the PDF
 119      # report, which cannot embed the live interactive HTML directly -
built
 120      # as an independent figure (no updatemenus/dropdown at all) rather
than
 121      # copying the interactive figure's layout, since dropdown UI
chrome has
 122      # no function in a static image and its "active" label does not
track
 123      # which traces were actually made visible below.
 124      snap_heatmap = go.Heatmap(heatmap_traces[1])
 125      snap_heatmap.visible = True
 126      snap_bar = go.Bar(bar_traces[1])
 127      snap_bar.visible = True
 128      snapshot = go.Figure(data=[snap_heatmap, snap_bar])
```

```
 129      snapshot.update_layout(
 130          title="ER Operations Dashboard (Prototype) - mean_wait_minutes
view",
 131          grid=dict(rows=1, columns=2, pattern="independent"),
 132          xaxis=dict(domain=[0.0, 0.42], title="Staffing capacity"),
 133          yaxis=dict(domain=[0.0, 1.0], title="Arrival-rate
multiplier"),
 134          xaxis2=dict(domain=[0.55, 1.0], title="Category",
tickangle=30),
 135          yaxis2=dict(domain=[0.0, 1.0], title="Mondrian interval
width"),
 136          width=1150, height=520, template="plotly_white",
 137      )
 138
snapshot.write_image("reports/assignments/figures/ops_dashboard_snapshot.png
", scale=2)
 139      print("Saved
reports/assignments/figures/ops_dashboard_snapshot.png")
 140
 141
 142  if __name__ == "__main__":
 143      main()
```

src/deployment/capacity_optimization.py - Conformal-interval-constrained capacity optimization (Chapter 10)

```
   1  """
   2  Prescriptive capacity allocation under uncertainty: formulates
choosing
   3  staffing capacity as a constrained optimization problem where the
   4  constraint is expressed directly in terms of this project's own
Mondrian
   5  conformal prediction intervals, rather than the surrogate's raw point
   6  prediction - a concrete, worked example of what "using a conformal
   7  interval as an explicit constraint set" (rather than only a reported
   8  number) looks like in an operational decision.
   9
  10  Problem, for a given arrival-rate scenario a and a policy wait-time
  11  ceiling W_max:
  12
  13      minimize    n_capacity                              (staffing cost
proxy)
  14      subject to  yhat(n_capacity, a) + q_cat(n_capacity, a) <= W_max
  15
  16  i.e. choose the *cheapest* staffing level whose Mondrian conformal
upper
  17  bound - not its point prediction - stays under the ceiling. Using the
  18  interval's upper bound rather than the point prediction is what makes
this
  19  a genuinely conservative, risk-aware plan: because Mondrian's interval
  20  width varies by category (Section on the core finding), the safety
margin
  21  this constraint enforces is automatically larger in the categories
this
  22  project's own results show are least reliably predicted, and smaller
  23  where they are not - the optimization inherits that structure directly
  24  from the calibrated intervals rather than needing it encoded
separately.
  25
  26  Solved by direct grid search over the small, integer n_capacity domain
```

```
this project's DES already uses (15-45) - exact for a monotone-ish
constraint on a small discrete domain, no need for a general-purpose
nonlinear solver.
"""

import numpy as np
import pandas as pd
import joblib

from src.uq.mondrian_cp import LEVEL_NAMES, assign_categories,
conformal_quantile

MODELS_DIR = "models"
CALIBRATION_DATA_PATH = "data/processed/cp_calibration_data.parquet"
OUT_PATH = "results/tables/capacity_optimization.csv"

FEATURES = ["n_capacity", "arrival_rate_multiplier"]
TARGET = "p95_wait_minutes"
ALPHA = 0.1

CAPACITY_DOMAIN = np.arange(15, 46)
ARRIVAL_SCENARIOS = [0.8, 0.9, 1.0, 1.1, 1.2, 1.3]
W_MAX_SWEEP = [60, 90, 120, 180, 240]  # policy wait-time ceilings
(minutes), swept rather than a single asserted value


def main():
    cal_df = pd.read_parquet(CALIBRATION_DATA_PATH)
    cap_bounds = np.quantile(cal_df["n_capacity"], [1 / 3, 2 / 3])
    arr_bounds = np.quantile(cal_df["arrival_rate_multiplier"], [1 /
3, 2 / 3])
    cal_cat = assign_categories(cal_df, cap_bounds, arr_bounds)

    model = joblib.load(f"{MODELS_DIR}/surrogate_{TARGET}.joblib")
    yhat_cal = model.predict(cal_df[FEATURES])
    abs_resid_cal = np.abs(cal_df[TARGET].values - yhat_cal)
    cat_quantile = {cat: conformal_quantile(abs_resid_cal[cal_cat ==
cat], ALPHA) for cat in set(cal_cat)}

    grid_df = pd.DataFrame(
        [(c, a) for c in CAPACITY_DOMAIN for a in ARRIVAL_SCENARIOS],
        columns=FEATURES,
    )
    grid_df["predicted"] = model.predict(grid_df[FEATURES])
    grid_df["category"] = assign_categories(grid_df, cap_bounds,
arr_bounds)
    grid_df["q"] = grid_df["category"].map(cat_quantile)
    grid_df["upper_bound"] = grid_df["predicted"] + grid_df["q"]

    rows = []
    for w_max in W_MAX_SWEEP:
        for arrival in ARRIVAL_SCENARIOS:
            sub = grid_df[grid_df["arrival_rate_multiplier"] ==
arrival].sort_values("n_capacity")
            feasible = sub[sub["upper_bound"] <= w_max]
            # for comparison: capacity needed if using the raw point
prediction instead of the CP upper bound
            feasible_point_only = sub[sub["predicted"] <= w_max]

            n_star_cp = int(feasible["n_capacity"].min()) if
len(feasible) else None
            n_star_point =
int(feasible_point_only["n_capacity"].min()) if len(feasible_point_only)
else None
```

```
            rows.append({
                "W_max_minutes": w_max,
                "arrival_rate_multiplier": arrival,
                "n_capacity_star_cp_constrained": n_star_cp,
                "n_capacity_star_point_prediction_only": n_star_point,
                "extra_capacity_from_using_cp_bound": (n_star_cp -
n_star_point) if (n_star_cp is not None and n_star_point is not None) else
None,
            })

    out_df = pd.DataFrame(rows)
    out_df.to_csv(OUT_PATH, index=False)
    print(out_df.to_string(index=False))
    print(f"\nSaved {OUT_PATH}")


if __name__ == "__main__":
    main()
```

APPENDIX B

Supplementary Result Tables

Full per-category detail tables (all nine categories, not only the worst case shown in Chapter 7) for the core Mondrian CP comparison, for completeness.

Table B.1: Full 9-category detail: mean_total_minutes.

Category	n_cal	n_test	Pooled cov.	Pooled width	Mondrian cov.	Mondrian width
staff=High/arrival=High	126	142	93.7%	46.4	91.5%	44.5
staff=High/arrival=Low	136	118	100.0%	46.4	95.8%	16.8
staff=High/arrival=Med	157	105	99.0%	46.4	87.6%	19.8
staff=Low/arrival=High	132	88	80.7%	46.4	92.0%	58.5
staff=Low/arrival=Low	128	117	85.5%	46.4	88.0%	50.9
staff=Low/arrival=Med	121	100	82.0%	46.4	97.0%	70.0
staff=Med/arrival=High	142	114	86.0%	46.4	86.8%	47.8
staff=Med/arrival=Low	136	109	90.8%	46.4	91.7%	50.0
staff=Med/arrival=Med	122	107	90.7%	46.4	96.3%	56.8

Table B.2: Full 9-category detail: mean_wait_minutes.

Category	n_cal	n_test	Pooled cov.	Pooled width	Mondrian cov.	Mondrian width
staff=High/arrival=Hi	126	142	95.1%	47.2	92.3%	39.1

gh						
staff=High/arrival=Low	136	118	100.0%	47.2	94.9%	2.0
staff=High/arrival=Med	157	105	99.0%	47.2	88.6%	6.5
staff=Low/arrival=High	132	88	68.2%	47.2	90.9%	65.4
staff=Low/arrival=Low	128	117	82.9%	47.2	85.5%	51.0
staff=Low/arrival=Med	121	100	79.0%	47.2	98.0%	65.2
staff=Med/arrival=High	142	114	87.7%	47.2	88.6%	48.6
staff=Med/arrival=Low	136	109	91.7%	47.2	89.9%	40.2
staff=Med/arrival=Med	122	107	90.7%	47.2	94.4%	51.5

Table B.3: Full 9-category detail: n_patients.

Category	n_cal	n_test	Pooled cov.	Pooled width	Mondrian cov.	Mondrian width
staff=High/arrival=High	126	142	87.3%	43.6	95.1%	53.8
staff=High/arrival=Low	136	118	88.1%	43.6	89.0%	47.1
staff=High/arrival=Med	157	105	89.5%	43.6	92.4%	50.0
staff=Low/arrival=High	132	88	98.9%	43.6	96.6%	35.0
staff=Low/arrival=Low	128	117	94.9%	43.6	88.0%	34.2
staff=Low/arrival=Med	121	100	97.0%	43.6	87.0%	29.1

staff=Med/arrival=High	142	114	89.5%	43.6	83.3%	38.0
staff=Med/arrival=Low	136	109	94.5%	43.6	97.2%	49.5
staff=Med/arrival=Med	122	107	92.5%	43.6	97.2%	50.1

Table B.4: Full 9-category detail: p95_wait_minutes.

Category	n_cal	n_test	Pooled cov.	Pooled width	Mondrian cov.	Mondrian width
staff=High/arrival=High	126	142	96.5%	362.9	93.0%	305.6
staff=High/arrival=Low	136	118	100.0%	362.9	92.4%	16.8
staff=High/arrival=Med	157	105	99.0%	362.9	87.6%	41.5
staff=Low/arrival=High	132	88	72.7%	362.9	92.0%	650.9
staff=Low/arrival=Low	128	117	84.6%	362.9	86.3%	379.4
staff=Low/arrival=Med	121	100	78.0%	362.9	92.0%	495.2
staff=Med/arrival=High	142	114	92.1%	362.9	93.0%	375.6
staff=Med/arrival=Low	136	109	94.5%	362.9	90.8%	285.1
staff=Med/arrival=Med	122	107	93.5%	362.9	94.4%	368.1